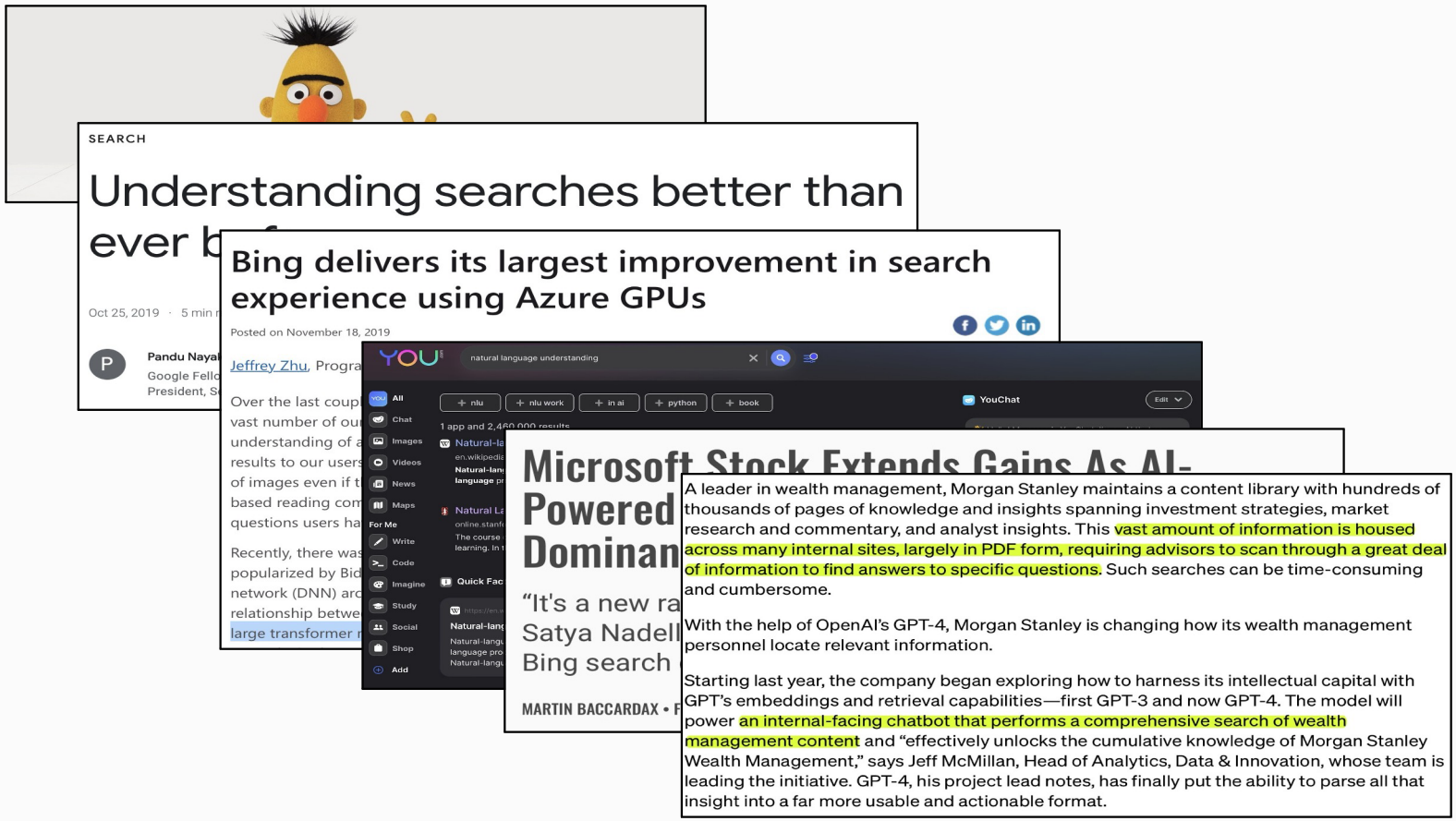
A short course on Retrieval Augmented Generation (RAG)

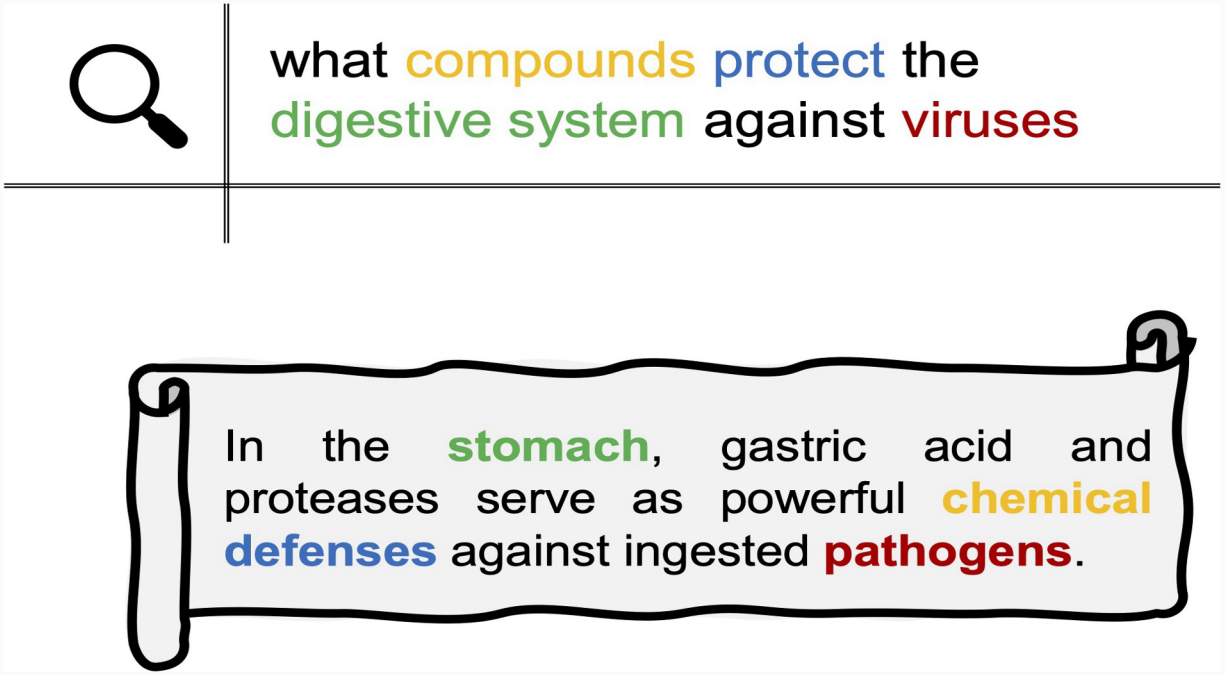
- Motivation for Neural Information Retrieval
- Transformer & BERT
- Neural Information Retrieval

(Classic/Neural) Information Retrieval

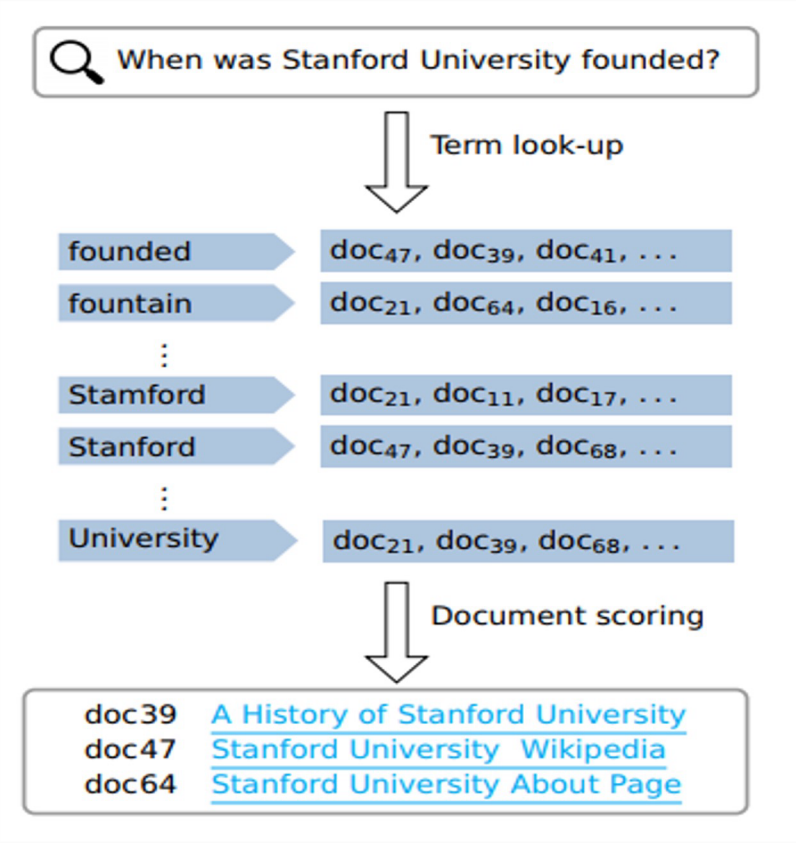
NLP is revolutionizing Information Retrieval (IR)



Classic search engines do not retrieve such a document



Classical IR



Web Indexing

- 1 billion pages. Build “inverted index”
  - Input documents:

    1: cat chases dog

    2: dog loves cat

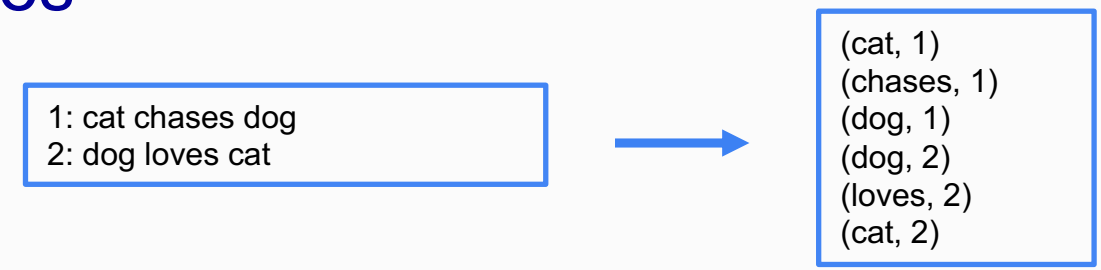
    ...
  - Output index:

    cat 1,2,5,10,20

    dog 1,2,3,8,9
- Q: How can we do this?

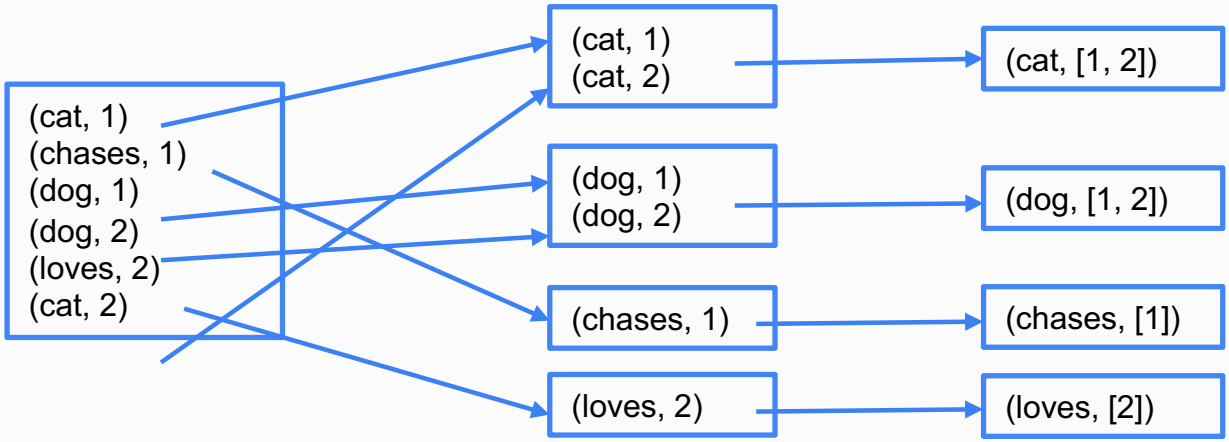
Web Indexing (1)

- Step 1: “Transform” every document into (word, doc_id) tuples



Web Indexing (2)

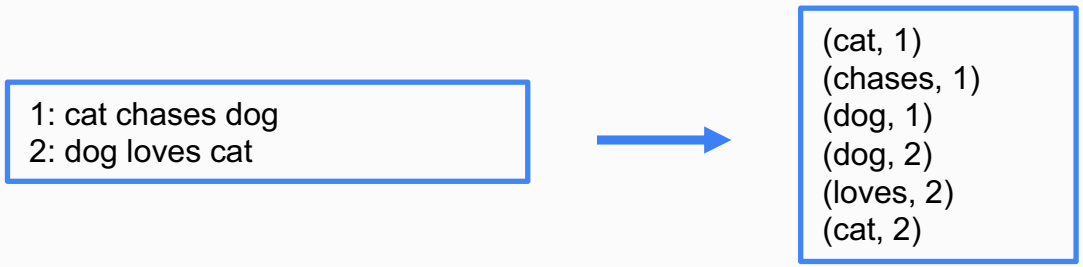
- Step 2: Collect all tuples with the same word and "aggregate" (or concatenate) the doc_id's



- Q: How can we parallelize the two steps on multiple machines?

Web Indexing (3)

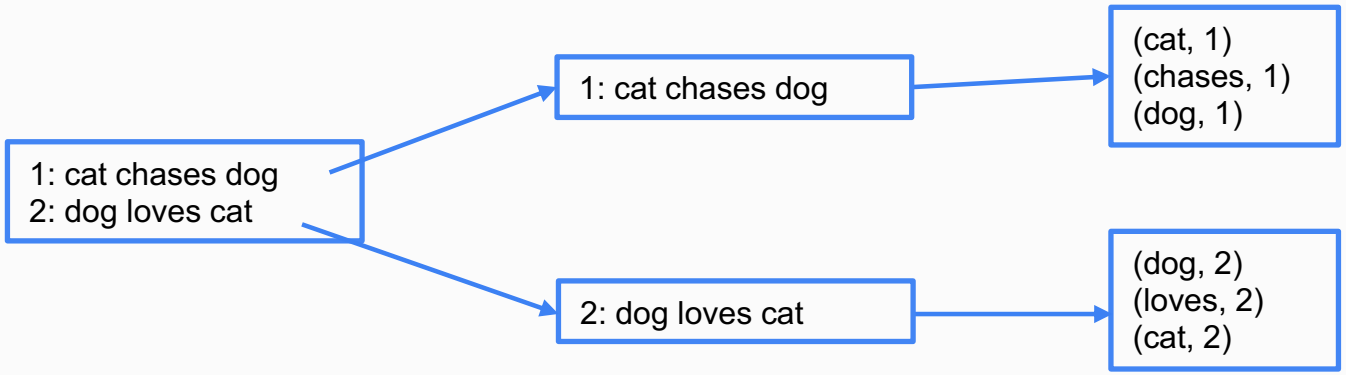
- Step 1: “Transform” every document into (word, doc_id) tuples



- Q: Can the transformation of each document be done independently of each other?

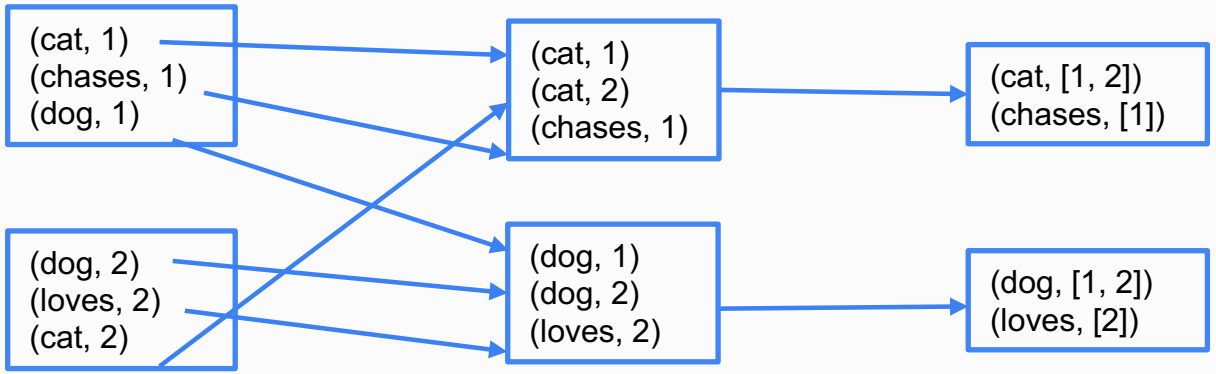
Web Indexing (4)

- Step 1: For parallel processing
  - Split input data into multiple independent chunks
  - Move each chunk to separate machine
  - Perform “transformation” on multiple machines in parallel

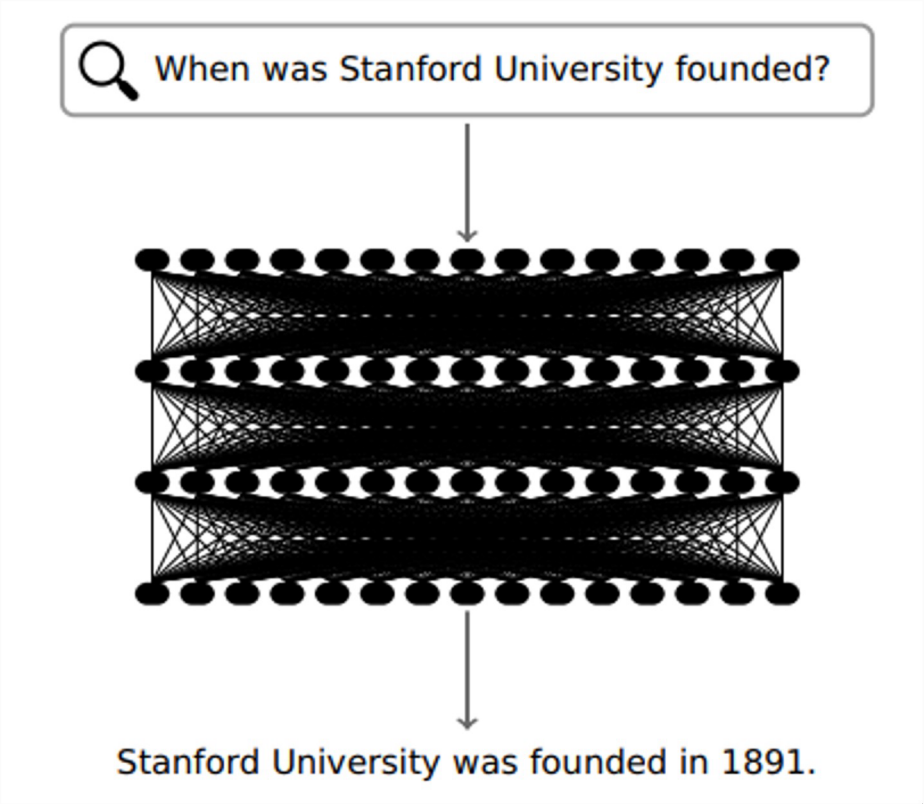


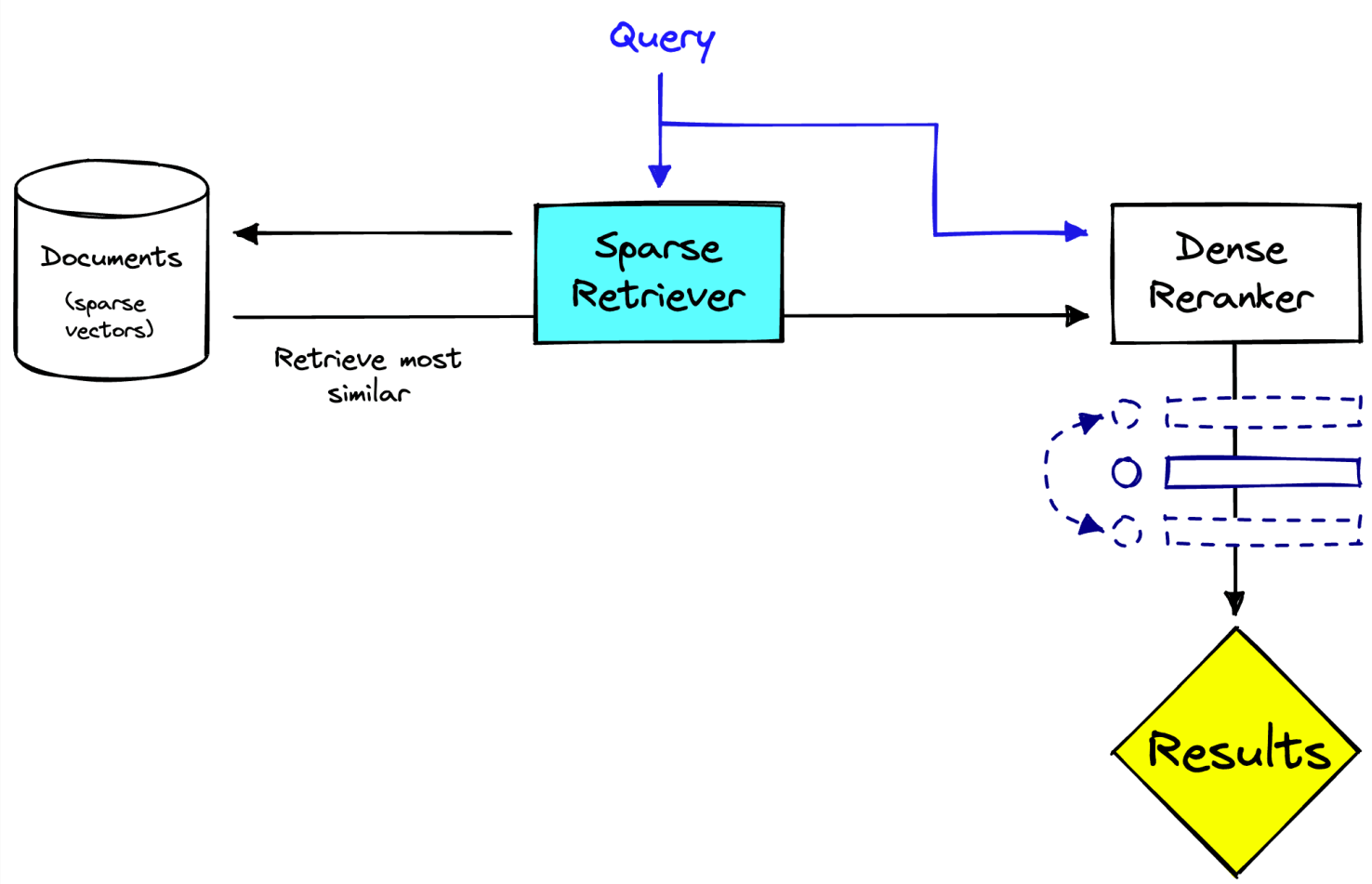
Web Indexing (5)

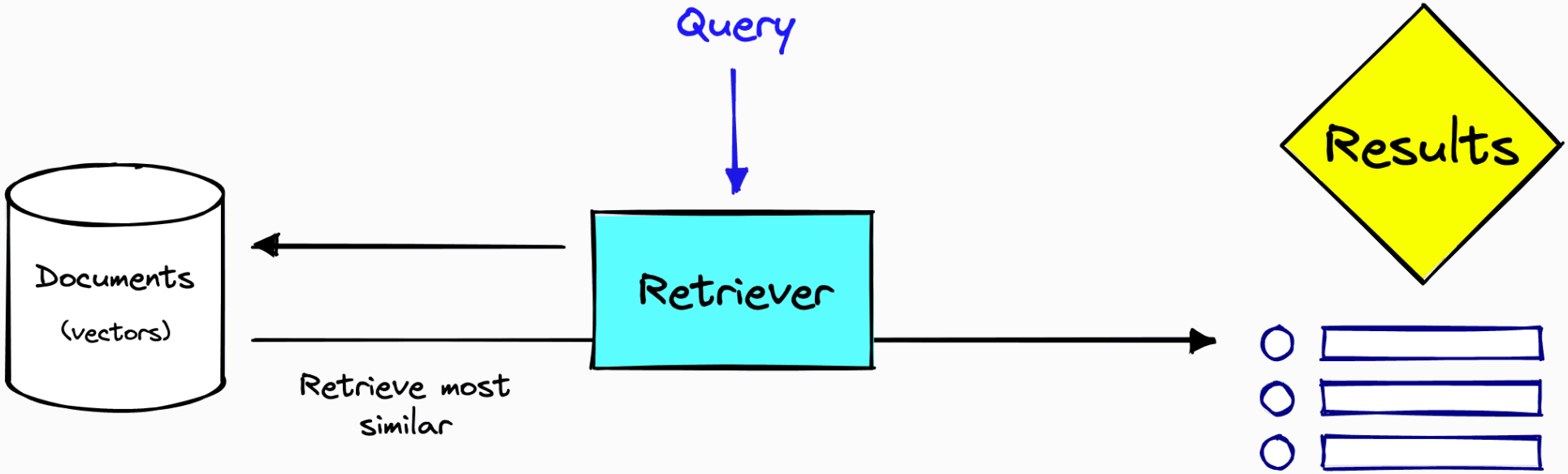
- Q: How can we parallelize second “concatenation step”?
- Step 2: For parallel processing
  - Move the tuples with the same word to the same machine
  - Perform aggregation on multiple machine in parallel



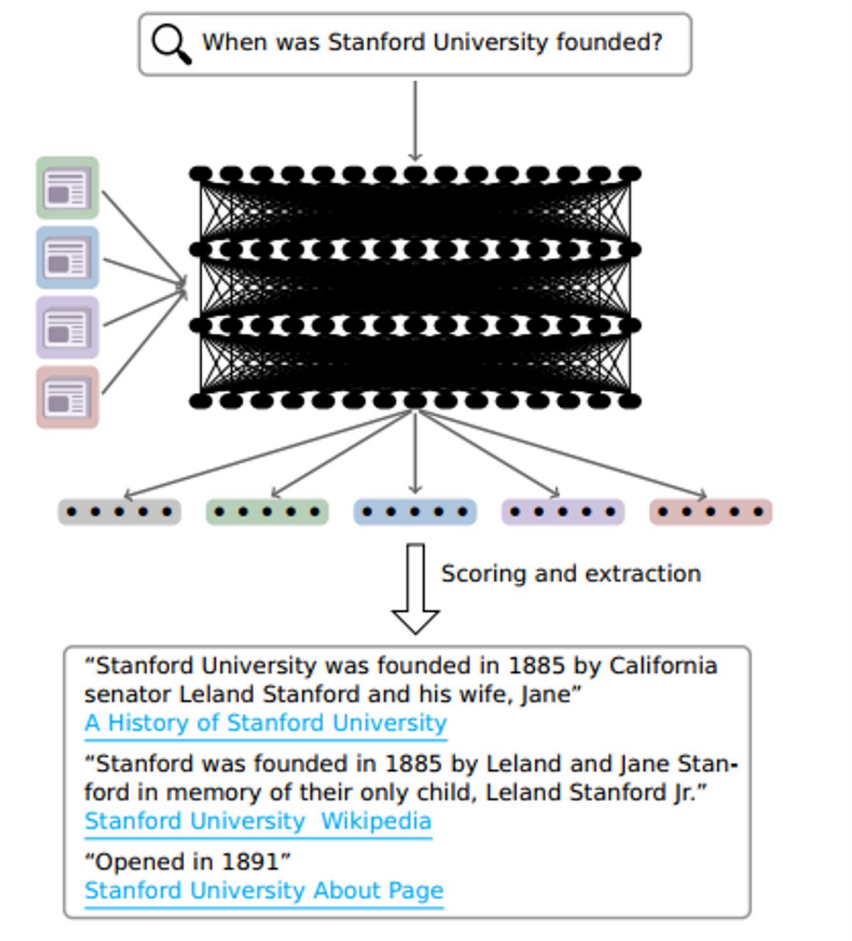
“LLMs for everything” for replacing search engine




Query
Documents
(sparse vectors)
Sparse Retriever
Dense Reranker
Retrieve most similar
Results


Query
Results
Documents
(vectors)
Retriever
Retrieve most
similar

Neural IR


When was Stanford University founded?
Scoring and extraction
"Stanford University was founded in 1885 by California senator Leland Stanford and his wife, Jane"
A History of Stanford University
"Stanford was founded in 1885 by Leland and Jane Stanford in memory of their only child, Leland Stanford Jr."
Stanford University Wikipedia
"Opened in 1891"
Stanford University About Page

Classical IR

The term–document matrix

	d1	d2	d3	d4	d5	d6	d7	d8	d9	d10	···
against	0	0	0	1	0	0	3	2	3	0	
age	0	0	0	1	0	3	1	0	4	0	
agent	0	0	0	0	0	0	0	0	0	0	
ages	0	0	0	0	0	2	0	0	0	0	
ago	0	0	0	2	0	0	0	0	3	0	
agree	0	1	0	0	0	0	0	0	0	0	
ahead	0	0	0	1	0	0	0	0	0	0	
ain’t	0	0	0	0	0	0	0	0	0	0	
air	0	0	0	0	0	0	0	0	0	0	
aka	0	0	0	1	0	0	0	0	0	0	
⋮											

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

what
candy
nlp
Is
language
…

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

what	0.33
candy	0
nlp	0.33
Is	0.33
language	0
…	…

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

d_1 = what is life ?

candy is life !

$$\begin{matrix} \text{what} \\ \text{candy} \\ \text{nlp} \\ \text{Is} \\ \text{language} \\ \dots \end{matrix} \begin{pmatrix} 0.33 \\ 0 \\ 0.33 \\ 0.33 \\ 0 \\ \dots \end{pmatrix}$$

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

d_1 = what is life ? candy is life !

d_2 = nlp is an acronym for natural language processing

what	0.33
candy	0
nlp	0.33
Is	0.33
language	0
…	…

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)

q=what is nlp

d_1 = what is life ? candy is life !

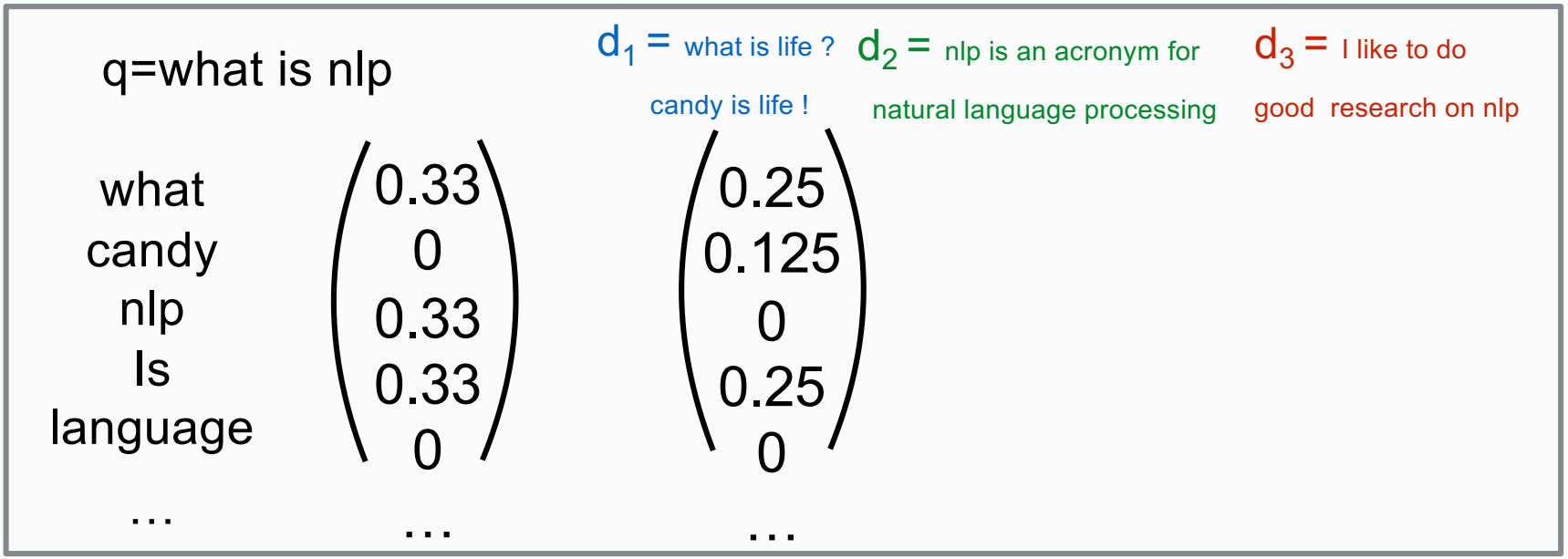
d_2 = nlp is an acronym for natural language processing

d_3 = I like to do good research on nlp

what	0.33
candy	0
nlp	0.33
Is	0.33
language	0
…	…

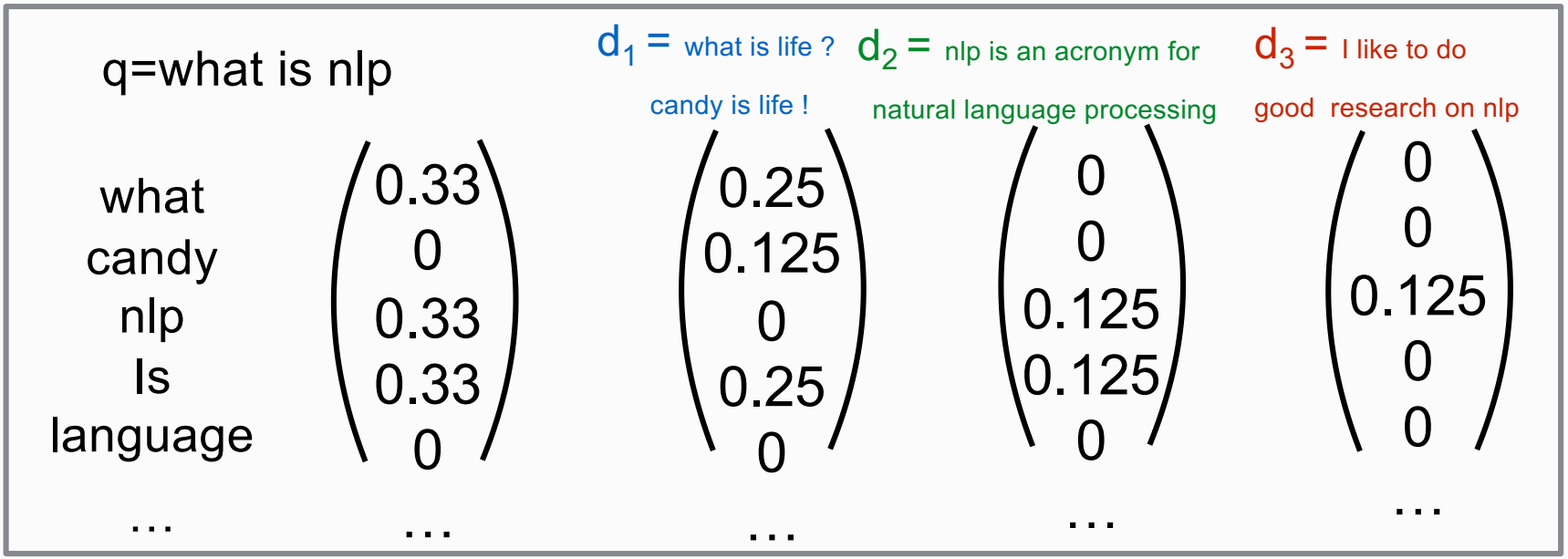
Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)



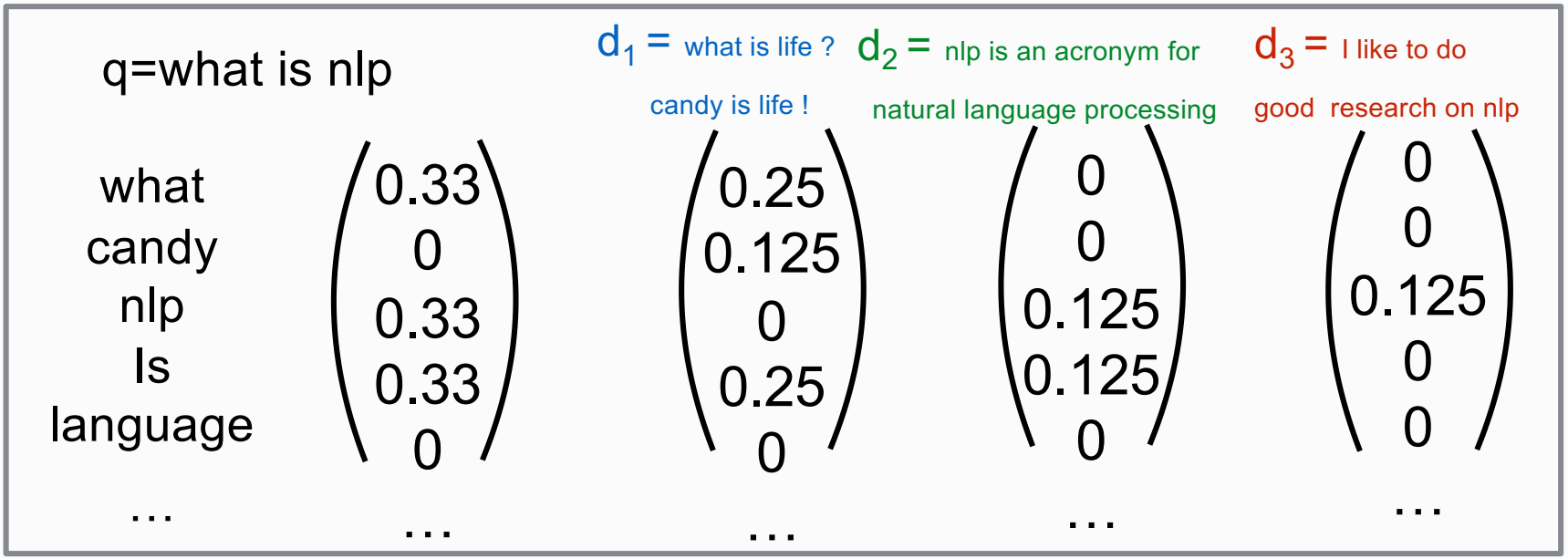
Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)



Sparse Retrieval

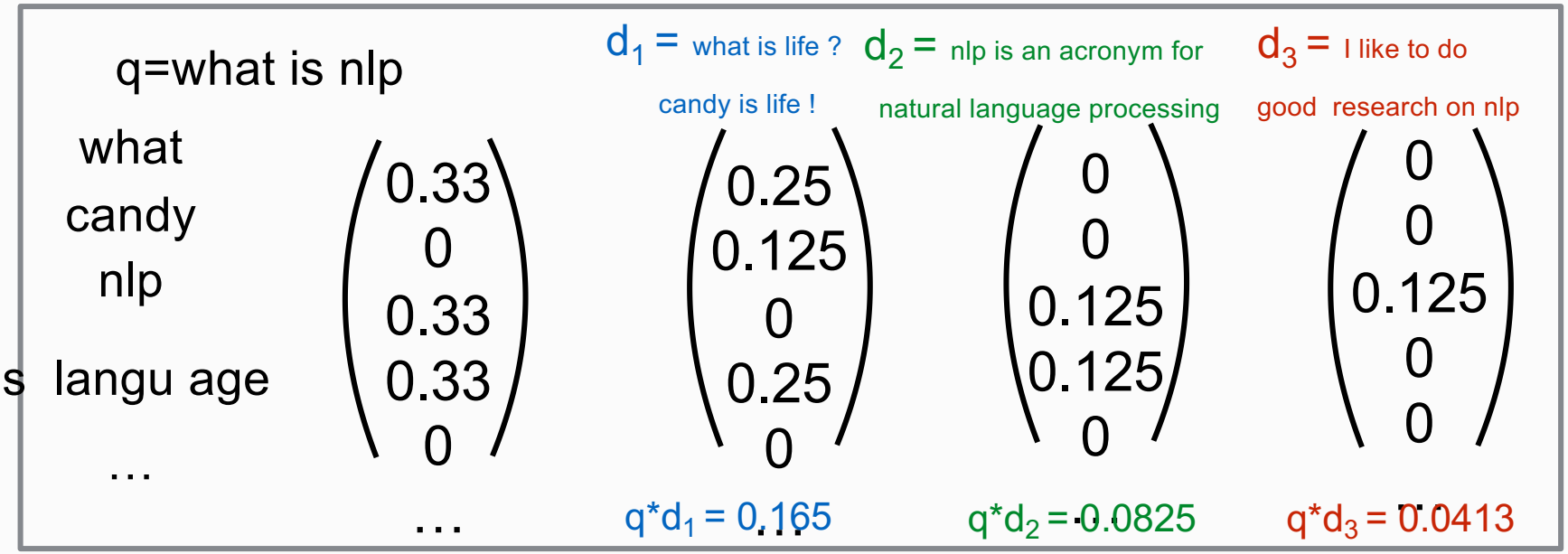
- Express the query and document as a sparse word frequency vector (usually normalized by length)



- Find the document with the highest inner-product or cosine similarity in the document collection

Sparse Retrieval

- Express the query and document as a sparse word frequency vector (usually normalized by length)



- Find the document with the highest inner-product or cosine similarity in the document collection

TF-IDF

For a corpus of documents D:

- Term frequency: $\mathbf{TF}(w, \text{doc}) = \frac{\mathbf{count}(w, \text{doc})}{|\text{doc}|}$
- Document frequency: $\mathbf{df}(w, D) = \left|\{\text{doc} \in D : w \in \text{doc}\}\right|$
- Inverse document frequency: $\text{IDF}(w, D) = \log_e\left(\frac{|D|}{\mathbf{df}(w,D)}\right)$
- $\text{TF-IDF}(w, \text{doc}, D) = \text{TF}(w, \text{doc}) \cdot \text{IDF}(w, D)$

	doc_1	doc_2	doc_3	doc_4
A	10	10	10	10
B	10	10	10	0
C	10	10	0	0
D	0	0	0	1

⇓

⇒

	IDF
A	0.00
B	0.29
C	0.69
D	1.39

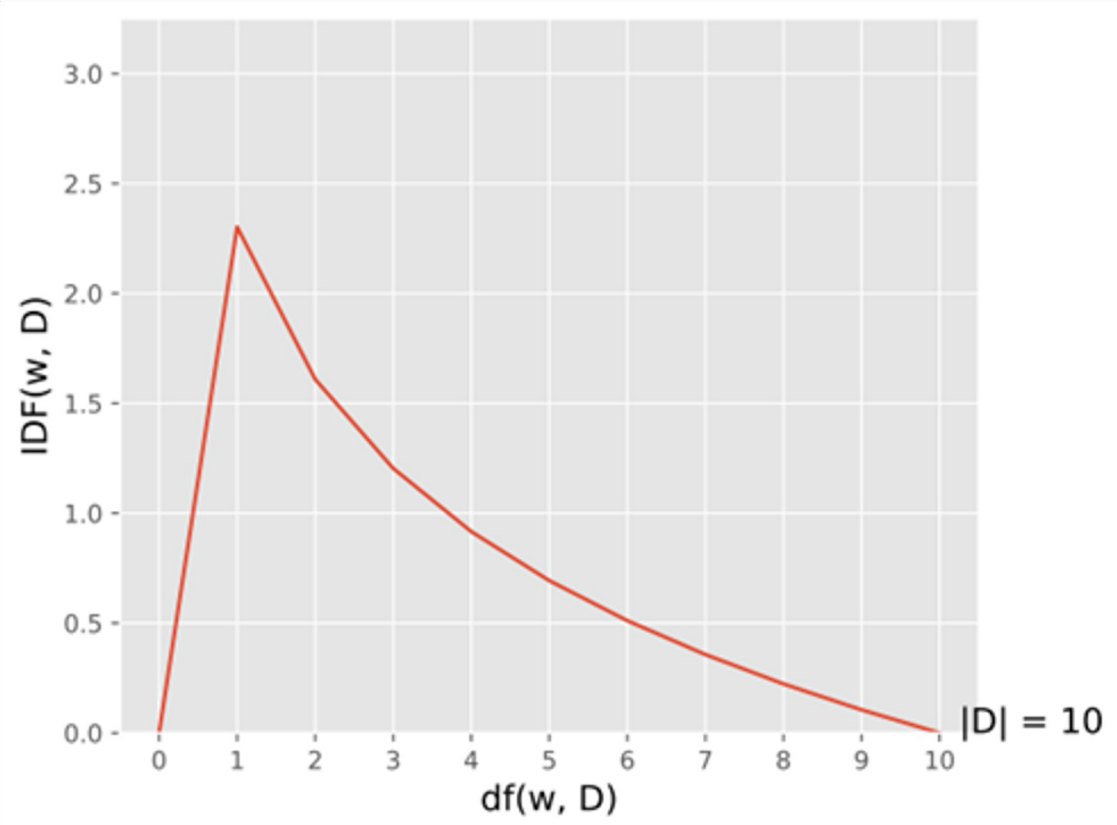
TF

	doc_1	doc_2	doc_3	doc_4
A	0.33	0.33	0.50	0.91
B	0.33	0.33	0.50	0.00
C	0.33	0.33	0.00	0.00
D	0.00	0.00	0.00	0.09

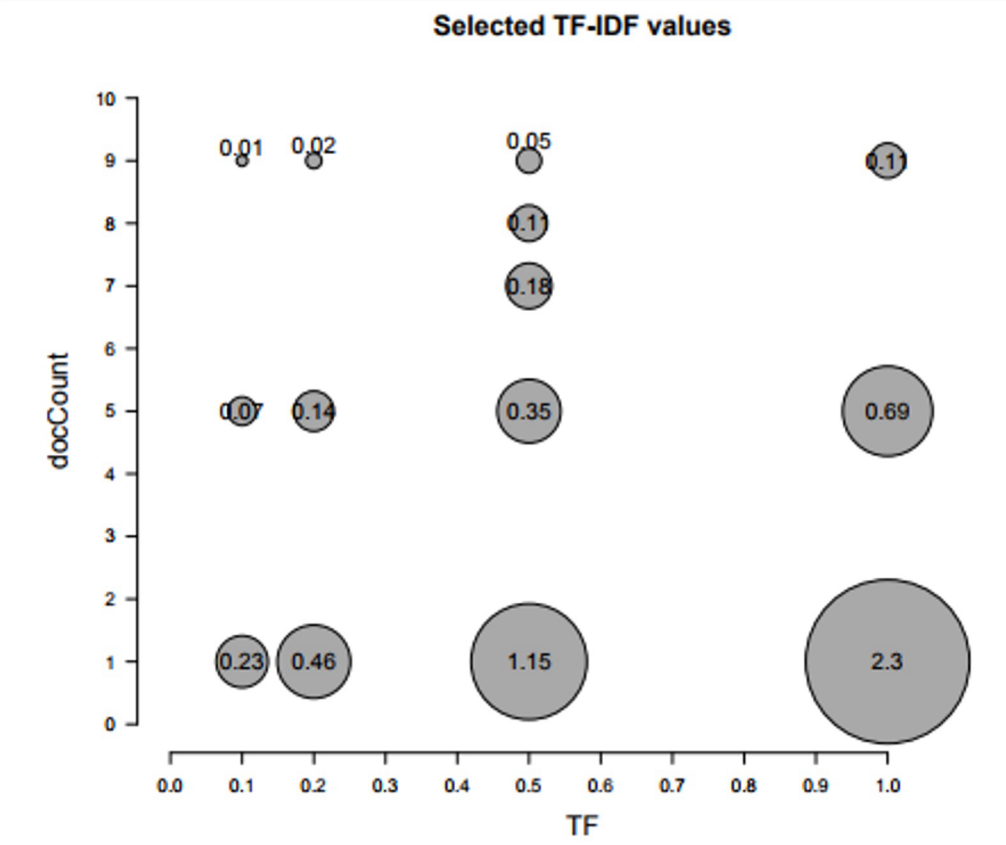
TF-IDF

	doc_1	doc_2	doc_3	doc_4
A	0.00	0.00	0.00	0.00
B	0.10	0.10	0.14	0.00
C	0.23	0.23	0.00	0.00
D	0.00	0.00	0.00	0.13

IDF values



Selected TF-IDF values



Relevance scores

$$\textbf{RelevanceScore}\ (q, \text{doc}, D) = \sum_{w \in q} \textbf{Weight}(w, \text{doc}, D)$$

where **Weight** is often TF-IDF.

BM25

Smoothed IDF

$$\text{IDF}_{\text{BM25}}(w, D) = \log_e\left(1 + \frac{|D| - \mathbf{df}(w, D) + 0.5}{\mathbf{df}(w, D) + 0.5}\right)$$

Scoring

With $k = 1.2$ and $b = 0.75$ (or thereabouts):

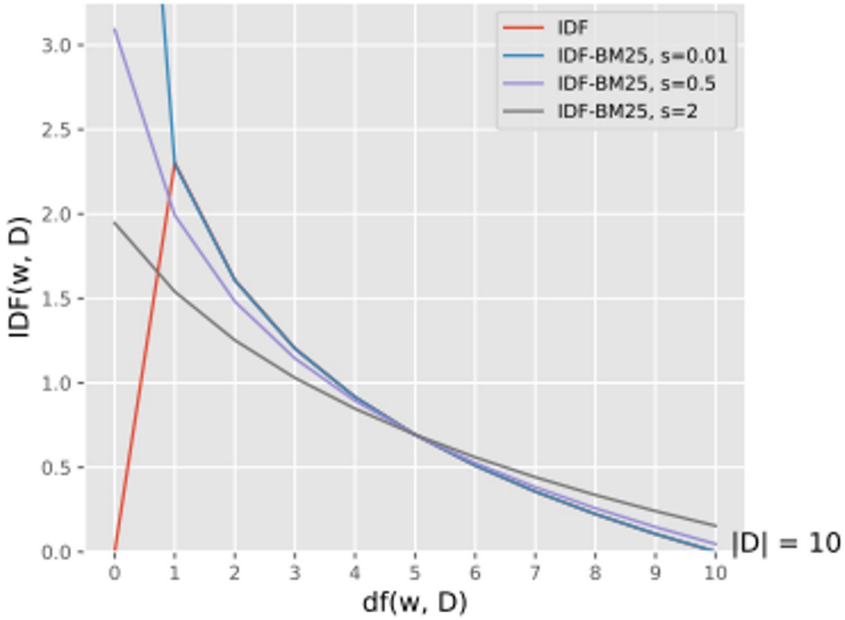
$$\text{Score}_{\text{BM25}}(w, doc) = \frac{\mathbf{TF}(w, \text{doc}) \cdot (k + 1)}{\mathbf{TF}(w, \text{doc}) + k \cdot \left(1 - b + b \cdot \frac{|\text{doc}|}{\text{avgdoclen}}\right)}$$

BM25 Weight

$$\text{BM25}(w, \text{doc}, D) = \text{Score}_{\text{BM25}}(w, \text{doc}) \cdot \text{IDF}_{\text{BM25}}(w, D)$$

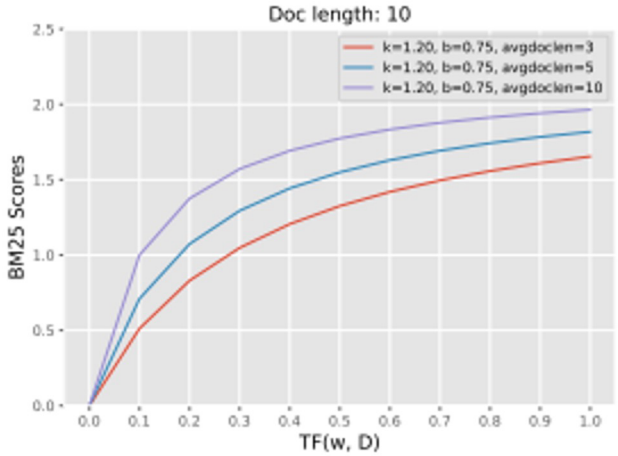
Best Match, Attempt #25; Robertson and Zaragoza 2009

BM25 IDF values



$$\mathrm{IDF}_{\mathrm{BM25}}(w, D) = \log_e\left(1 + \frac{|D| - \mathbf{df}(w, D) + s}{\mathbf{df}(w, D) + s}\right)$$

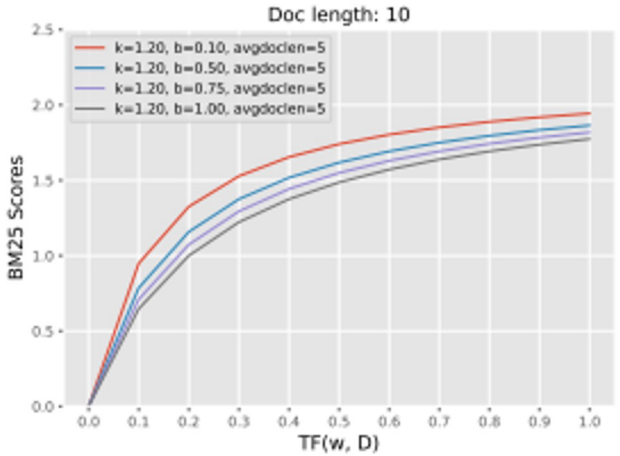
BM25 Scores: avgdoclen

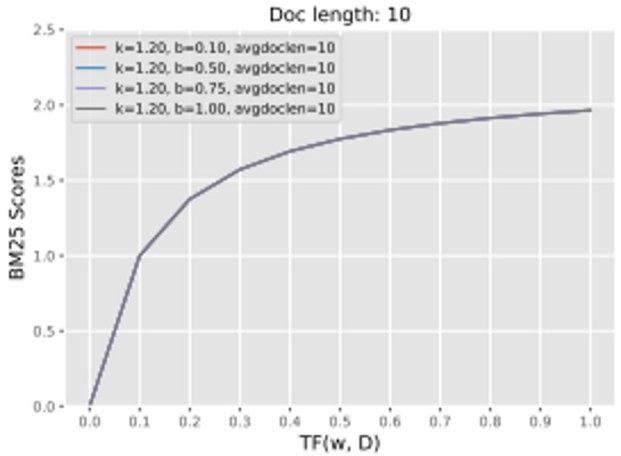


$$\text{Score}_{\text{BM25}}(w, doc) = \frac{\mathbf{TF}(w, \text{doc}) \cdot (k+1)}{\mathbf{TF}(w, \text{doc}) + k \cdot \left(1 - b + b \cdot \frac{|\text{doc}|}{\text{avgdoclen}}\right)}$$

Penalizes long documents

BM25 Scores: b

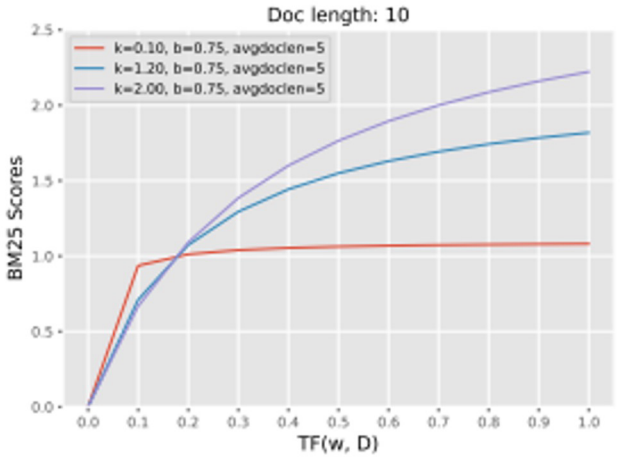


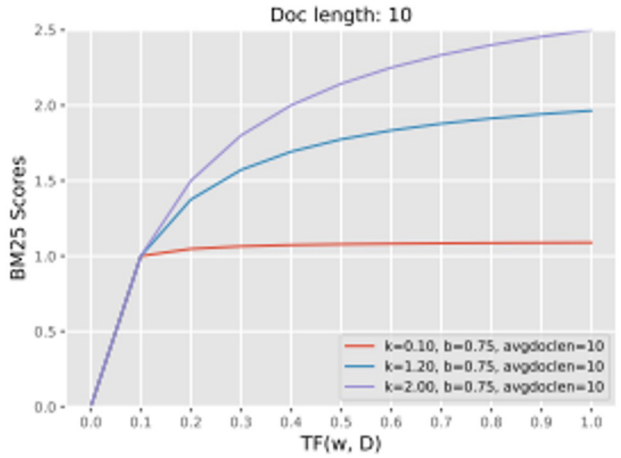


$$\text{Score}_{\text{BM25}}(w, doc) = \frac{\mathbf{TF}(w, \text{doc}) \cdot (k+1)}{\mathbf{TF}(w, \text{doc}) + k \cdot \left(1 - b + b \cdot \frac{|\text{doc}|}{\text{avgdoclen}}\right)}$$

b controls the doc length penalty

BM25 Scores: k

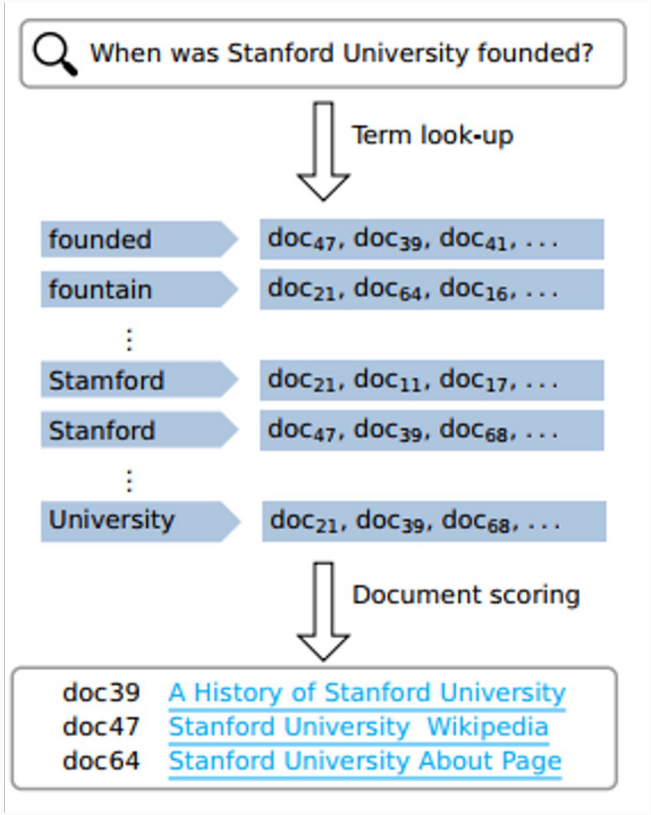




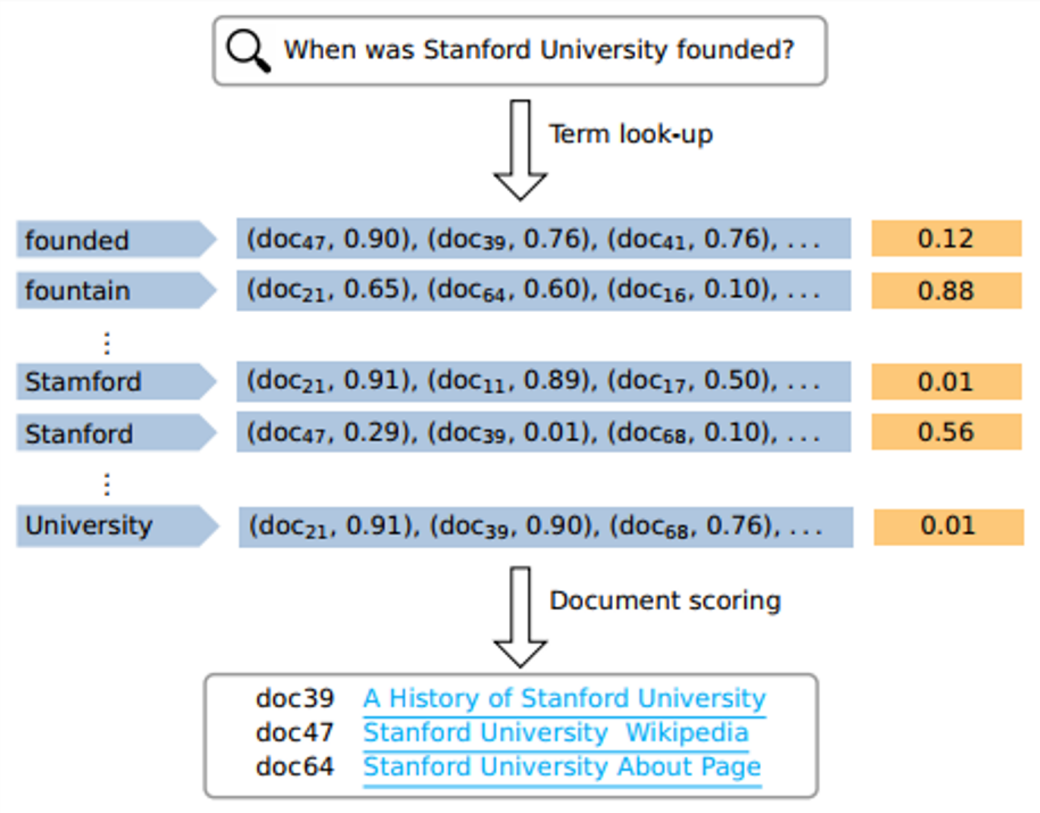
$$\text{Score}_{\text{BM25}}(w, doc) = \frac{\mathbf{TF}(w, \text{doc}) \cdot (k+1)}{\mathbf{TF}(w, \text{doc}) + k \cdot \left(1 - b + b \cdot \frac{|\text{doc}|}{\text{avgdoclen}}\right)}$$

Flattens out higher frequencies

Inverted indices



Inverted indices



IR metrics

Many dimensions

1. Accuracy-style metrics: These will be our focus.
2. Latency: Time to execute a single query.
3. Throughput: Total queries served in a fixed time, perhaps via batch processing.
4. FLOPs: Hardware agnostic measure of compute resources.
5. Disk usage: For the model, index, etc.
6. Memory usage: For the model, index, etc.
7. Cost: Total cost of deployment for a system.

Relevance data types

Given a query *q* and a collection of *N* documents *D*:

1. A complete partial gold ranking **D** = $[\mathrm{doc}_1, \ldots, \mathrm{doc}_N]$ of *D* with respect to *q*.
   - Unlikely unless D was automatically generated.
2. An incomplete partial ranking of *D* with respect to *q*.
3. Labels for which passages in *D* are relevant to *q*.
   - Could be based in a weak supervision heuristic like whether each doc_i contains *q* as a substring.
4. A tuple consisting of one positive document doc^+ for *q* and one or more negatives doc^- for *q*.

Success and Reciprocal Rank

Rank

For a ranking $\mathbf{D} = [\text{doc}_1, \ldots, \text{doc}_N]$, let

$$\textbf{Rank}(q, \mathbf{D}) \in \{1, 2, 3, \ldots\}$$

be the position of the **first** relevant document for q in $\mathbf{D}$.

Success

$$\text{Success@K}(q, \mathbf{D}) = \begin{cases} 1 & \text{if } \textbf{Rank}(q, \mathbf{D}) \leqslant K \\ 0 & \text{otherwise} \end{cases}$$

Reciprocal Rank

$$\text{RR@K}(q, \mathbf{D}) = \begin{cases} \frac{1}{\textbf{Rank}(q, \mathbf{D})} & \text{if } \textbf{Rank}(q, \mathbf{D}) \leqslant K \\ 0 & \text{otherwise} \end{cases}$$

MRR@K is the average of this over multiple queries.

Success and Reciprocal Rank: A comparison

$\mathbf{D}_1$ for q		
1	doc_C	⋆
2	doc_E	⋆
3	doc_D	
4	doc_B	
5	doc_A	
6	doc_F	⋆

- Success@2$(q, \mathbf{D}_1) = 1$
- RR@2$(q, \mathbf{D}_1) = 1/1$

$\mathbf{D}_2$ for q		
1	doc_A	
2	doc_C	⋆
3	doc_G	
4	doc_B	
5	doc_E	⋆
6	doc_F	⋆

- Success@2$(q, \mathbf{D}_2) = 1$
- RR@2$(q, \mathbf{D}_2) = 1/2$

$\mathbf{D}_3$ for q		
1	doc_D	
2	doc_B	
3	doc_E	⋆
4	doc_C	⋆
5	doc_F	⋆
6	doc_A	

- Success@2$(q, \mathbf{D}_3) = 0$
- RR@2$(q, \mathbf{D}_3) = 0$

Precision and Recall

Ret($\mathbf{D}, K$) is the set of documents at or above K in $\mathbf{D}$.

Rel($\mathbf{D}, q$) is the set of all documents that are relevant q.

Precision

$$\text{Prec@K}(q, \mathbf{D}) = \frac{|\text{Ret}(\mathbf{D}, K) \cap \text{Rel}(\mathbf{D}, q)|}{K}$$

Recall

$$\text{Rec@K}(q, \mathbf{D}) = \frac{|\text{Ret}(\mathbf{D}, K) \cap \text{Rel}(\mathbf{D}, q)|}{|\text{Rel}(\mathbf{D}, q)|}$$

Precision and Recall examples

$\mathbf{D}_1$ for q		
1	doc_C	★
2	doc_E	★
3	doc_D	
4	doc_B	
5	doc_A	
6	doc_F	★

- $\text{Prec@2}(q, \mathbf{D}_1) = 2/2$
- $\text{Rec@2}(q, \mathbf{D}_1) = 2/3$

$\mathbf{D}_2$ for q		
1	doc_A	
2	doc_C	★
3	doc_G	
4	doc_B	
5	doc_E	★
6	doc_F	★

- $\text{Prec@2}(q, \mathbf{D}_2) = 1/2$
- $\text{Rec@2}(q, \mathbf{D}_2) = 1/3$

$\mathbf{D}_3$ for q		
1	doc_D	
2	doc_B	
3	doc_E	★
4	doc_C	★
5	doc_F	★
6	doc_A	

- $\text{Prec@2}(q, \mathbf{D}_3) = 0/2$
- $\text{Rec@2}(q, \mathbf{D}_3) = 0/3$

Precision and Recall examples

$\mathbf{D}_1$ for q		
1	doc_C	★
2	doc_E	★
3	doc_D	
4	doc_B	
5	doc_A	
6	doc_F	★

- $\text{Prec@5}(q, \mathbf{D}_1) = 2/5$
- $\text{Rec@5}(q, \mathbf{D}_1) = 2/3$

$\mathbf{D}_2$ for q		
1	doc_A	
2	doc_C	★
3	doc_G	
4	doc_B	
5	doc_E	★
6	doc_F	★

- $\text{Prec@5}(q, \mathbf{D}_2) = 2/5$
- $\text{Rec@5}(q, \mathbf{D}_2) = 2/3$

$\mathbf{D}_3$ for q		
1	doc_D	
2	doc_B	
3	doc_E	★
4	doc_C	★
5	doc_F	★
6	doc_A	

- $\text{Prec@5}(q, \mathbf{D}_3) = 3/5$
- $\text{Rec@5}(q, \mathbf{D}_3) = 3/3$

Average Precision

$$\text{AvgPrec}(q, \mathbf{D}) = \frac{\sum_{i=1}^{|\mathbf{D}|} \begin{cases} \text{Prec@i}(q, \mathbf{D}) & \text{if Rel}(q, \text{doc}_i) \\ 0 & \text{otherwise} \end{cases}}{|\text{Rel}(\mathbf{D}, q)|}$$

$\mathbf{D}_1$ for q		
1	doc_C	⋆
2	doc_E	⋆
3	doc_D	
4	doc_B	
5	doc_A	
6	doc_F	⋆

$\text{Prec@1}(q, \mathbf{D}) = 1/1 +$
$\text{Prec@2}(q, \mathbf{D}) = 2/2 +$
$\text{Prec@6}(q, \mathbf{D}) = 3/6 +$
2.5/3

$\mathbf{D}_2$ for q		
1	doc_A	
2	doc_C	⋆
3	doc_G	
4	doc_B	
5	doc_E	⋆
6	doc_F	⋆

$\text{Prec@2}(q, \mathbf{D}) = 1/2 +$
$\text{Prec@5}(q, \mathbf{D}) = 2/5 +$
$\text{Prec@6}(q, \mathbf{D}) = 3/6 +$
1.4/3

$\mathbf{D}_3$ for q		
1	doc_D	
2	doc_B	
3	doc_E	⋆
4	doc_C	⋆
5	doc_F	⋆
6	doc_A	

$\text{Prec@3}(q, \mathbf{D}) = 1/3 +$
$\text{Prec@4}(q, \mathbf{D}) = 2/4 +$
$\text{Prec@5}(q, \mathbf{D}) = 3/5 +$
1.43/3

Which metric? There is no single answer!

1. Is the cost of scrolling through K passages low? Then perhaps Success@K is fine-grained enough.
2. Are there multiple relevant documents per query? If so, Success@K and RR@K may be too coarse-grained.
3. Is it more important to find every relevant document? If so, favor Recall.
4. Is it more important to review only relevant documents? If so, favor Precision.
5. F1@K is the harmonic mean of Prec@K and Recall@K. It can be used where there are multiple relevant documents but their relative order above K doesn't matter that much.
6. AvgPrec will give the finest-grained distinctions of the metrics discussed here: it is sensitive to rank, precision, and recall.

Transformer & BERT

Outline

- Self-Attention
- Transformer Encoder
- Transformer Decoder
- BERT

Outline

- Transformer
- Bert
- Transformer Decoder
- Dense Retrieval
- Sparse Retrieval

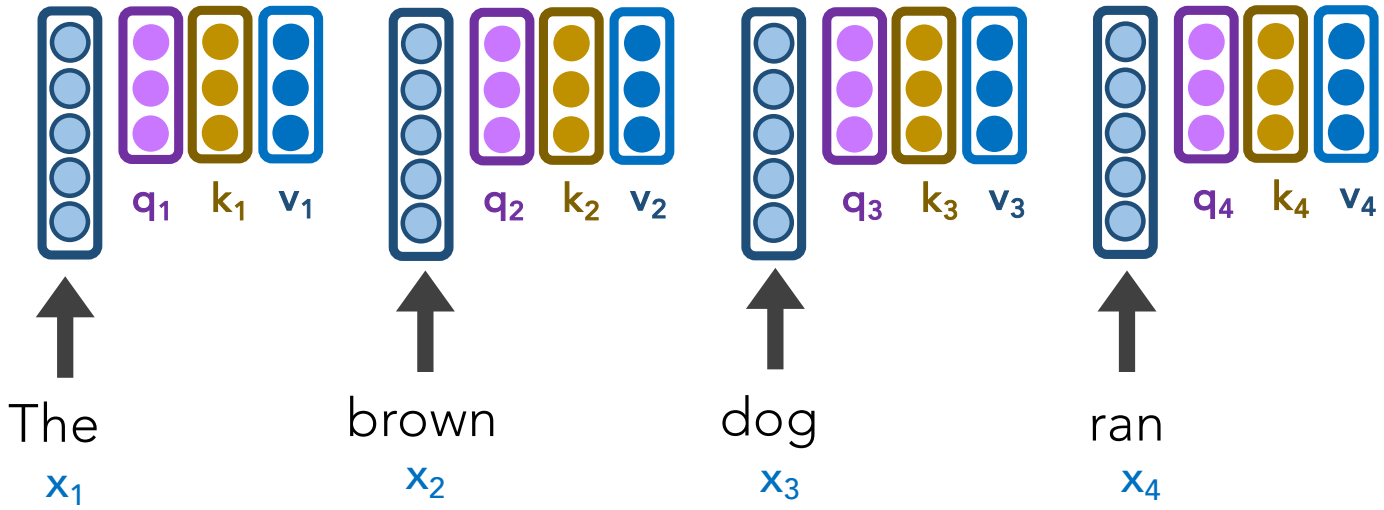
Self-Attention

Step 1: Our Self-Attention Head I has just 3 weight matrices W_q, W_k, W_v in total. These same 3 weight matrices are multiplied by each x_i to create all vectors:

$$q_i = w_q x_i$$
$$k_i = w_k x_i$$
$$v_i = w_v x_i$$

Under the hood, each x_i has 3 small, associated vectors. For example, x_1 has:

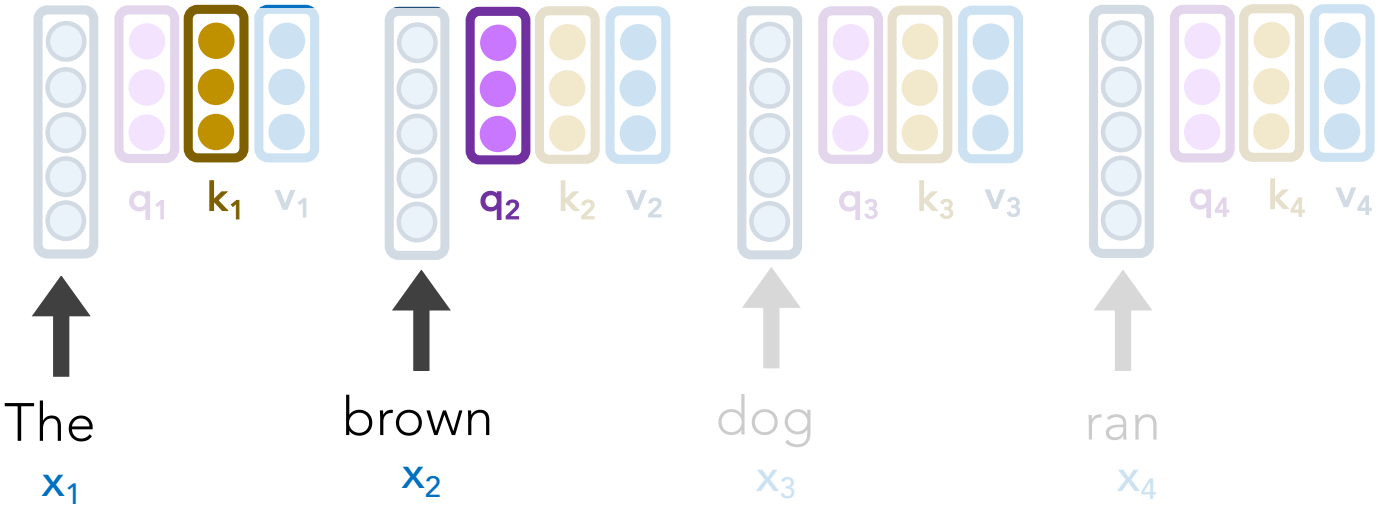
- Query q_1
- Key k_1
- Value v_1



Self-Attention

Step 2: For word x_2, let's calculate the scores s_1, s_2, s_3, s_4, which represent how much attention to pay to each respective "word" v_i

$s_1 = q_2 \cdot k_1 = 92$

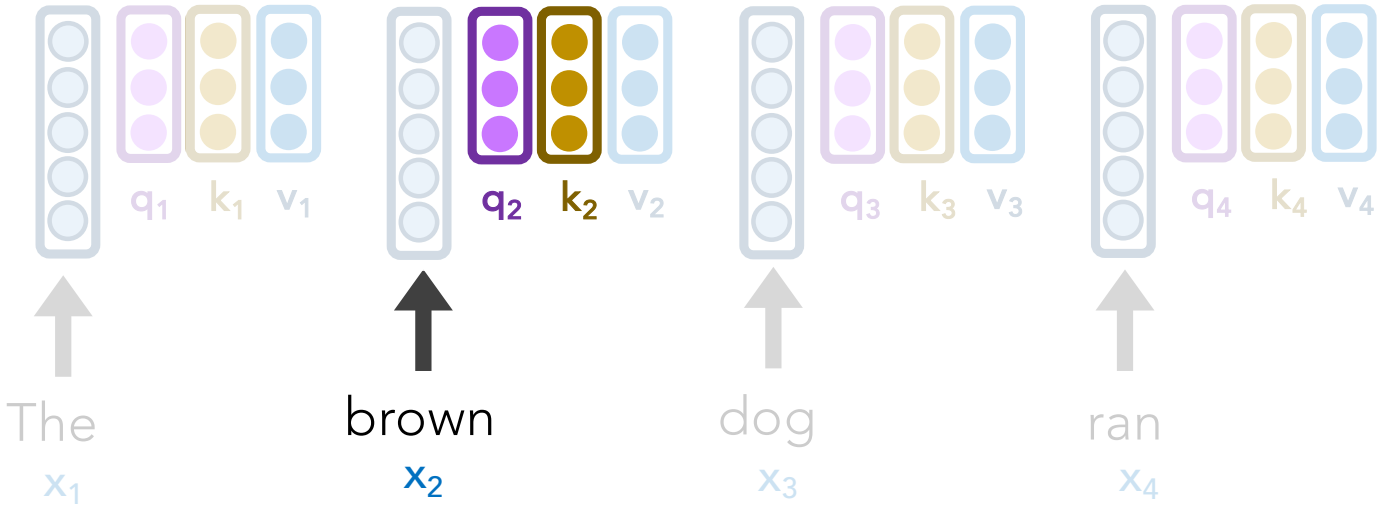


Self-Attention

Step 2: For word x_2, let's calculate the scores s_1, s_2, s_3, s_4, which represent how much attention to pay to each respective "word" v_i

$s_2 = q_2 \cdot k_2 = 124$

$s_1 = q_2 \cdot k_1 = 92$



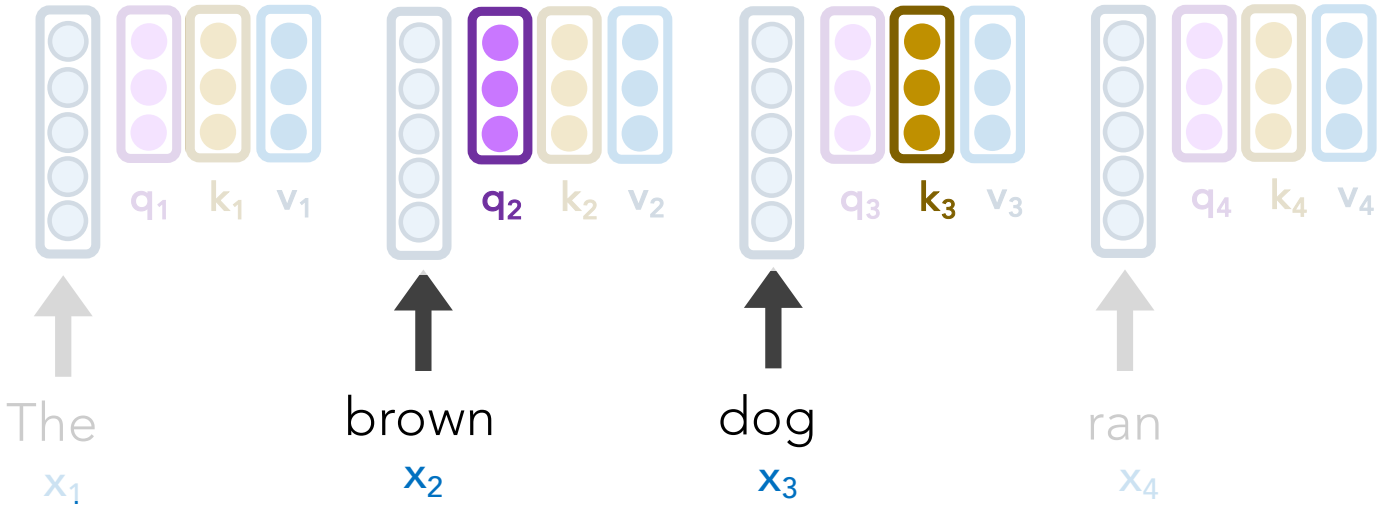
Self-Attention

Step 2: For word x_2, let's calculate the scores s_1, s_2, s_3, s_4, which represent how much attention to pay to each respective "word" v_i

$s_3 = q_2 \cdot k_3 = 22$

$s_2 = q_2 \cdot k_2 = 124$

$s_1 = q_2 \cdot k_1 = 92$



Self-Attention

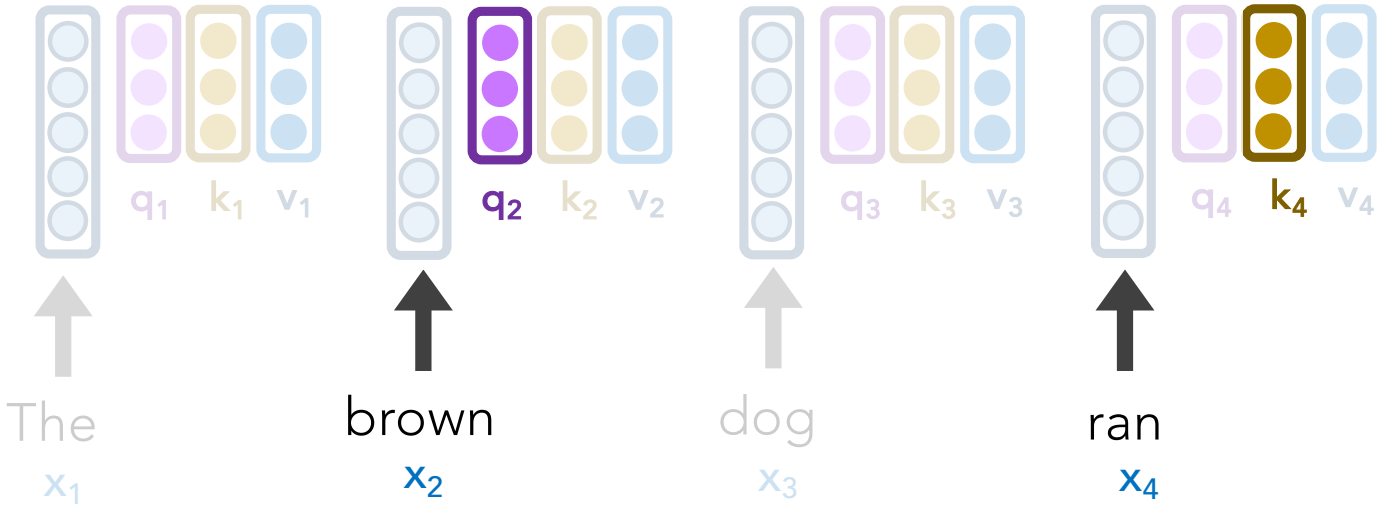
Step 2: For word x_2, let's calculate the scores s_1, s_2, s_3, s_4, which represent how much attention to pay to each respective "word" v_i

$s_4 = q_2 \cdot k_4 = 8$

$s_3 = q_2 \cdot k_3 = 22$

$s_2 = q_2 \cdot k_2 = 124$

$s_1 = q_2 \cdot k_1 = 92$



Self-Attention

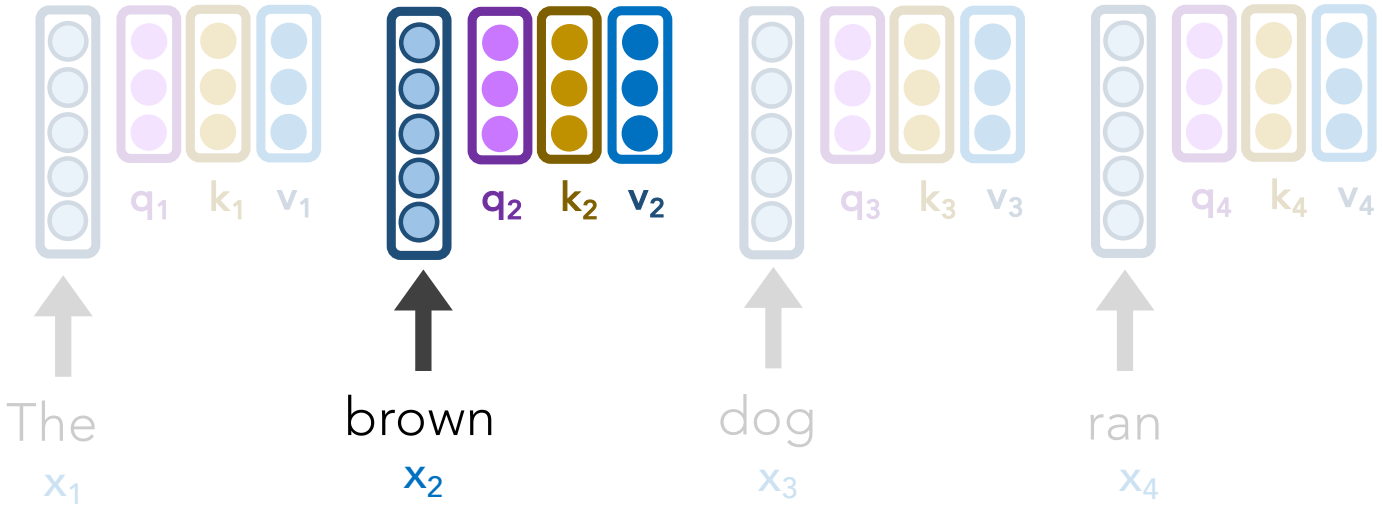
Step 3: Our scores s_1, s_2, s_3, s_4 don't sum to 1. Let's divide by $\sqrt{len(k_i)}$ and softmax it

$s_4 = q_2 \cdot k_4 = 8$ $\quad a_4 = \sigma(s_4/8) = 0$

$s_3 = q_2 \cdot k_3 = 22$ $\quad a_3 = \sigma(s_3/8) = .01$

$s_2 = q_2 \cdot k_2 = 124$ $\quad a_2 = \sigma(s_2/8) = .91$

$s_1 = q_2 \cdot k_1 = 92$ $\quad a_1 = \sigma(s_1/8) = .08$



Self-Attention

Step 3: Our scores s_1, s_2, s_3, s_4 don't sum to 1. Let's divide by $\sqrt{len(k_i)}$ and softmax it

$s_4 = q_2 \cdot k_4 = 8$

$s_3 = q_2 \cdot k_3 = 22$

$s_2 = q_2 \cdot k_2 = 124$

$s_1 = q_2 \cdot k_1 = 92$

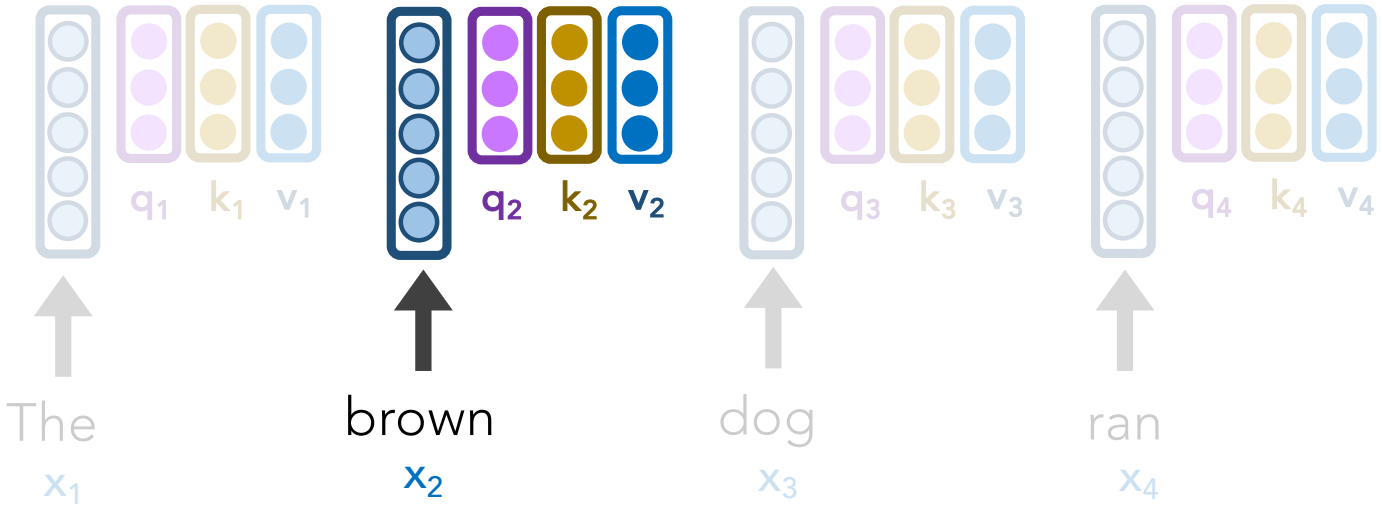
$a_4 = \sigma(s_4/8) = 0$

$a_3 = \sigma(s_3/8) = .01$

$a_2 = \sigma(s_2/8) = .91$

$a_1 = \sigma(s_1/8) = .08$

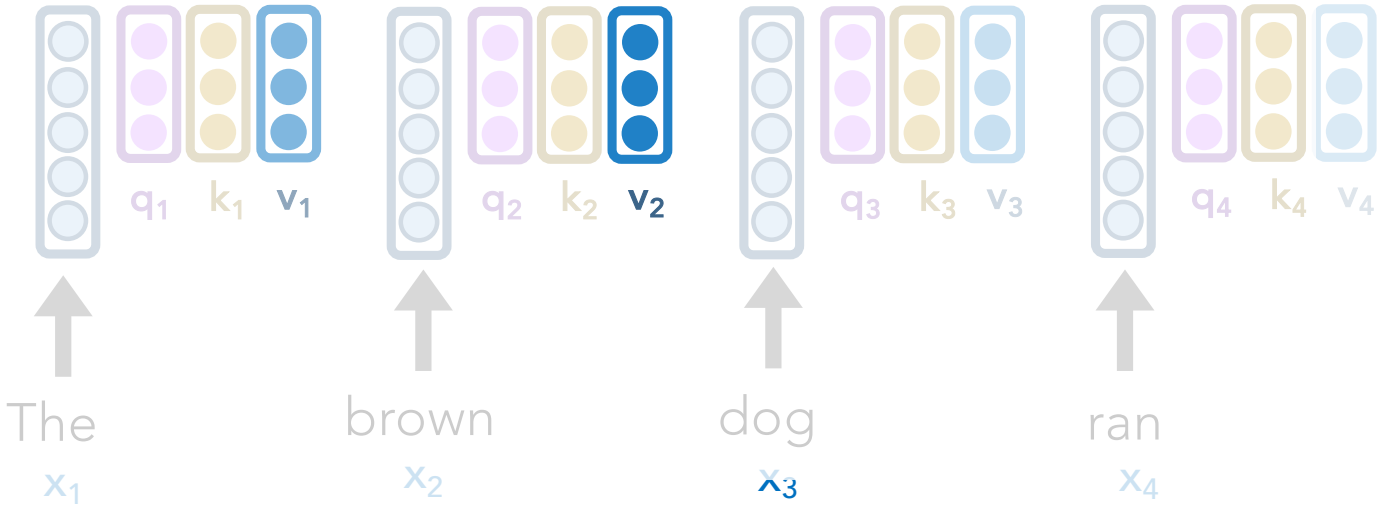
Instead of these a_i values directly weighting our original x_i word vectors, they directly weight our v_i vectors.



Self-Attention

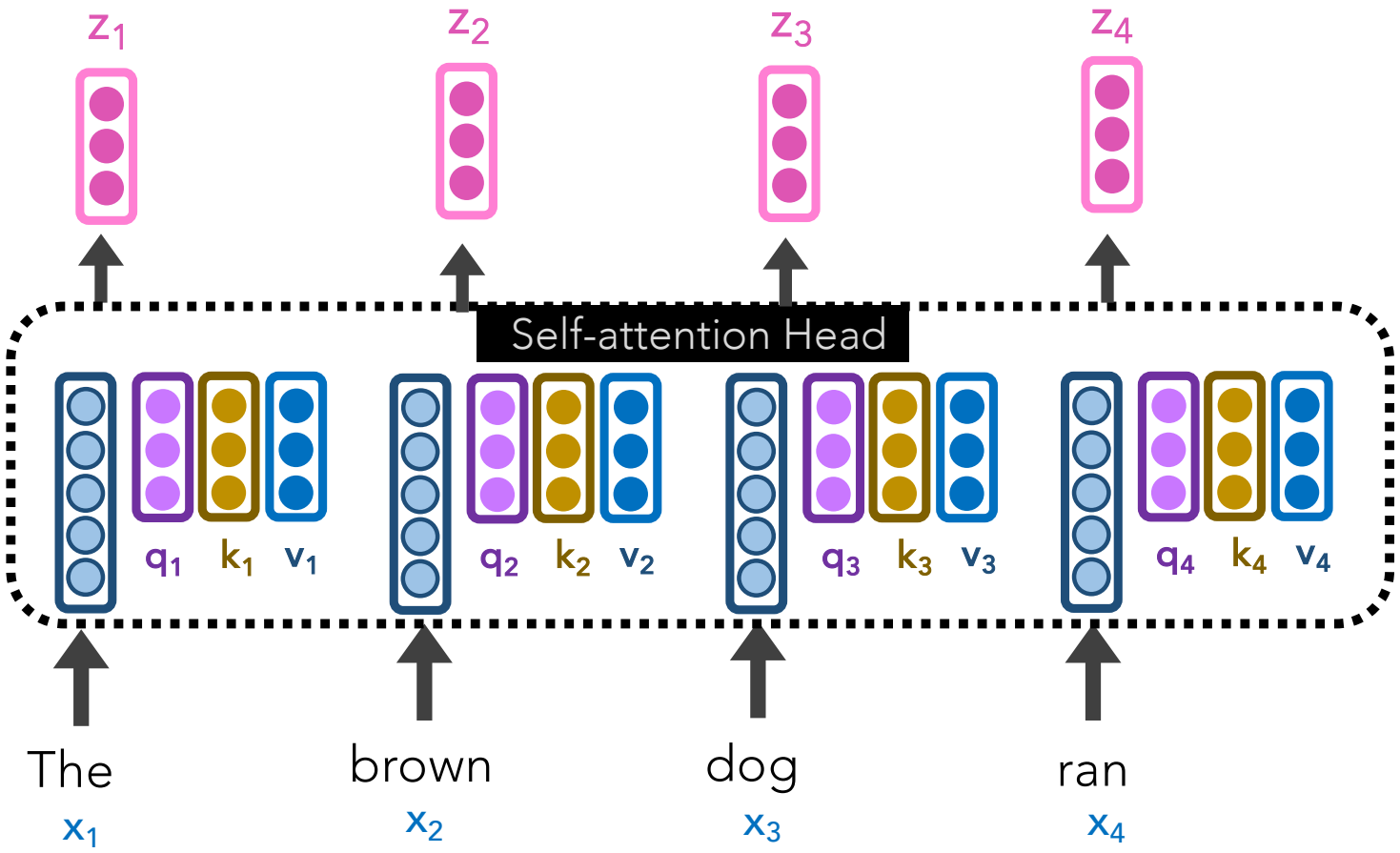
Step 4: Let's weight our v_i vectors and simply sum them up!

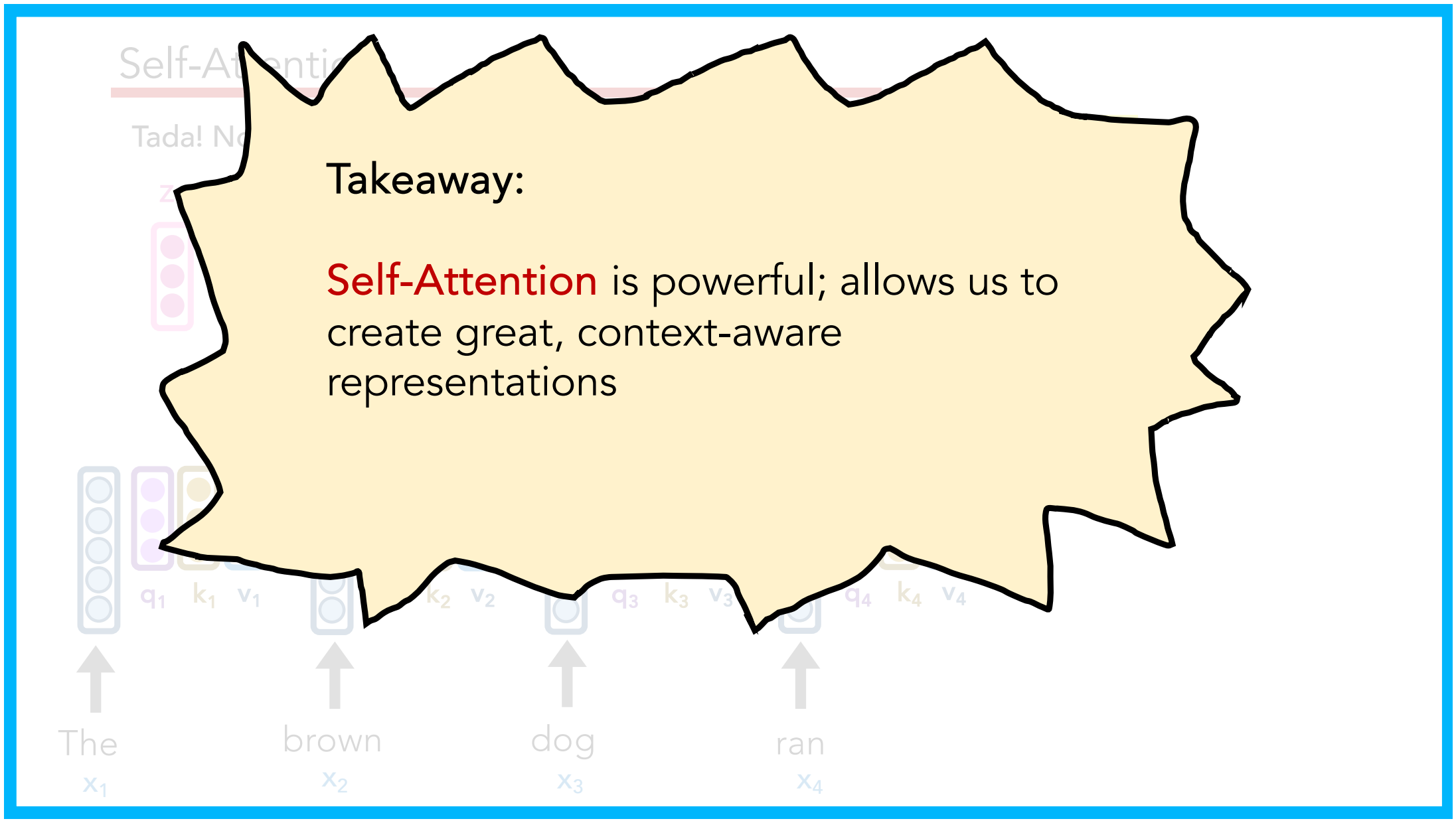
$$z_2 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$
$$= 0.08 \cdot v_1 + 0.91 \cdot v_2 + 0.01 \cdot v_3 + 0 \cdot v_4$$



Self-Attention

Tada! Now we have great, new representations z_i via a self-attention head




Takeaway:
Self-Attention is powerful; allows us to create great, context-aware representations

Outline

Self-Attention

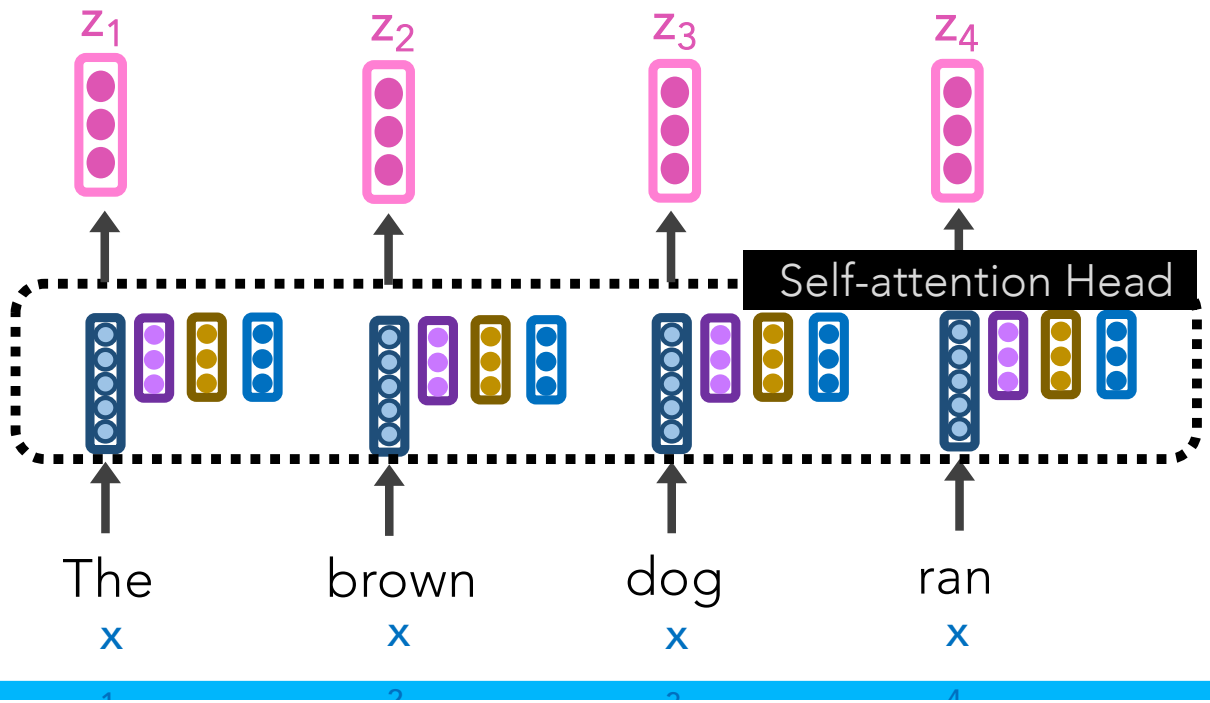
Transformer Encoder

Transformer Decoder

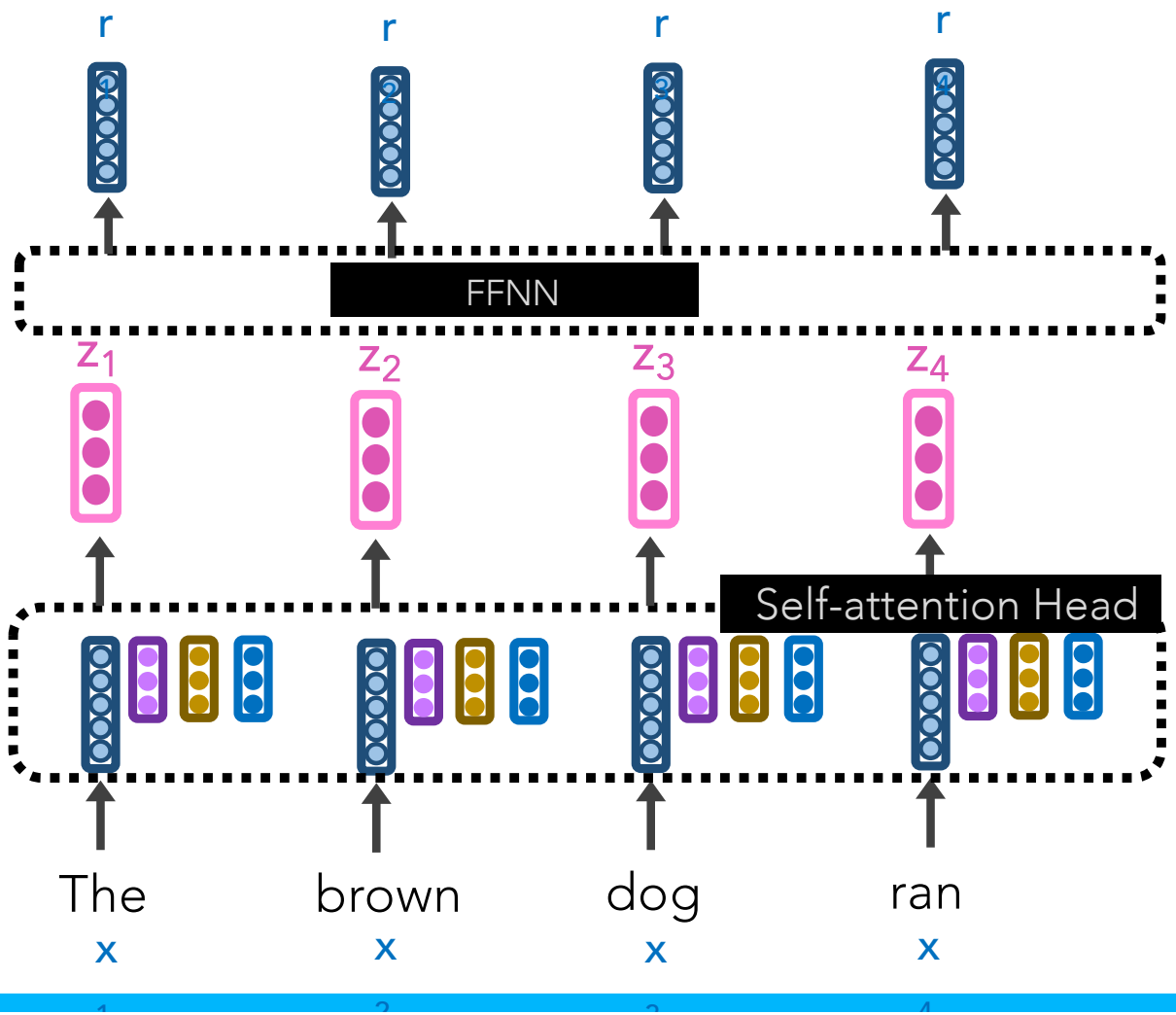
BERT

Self-Attention

Let's further pass each z_i through a FFNN

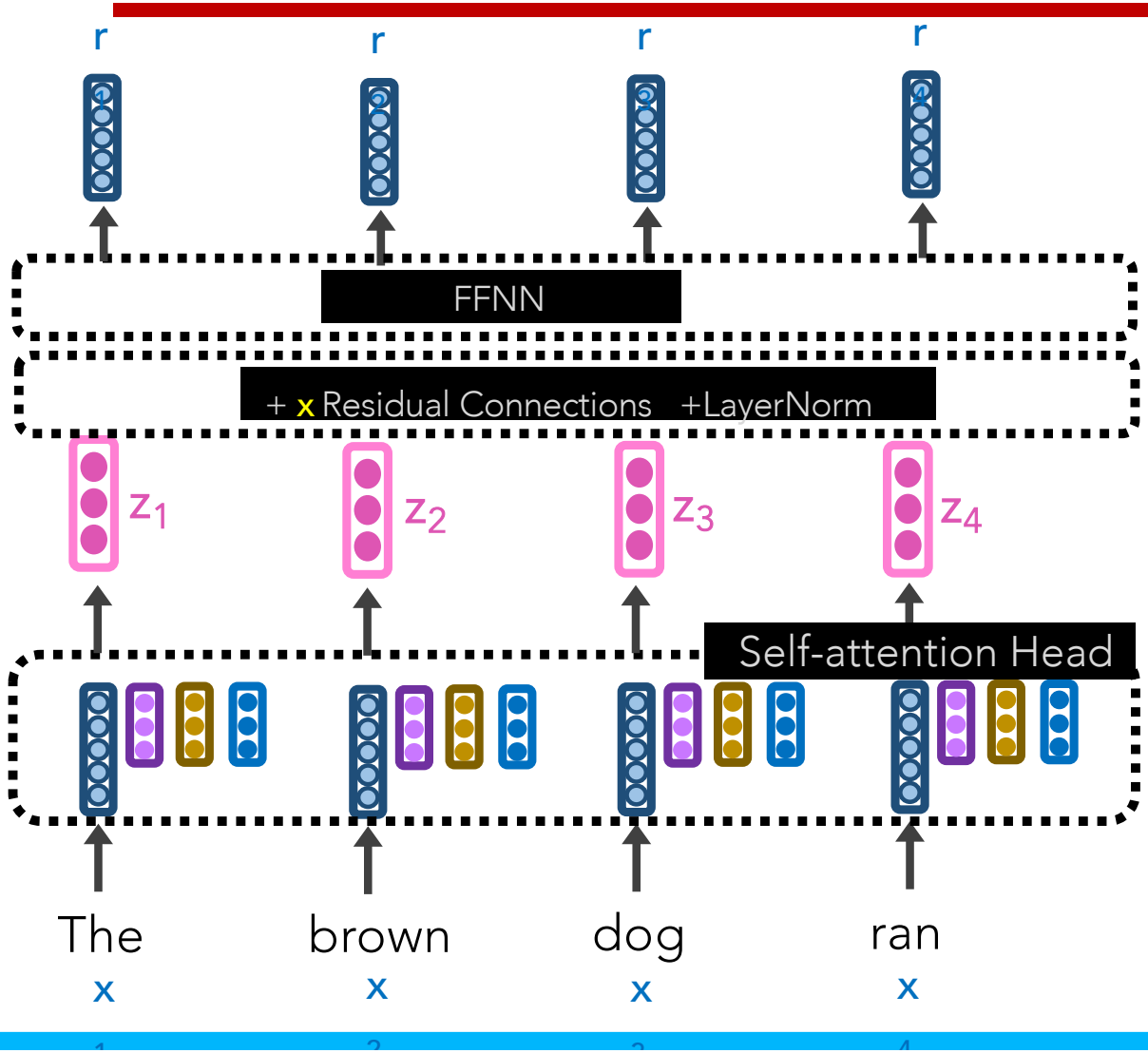


Self-Attention + FFNN



Let's further pass each z_i through a FFNN

Self-Attention + FFNN

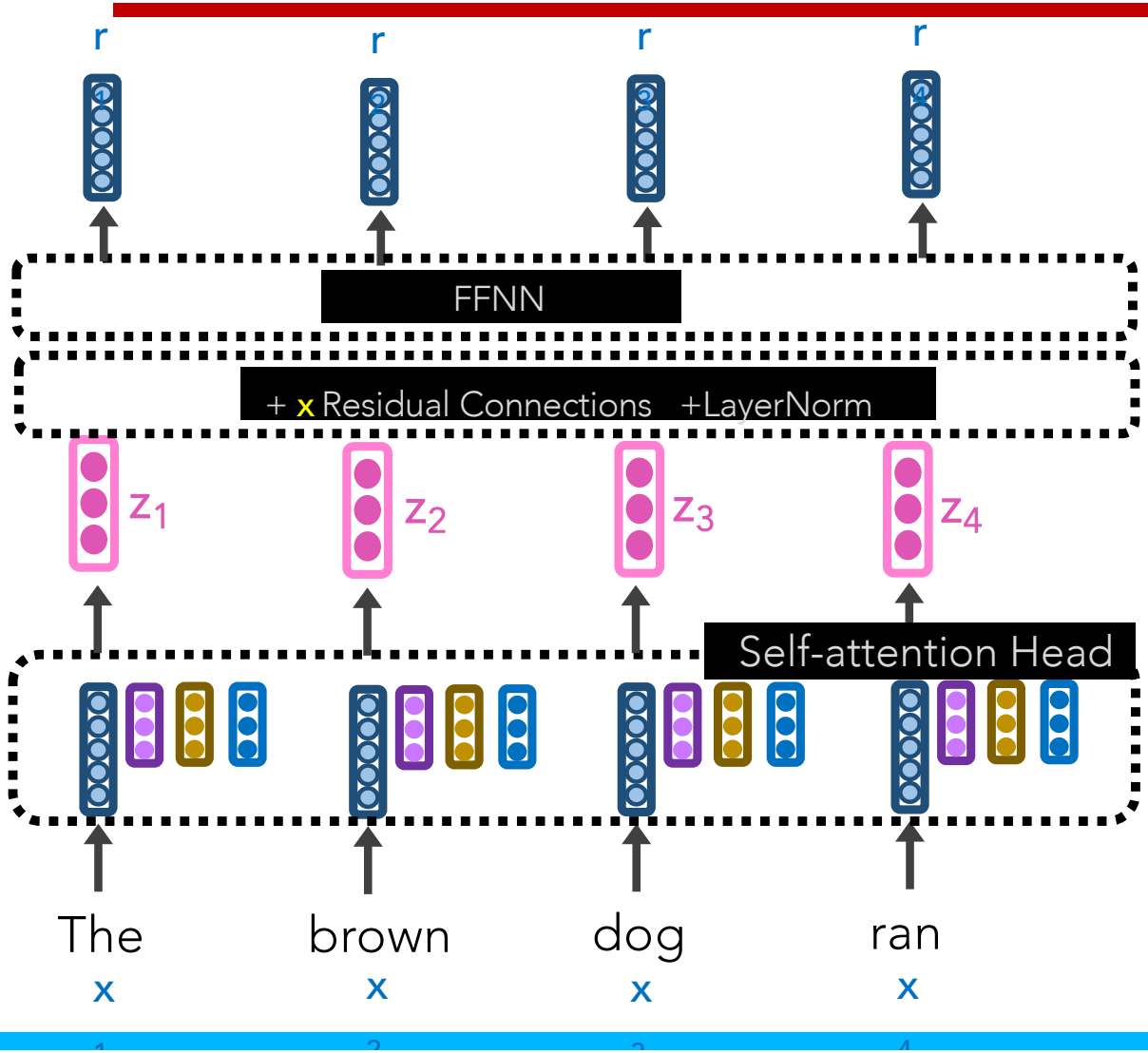


Let's further pass each z_i through a FFNN

We concat w/ a residual connection to help ensure relevant info is getting forward passed.

We perform LayerNorm to stabilize the network and allow for proper gradient flow. You should do this after the FFNN, too.

Self-Attention + FFNN



Let's further pass each z_i through a FFNN

We concat w/ a residual connection to help ensure relevant info is getting forward passed.

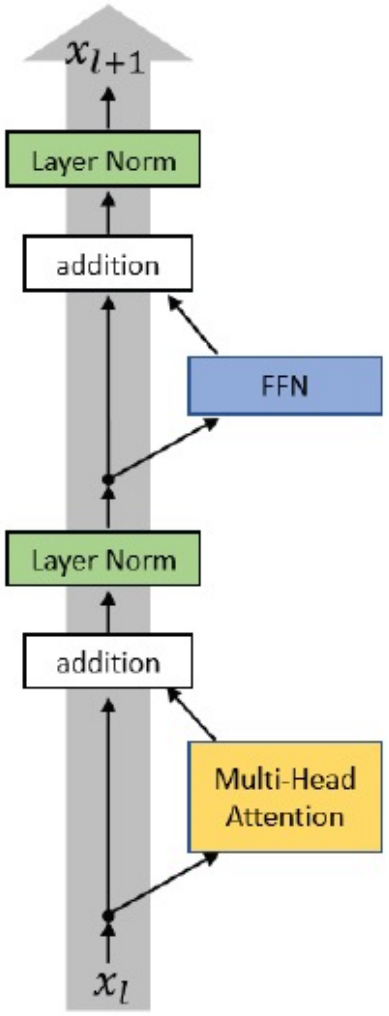
We perform LayerNorm to stabilize the network and allow for proper gradient flow. You should do this after the FFNN, too.

Each z_i can be computed in parallel, unlike LSTMs!

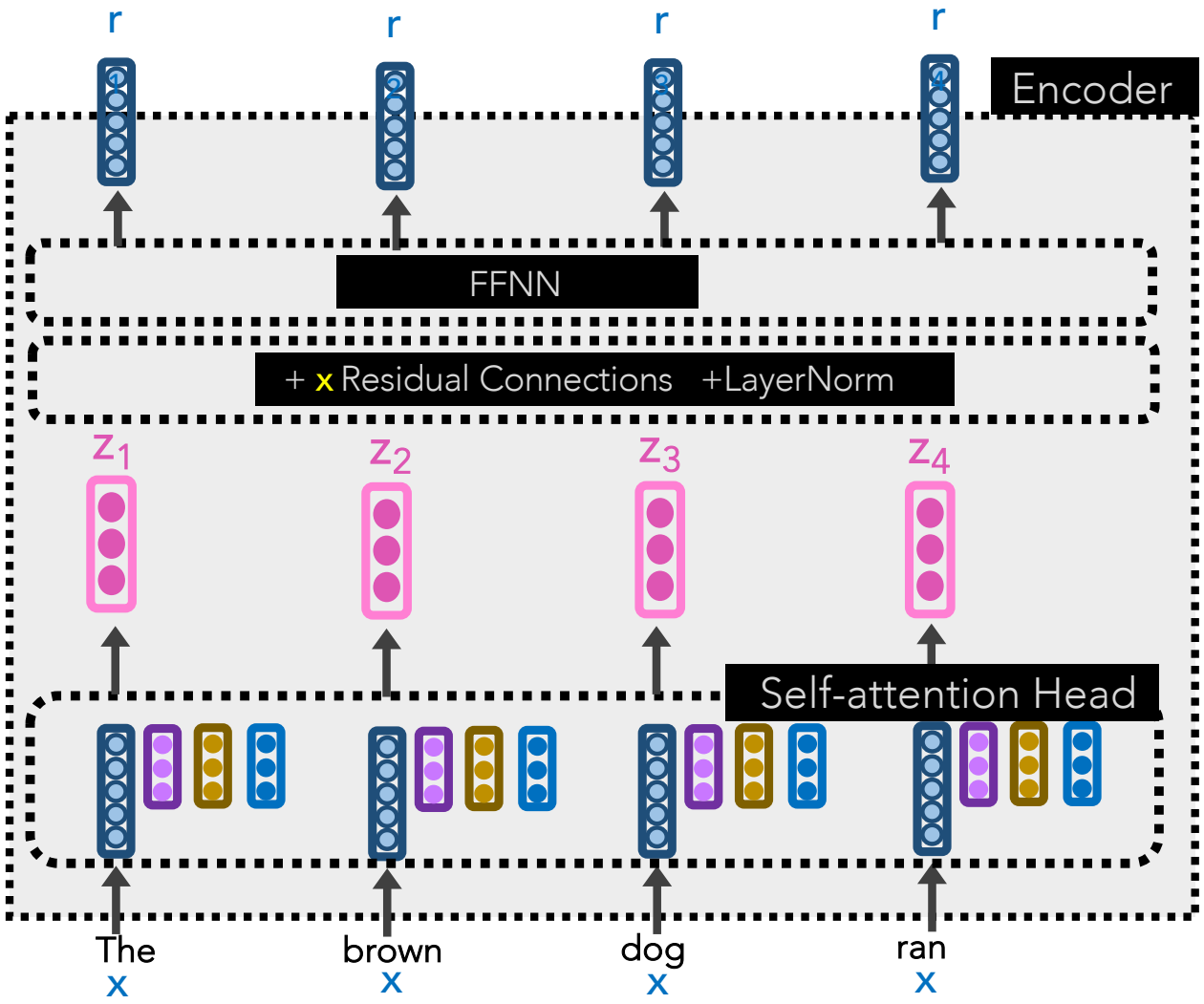
Post-LN Transformer

LayerNorm이 **평균 0·분산 1** 분포를 강제로 유지해 주기 때문에,

1. 다음 층이 예측하기 쉬운 안정적 공간에 머무르고
2. 역전파 기울기 크기가 제한되며
3. residual 합에서도 스케일 불일치가 해소되어
   결국 “stabilize the network and allow proper gradient flow”가 가능합니다.

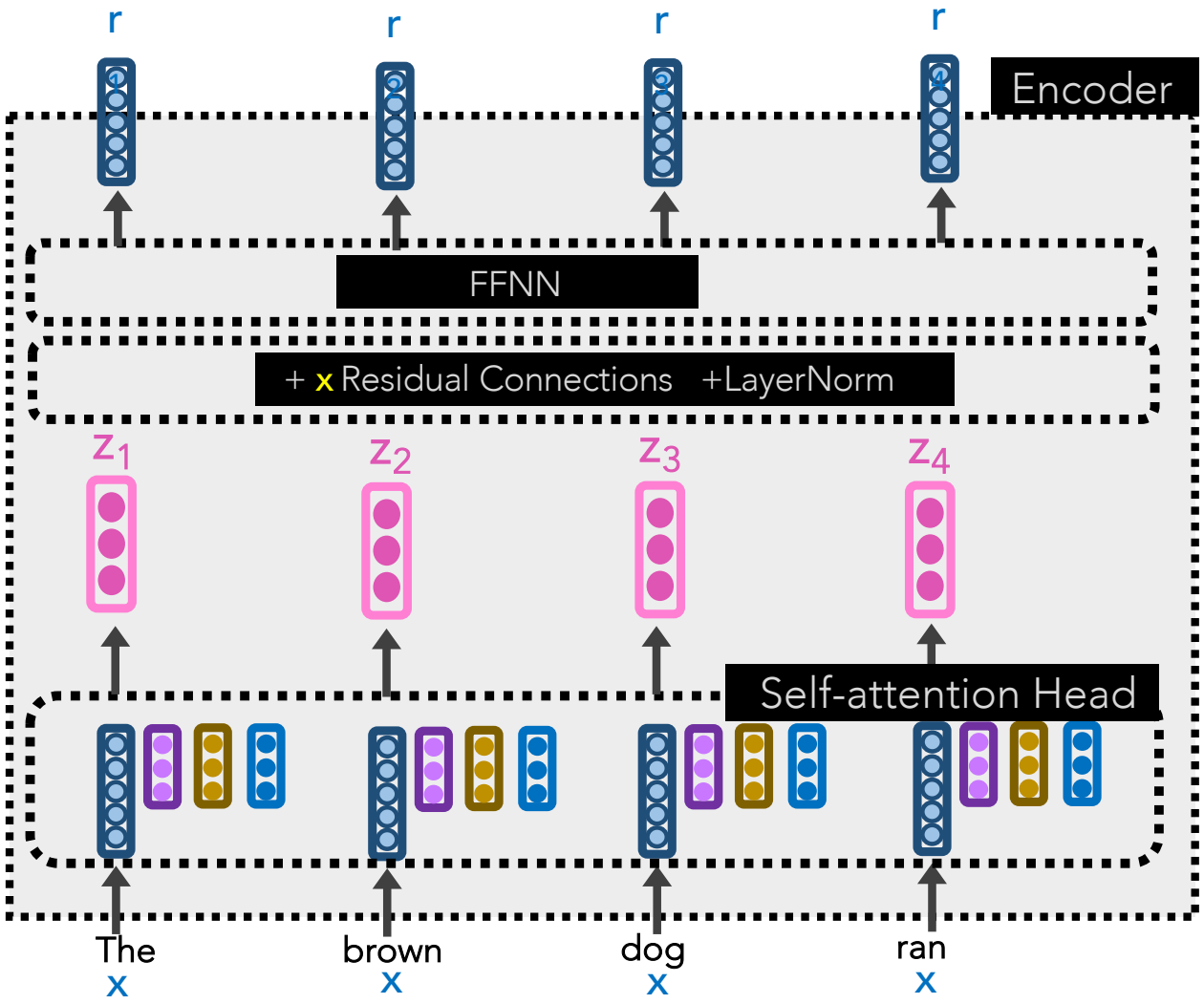


Transformer Encoder



Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

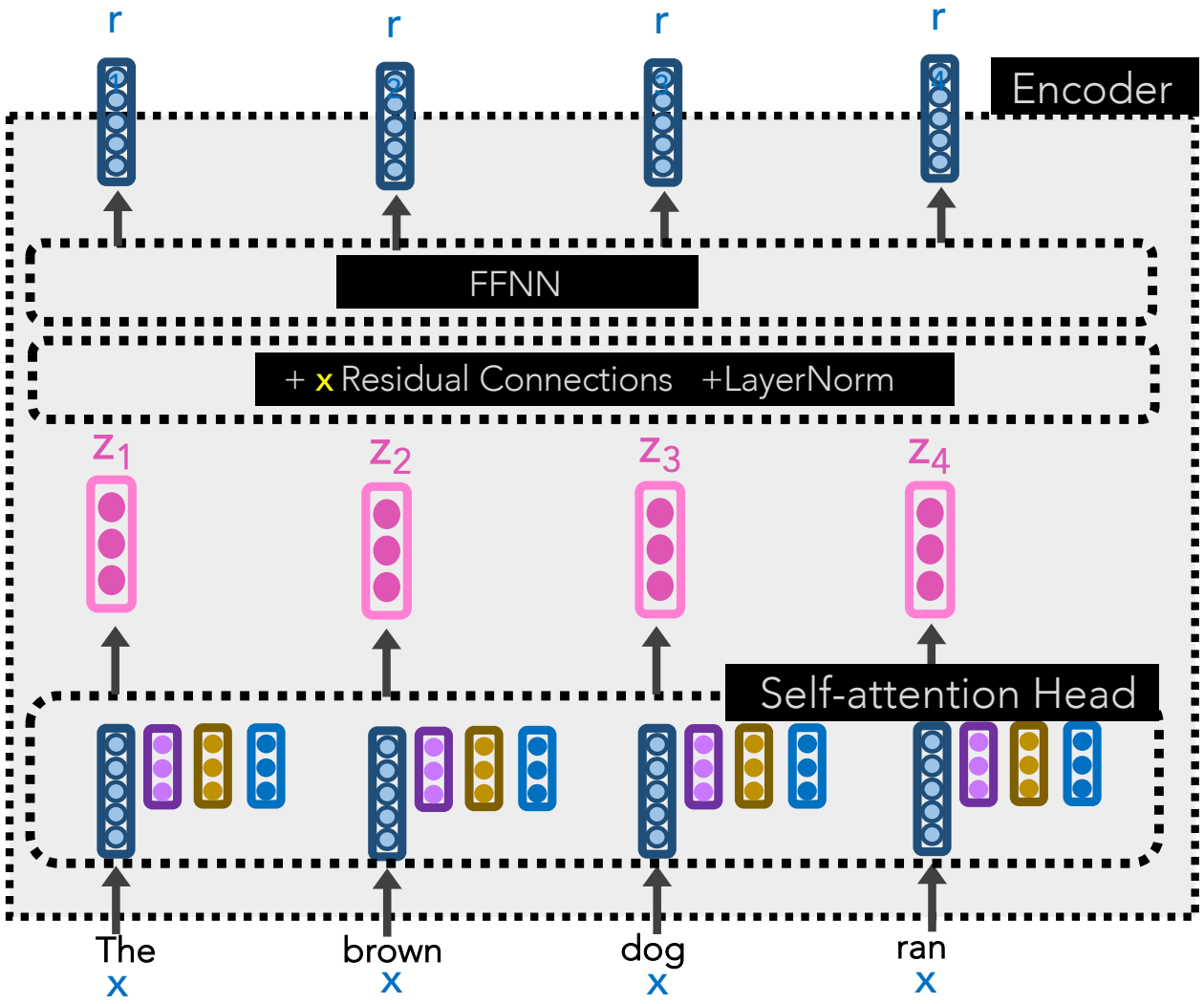
Transformer Encoder



Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

Problem: there is no concept of positionality. Words are weighted as if a "bag of words"

Transformer Encoder

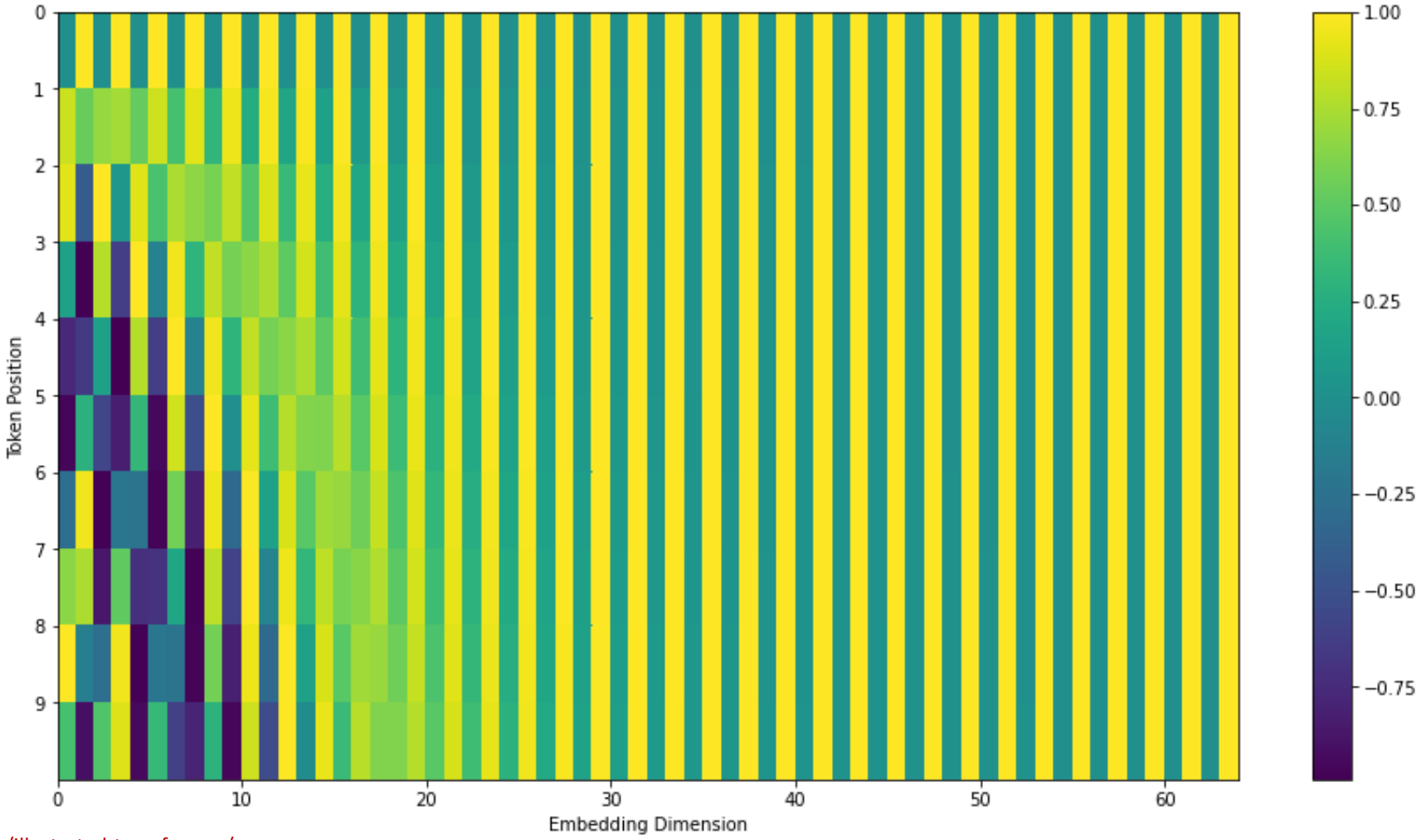


Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

Problem: there is no concept of positionality. Words are weighted as if a "bag of words"

Solution: add to each input word x_i a positional encoding $\sim \sin(i)\cos(i)$

Position Encodings



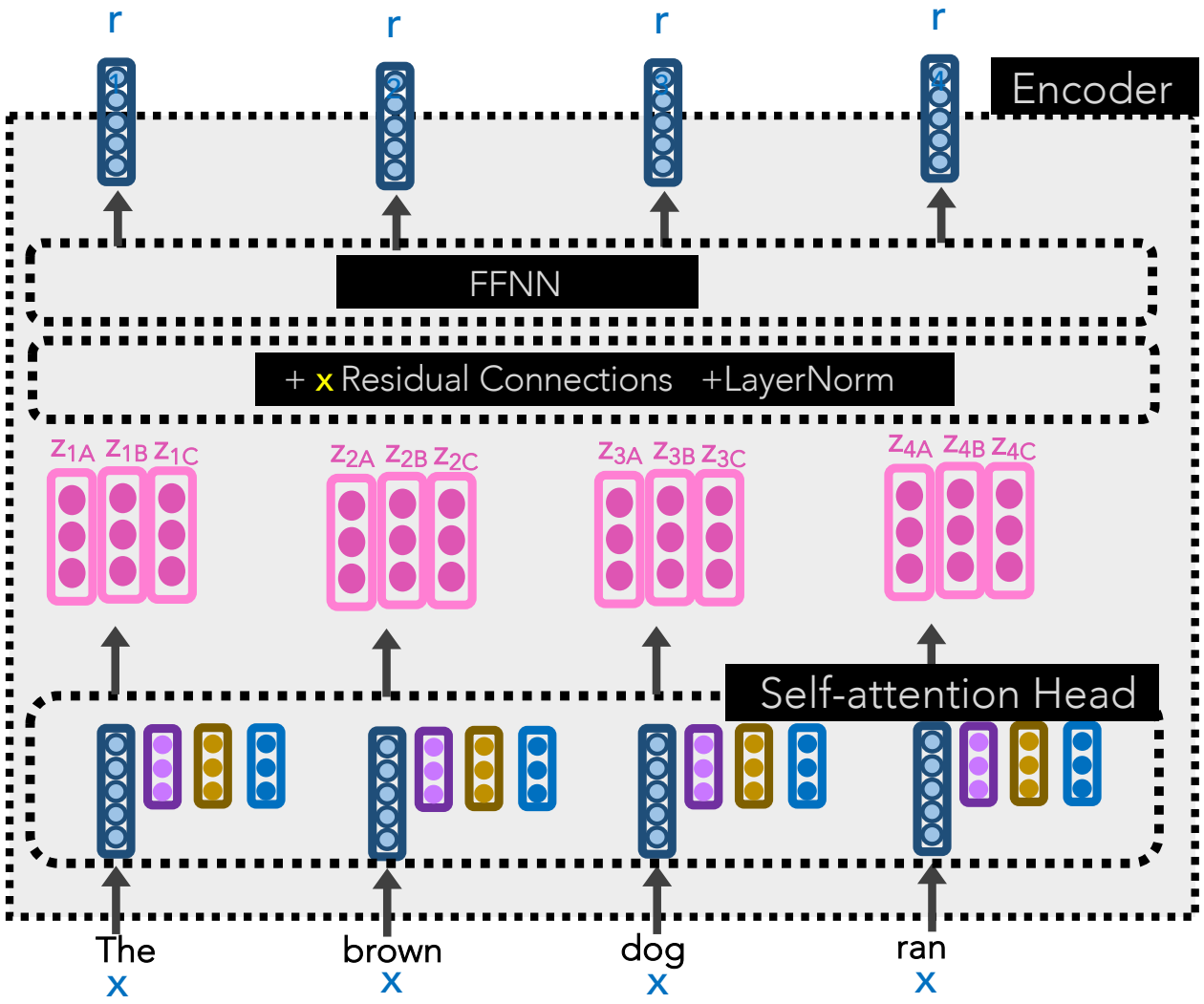
https://jalammar.github.io/illustrated-transformer/

A Self-Attention Head has just one set of query/key/value weight matrices w_q, w_k, w_v

Words can relate in many ways, so it's restrictive to rely on just one Self-Attention Head in the system.

Let's create Multi-headed Self-Attention

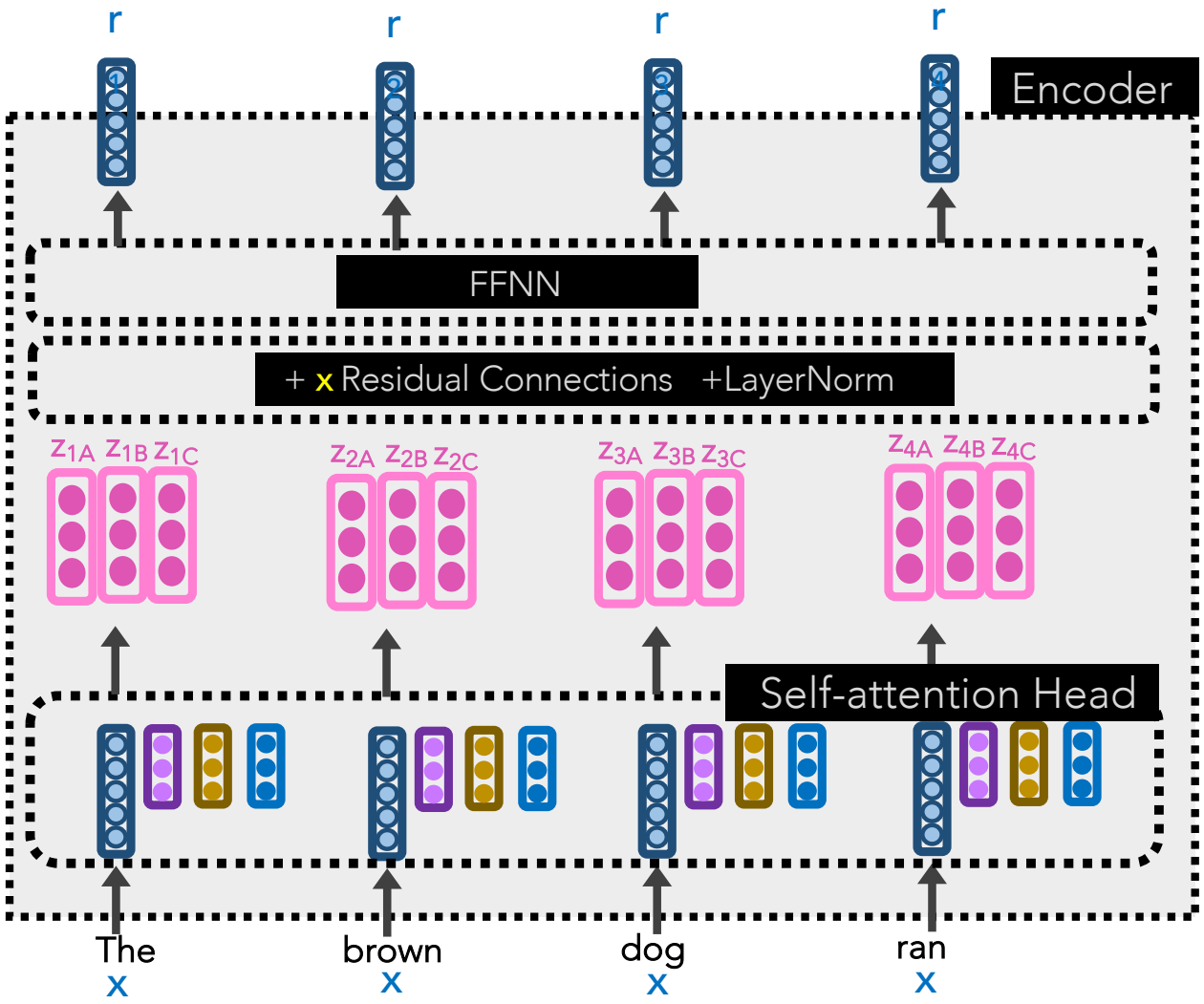
Transformer Encoder



Each Self-Attention Head produces a z_i vector.

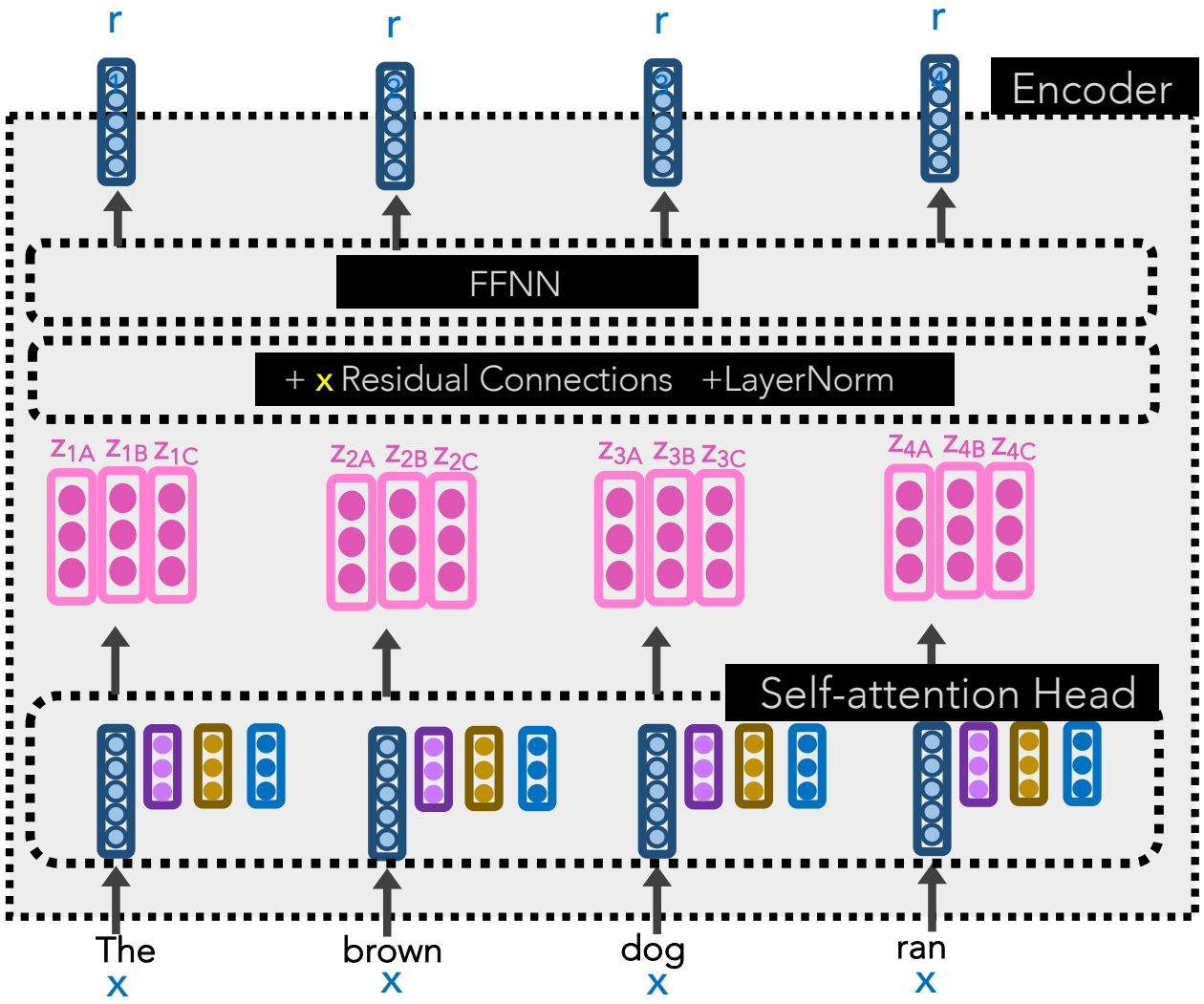
We can, in parallel, use multiple heads and concat the z_i's.

Transformer Encoder

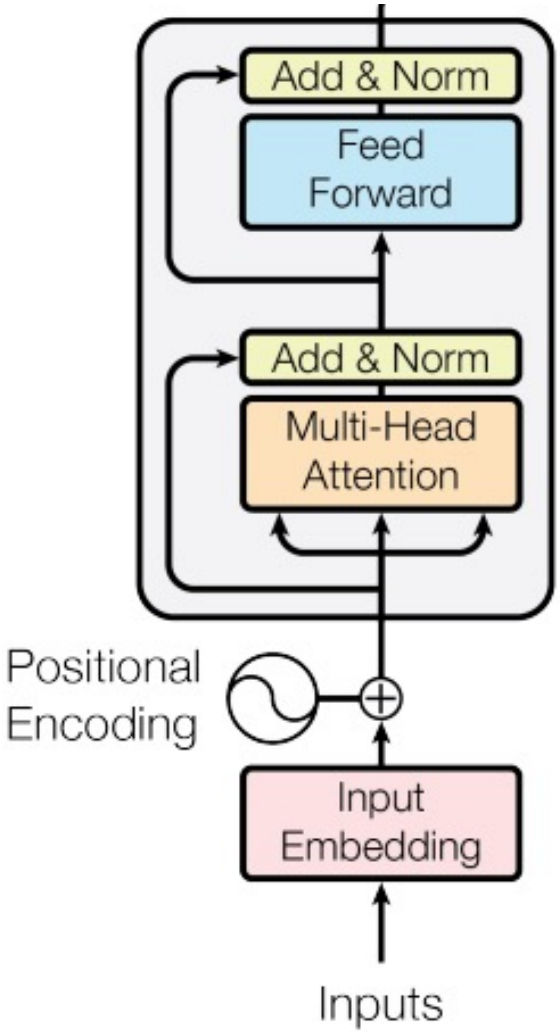


To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

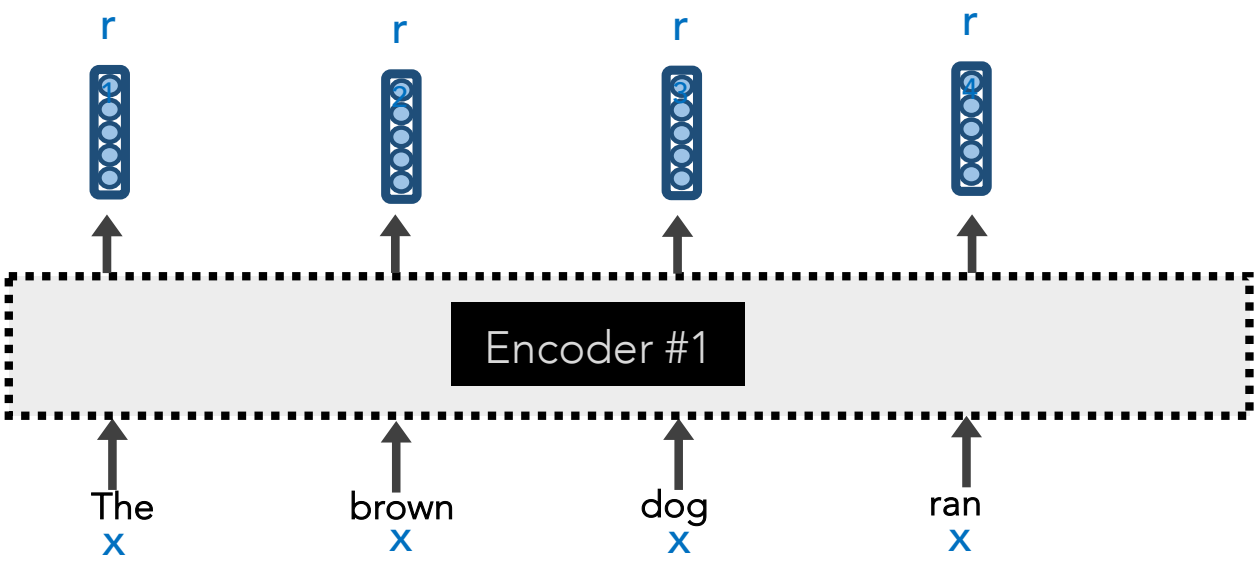
Transformer Encoder



=

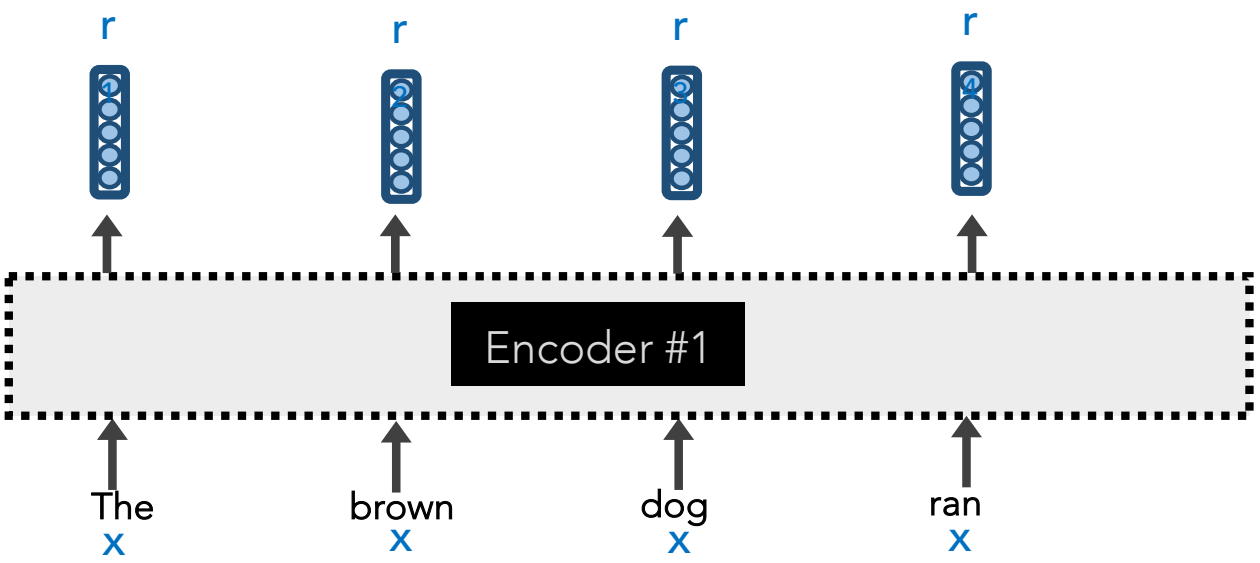


Transformer Encoder



To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

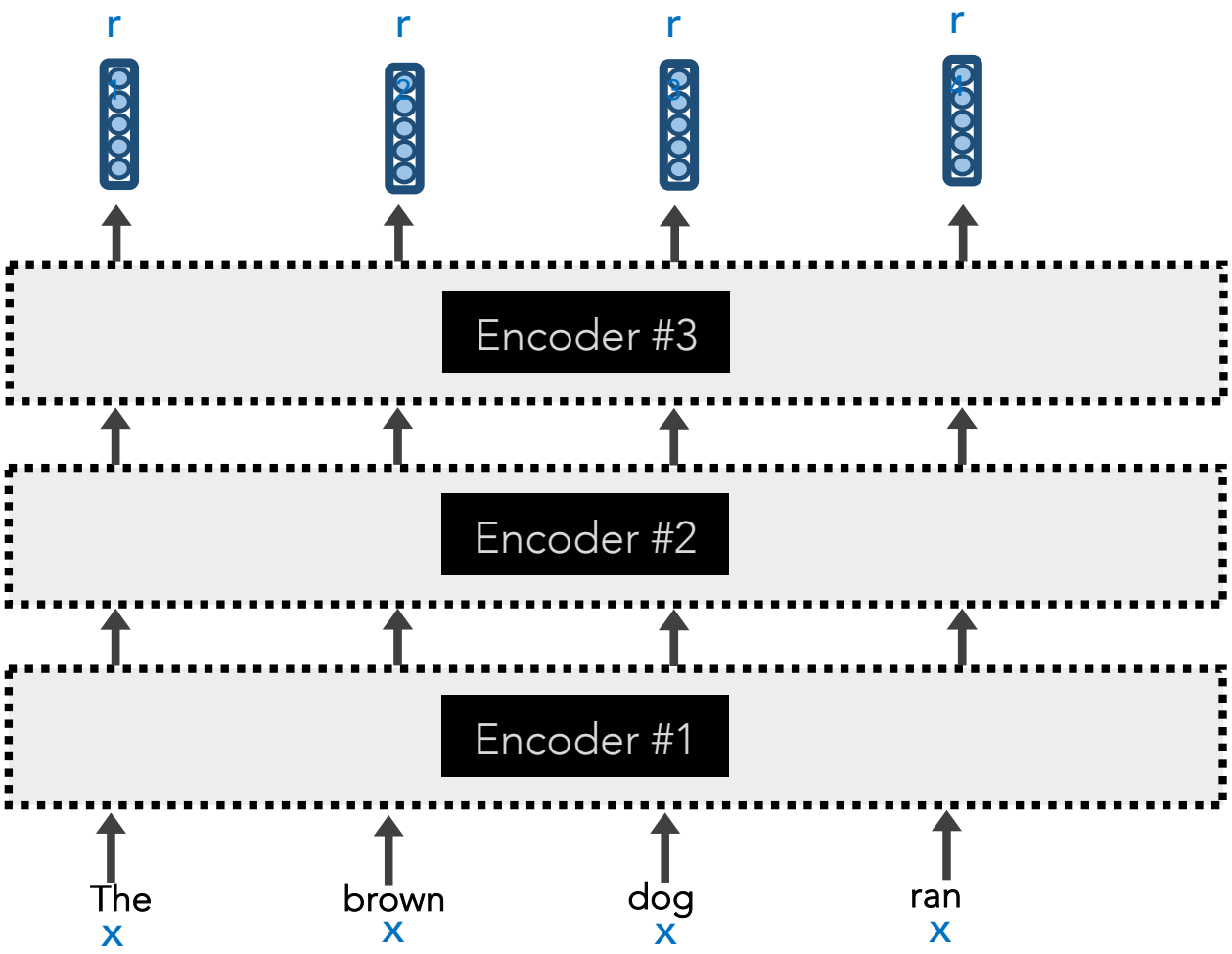
Transformer Encoder



To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

Why stop with just 1 Transformer Encoder?
We could stack several!

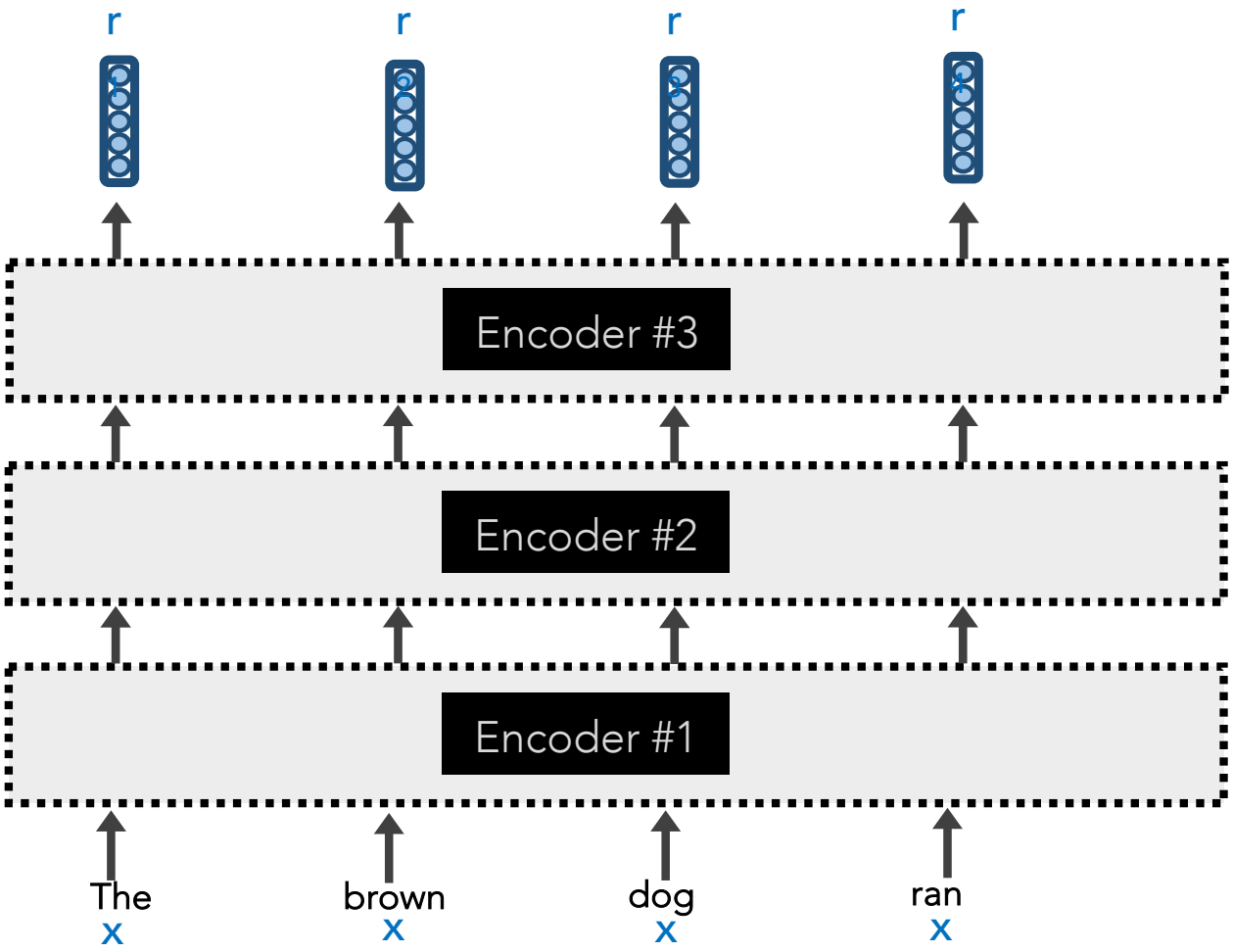
Transformer Encoder



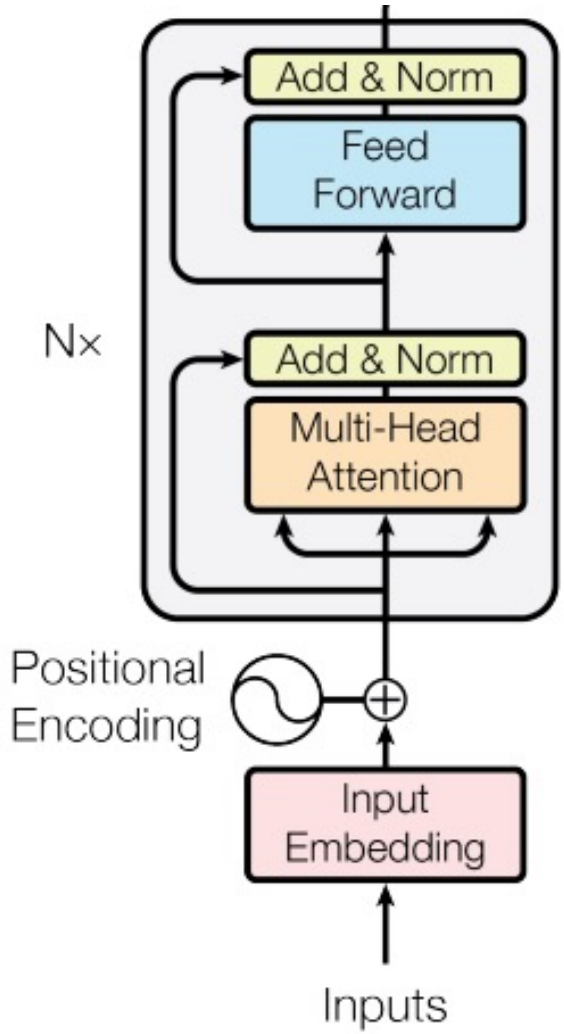
To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

Why stop with just 1 Transformer Encoder? We could stack several!

Transformer Encoder



=



The original Transformer model was intended for Machine Translation, so it had Decoders, too

Outline

Self-Attention

Transformer Encoder

Transformer Decoder

BERT

Outline

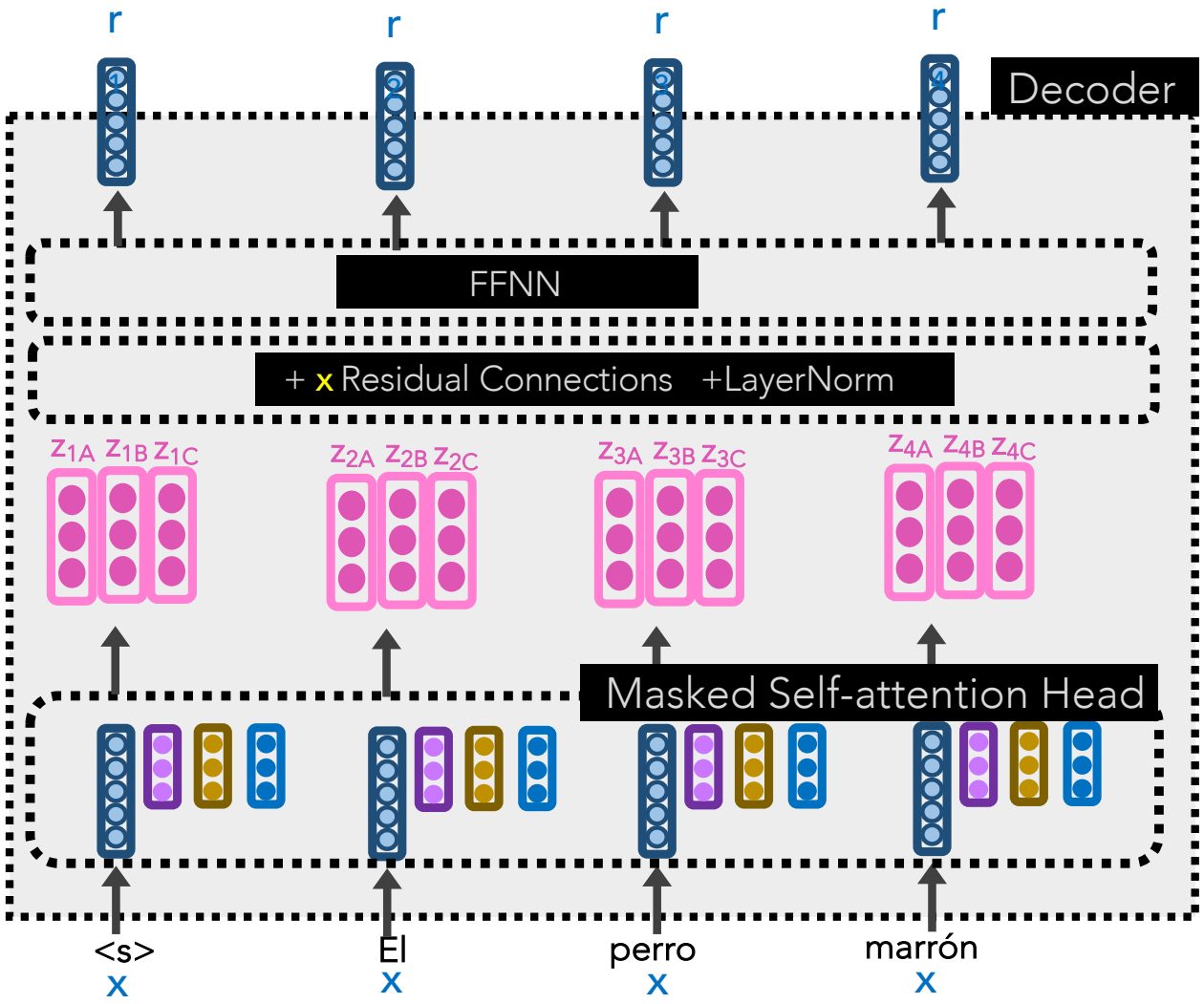
Self-Attention

Transformer Encoder

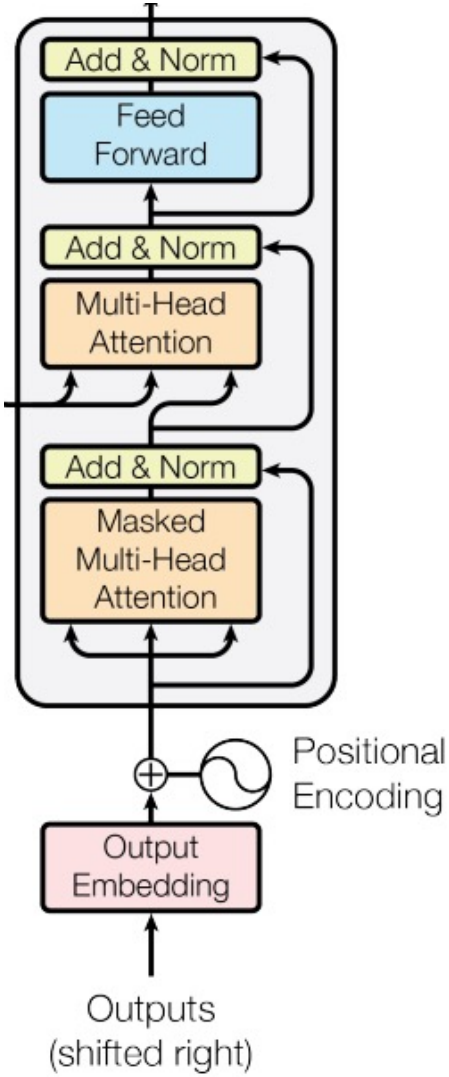
Transformer Decoder

BERT

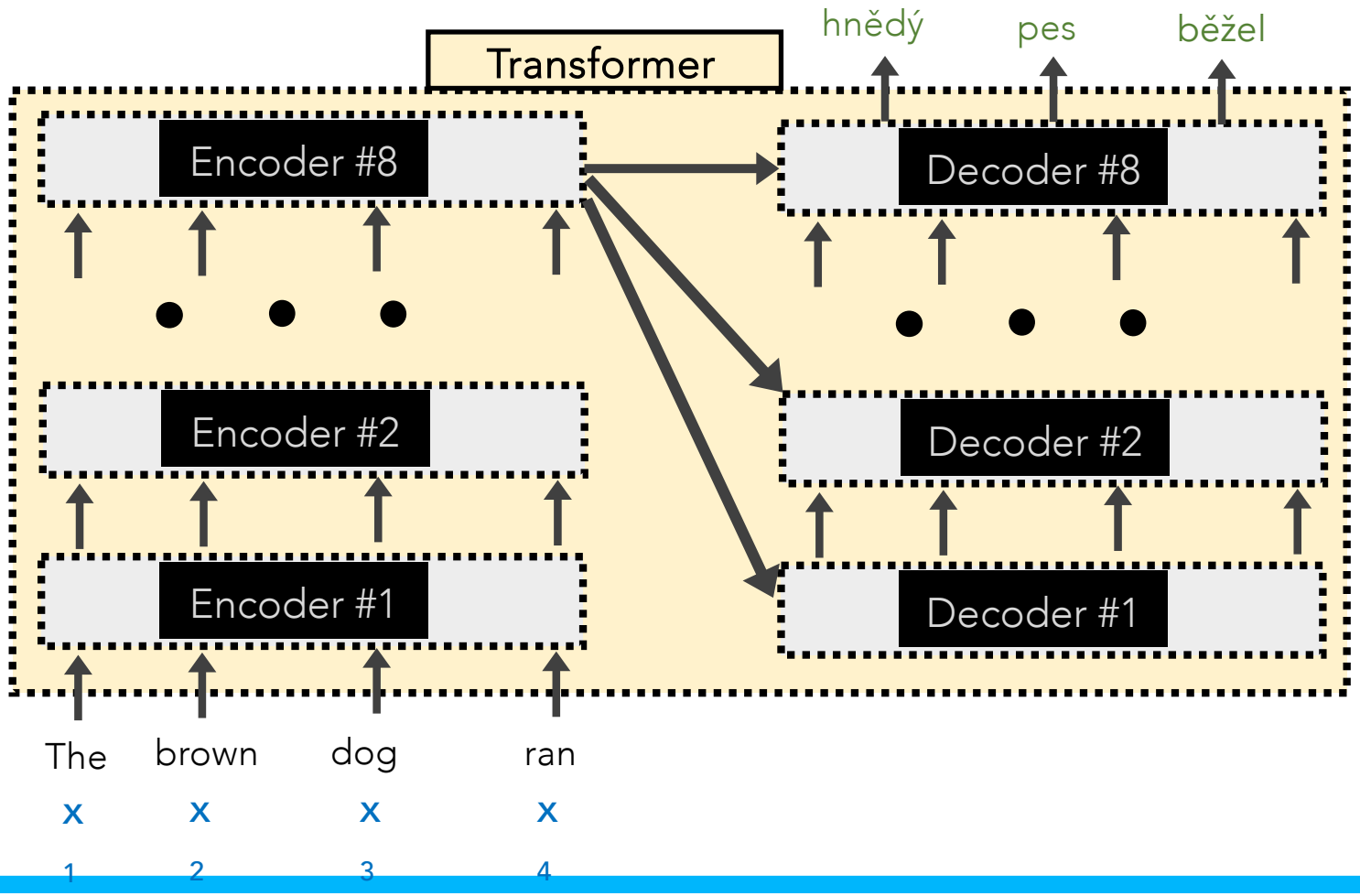
Transformer Decoder



=



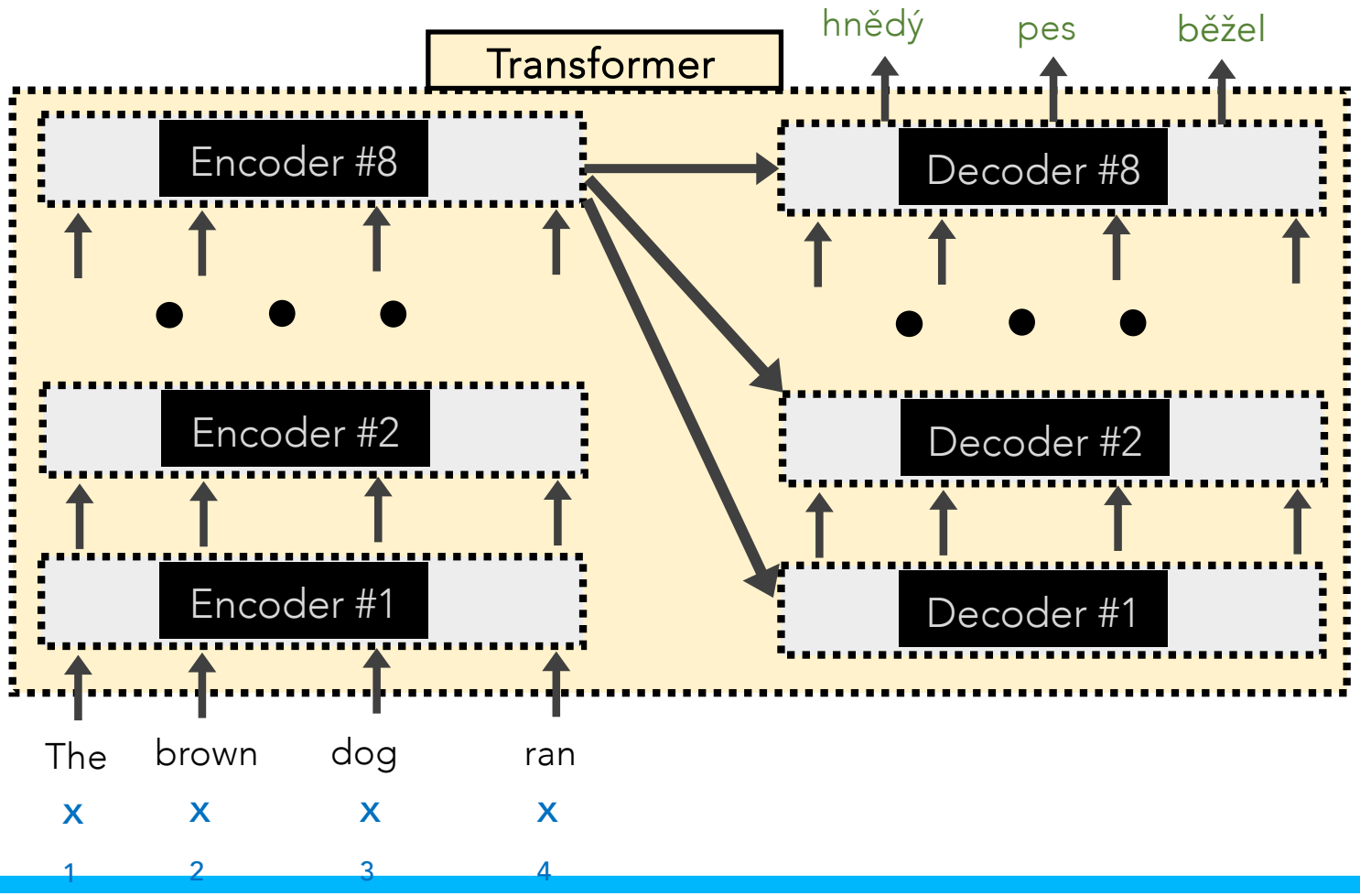
Transformer Encoders and Decoders



Transformer Encoders produce **contextualized embeddings** of each word

Transformer Decoders generate new sequences of text

Transformer Encoders and Decoders

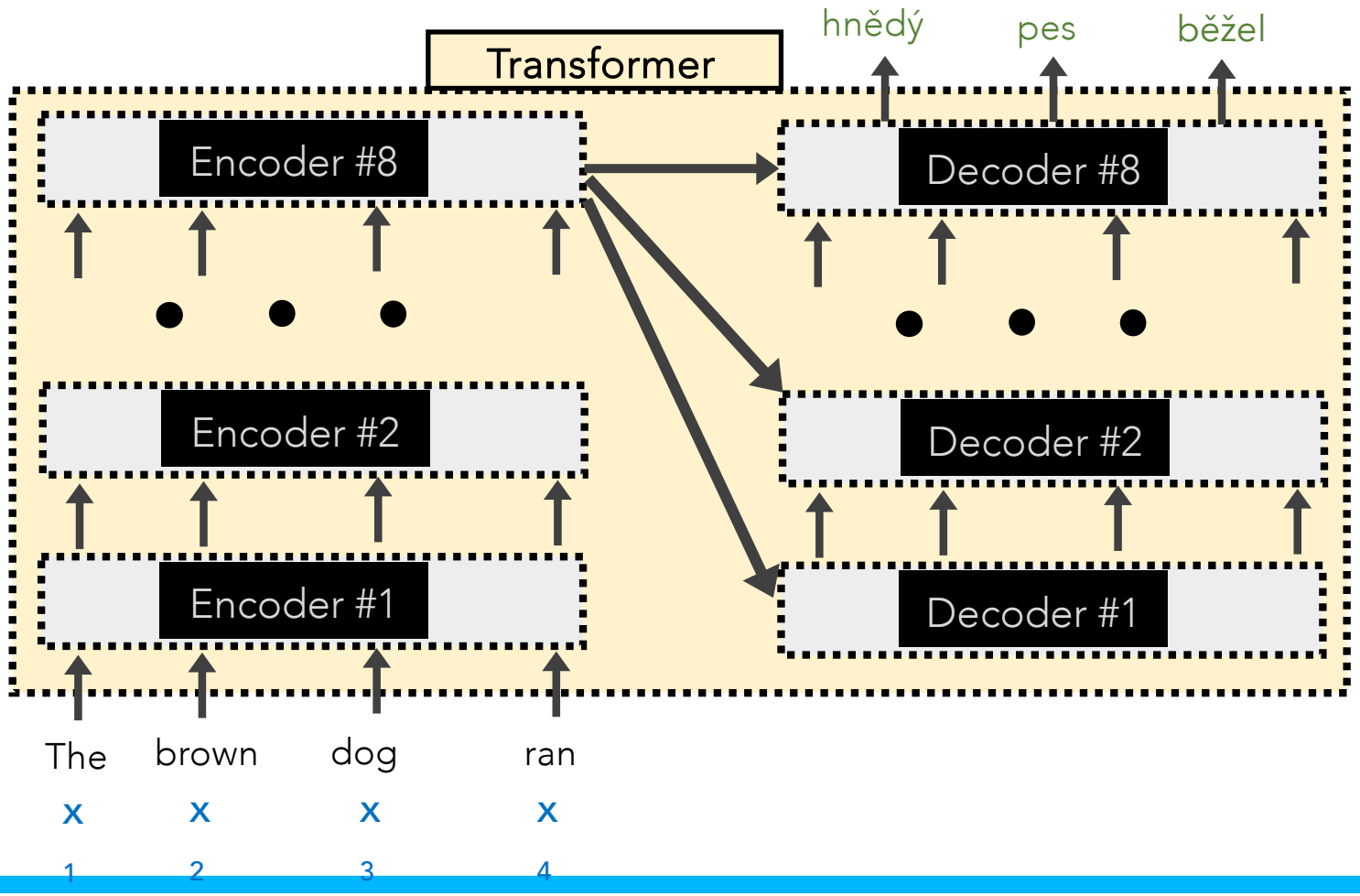


NOTE

Transformer Decoders are identical to the Encoders, except they have an additional Attention Head in between the Self-Attention and FFNN layers.

This additional Attention Head **focuses on parts of the encoder's representations.**

Transformer Encoders and Decoders

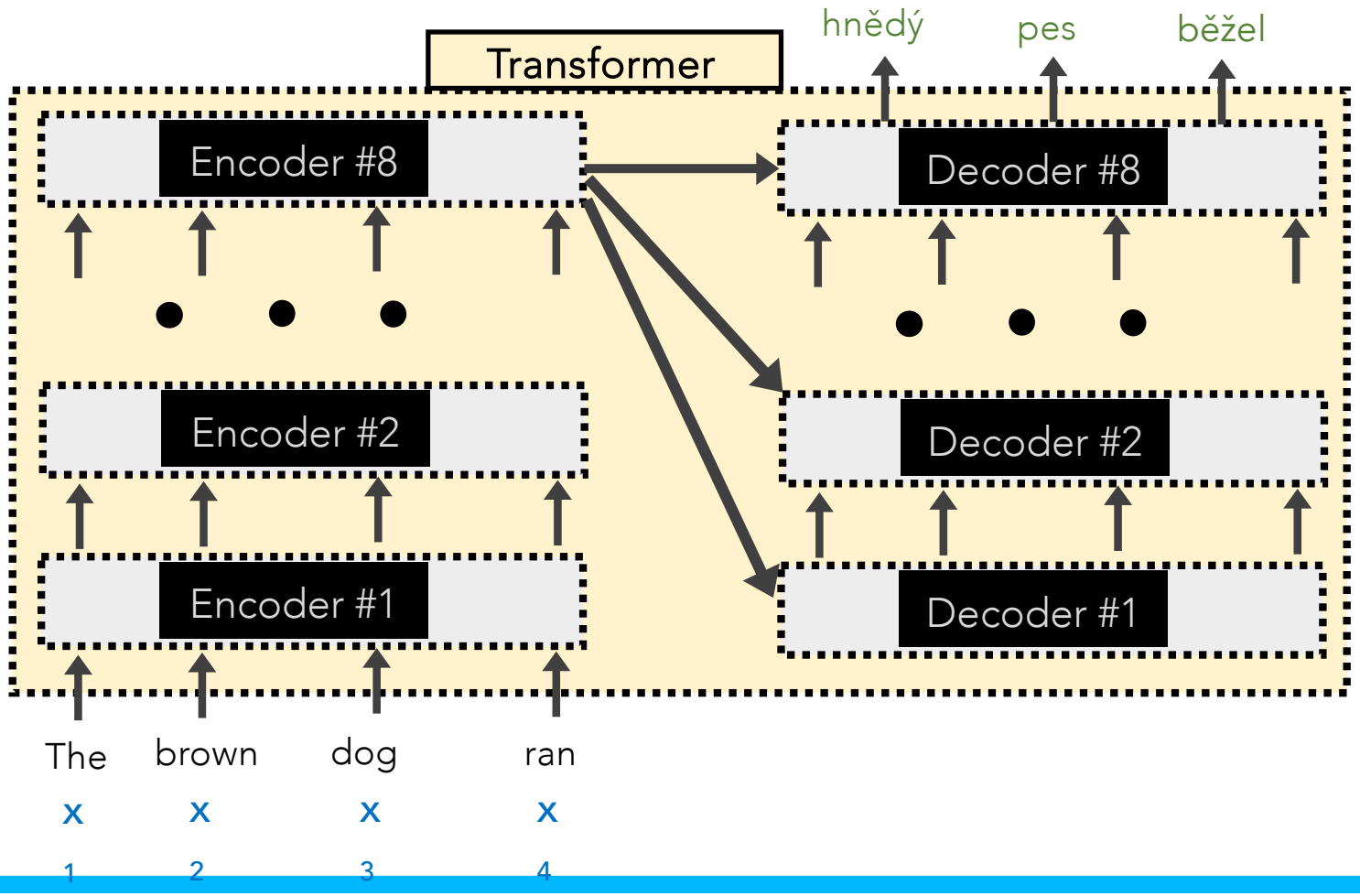


NOTE

The query vector for a Transformer Decoder's Attention Head (not Self-Attention Head) is from the output of the previous decoder layer.

However, the key and value vectors are from the Transformer Encoders' outputs.

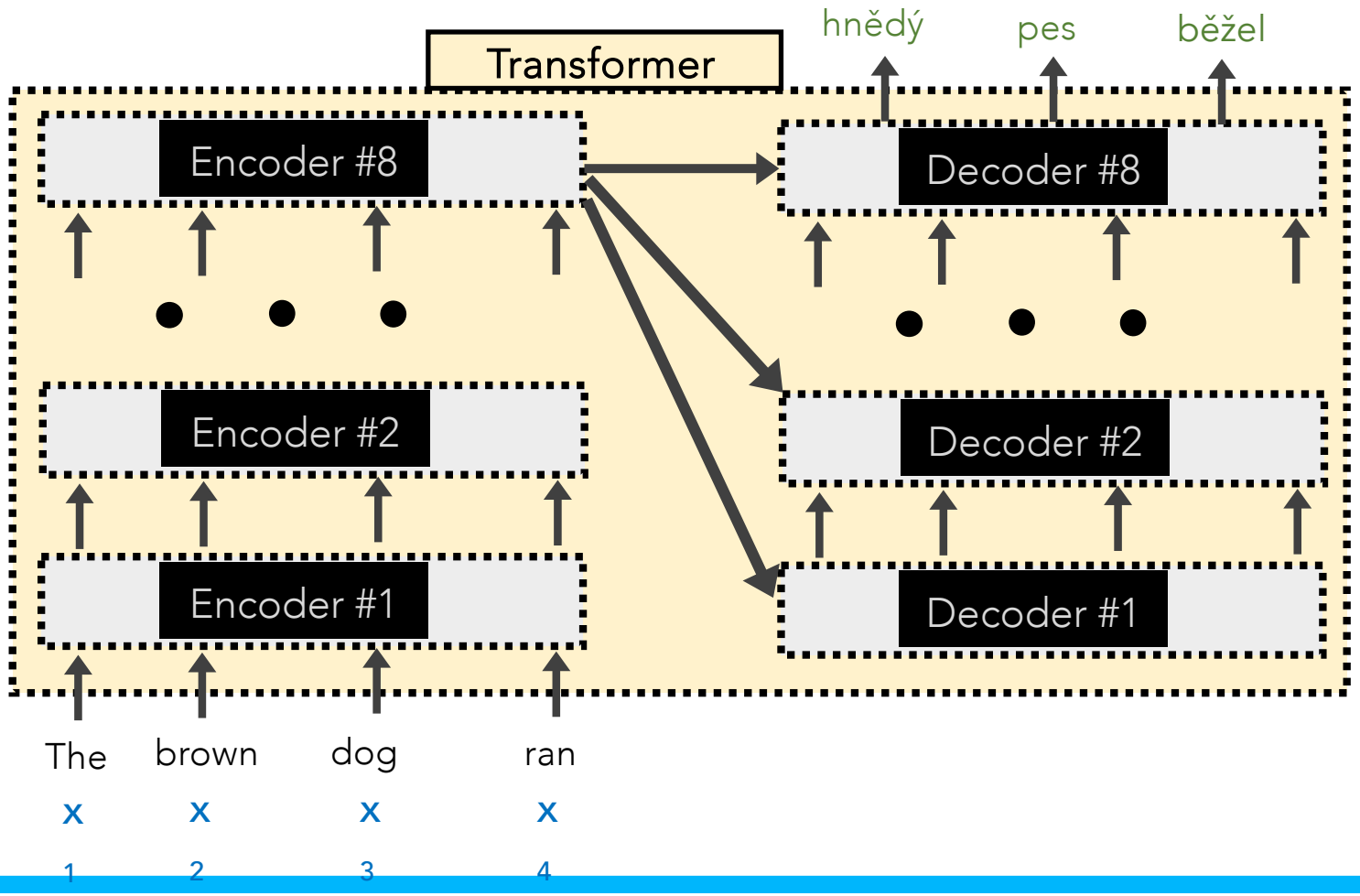
Transformer Encoders and Decoders



NOTE

The query, key, and value vectors for a Transformer Decoder's Self-Attention Head (not Attention Head) are all from the output of the previous decoder layer.

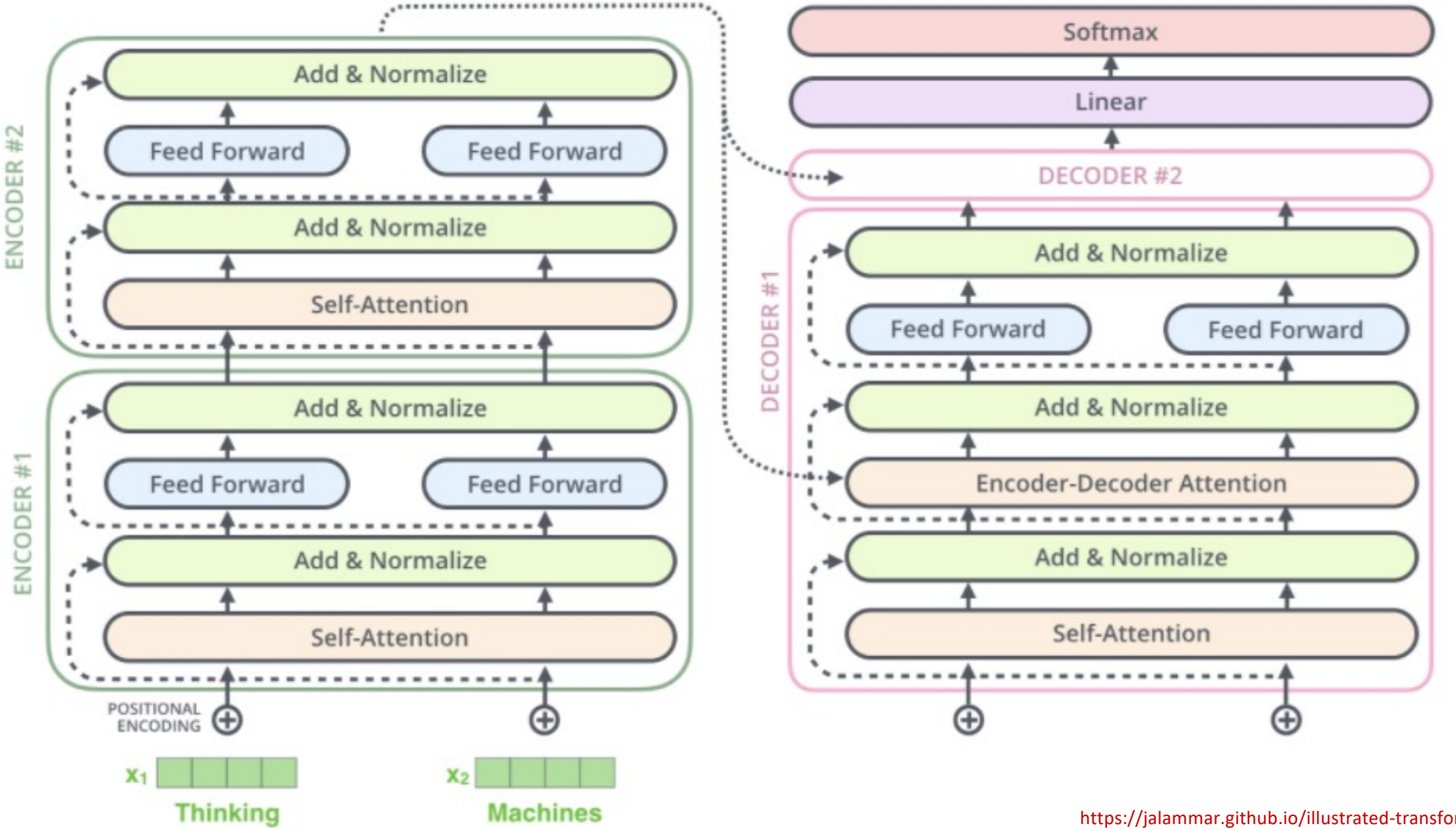
Transformer Encoders and Decoders



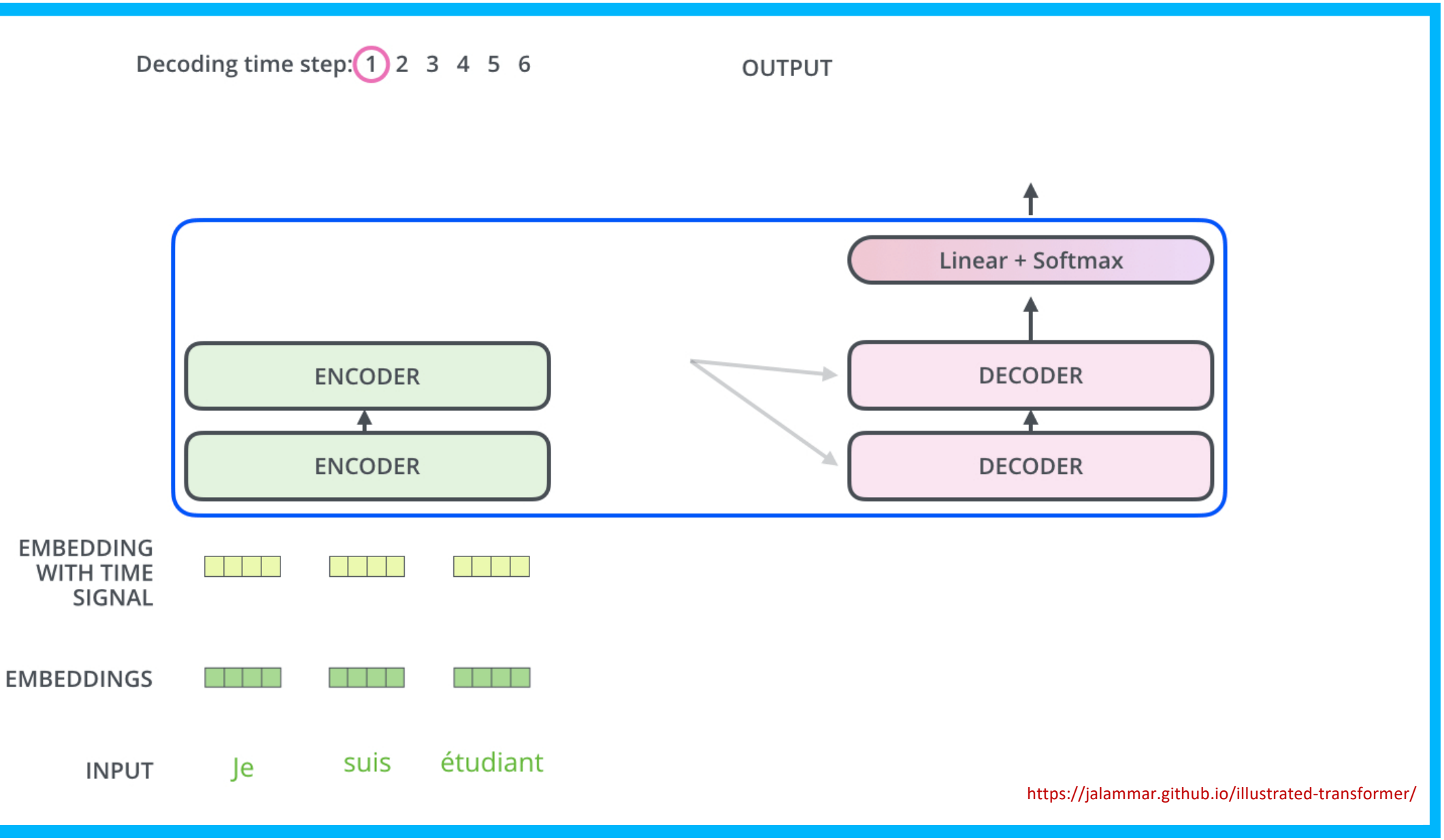
IMPORTANT

The Transformer Decoders have positional embeddings, too, just like the Encoders.

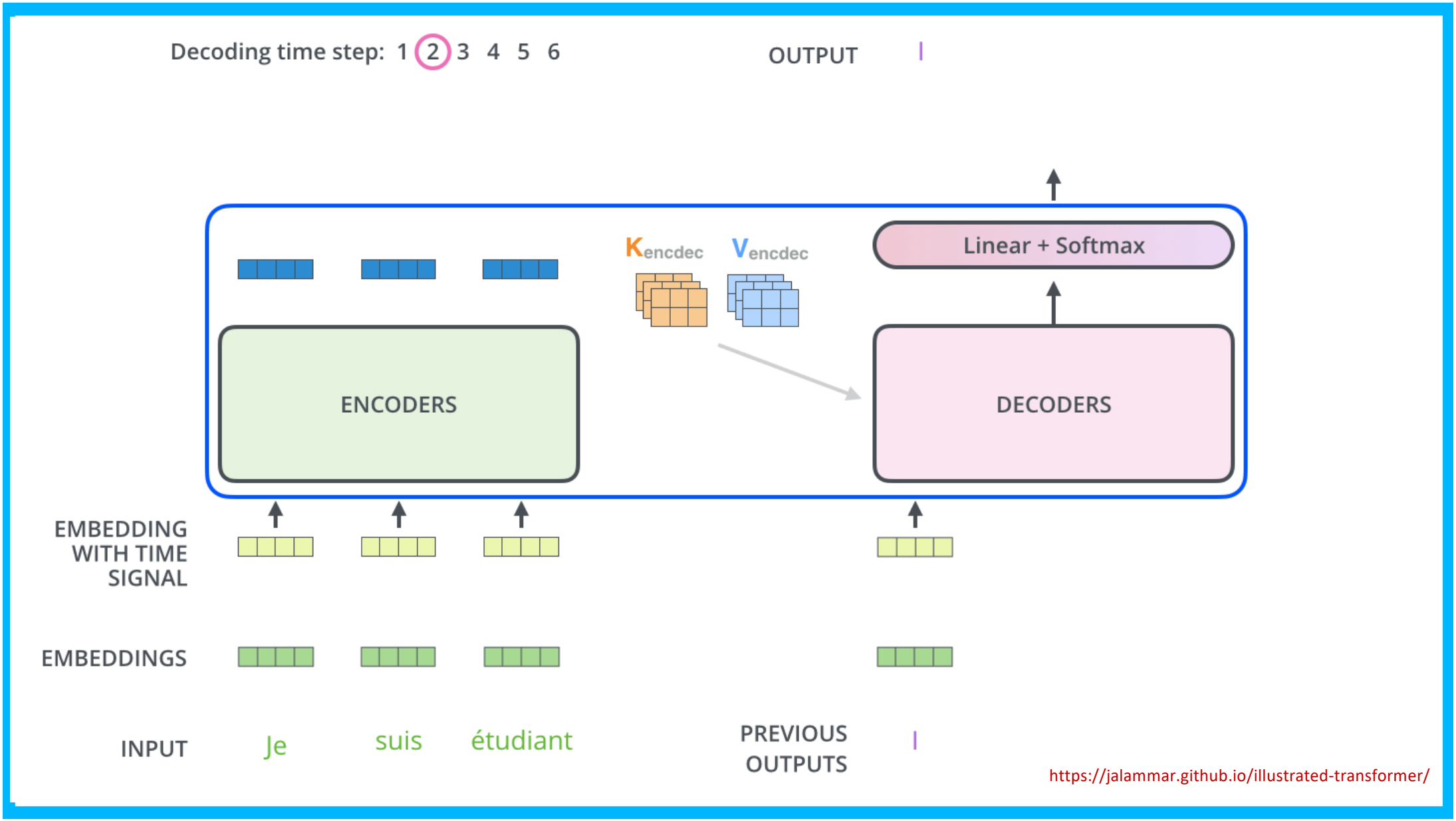
Critically, each position is only allowed to attend to the previous indices. This *masked* Attention preserves it as being an auto-regressive LM.



https://jalammar.github.io/illustrated-transformer/



https://jalammar.github.io/illustrated-transformer/


Decoding time step: 1 2 3 4 5 6
OUTPUT
I
Linear + Softmax
K encdec
V encdec
ENCODERS
DECODERS
EMBEDDING WITH TIME SIGNAL
EMBEDDINGS
INPUT
Je
suis
étudiant
PREVIOUS OUTPUTS
I
https://jalammar.github.io/illustrated-transformer/

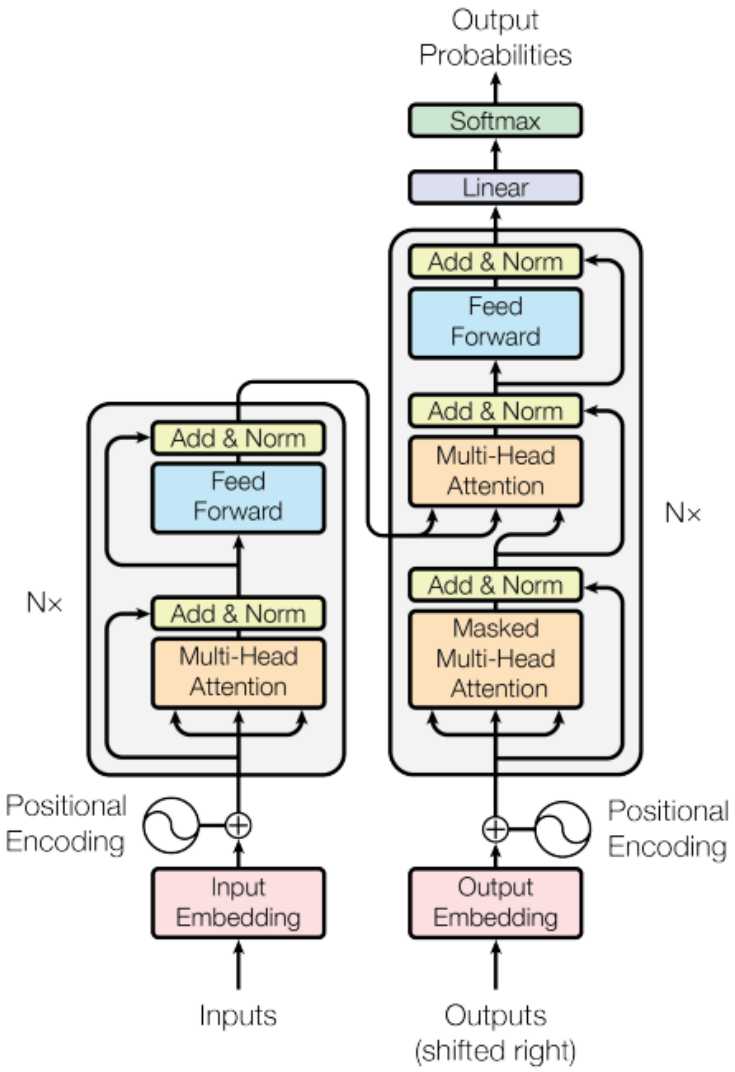


Figure 1: The Transformer - model architecture.

Attention is All you Need (2017) https://arxiv.org/pdf/1706.03762.pdf

- What if we don't want to decode/translate?
- Just want to perform a particular task (e.g., classification)
- Want even more robust, flexible, rich representation!
- Want positionality to play a more explicit role, while not being restricted to a particular form (e.g., CNNs)

Outline

Self-Attention

Transformer Encoder

Transformer Decoder

BERT

Outline

Self-Attention

Transformer Encoder

Transformer Decoder

BERT

BERT

Bidirectional Encoder Representations from Transformers

BERT

Bidirectional Encoder Representations from Transformers

Like *Bidirectional LSTMs*, let's look in **both** directions

BERT

Bidirectional Encoder Representations from Transformers

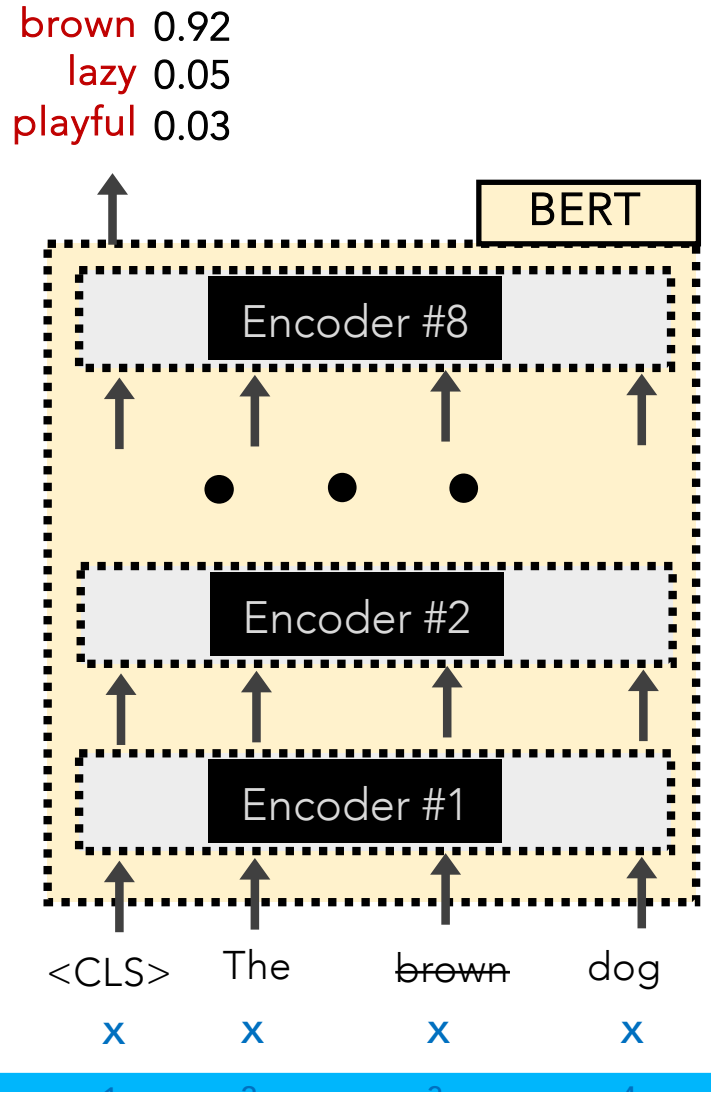
Let's only use Transformer *Encoders*, no Decoders

BERT

Bidirectional Encoder Representations from Transformers

It's a language model that builds rich representations

BERT



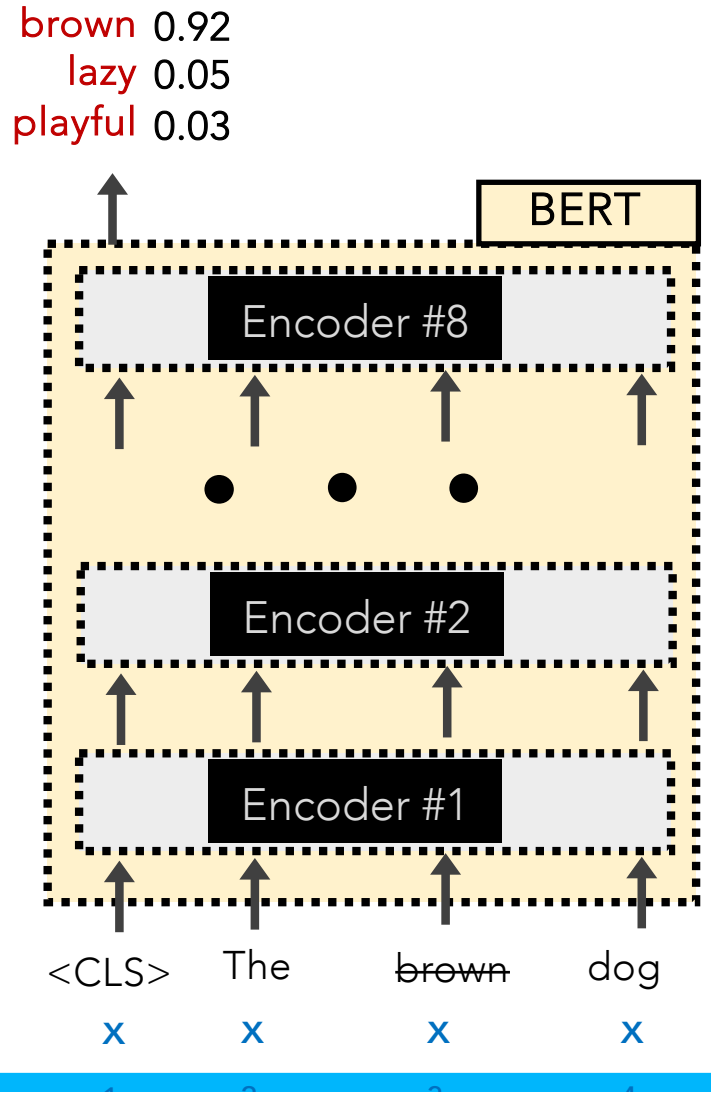
BERT has 2 training objectives:

1. Predict the **Masked word** (a la CBOW)

15% of all input words are randomly masked.

- 80% become [MASK]
- 10% become revert back
- 10% become are deliberately corrupted as wrong words

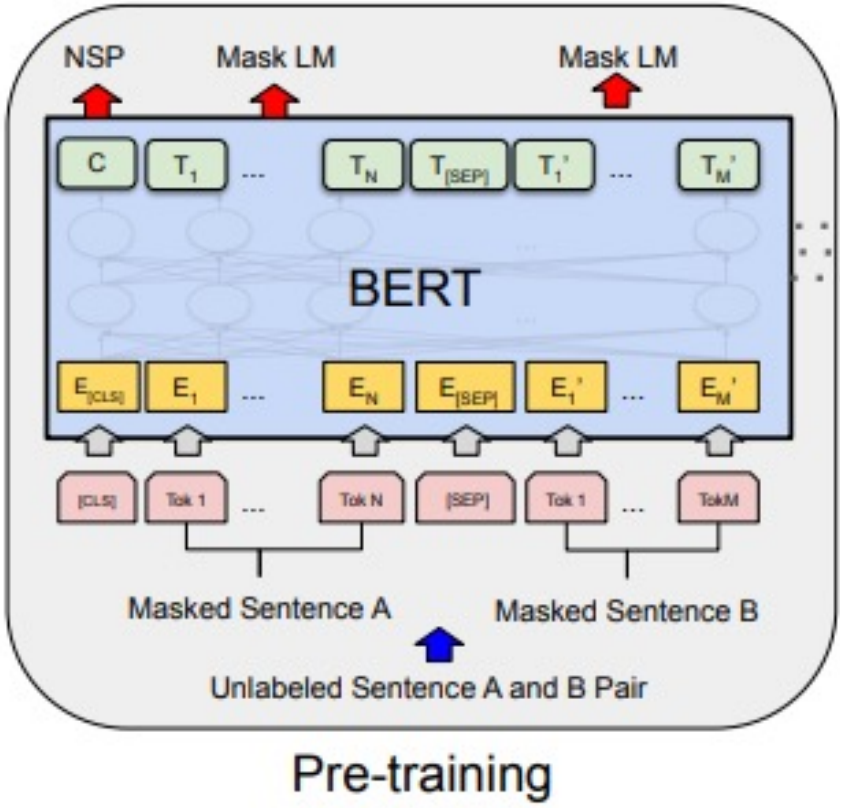
BERT



BERT has 2 training objectives:

2. Two sentences are fed in at a time. Predict the if the second sentence of input truly follows the first one or not.

BERT



Every two sentences are separated by a <SEP> token.

50% of the time, the 2nd sentence is a randomly selected sentence from the corpus.

50% of the time, it truly follows the first sentence in the corpus.

Source: original BERT paper: https://arxiv.org/pdf/1810.04805.pdf

Example of Training set

샘플 i	Sentence A	Sentence B	NSP Label	Masked Tokens
1	"The cat"	"sat on the mat"	IsNext	"cat"
2	"Alice loves Bob"	"Bananas are yellow"	NotNext	"loves"
3	"Quantum physics"	"is very complex"	IsNext	"complex"
4	"Dogs are great companions"	"He drinks coffee"	NotNext	"great"

4️⃣ 최종 Loss 및 Backward

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$$

- optimizer (e.g. Adam)로 가중치 업데이트
- gradient는 encoder + NSP classifier + MLM classifier로 모두 전달됨

(a) MLM Loss (Token-level Loss)

- 각 샘플마다 **masked 위치의 예측 결과만** 골라서 정답과 비교
- CrossEntropy Loss 적용:

$$\mathcal{L}_{\text{MLM}} = \frac{1}{\#\text{masked tokens}} \sum_{i} \text{CE}(\text{logits}_i, \text{target}_i)$$

예:

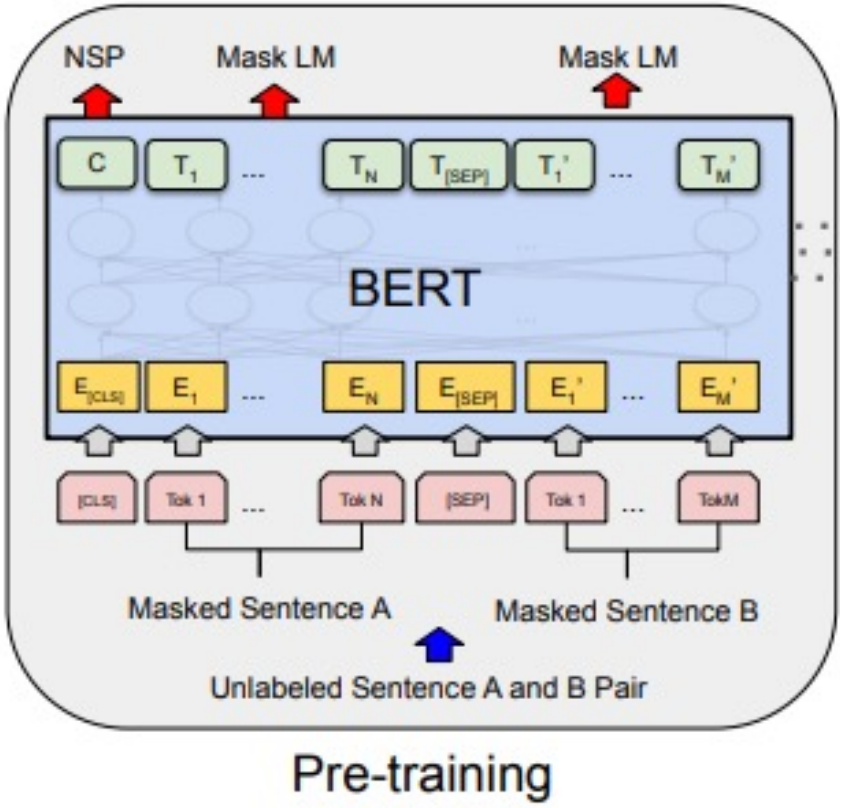
- 샘플 1 → `cat`을 예측해야 함
- 샘플 2 → `loves` 예측

(b) NSP Loss (Sentence-level Loss)

- 각 샘플의 **[CLS] 임베딩** → 2-class classifier (IsNext / NotNext)
- CrossEntropy Loss로 NSP label 예측

$$\mathcal{L}_{\text{NSP}} = \frac{1}{N} \sum_{i=1}^{N} \text{CE}(\text{NSP_logits}_i, \text{label}_i)$$

BERT



Pre-training

NOTE: BERT also embeds the inputs by their WordPiece embeddings.

WordPiece is a sub-word tokenization learns to merge and use characters based on which pairs maximize the likelihood of the training data if added to the vocab.

Source: original BERT paper: https://arxiv.org/pdf/1810.04805.pdf

Outline

- Transformer Decoder
- Learning/Data/Tasks
- BERT
- BERT Fine-Tuning
- Extensions

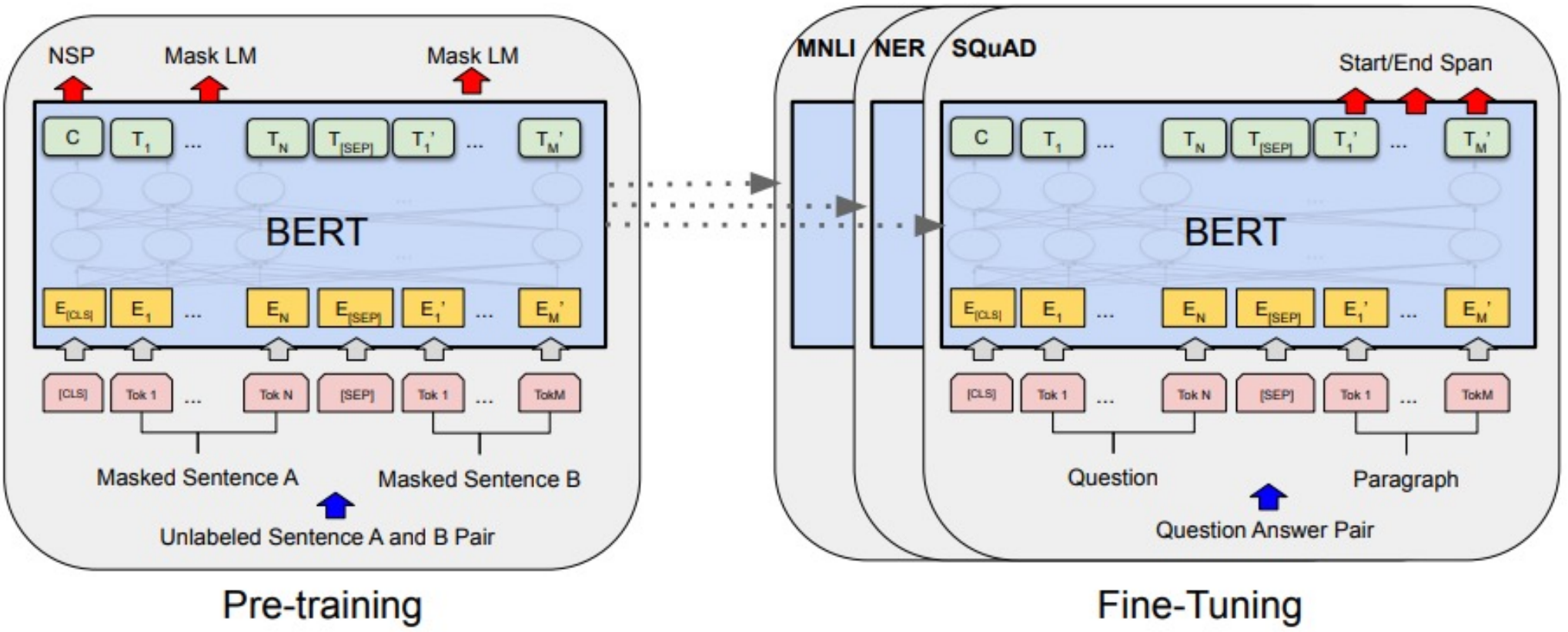
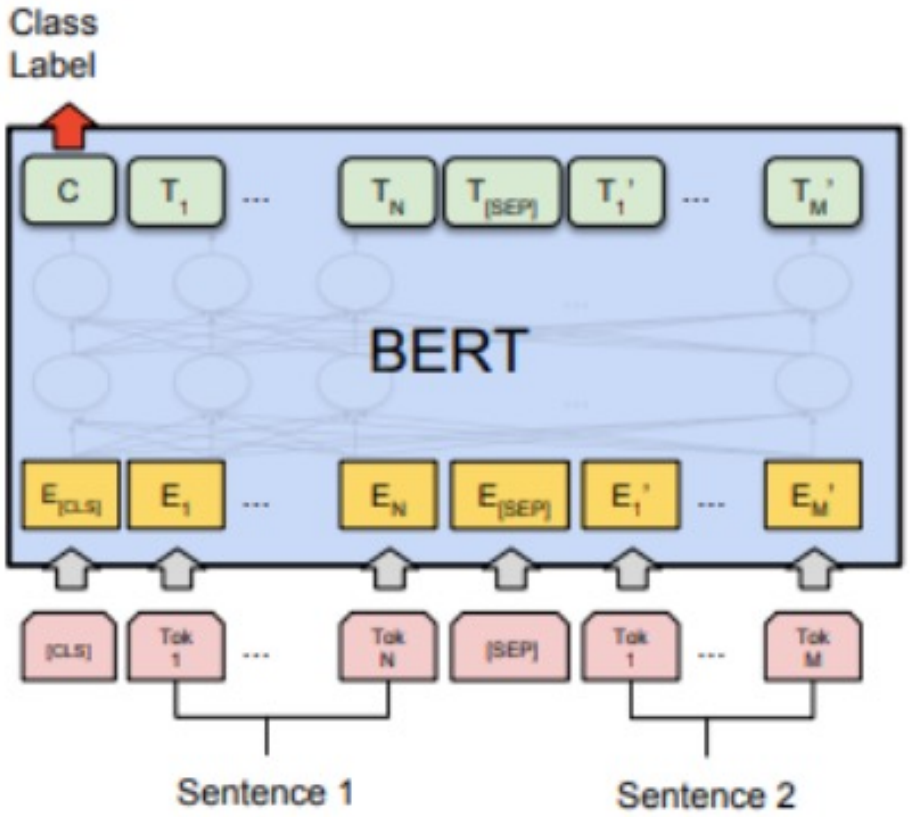


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. `[CLS]` is a special symbol added in front of every input example, and `[SEP]` is a special separator token (e.g. separating questions/answers).

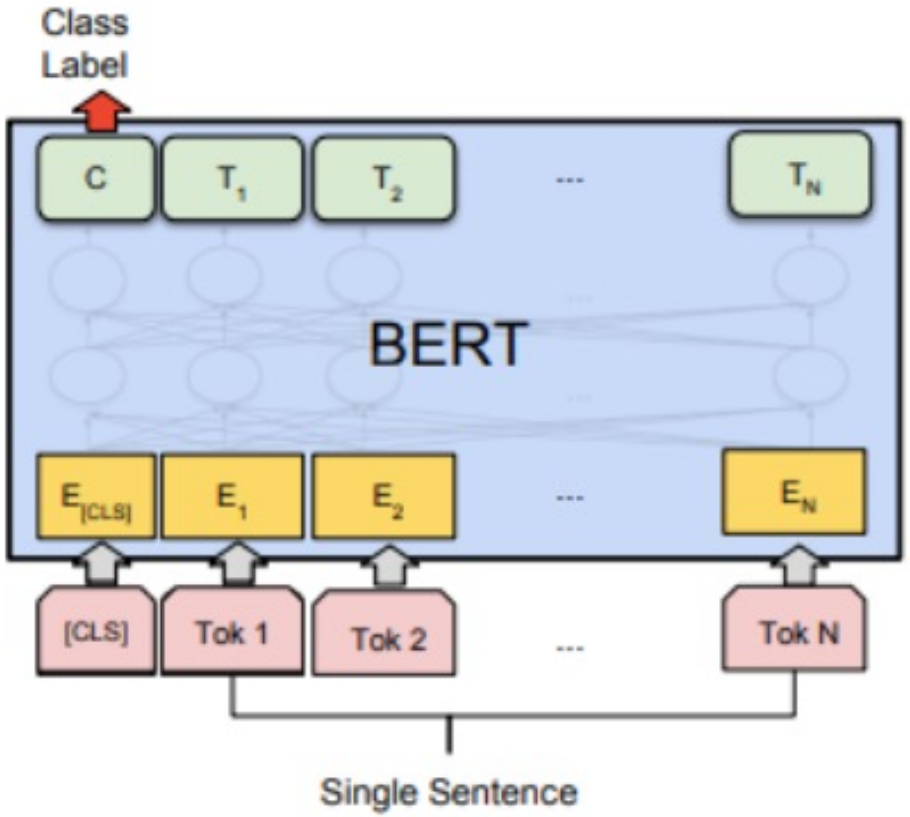
https://aclanthology.org/N19-1423.pdf

BERT fine-tuning



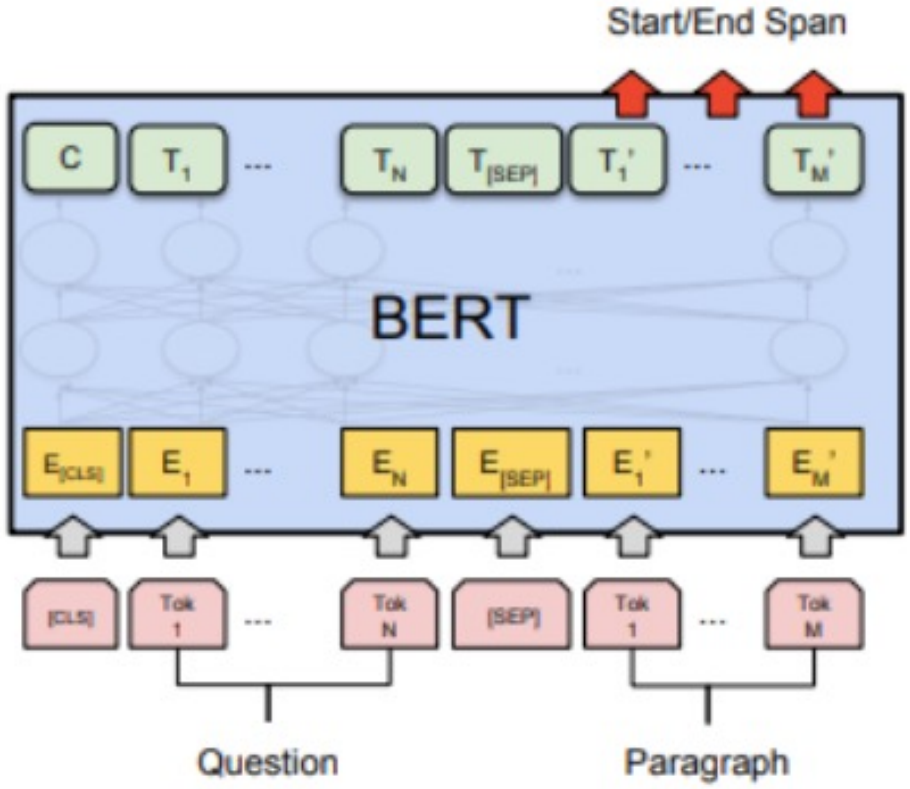
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

BERT fine-tuning



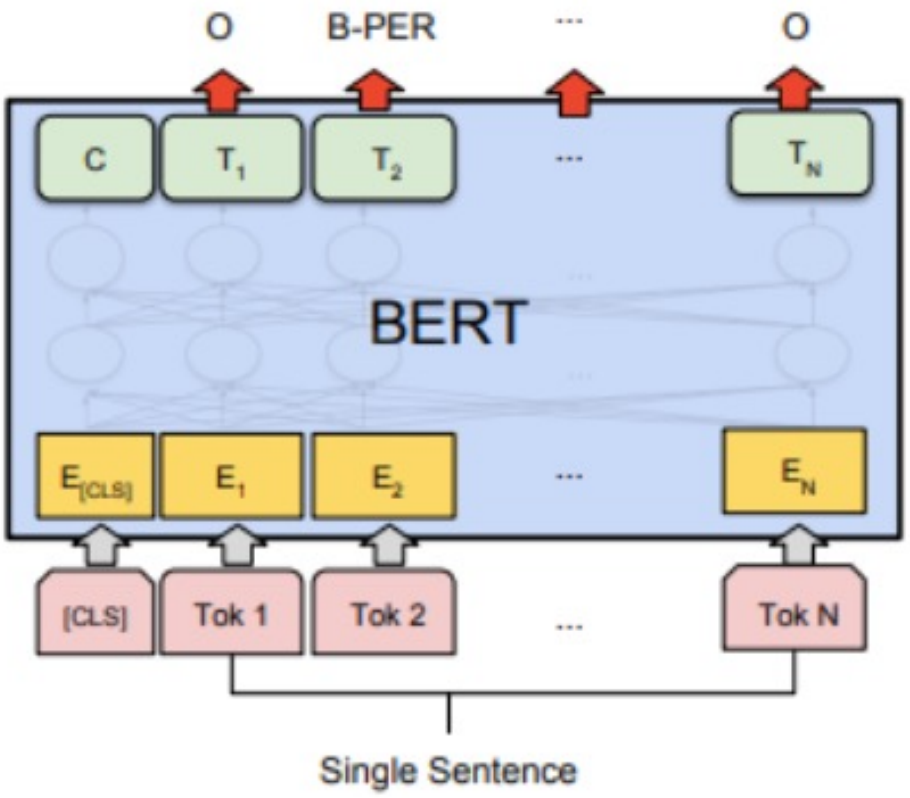
(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT fine-tuning



(c) Question Answering Tasks:
SQuAD v1.1

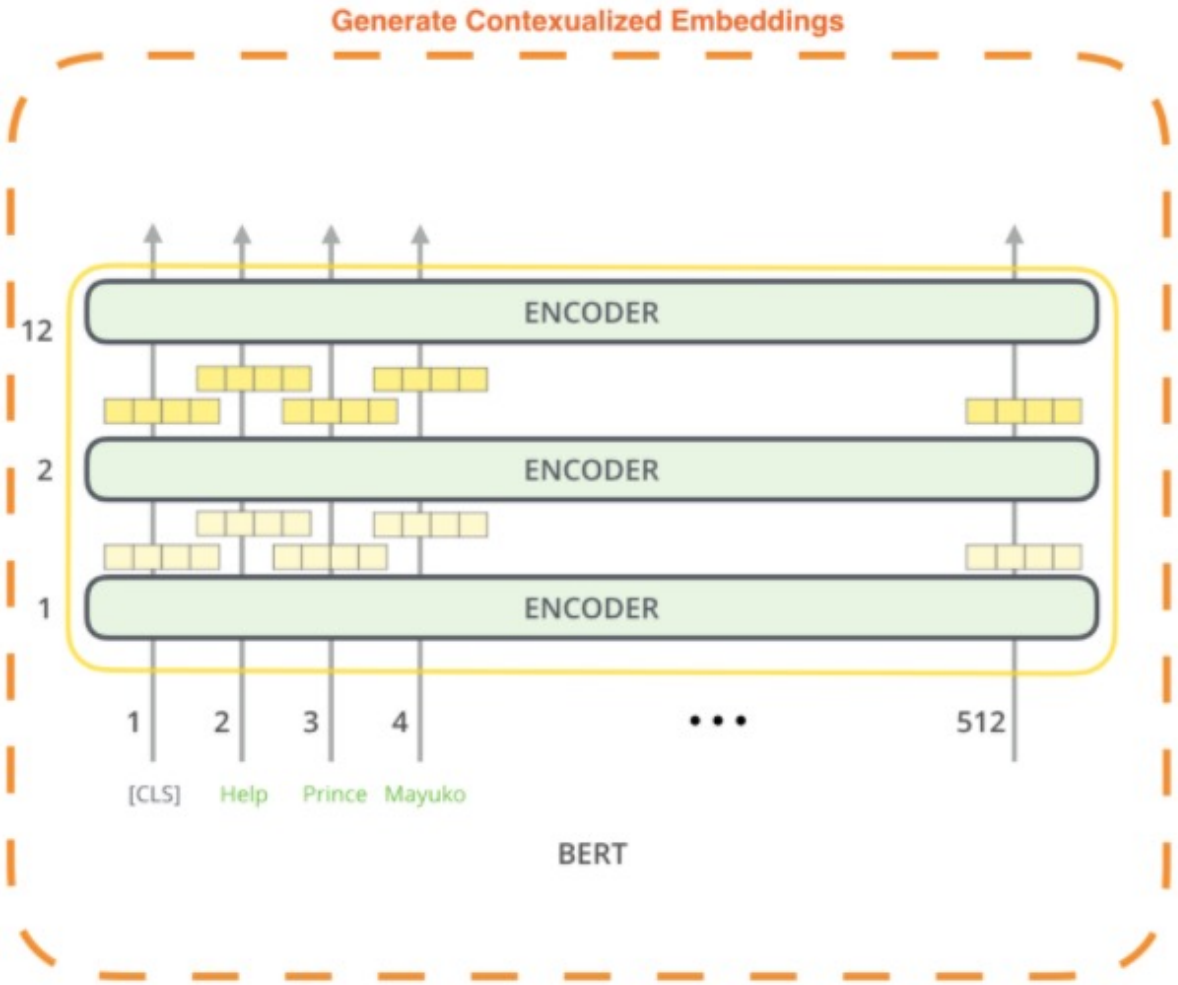
BERT fine-tuning



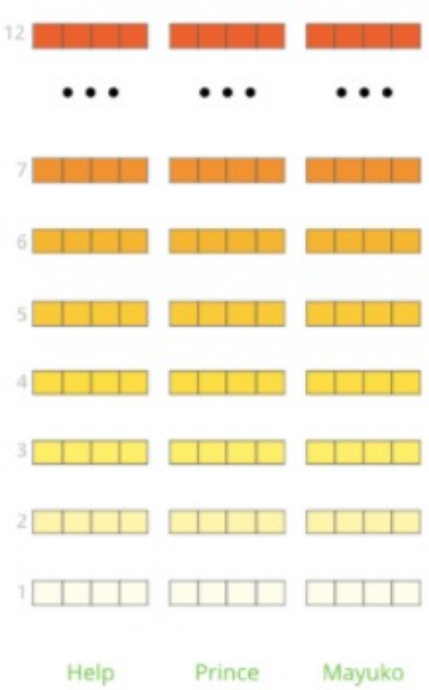
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT

Or, one could extract the contextualized embeddings



The output of each encoder layer along each token's path can be used as a feature representing that token.

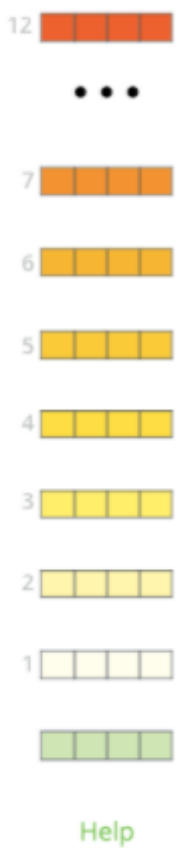


But which one should we use?

Picture: https://jalammar.github.io/illustrated-bert/

BERT

Later layers have the best contextualized embeddings



		Dev F1 Score
First Layer	Embedding	91.0
Last Hidden Layer	12	94.9
Sum All 12 Layers	12 + ... + 2 + 1 =	95.5
Second-to-Last Hidden Layer	11	95.6
Sum Last Four Hidden	12 + 11 + 10 + 9 =	95.9
Concat Last Four Hidden	9 10 11 12	96.1

Picture: https://jalammar.github.io/illustrated-bert/

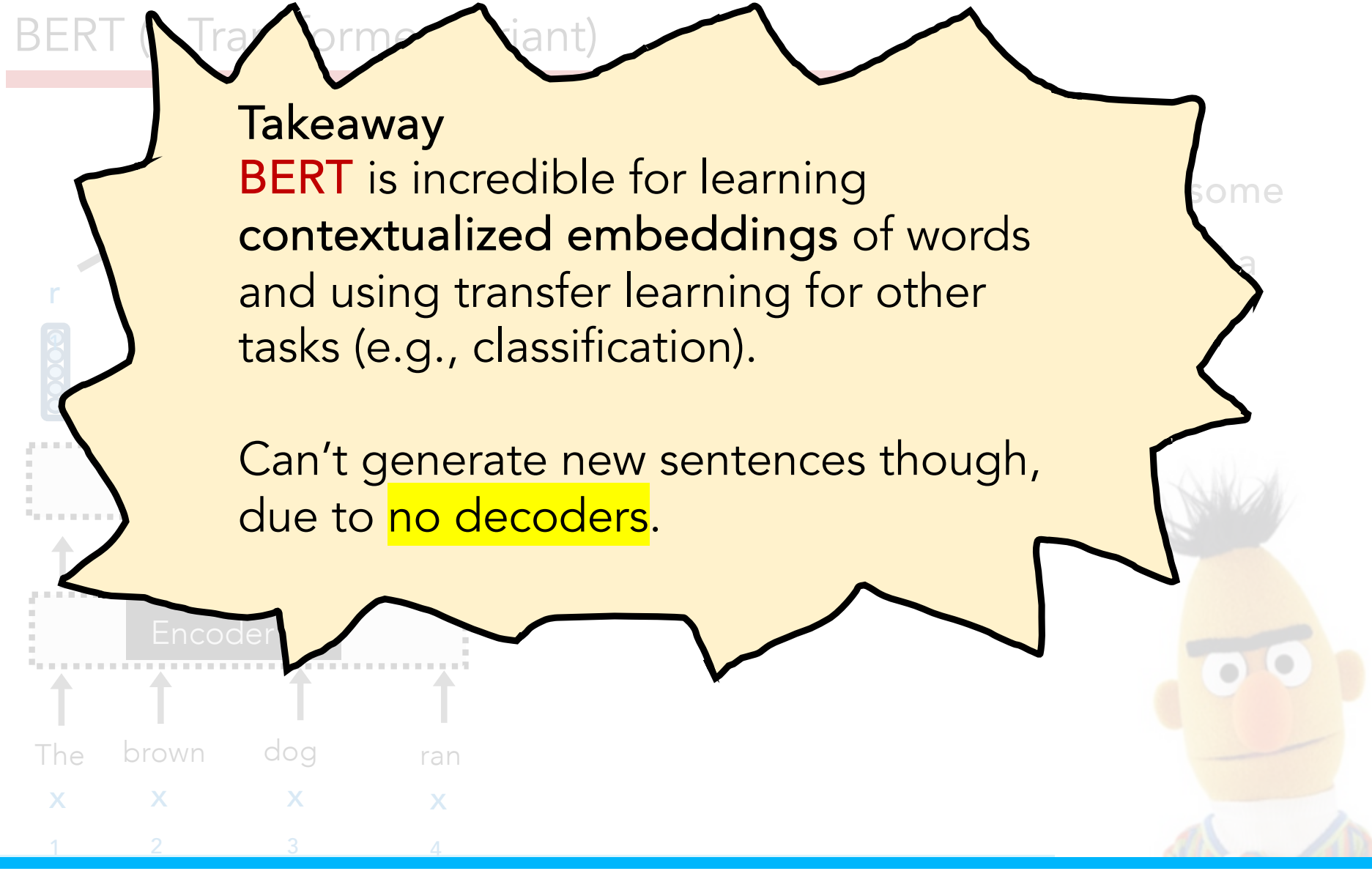
BERT

BERT yields state-of-the-art (SOTA) results on many tasks

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

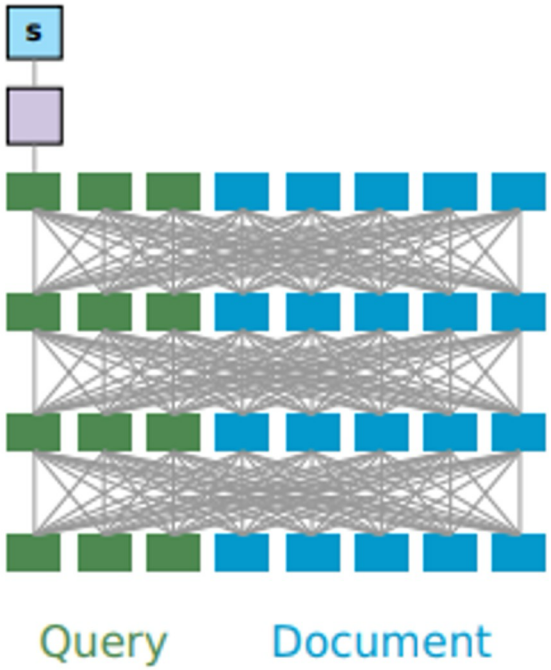
Table 1: GLUE Test results, scored by the evaluation server (https://gluebenchmark.com/leaderboard).

Source: original BERT paper: https://arxiv.org/pdf/1810.04805.pdf


Takeaway
BERT is incredible for learning contextualized embeddings of words and using transfer learning for other tasks (e.g., classification).
Can't generate new sentences though, due to no decoders.

Dense Retrieval

Cross-encoders



1. Examples: $\langle q_i, \text{doc}_i^+, \{\text{doc}_{i,k}^-\}\rangle$
2. For a BERT-style encoder with N layers:

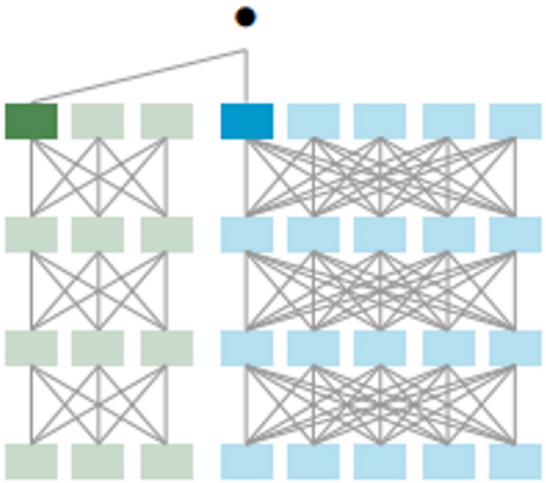
$$\textbf{Rep}(q, \text{doc}) = \text{Dense}\left(\textbf{Enc}([q; \text{doc}]_{N,0})\right)$$

3. Loss: negative log-likelihood of the positive passage

$$-\log \frac{\exp\left(\textbf{Rep}(q_i, \text{doc}_i^+)\right)}{\exp\left(\textbf{Rep}(q_i, \text{doc}_i^+)\right) + \sum_{j=1}^{n} \exp\left(\textbf{Rep}(q_i, \text{doc}_{i,j}^-)\right)}$$

Incredibly rich, but won't scale!

DPR



1. Examples: $\langle q_i, \text{doc}_i^+, \{\text{doc}_{i,k}^-\}\rangle$
2. For a BERT-style encoder with N layers:

   $$\textbf{Sim}(q, \text{doc}) = \textbf{EncQ}(q)_{N,0}{}^{\top}\textbf{EncD}(\text{doc})_{N,0}$$

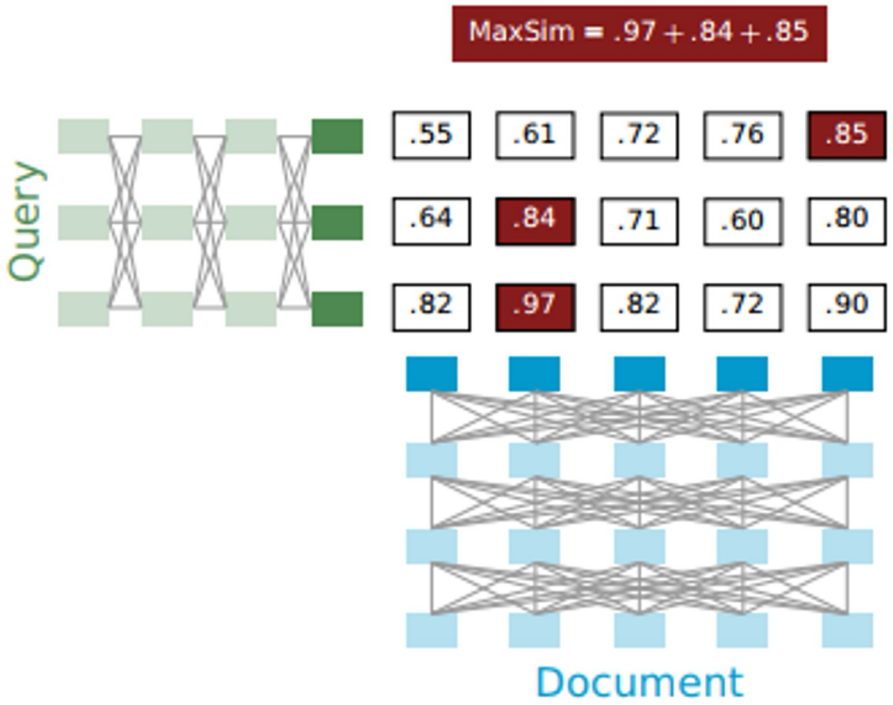
3. Loss: negative log-likelihood of the positive passage

$$-\log \frac{\exp\left(\textbf{Sim}(q_i, \text{doc}_i^+)\right)}{\exp\left(\textbf{Sim}(q_i, \text{doc}_i^+)\right) + \sum_{j=1}^{n} \exp\left(\textbf{Sim}(q_i, \text{doc}_{i,j}^-)\right)}$$

Highly scalable, but limited query/doc interactions!

Karpukhin et al. 2020

ColBERT



1. Examples: $\langle q_i, \text{doc}_i^+, \{\text{doc}_{i,k}^-\}\rangle$
2. Loss: negative log-likelihood of the positive passage, with **MaxSim** as the basis.

Highly scalable with late, contextual interactions!

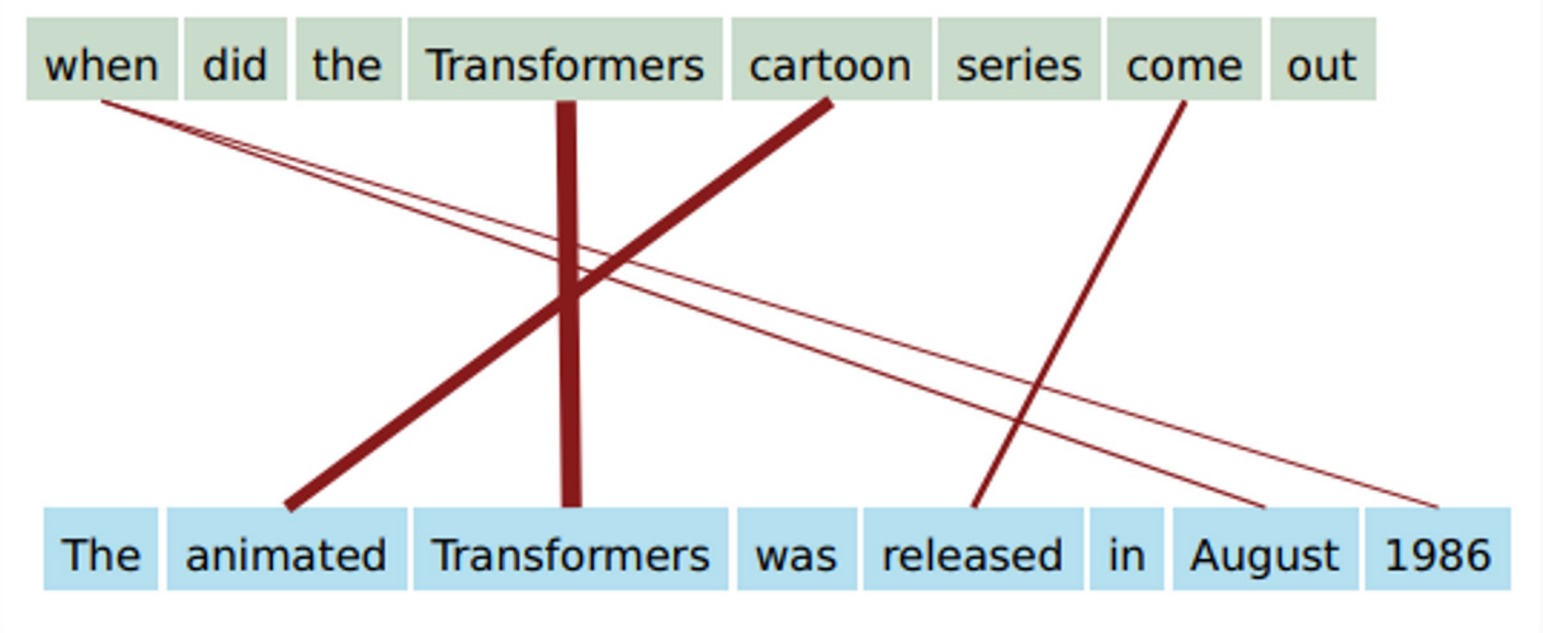
For a BERT-style encoder with N layers:

$$\textbf{MaxSim}(q, \text{doc}) = \sum_{i}^{L} \max_{j}^{M} \textbf{Enc}(q)_{N,i}{}^{\mathsf{T}} \textbf{Enc}(\text{doc})_{N,j}$$

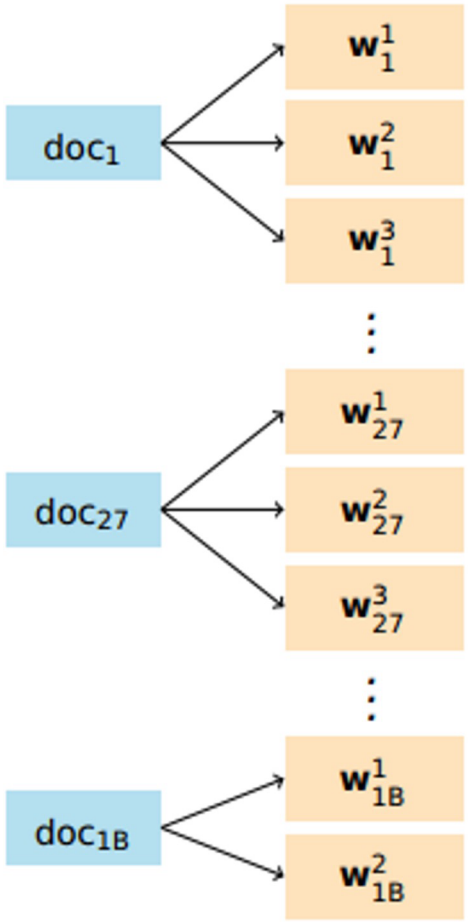
with L is the length of q, M the length of doc.

Khattab and Zaharia 2020

Soft alignment with ColBERT



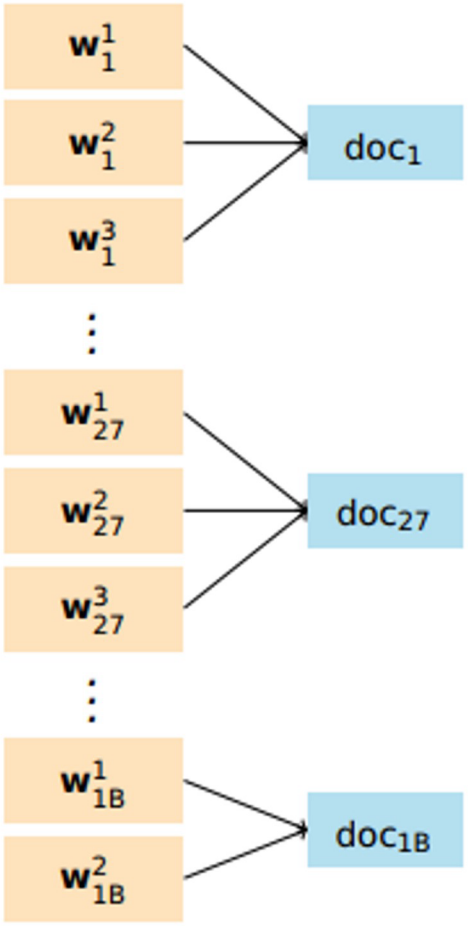
ColBERT as a reranker



Given query $q = [w^1, \ldots, w^M]$:

1. Get the top K documents for q using a fast, term-based model like BM25.
2. Score each of those top K documents using ColBERT.

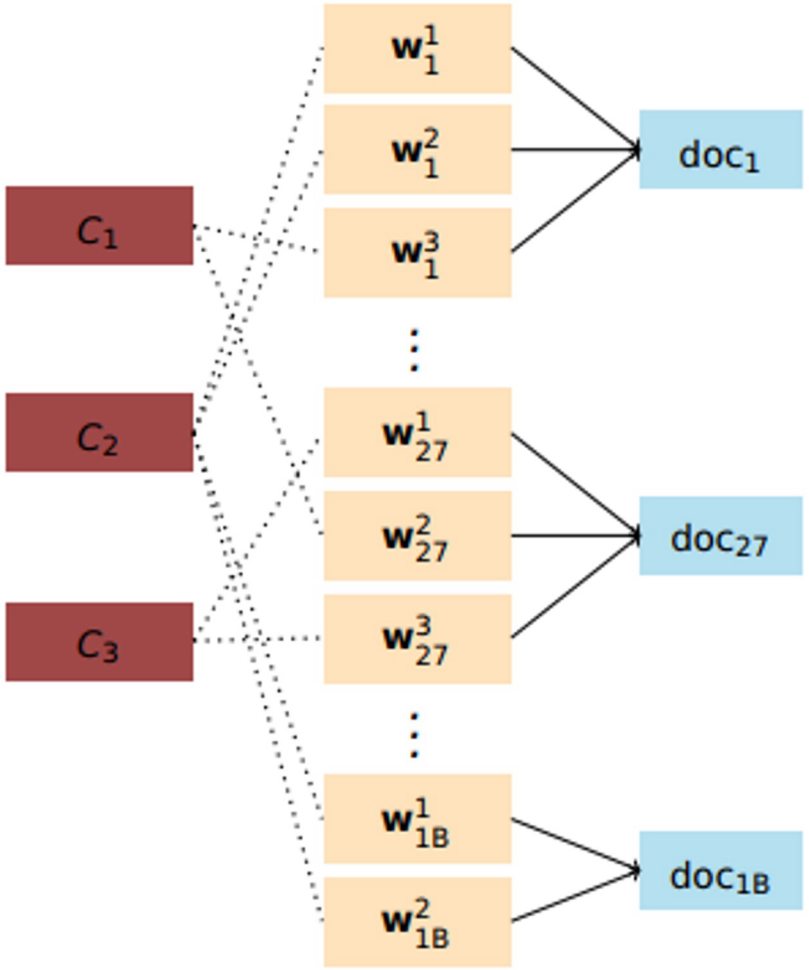
Beyond reranking for ColBERT



Given query q encoded as vectors $[\mathbf{w}^1, \ldots, \mathbf{w}^M]$, for each query vector $\mathbf{w}^i$:

1. Retrieve the p most similar token vectors $\mathbf{w}_j^k$ to $\mathbf{w}^i$.
2. Score each doc_j using ColBERT.

Centroid-based ranking



Given q encoded as $[\mathbf{w}^1, \ldots, \mathbf{w}^M]$, for each vector $\mathbf{w}^i$:

1. Retrieve the p centroids closest to $\mathbf{w}^i$.
2. Retrieve the t most similar token vectors $\mathbf{w}_j^k$ to any of the centroids.
3. Score each doc_j using ColBERT.

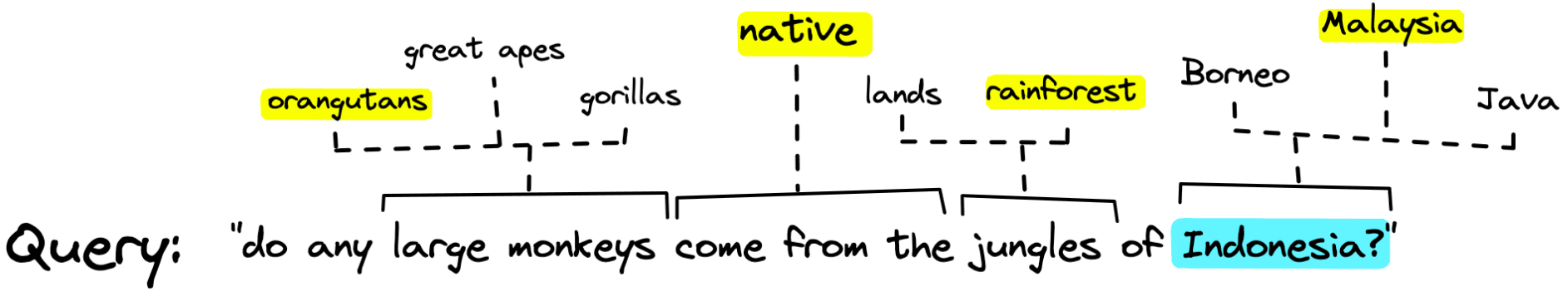
Sparse Retrieval

Problems with Existing Sparse Retrieval

Query: "do any large monkeys come from the jungles of Indonesia?"

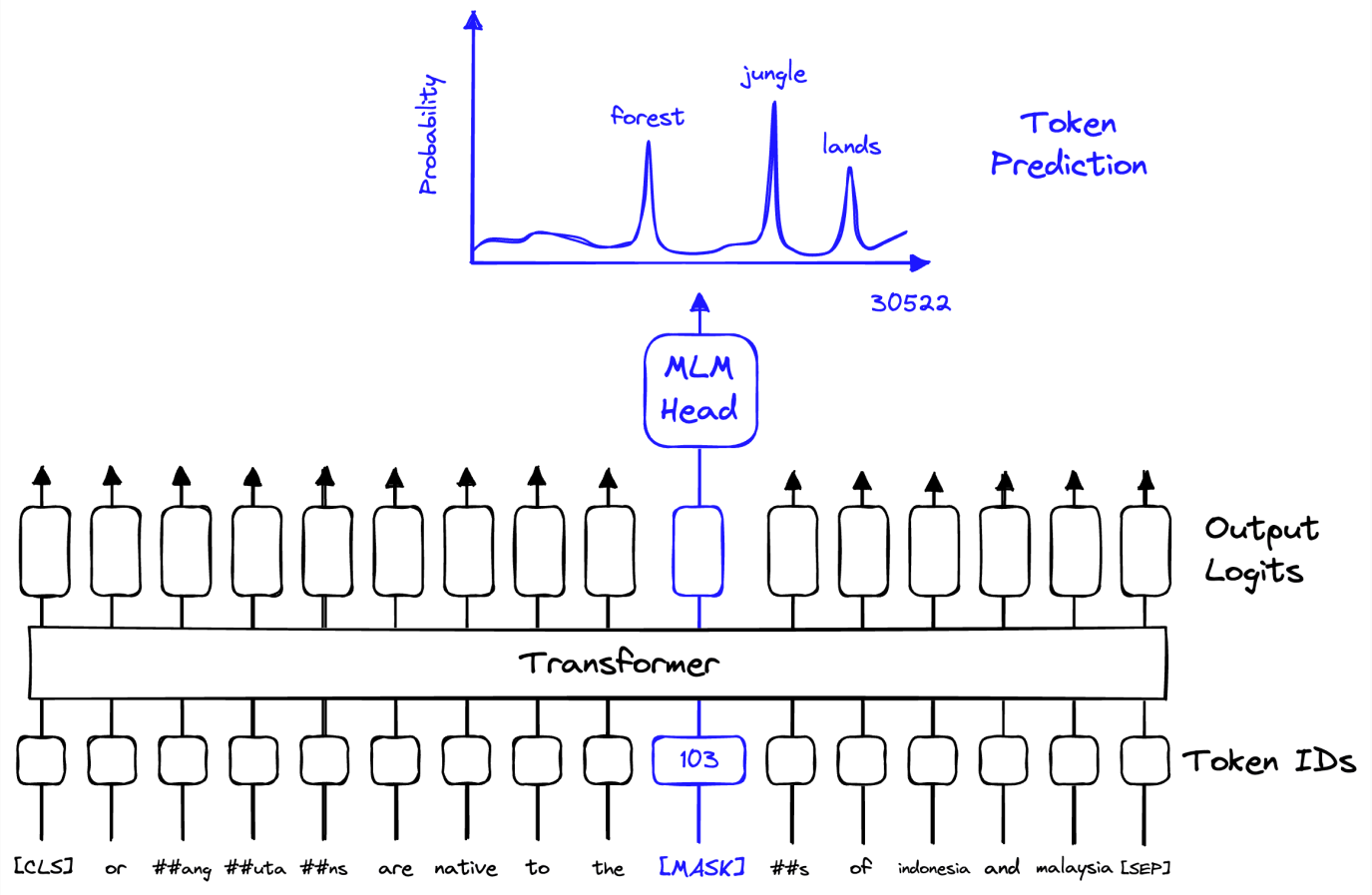
term overlap

Doc: "Orangutans are native to the rainforests of Indonesia and Malaysia"

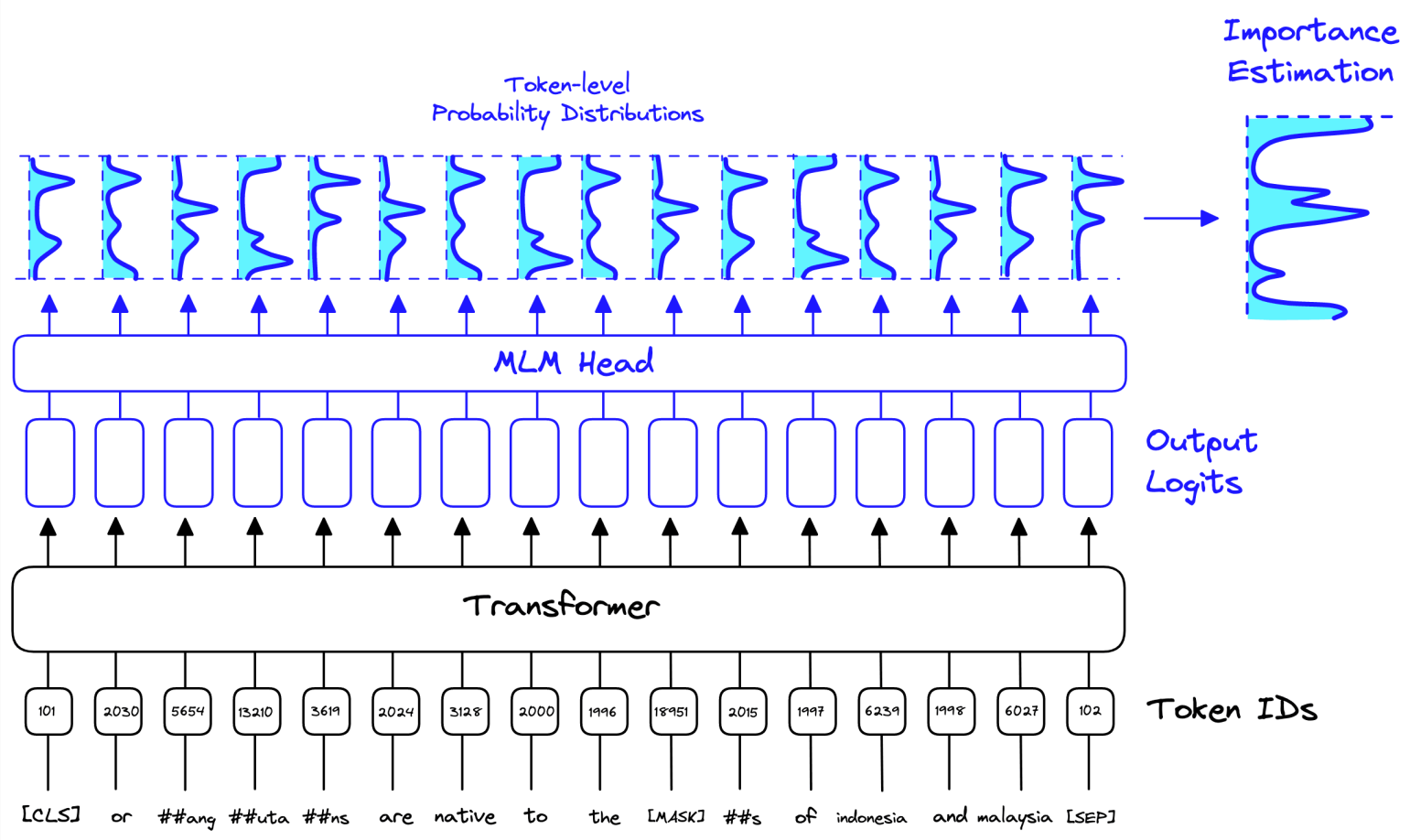


- Thibault Formal *et al.*, "SPLADE: Sparse Lexical and Expansion Model," SIGIR 2021.
- Carlos Lassance & Stéphane Clinchant, "An Efficiency Study for SPLADE Models," arXiv 2207.03834.
- Pinecone Blog, "SPLADE for Sparse Vector Search Explained," 2023.

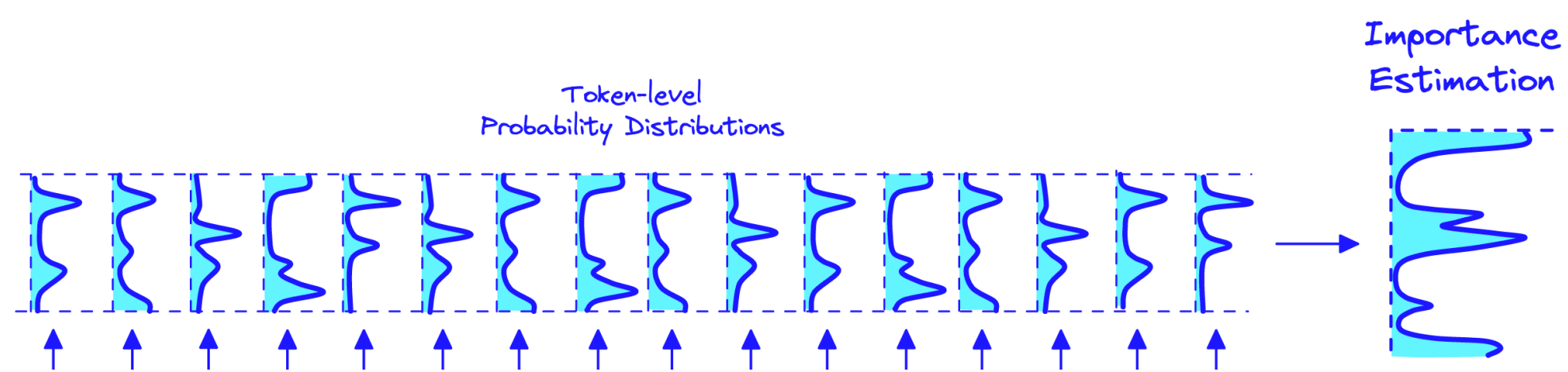
MLM head produces a probability distribution from each output logit



MLM and Sparse Vectors



Reducing the number of query vector non-zero values



1. Document expansion only
2. max pooling

$$w_j = \sum_{i \in t} log(1 + ReLU(w_{ij})) \quad \Rightarrow \quad w_j = max_{i \in t} log(1 + ReLU(w_{ij}))$$

Datasets

TREC

1. **T**ext **RE**trieval **C**onference (TREC) has annual competitions for comparing IR systems.
2. The 2023 iteration has a number of tracks:

   https://trec.nist.gov/pubs/call2023.html
3. TREC tends to emphasize careful evaluation with a very small set of queries (e.g., 50 queries, each with >100 annotated documents).
4. Having few test queries does not imply few documents!

MS MARCO ranking tasks

1. MS MARCO Ranking is the largest public IR benchmark.
2. It is adapted from a Question Answering dataset
3. It consists of more than 500k Bing search queries
4. Sparse labels: approx. one relevance label per query!
5. Fantastic for training IR models!
6. Passage Ranking: 9M short passages; sparse labels
7. Document Ranking: 3M long documents; sparse labels

BEIR: Benchmarking IR

For testing models in zero-shot scenarios:

Split (→)					Train	Dev	Test			Avg. Word Lengths	
Task (↓)	Domain (↓)	Dataset (↓)	Title	Relevancy	#Pairs	#Query	#Query	#Corpus	Avg. D / Q	Query	Document
Passage-Retrieval	Misc.	MS MARCO [45]	✗	Binary	532,761	—	6,980	8,841,823	1.1	5.96	55.98
Bio-Medical	Bio-Medical	TREC-COVID [65]	✓	3-level	—	—	50	171,332	493.5	10.60	160.77
Information	Bio-Medical	NFCorpus [7]	✓	3-level	110,575	324	323	3,633	38.2	3.30	232.26
Retrieval (IR)	Bio-Medical	BioASQ [61]	✓	Binary	32,916	—	500	14,914,602	4.7	8.05	202.61
Question	Wikipedia	NQ [34]	✓	Binary	132,803	—	3,452	2,681,468	1.2	9.16	78.88
Answering	Wikipedia	HotpotQA [76]	✓	Binary	170,000	5,447	7,405	5,233,329	2.0	17.61	46.30
(QA)	Finance	FiQA-2018 [44]	✗	Binary	14,166	500	648	57,638	2.6	10.77	132.32
Tweet-Retrieval	Twitter	Signal-1M (RT) [59]	✗	3-level	—	—	97	2,866,316	19.6	9.30	13.93
News	News	TREC-NEWS [58]	✓	5-level	—	—	57	594,977	19.6	11.14	634.79
Retrieval	News	Robust04 [64]	✗	3-level	—	—	249	528,155	69.9	15.27	466.40
Argument	Misc.	ArguAna [67]	✓	Binary	—	—	1,406	8,674	1.0	192.98	166.80
Retrieval	Misc.	Touché-2020 [6]	✓	3-level	—	—	49	382,545	19.0	6.55	292.37
Duplicate-Question	StackEx.	CQADupStack [25]	✓	Binary	—	—	13,145	457,199	1.4	8.59	129.09
Retrieval	Quora	Quora	✗	Binary	—	5,000	10,000	522,931	1.6	9.53	11.44
Entity-Retrieval	Wikipedia	DBPedia [21]	✓	3-level	—	67	400	4,635,922	38.2	5.39	49.68
Citation-Prediction	Scientific	SCIDOCS [9]	✓	Binary	—	—	1,000	25,657	4.9	9.38	176.19
	Wikipedia	FEVER [60]	✓	Binary	140,085	6,666	6,666	5,416,568	1.2	8.13	84.76
Fact Checking	Wikipedia	Climate-FEVER [14]	✓	Binary	—	—	1,535	5,416,593	3.0	20.13	84.76
	Scientific	SciFact [68]	✓	Binary	920	—	300	5,183	1.1	12.37	213.63

Thakur et al. 2021

Appendix

Retrieval-Augmented Generation

Standard Prompting

- Combine a prompt template together with an input

Standard Prompting

- Combine a prompt template together with an input

Please answer this question:
I think Vin Diesel has been a voice actor for several characters in TV series, do you know what their names are?

Problems of LLM

Problems

- *Accuracy issues:*
  - **Knowledge cutoffs:** parameters are usually only updated to a particular time

Problems

- *Accuracy issues:*
  - **Knowledge cutoffs:** parameters are usually only updated to a particular time
  - **Private data:** data stored in private text or data repositories not suitable for training

Problems

- *Accuracy issues:*
  - **Knowledge cutoffs:** parameters are usually only updated to a particular time
  - **Private data:** data stored in private text or data repositories not suitable for training
  - **Learning failures:** even for data that the model was trained on, it might not be sufficient to get the right answer

Problems

- *Accuracy issues:*
  - **Knowledge cutoffs:** parameters are usually only updated to a particular time
  - **Private data:** data stored in private text or data repositories not suitable for training
  - **Learning failures:** even for data that the model was trained on, it might not be sufficient to get the right answer
- *Verifiability issues:* It is hard to tell if the answer is correct

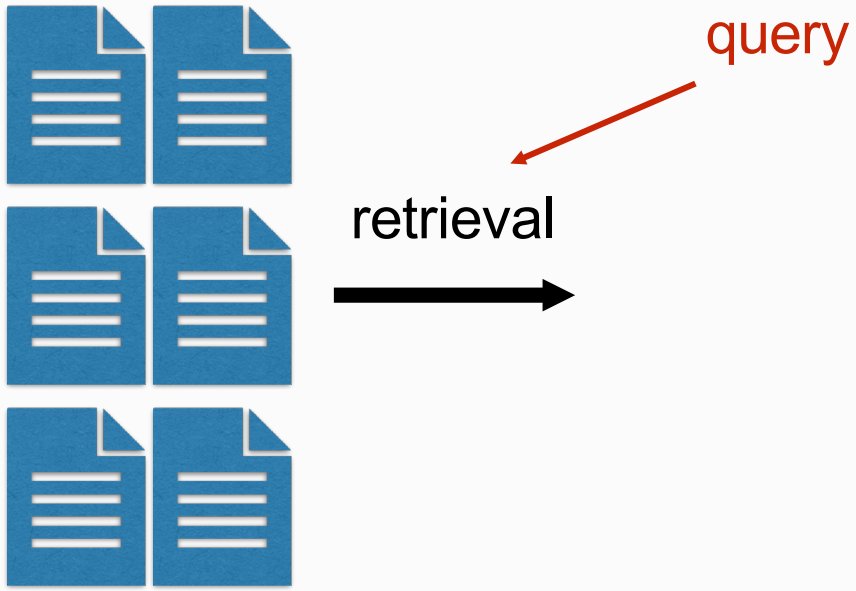
Retrieval-augmented Generation
(Chen et al. 2017)

Retrieval-augmented Generation
(Chen et al. 2017)

- **Retrieve** relevant passages efficiently
- **Read** the passages to answer the query

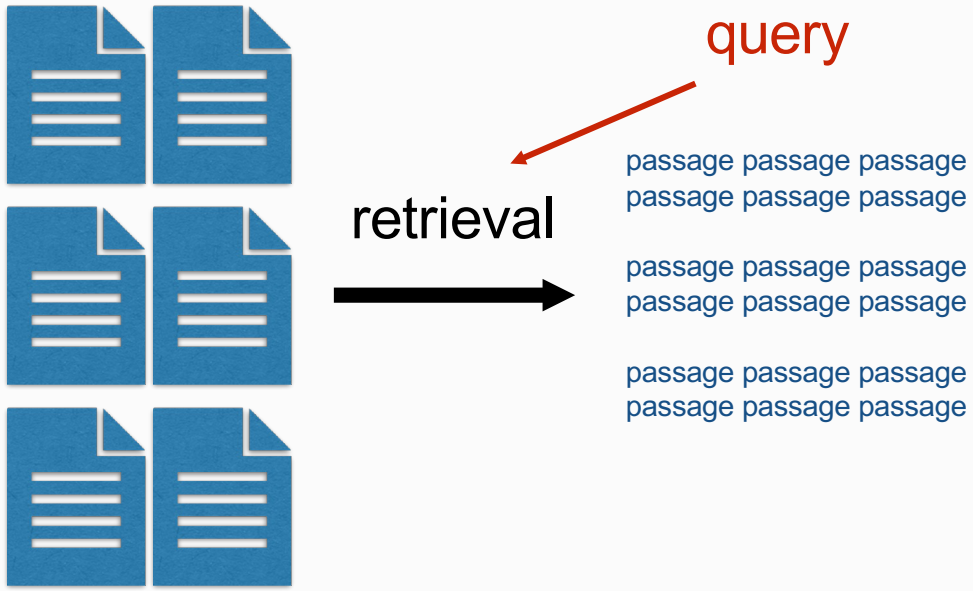
Retrieval-augmented Generation
(Chen et al. 2017)

- **Retrieve** relevant passages efficiently
- **Read** the passages to answer the query



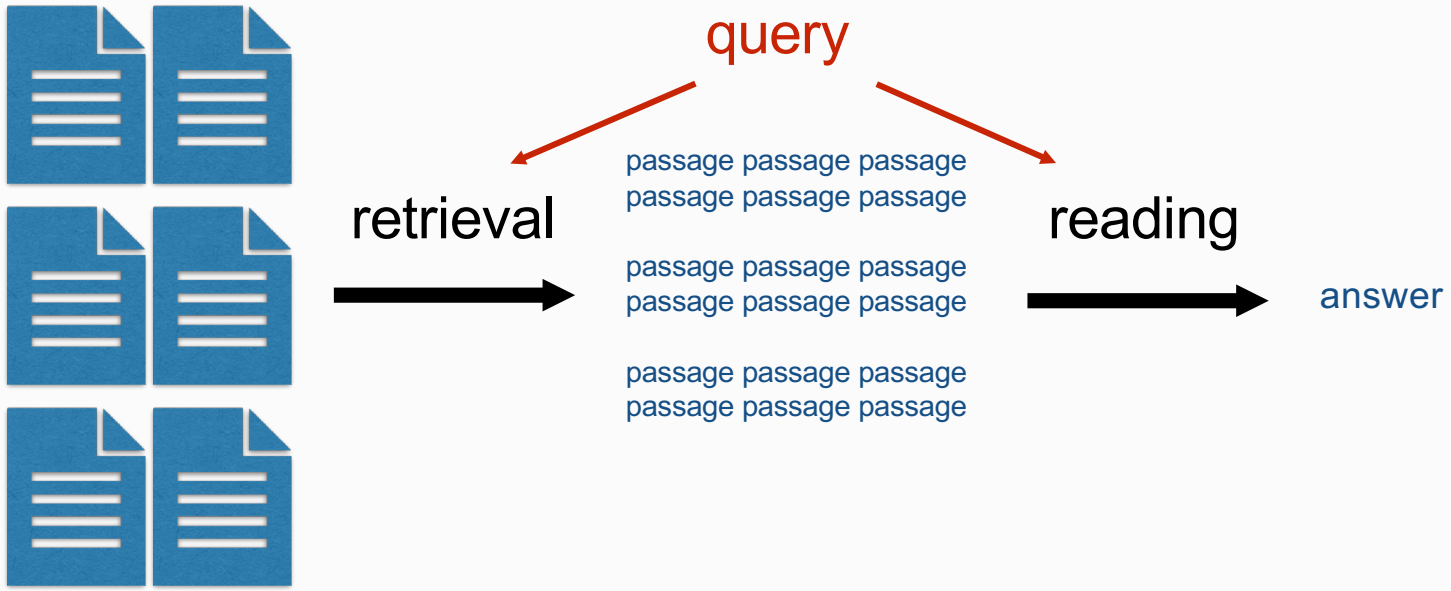
Retrieval-augmented Generation
(Chen et al. 2017)

- **Retrieve** relevant passages efficiently
- **Read** the passages to answer the query



Retrieval-augmented Generation
(Chen et al. 2017)

- **Retrieve** relevant passages efficiently
- **Read** the passages to answer the query



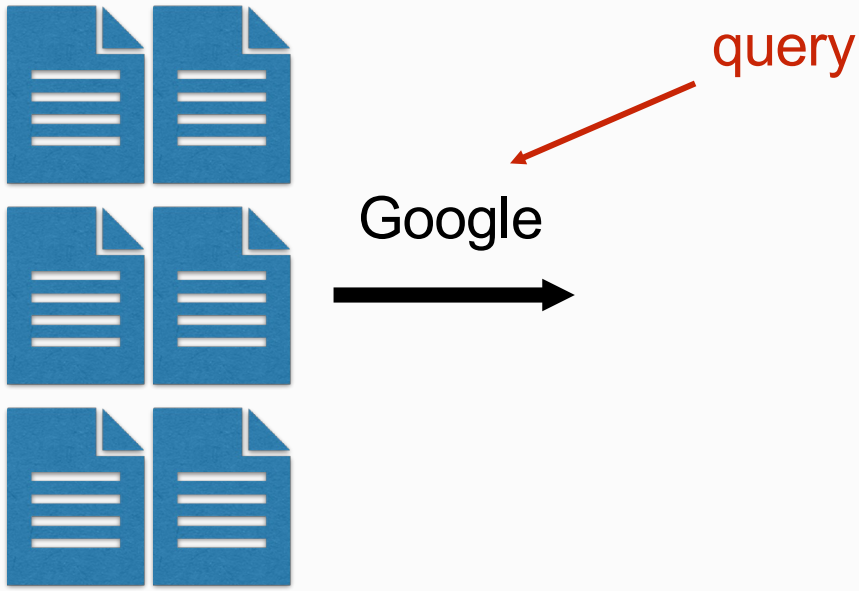
Retriever-Reader Models

Simple: Just Chain Retrieval+Reading

- Use an out-of-the-box retriever and out-of-the-box reader

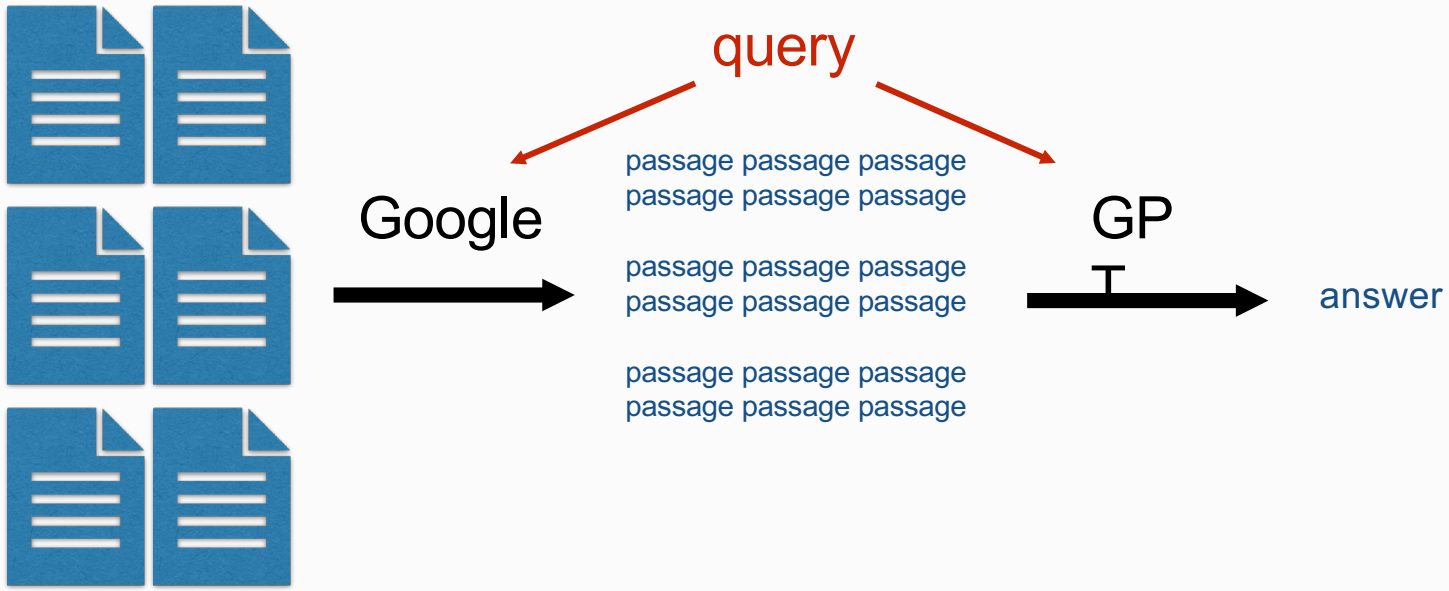
Simple: Just Chain Retrieval+Reading

- Use an out-of-the-box retriever and out-of-the-box reader



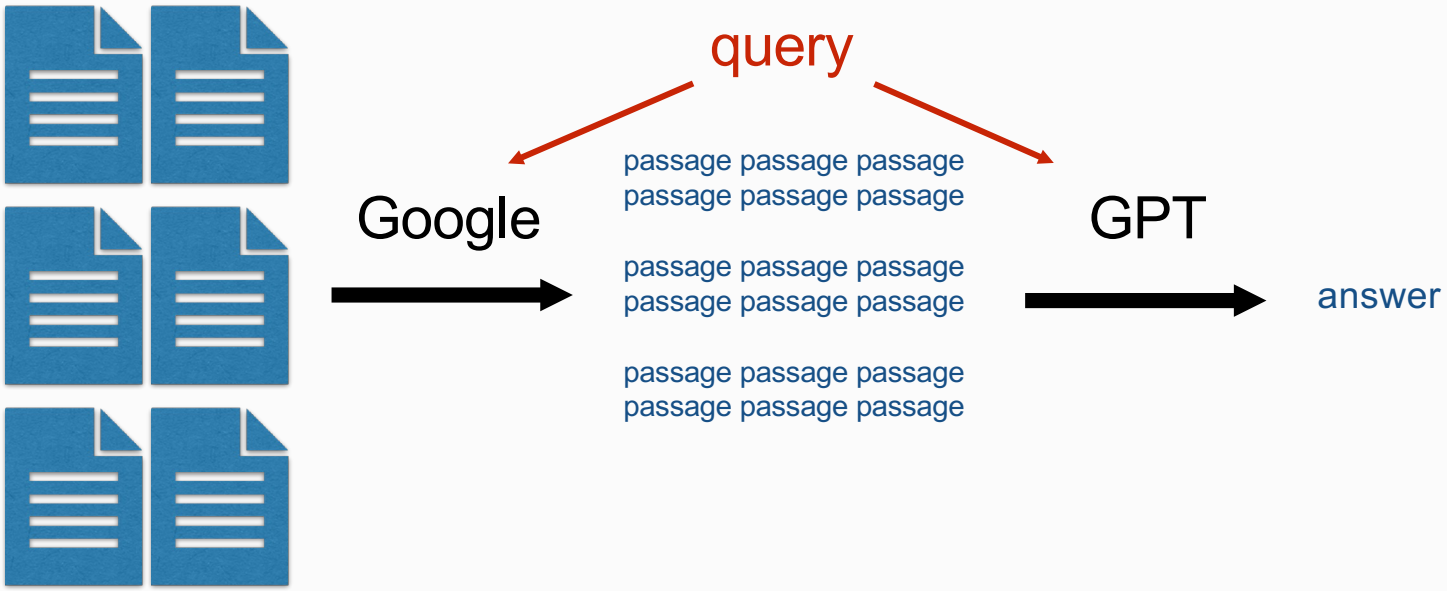
Simple: Just Chain Retrieval+Reading

- Use an out-of-the-box retriever and out-of-the-box reader



Simple: Just Chain Retrieval+Reading

- Use an out-of-the-box retriever and out-of-the-box reader



- Passages are concatenated to the context

Retriever + Generator End-to-end Training (“RAG”)

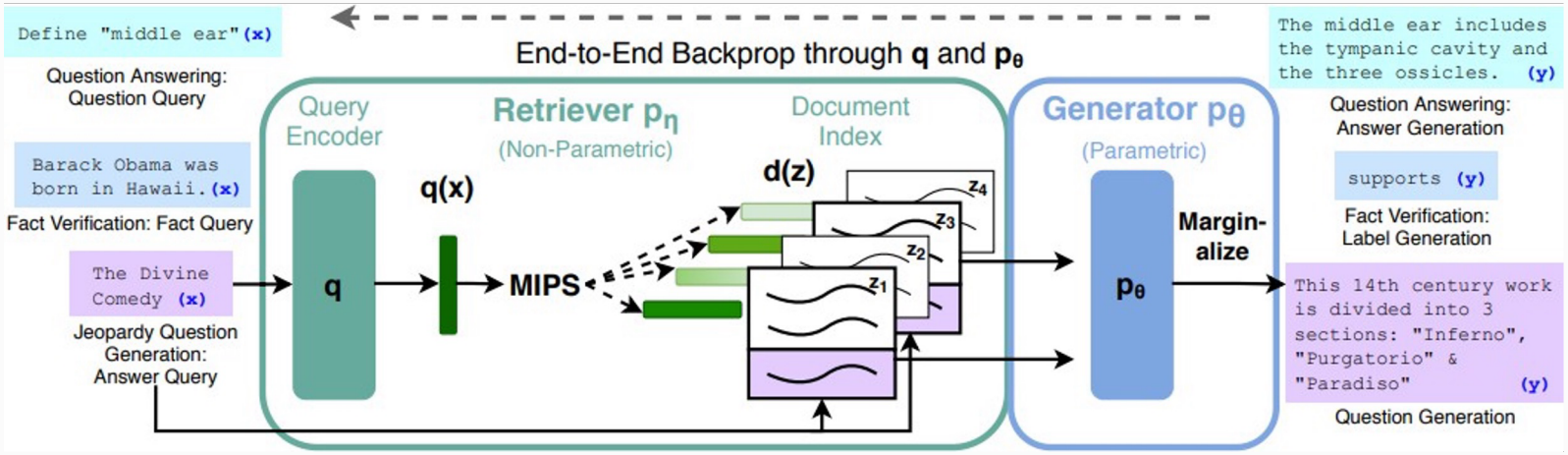
(Lewis et al. 2020)

Retriever + Generator End-to-end Training (“RAG”)
(Lewis et al. 2020)

- Train the retriever and reader to improve accuracy
- **Reader:** Maximize generation likelihood given single retrieved document

Retriever + Generator End-to-end Training ("RAG")
(Lewis et al. 2020)

- Train the retriever and reader to improve accuracy
- **Reader:** Maximize generation likelihood given single retrieved document
- **Retriever:** Maximize overall likelihood by optimizing mixture weights over documents



End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z, y_{1:i-1})$$

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z, y_{1:i-1})$$

Retriever Generator

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

- Adjusts retriever to give higher similarities helpful docs

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

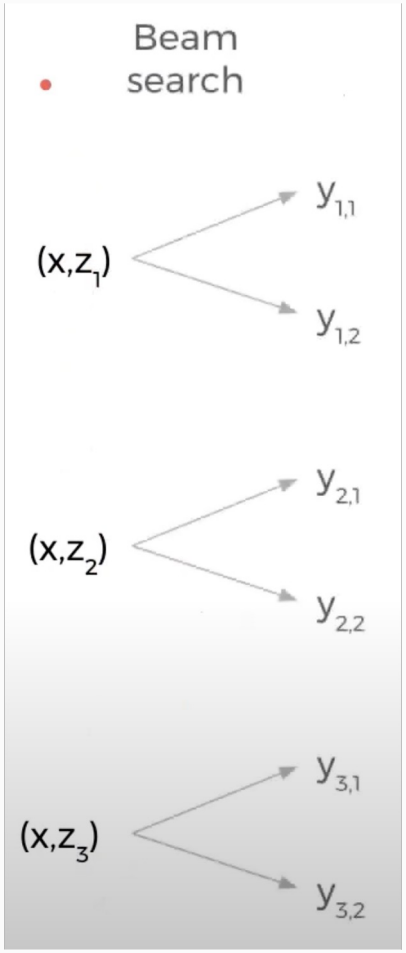
$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

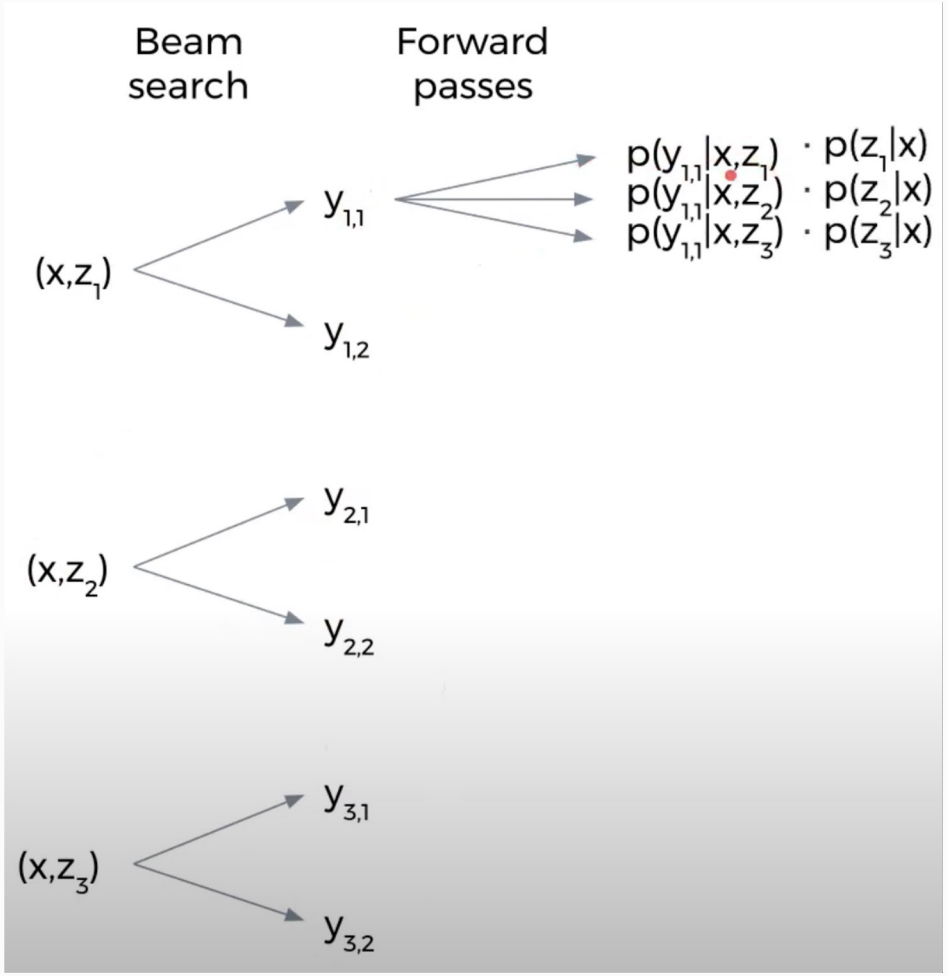
- Probability of the retriever is based on embeddings

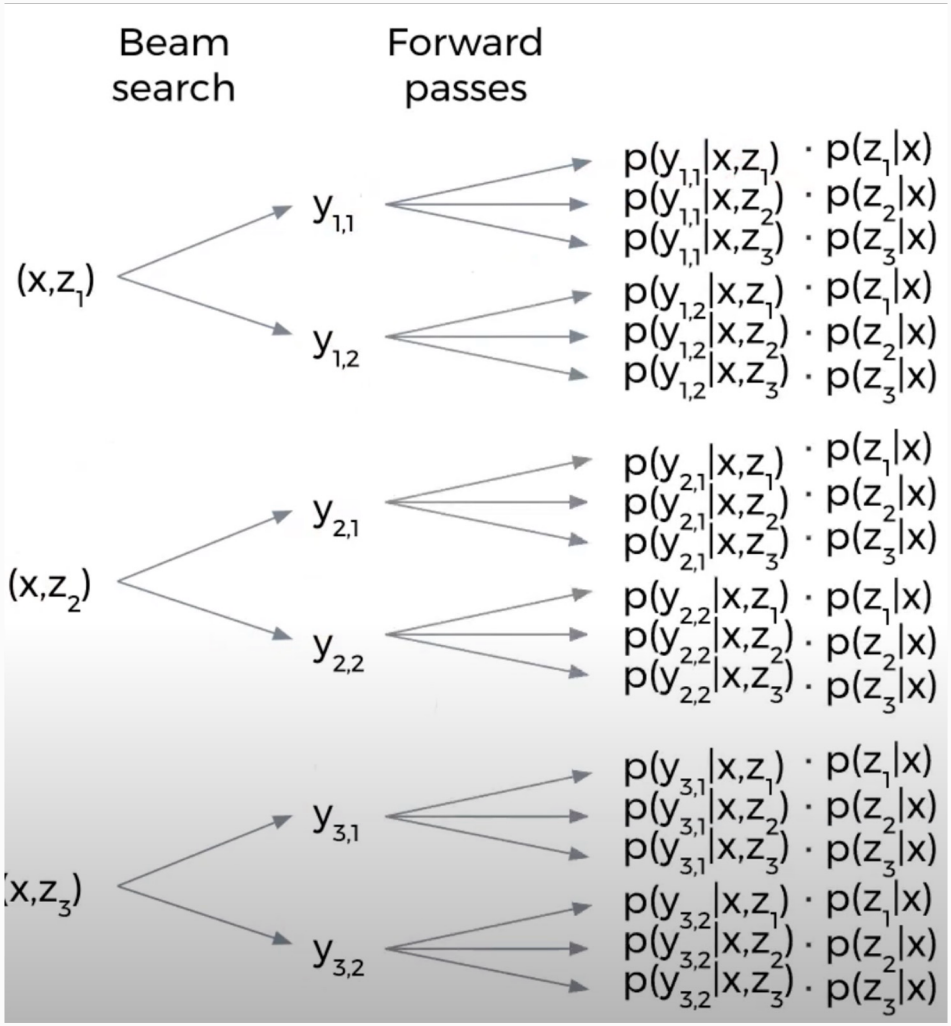
$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

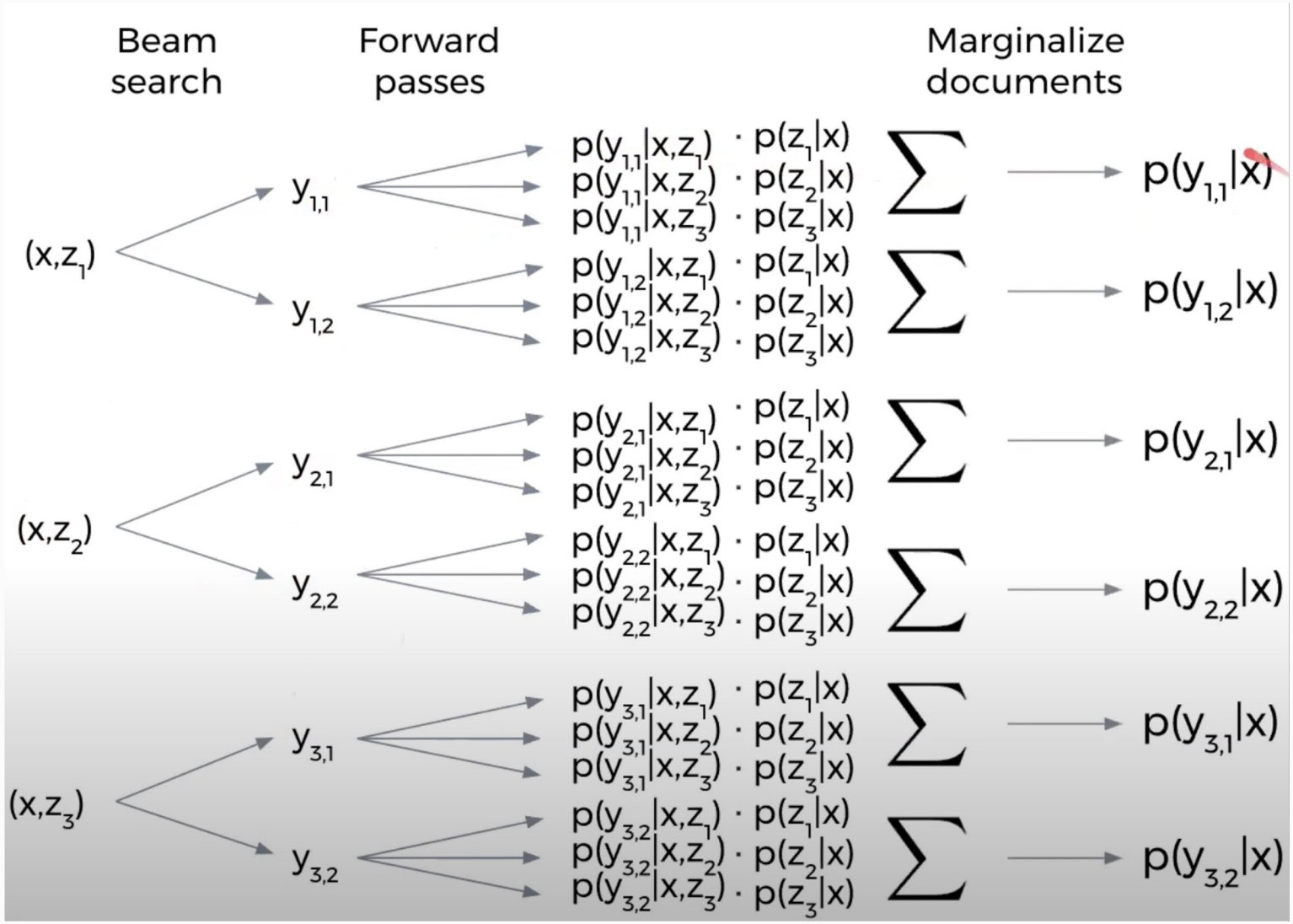
- Adjusts retriever to give higher similarities helpful docs
- Issue: search index becomes stale → can only train q(x)

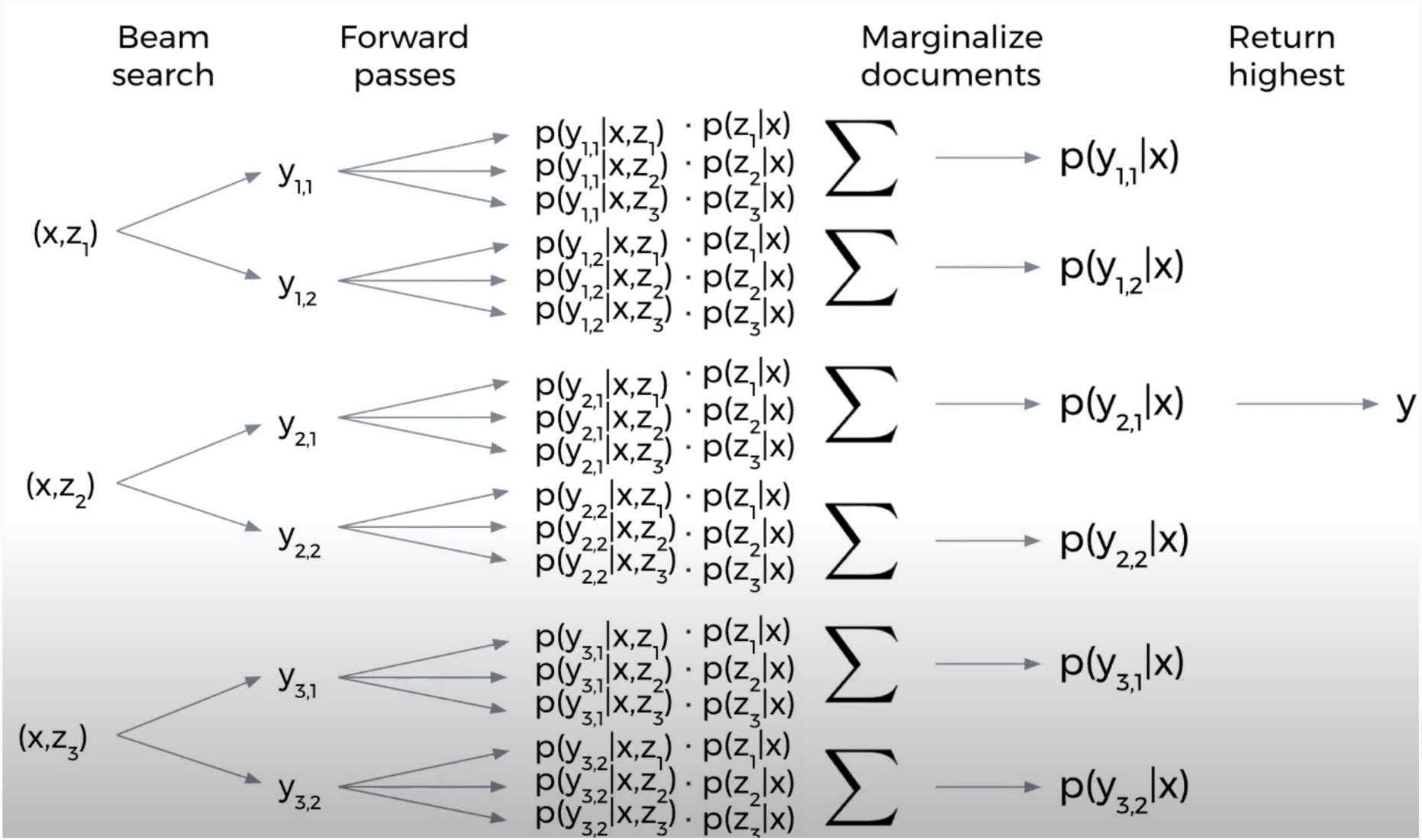
Decoding step



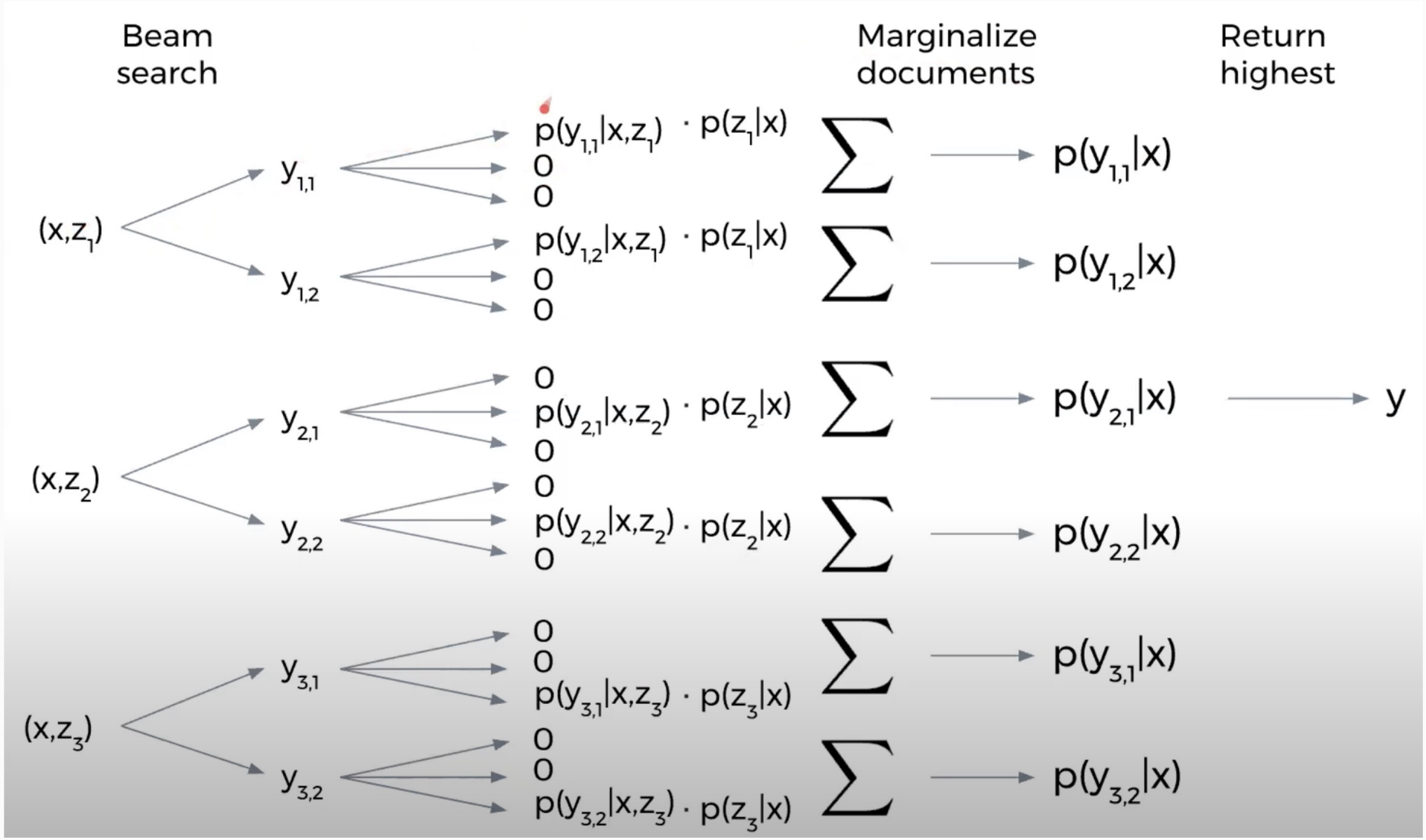

Beam search
Forward passes
(x,z_1)
$y_{1,1}$
$y_{1,2}$
$p(y_{1,1}|x,z_1) \cdot p(z_1|x)$
$p(y_{1,1}|x,z_2) \cdot p(z_2|x)$
$p(y_{1,1}|x,z_3) \cdot p(z_3|x)$
(x,z_2)
$y_{2,1}$
$y_{2,2}$
(x,z_3)
$y_{3,1}$
$y_{3,2}$


Beam search
Forward passes
(x,z_1)
(x,z_2)
(x,z_3)
$y_{1,1}$
$y_{1,2}$
$y_{2,1}$
$y_{2,2}$
$y_{3,1}$
$y_{3,2}$
$p(y_{1,1}|x,z_1) \cdot p(z_1|x)$
$p(y_{1,1}|x,z_2) \cdot p(z_2|x)$
$p(y_{1,1}|x,z_3) \cdot p(z_3|x)$
$p(y_{1,2}|x,z_1) \cdot p(z_1|x)$
$p(y_{1,2}|x,z_2) \cdot p(z_2|x)$
$p(y_{1,2}|x,z_3) \cdot p(z_3|x)$
$p(y_{2,1}|x,z_1) \cdot p(z_1|x)$
$p(y_{2,1}|x,z_2) \cdot p(z_2|x)$
$p(y_{2,1}|x,z_3) \cdot p(z_3|x)$
$p(y_{2,2}|x,z_1) \cdot p(z_1|x)$
$p(y_{2,2}|x,z_2) \cdot p(z_2|x)$
$p(y_{2,2}|x,z_3) \cdot p(z_3|x)$
$p(y_{3,1}|x,z_1) \cdot p(z_1|x)$
$p(y_{3,1}|x,z_2) \cdot p(z_2|x)$
$p(y_{3,1}|x,z_3) \cdot p(z_3|x)$
$p(y_{3,2}|x,z_1) \cdot p(z_1|x)$
$p(y_{3,2}|x,z_2) \cdot p(z_2|x)$
$p(y_{3,2}|x,z_3) \cdot p(z_3|x)$


Beam search
Forward passes
Marginalize documents
(x,z1)
(x,z2)
(x,z3)
y1,1
y1,2
y2,1
y2,2
y3,1
y3,2
p(y1,1|x,z1) · p(z1|x)
p(y1,1|x,z2) · p(z2|x)
p(y1,1|x,z3) · p(z3|x)
p(y1,2|x,z1) · p(z1|x)
p(y1,2|x,z2) · p(z2|x)
p(y1,2|x,z3) · p(z3|x)
p(y2,1|x,z1) · p(z1|x)
p(y2,1|x,z2) · p(z2|x)
p(y2,1|x,z3) · p(z3|x)
p(y2,2|x,z1) · p(z1|x)
p(y2,2|x,z2) · p(z2|x)
p(y2,2|x,z3) · p(z3|x)
p(y3,1|x,z1) · p(z1|x)
p(y3,1|x,z2) · p(z2|x)
p(y3,1|x,z3) · p(z3|x)
p(y3,2|x,z1) · p(z1|x)
p(y3,2|x,z2) · p(z2|x)
p(y3,2|x,z3) · p(z3|x)
p(y1,1|x)
p(y1,2|x)
p(y2,1|x)
p(y2,2|x)
p(y3,1|x)
p(y3,2|x)


Beam search
Forward passes
Marginalize documents
Return highest
(x,z1)
(x,z2)
(x,z3)
y1,1
y1,2
y2,1
y2,2
y3,1
y3,2
p(y1,1|x,z1) · p(z1|x)
p(y1,1|x,z2) · p(z2|x)
p(y1,1|x,z3) · p(z3|x)
p(y1,2|x,z1) · p(z1|x)
p(y1,2|x,z2) · p(z2|x)
p(y1,2|x,z3) · p(z3|x)
p(y2,1|x,z1) · p(z1|x)
p(y2,1|x,z2) · p(z2|x)
p(y2,1|x,z3) · p(z3|x)
p(y2,2|x,z1) · p(z1|x)
p(y2,2|x,z2) · p(z2|x)
p(y2,2|x,z3) · p(z3|x)
p(y3,1|x,z1) · p(z1|x)
p(y3,1|x,z2) · p(z2|x)
p(y3,1|x,z3) · p(z3|x)
p(y3,2|x,z1) · p(z1|x)
p(y3,2|x,z2) · p(z2|x)
p(y3,2|x,z3) · p(z3|x)
p(y1,1|x)
p(y1,2|x)
p(y2,1|x)
p(y2,2|x)
p(y3,1|x)
p(y3,2|x)
y

Since the $|Y|$ may become larger, they use more efficient decoding, they make a further approximation that $p_\theta(y \mid x, z_i) \approx 0$ where y wasn't generated during beam search from x, z_i. They called it "Fast Decoding".


Beam search
Marginalize documents
Return highest
(x,z_1)
(x,z_2)
(x,z_3)
$y_{1,1}$
$y_{1,2}$
$y_{2,1}$
$y_{2,2}$
$y_{3,1}$
$y_{3,2}$
$p(y_{1,1}|x,z_1) \cdot p(z_1|x)$
$p(y_{1,2}|x,z_1) \cdot p(z_1|x)$
$p(y_{2,1}|x,z_2) \cdot p(z_2|x)$
$p(y_{2,2}|x,z_2) \cdot p(z_2|x)$
$p(y_{3,1}|x,z_3) \cdot p(z_3|x)$
$p(y_{3,2}|x,z_3) \cdot p(z_3|x)$
0
$\sum$
$p(y_{1,1}|x)$
$p(y_{1,2}|x)$
$p(y_{2,1}|x)$
$p(y_{2,2}|x)$
$p(y_{3,1}|x)$
$p(y_{3,2}|x)$
y

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

- Adjusts retriever to give higher similarities helpful docs

End-to-end Training Equations
(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_\eta(z|x)}_{\text{Retriever}} \underbrace{p_\theta(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

- Adjusts retriever to give higher similarities helpful docs
- Issue: search index becomes stale → can only train q(x)

When do we Retrieve?

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)
  - Search when the model is uncertain (Jiang et al. 2023)

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)
  - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)
  - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**
  - Find similar final embeddings (Khandelwal et al. 2019)

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)
  - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**
  - Find similar final embeddings (Khandelwal et al. 2019)
  - Approximate attention with nearest neighbors (Bertsch et al. 2023)

Triggering Retrieval w/ Tokens

- Toolformer (Schick et al. 2023) generates tokens that trigger retrieval (or other tools)

Triggering Retrieval w/ Tokens

- Toolformer (Schick et al. 2023) generates tokens that trigger retrieval (or other tools)

The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.

Triggering Retrieval w/ Tokens

- Toolformer (Schick et al. 2023) generates tokens that trigger retrieval (or other tools)
- Training is done in an iterative manner - generate and identify successful retrievals

The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

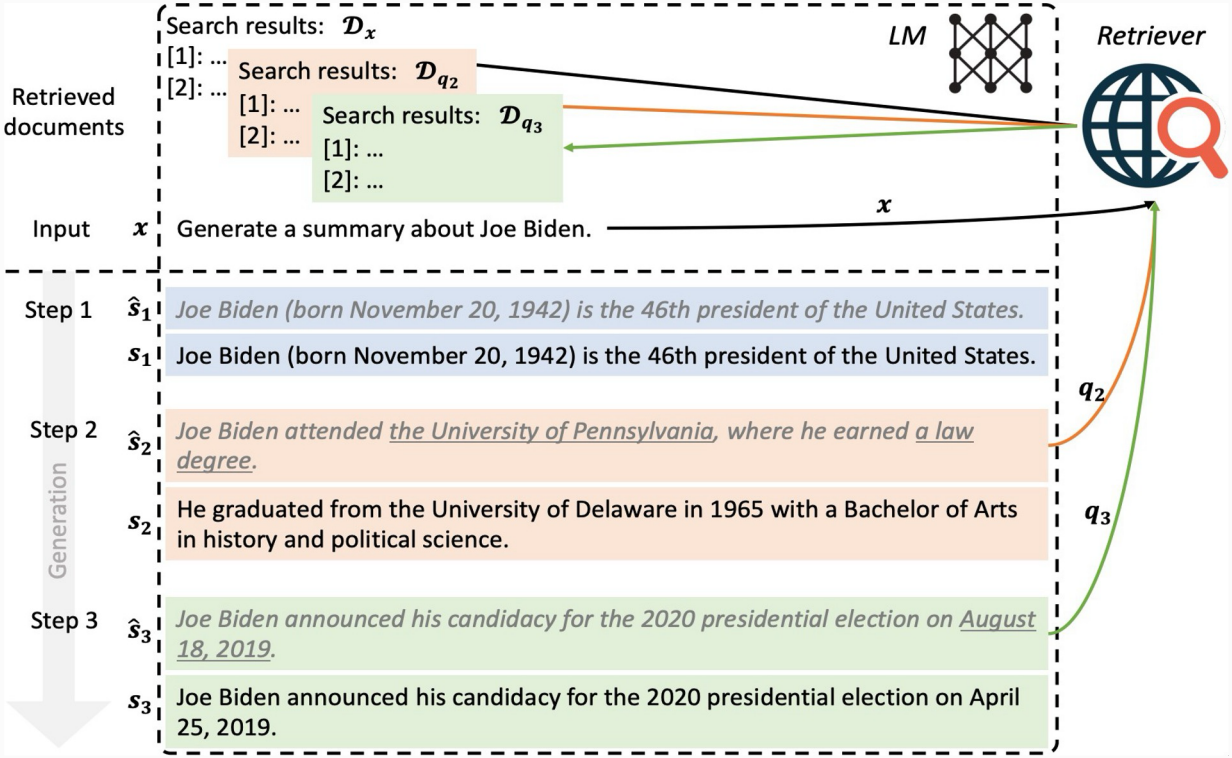
The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.

Triggering Retrieval w/ Uncertainty

- FLARE (Jiang et al. 2023) tries to generate content, then does retrieval if LM certainty is low

Triggering Retrieval w/ Uncertainty

- FLARE (Jiang et al. 2023) tries to generate content, then does retrieval if LM certainty is low

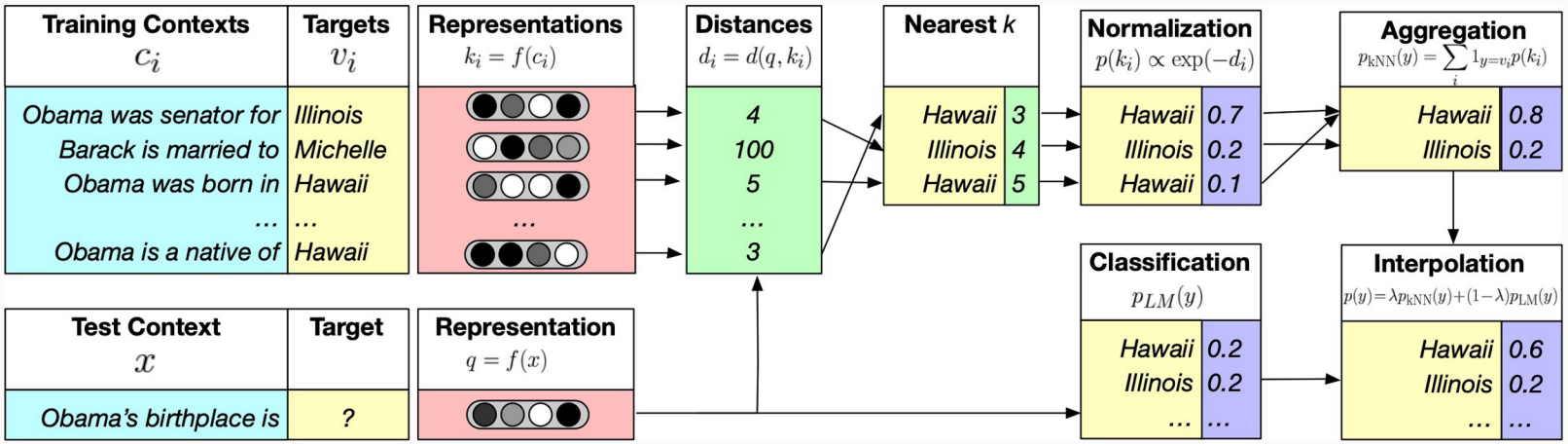


Token-level Softmax Modification

- kNN-LM (Khandelwal et al. 2019) retrieves similar examples, and uses the following token from them

Token-level Softmax Modification

- kNN-LM (Khandelwal et al. 2019) retrieves similar examples, and uses the following token from them



Token-level Approximate Attention

- Unlimiformer (Bertsch et al. 2023) notes that attention is an inner-product and does top-k attention

Token-level Approximate Attention

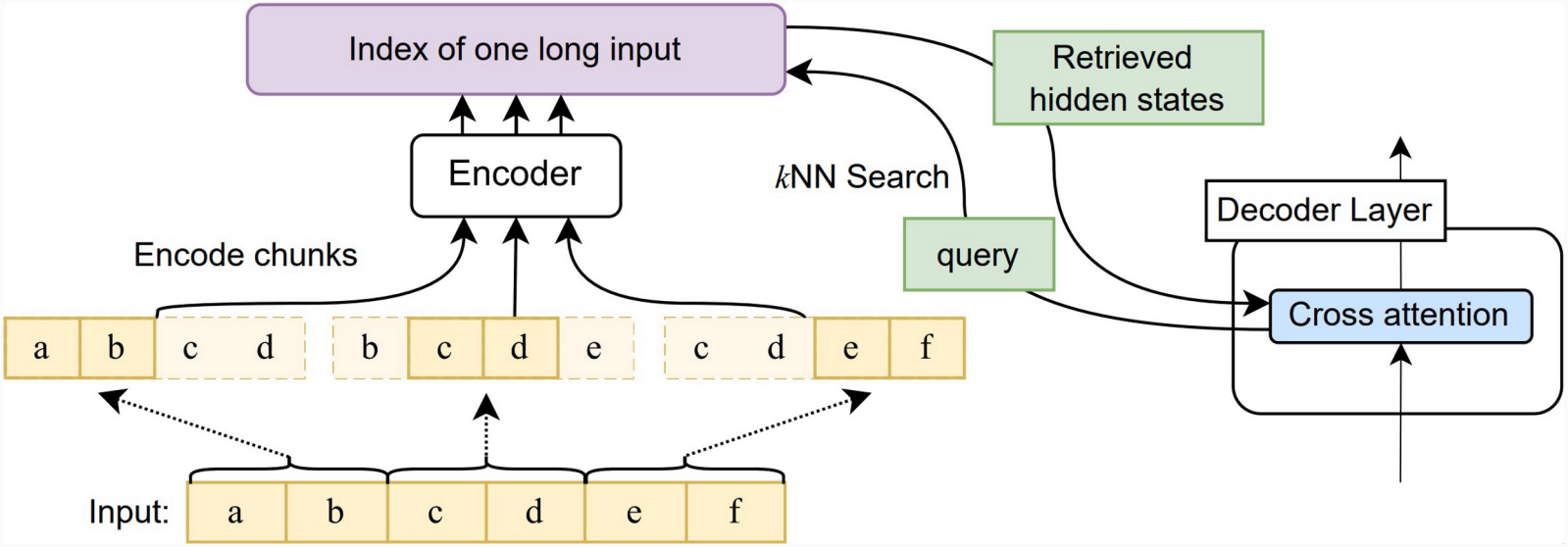
- Unlimiformer (Bertsch et al. 2023) notes that attention is an inner-product and does top-k attention
- First, process input with a sliding window

Token-level Approximate Attention

- Unlimiformer (Bertsch et al. 2023) notes that attention is an inner-product and does top-k attention
- First, process input with a sliding window
- Then perform attention using a vector index

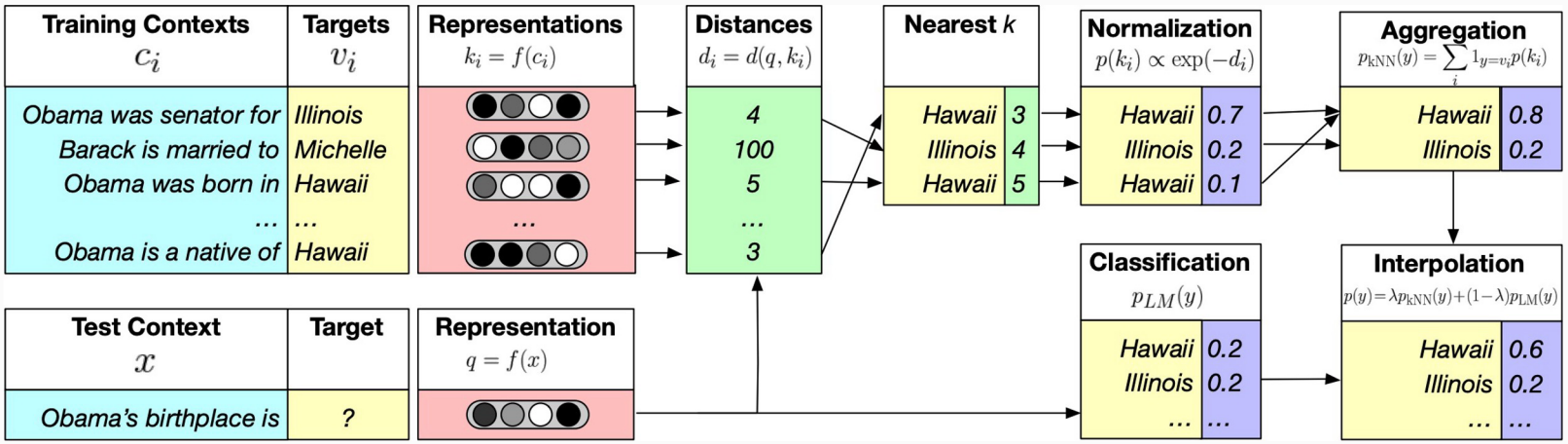
Token-level Approximate Attention

- Unlimiformer (Bertsch et al. 2023) notes that attention is an inner-product and does top-k attention
- First, process input with a sliding window
- Then perform attention using a vector index



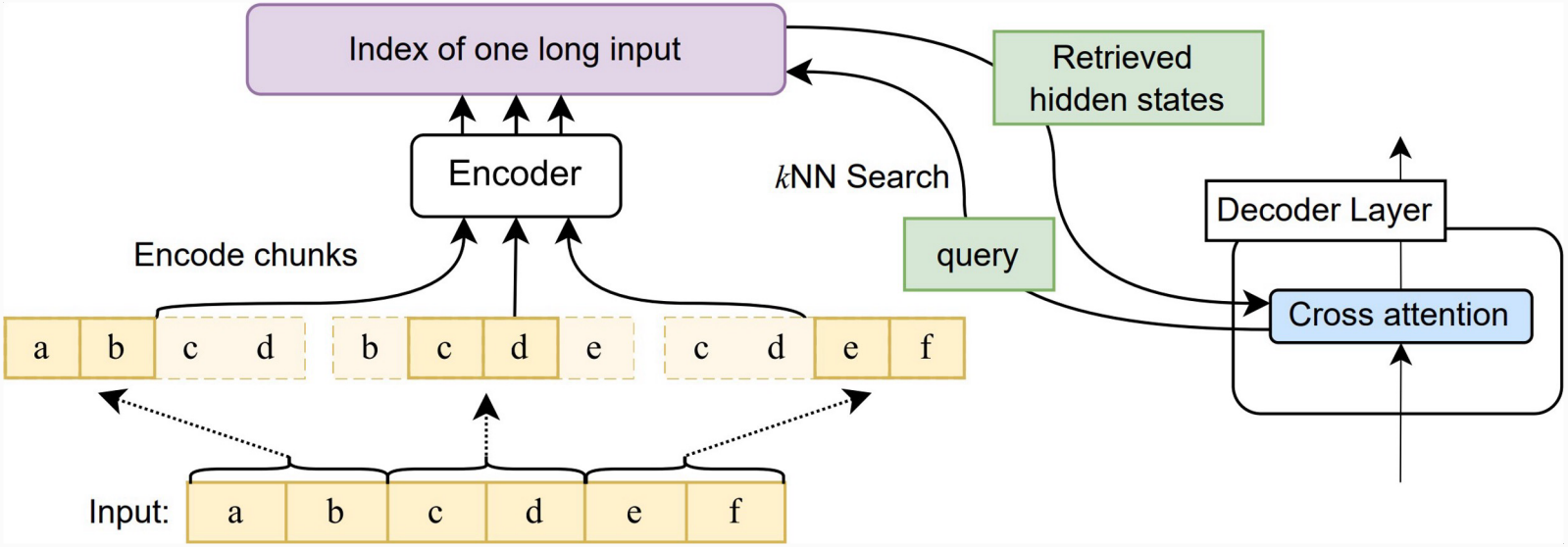
Token-level Softmax Modification

- kNN-LM (Khandelwal et al. 2019) retrieves similar examples, and uses the following token from them



Token-level Approximate Attention

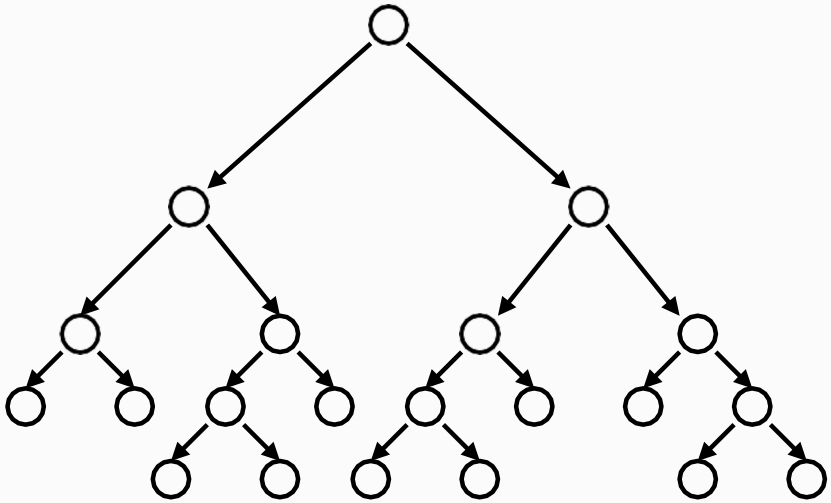
- Unlimiformer (Bertsch et al. 2023) notes that attention is an inner-product and does top-k attention
- First, process input with a sliding window
- Then perform attention using a vector index



Locality sensitive hashing for approximate NN search

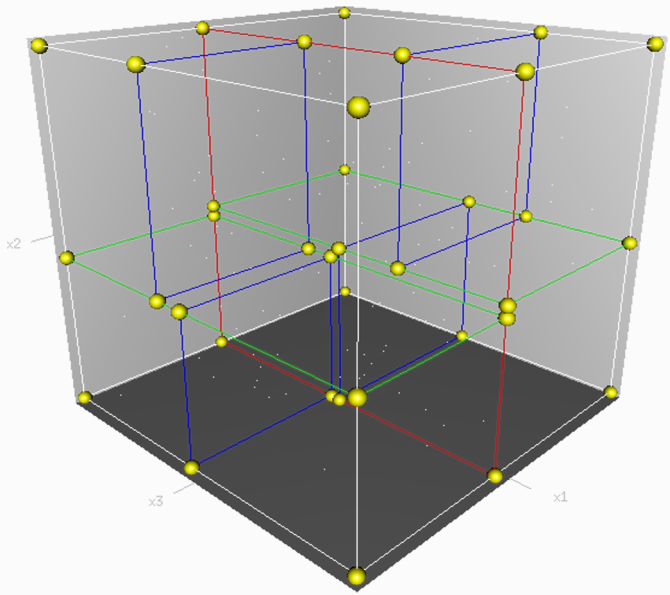
Motivating alternative approaches to approximate NN search

- KD-trees are cool, but…
  - Non-trivial to implement efficiently
  - Problems with high-dimensional data

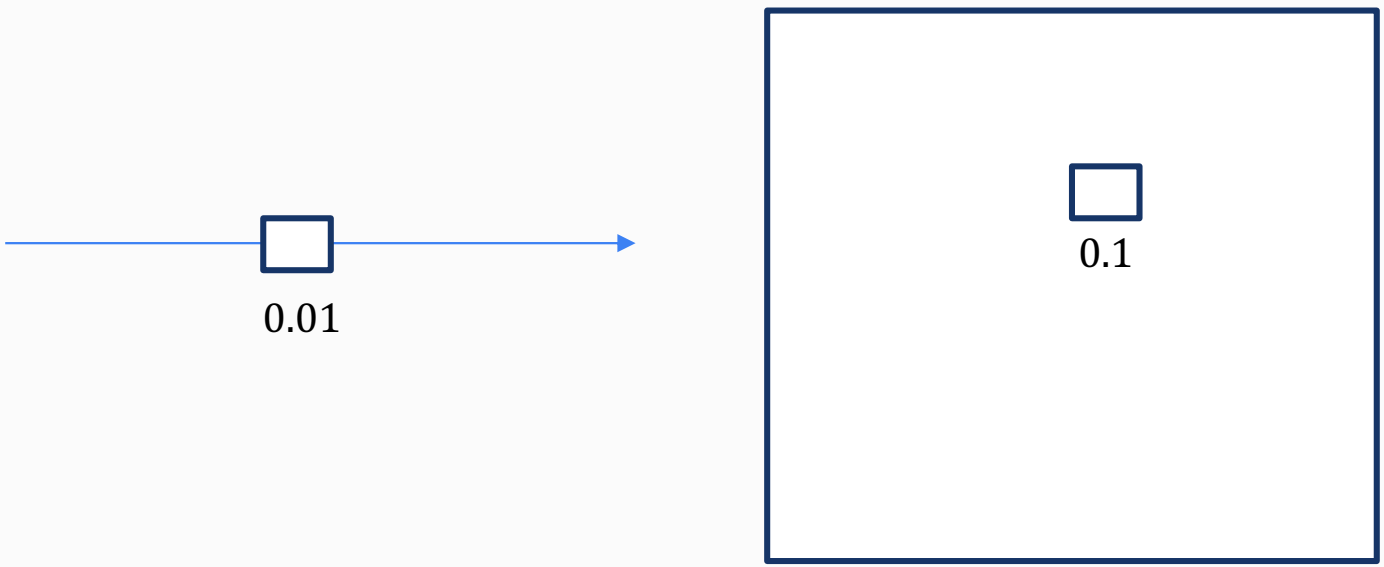


KD-trees in high dimensions

- Unlikely to have any data points close to query point
- Once "nearby" point is found, the search radius is likely to intersect many hypercubes in at least one dim
- Not many nodes can be pruned
- Can show under some conditions that you visit at least 2^d nodes



Curse of Dimension related to side length



Moving away from exact NN search

- Approximate neighbor finding…
  - Don’t find exact neighbor, but that’s okay for many applications

Out of millions of articles, do we need the closest article or just one that’s pretty similar?

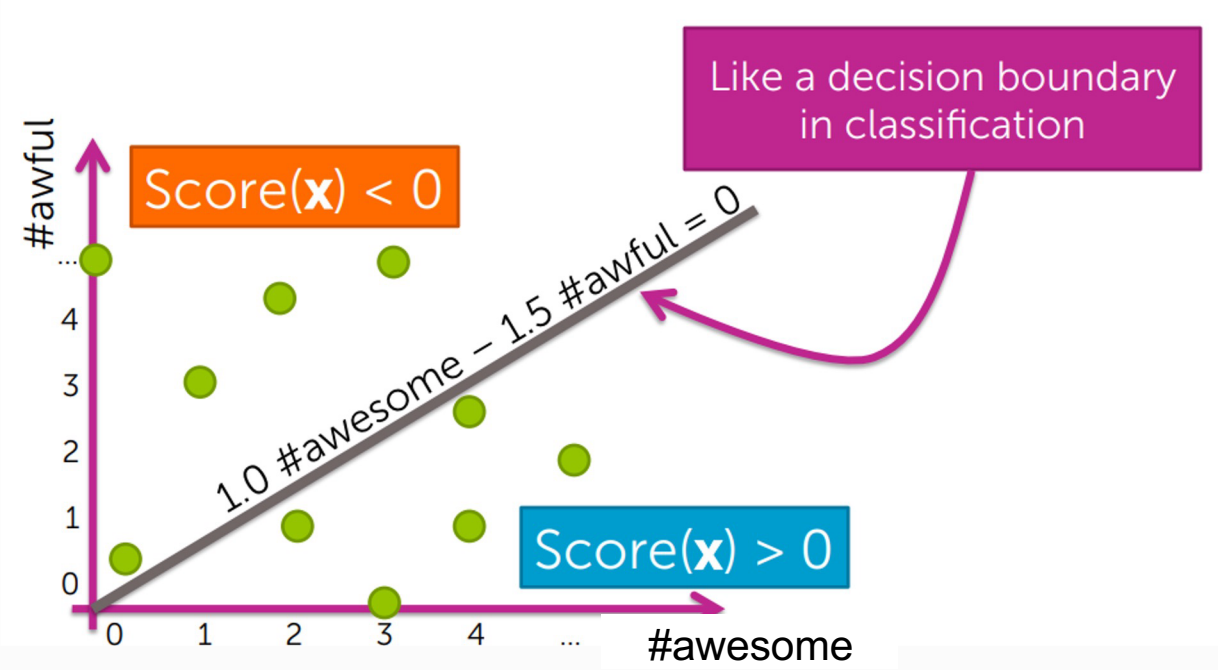
Do we even fully trust our measure of similarity???

- Focus on methods that provide good probabilistic guarantees on approximation

LSH as an alternative to KD-trees

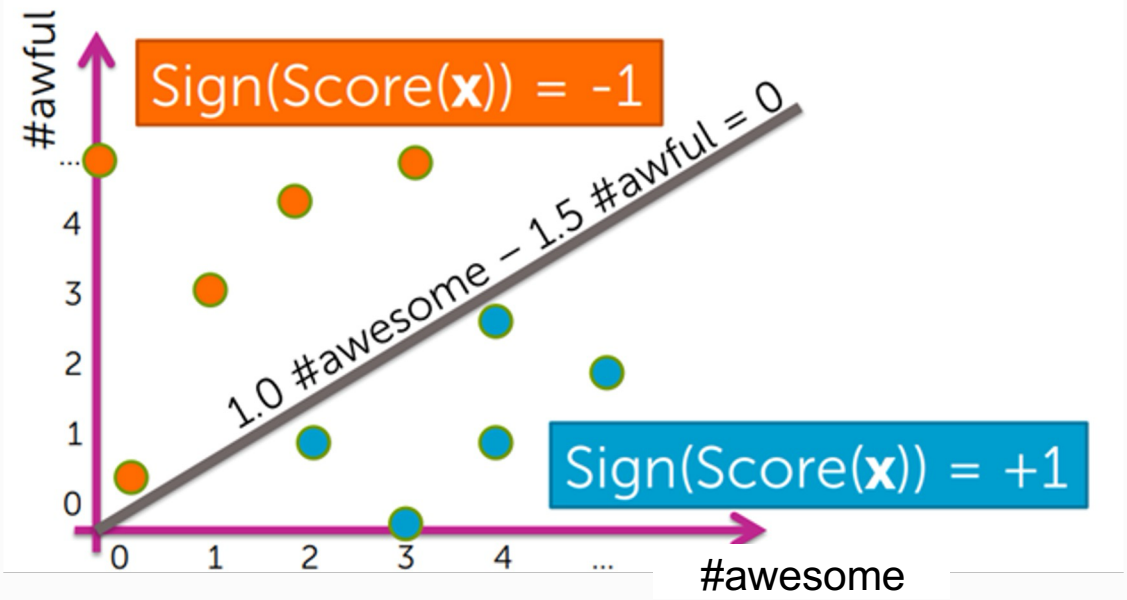
Simple “binning” of data into 2 bins

Score(x) = 1.0 #awesome – 1.5 #awful

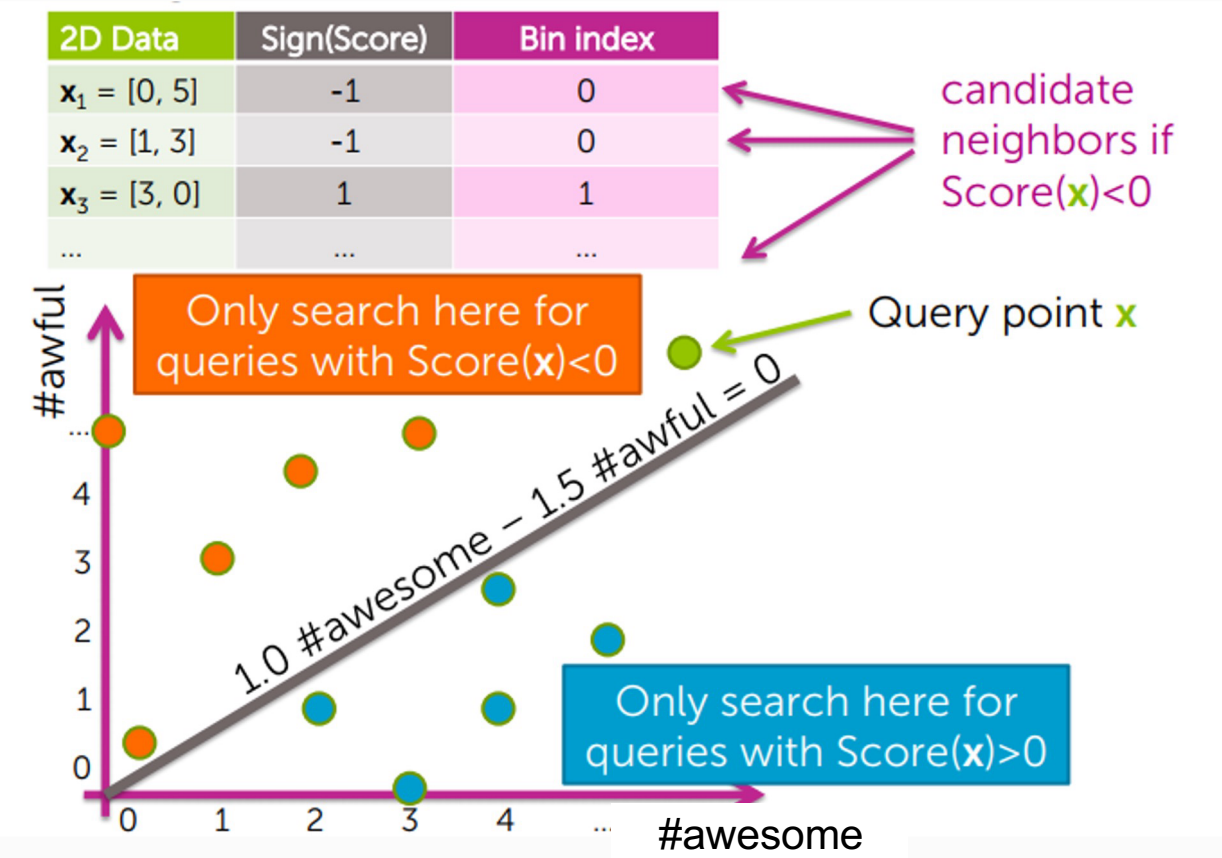


Simple “binning” of data into 2 bins

2D Data	Sign(Score)
$\mathbf{x}_1$ = [0, 5]	-1
$\mathbf{x}_2$ = [1, 3]	-1
$\mathbf{x}_3$ = [3, 0]	1
...	...



Using bins for NN search



Using score for NN search

2D Data	Sign(Score)	Bin index
x_1 = [0, 5]	-1	0
x_2 = [1, 3]	-1	0
x_3 = [3, 0]	1	1
...	...	...

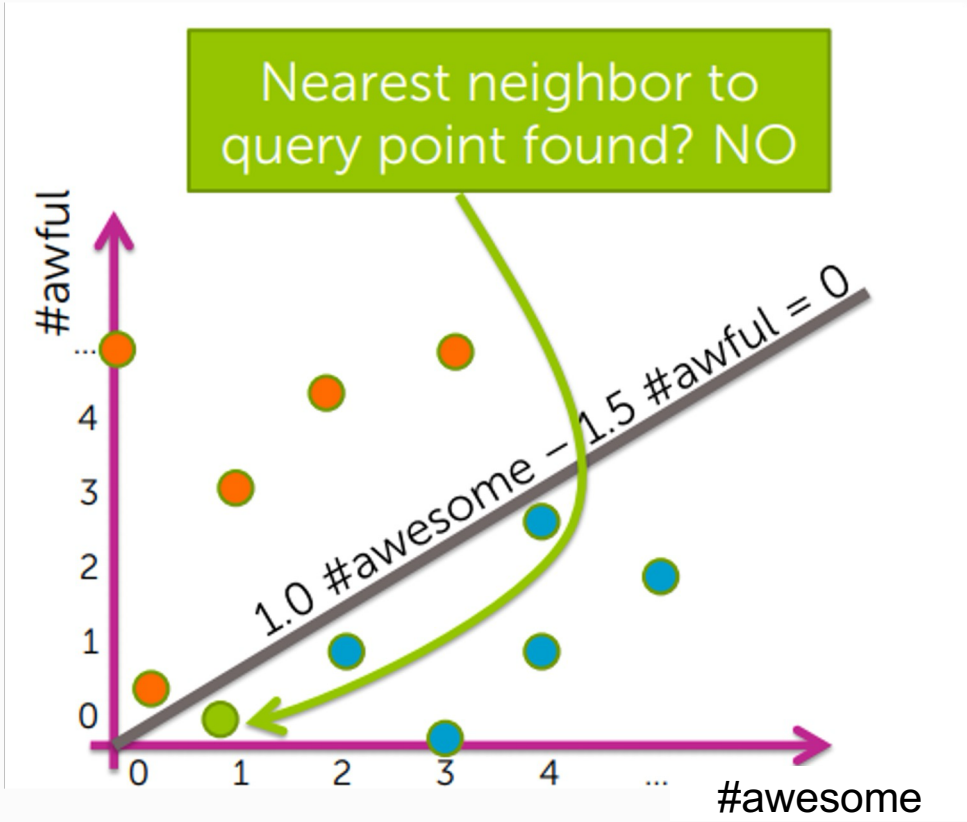
candidate neighbors if Score(x)<0

Bin	0	1
List containing indices of datapoints:	{1,2,4,7,...}	{3,5,6,8,...}

HASH TABLE

search for NN amongst this set

Provides approximate NN

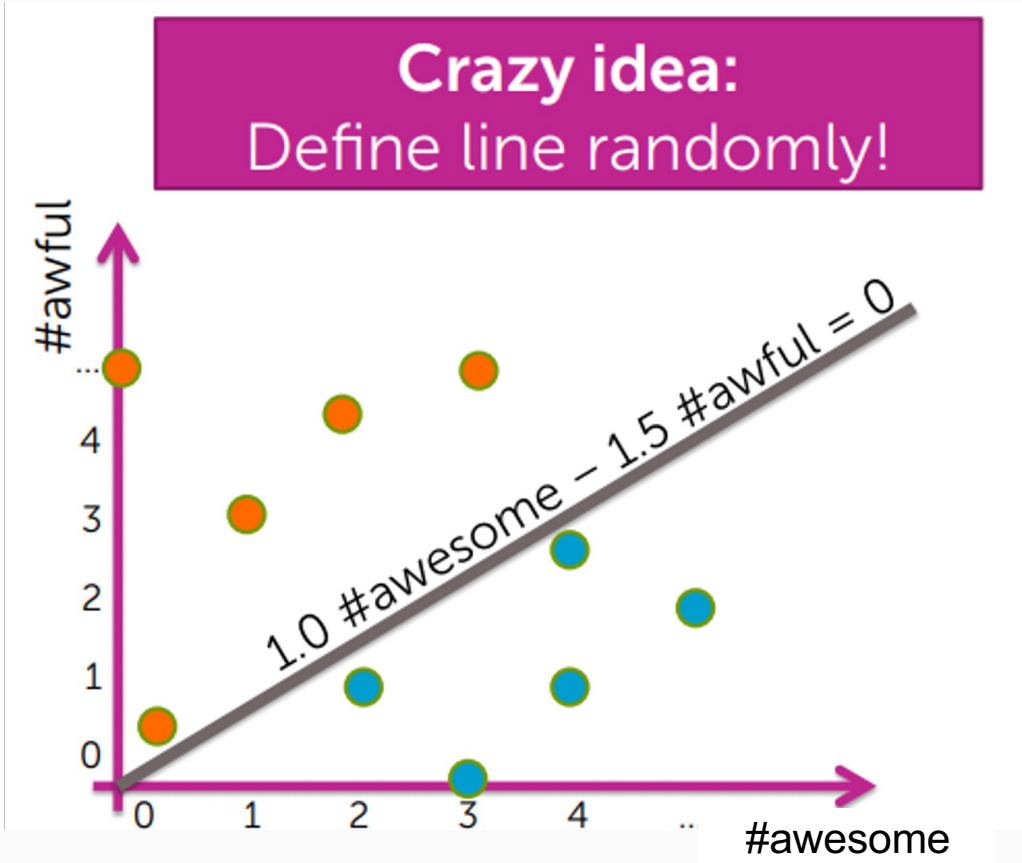


A practical implementation

Three potential issues with simple approach

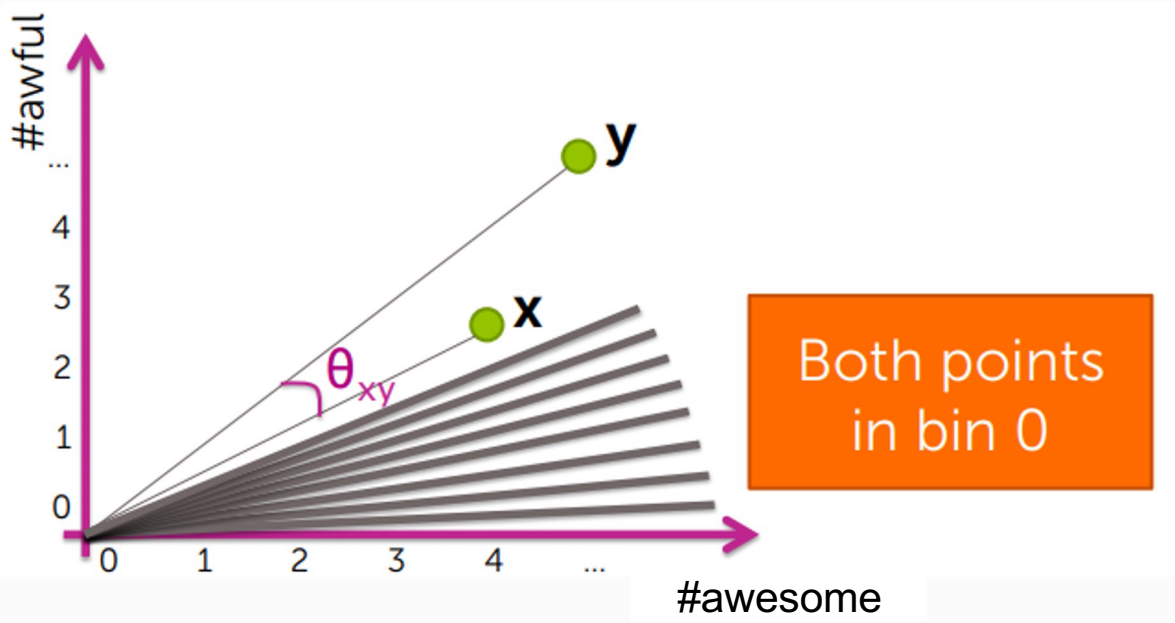
- Challenging to find good line
- Poor quality solution:
  - Points close together get split into separate bins
- Large computational cost:
  - Bins might contain many points, so still searching over large set for each NN query

How to define the line?



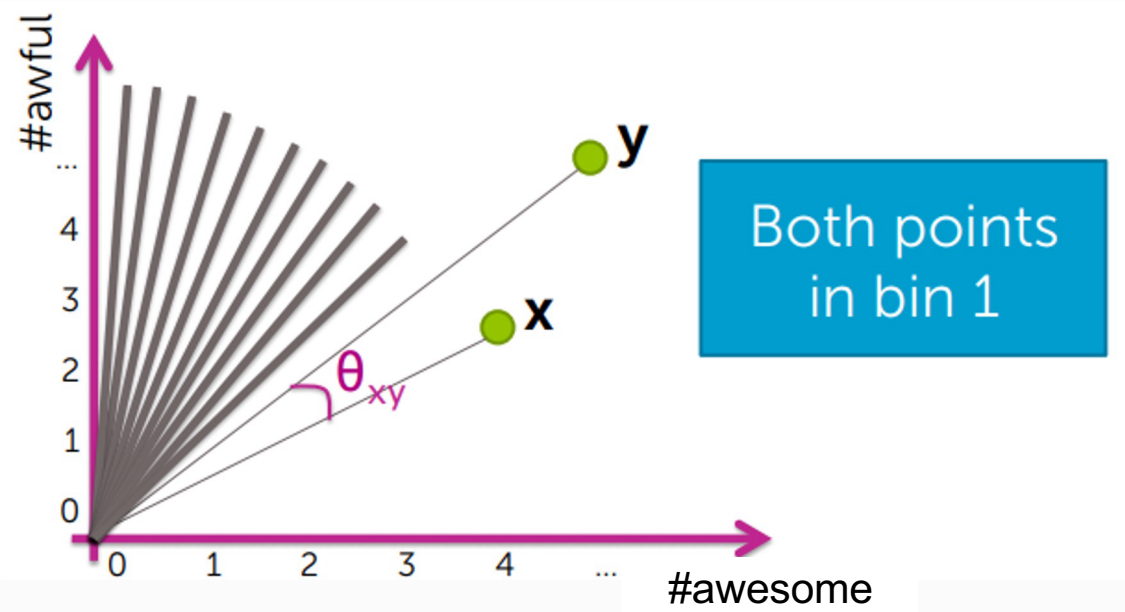
How bad can a random line be?

Goal: If x,y are close (according to cosine similarity),
want binned values to be the same.



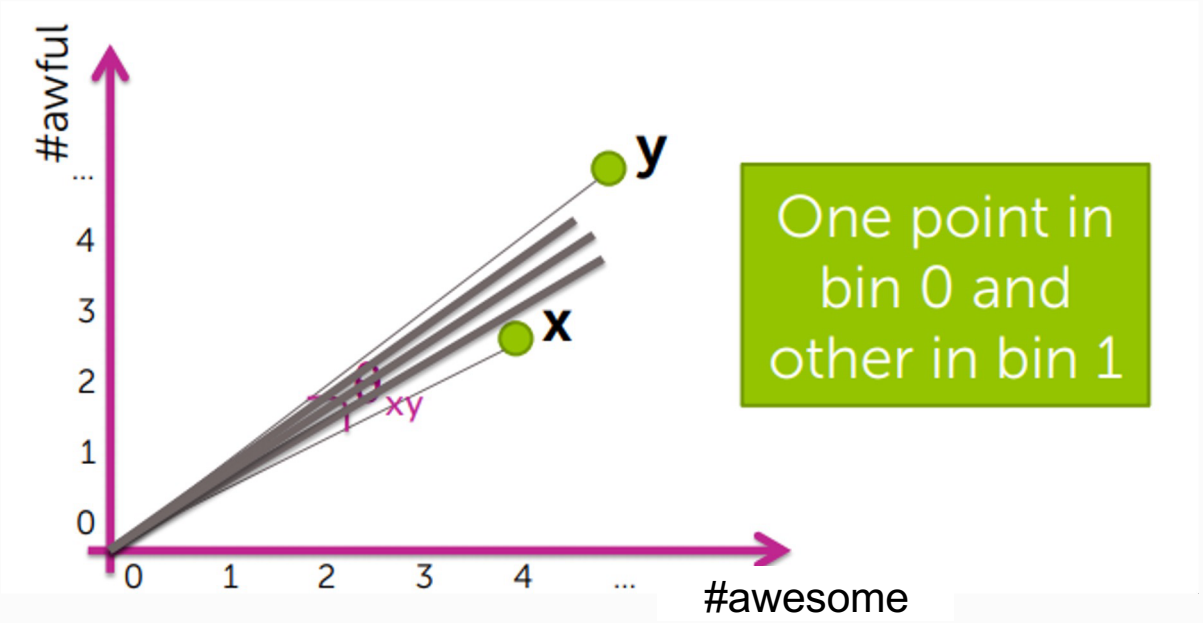
How bad can a random line be?

Goal: If x,y are close (according to cosine similarity), want binned values to be the same.



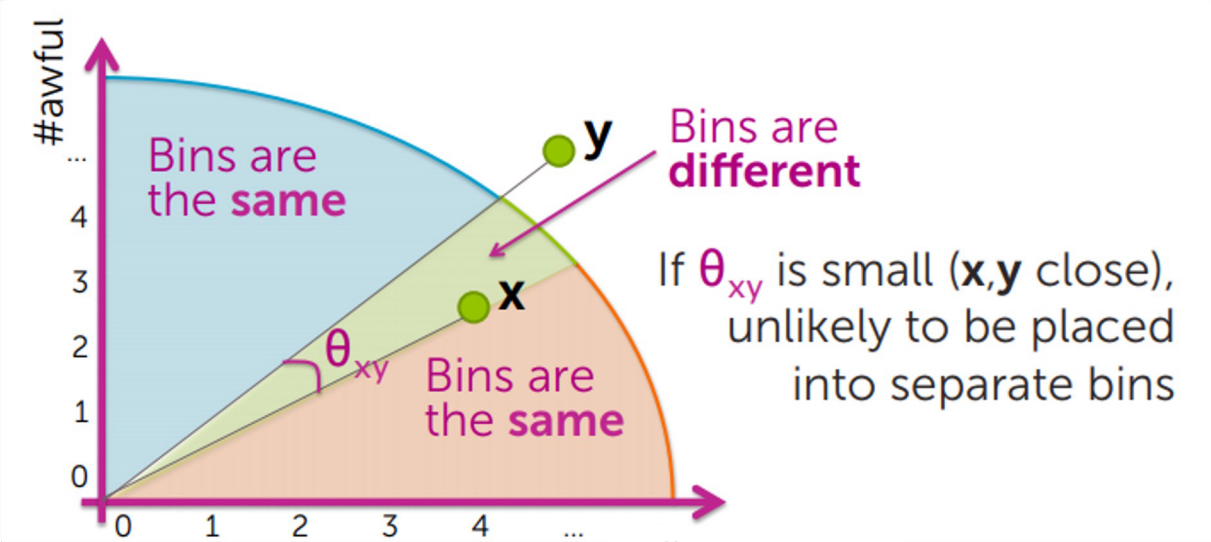
How bad can a random line be?

Goal: If x,y are close (according to cosine similarity), want binned values to be the same.



How bad can a random line be?

Goal: If x,y are close (according to cosine similarity), want binned values to be the same.



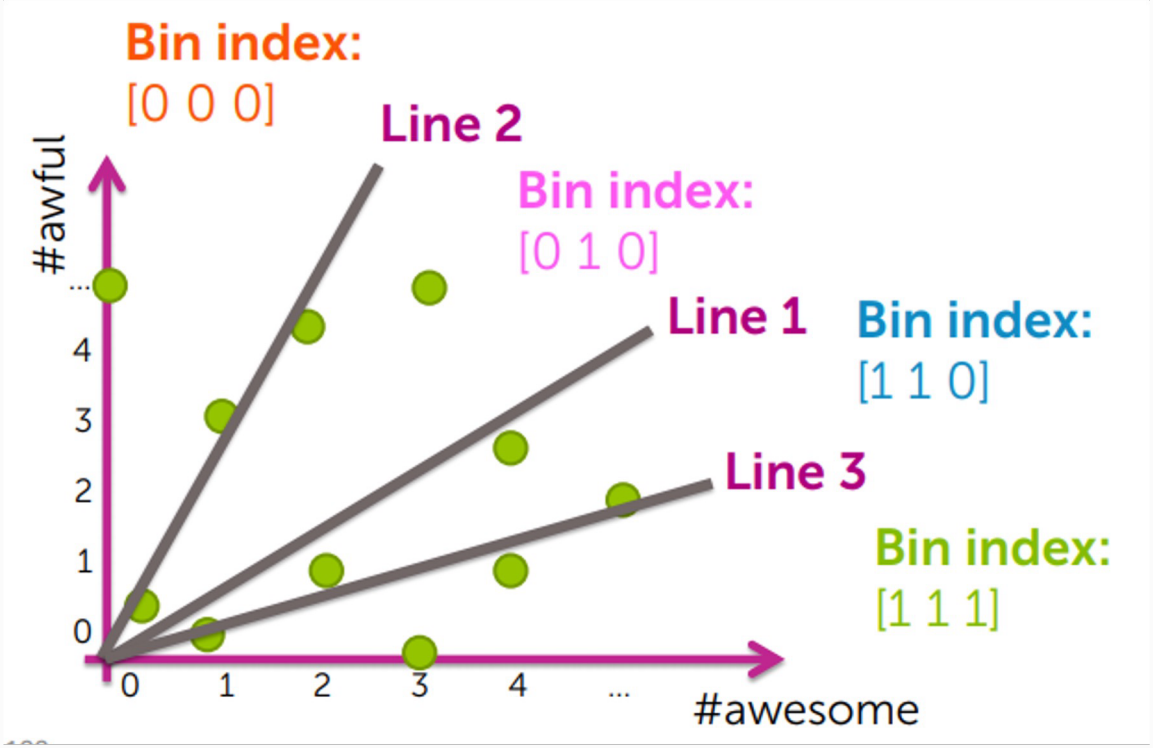
Three potential issues with simple approach

- Challenging to find good line
- Poor quality solution:
  - Points close together get split into separate bins
- Large computational cost:
  - Bins might contain many points, so still searching over large set for each NN query

Bin	0	1
List containing indices of datapoints:	{1,2,4,7,…}	{3,5,6,8,…}

Improving efficiency:
Reducing # points examined per query

Reducing search cost through more bins



Using score for NN search

2D Data	Sign ($Score_1$)	Bin 1 index	Sign ($Score_2$)	Bin 2 index	Sign ($Score_3$)	Bin 3 index
$\mathbf{x}_1$ = [0, 5]	-1	0	-1	0	-1	0
$\mathbf{x}_2$ = [1, 3]	-1	0	-1	0	-1	0
$\mathbf{x}_3$ = [3, 0]	1	1	1	1	1	1
...	...	...	...	...	...	...

Bin	[0 0 0] = 0	[0 0 1] = 1	[0 1 0] = 2	[0 1 1] = 3	[1 0 0] = 4	[1 0 1] = 5	[1 1 0] = 6	[1 1 1] = 7
Data indices:	{1,2}	--	{4,8,11}	--	--	--	{7,9,10}	{3,5,6}

search for NN amongst this set

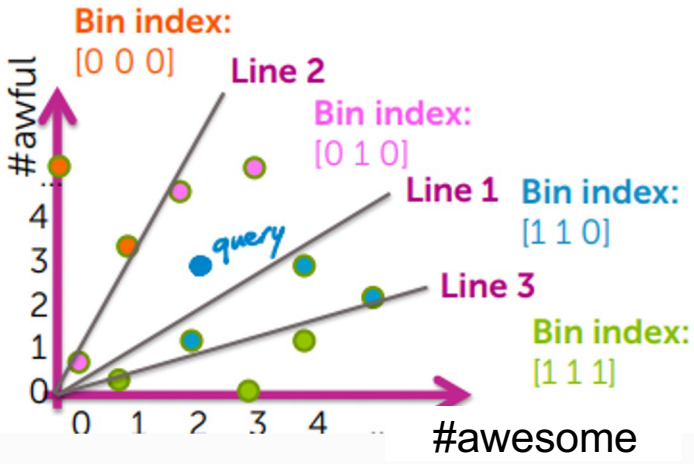
Improving search quality by searching neighboring bins

Bin	[0 0 0] = 0	[0 0 1] = 1	[0 1 0] = 2	[0 1 1] = 3	[1 0 0] = 4	[1 0 1] = 5	[1 1 0] = 6	[1 1 1] = 7
Data indices:	{1,2}	--	{4,8,11}	--	--	--	{7,9,10}	{3,5,6}

Query point here, but is NN?

Not necessarily

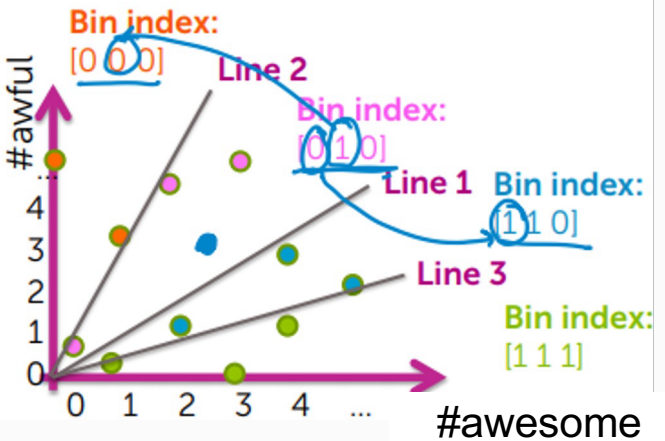
Even worse than before...Each line can split pts.
Sacrificing accuracy for speed



Improving search quality by searching neighboring bins

Bin	[0 0 0] = 0	[0 0 1] = 1	[0 1 0] = 2	[0 1 1] = 3	[1 0 0] = 4	[1 0 1] = 5	[1 1 0] = 6	[1 1 1] = 7
Data indices:	{1,2}	--	{4,8,11}	--	--	--	{7,9,10}	{3,5,6}

Next closest bins (flip 1 bit)

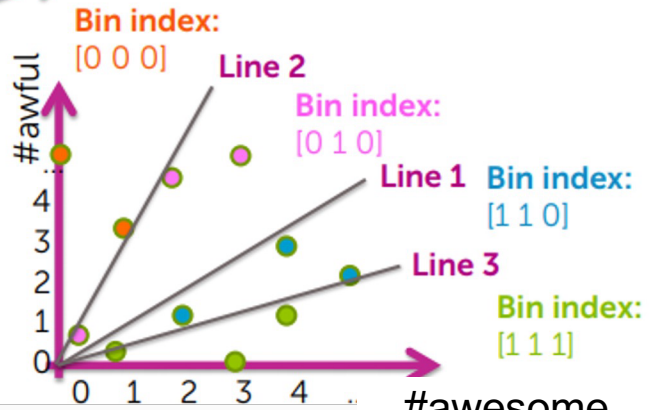


#awesome

Improving search quality by searching neighboring bins

Bin	[0 0 0] = 0	[0 0 1] = 1	[0 1 0] = 2	[0 1 1] = 3	[1 0 0] = 4	[1 0 1] = 5	[1 1 0] = 6	[1 1 1] = 7
Data indices:	{1,2}	--	{4,8,11}	--	--	--	{7,9,10}	{3,5,6}

Further bin
(flip 2 bits)



#awesome

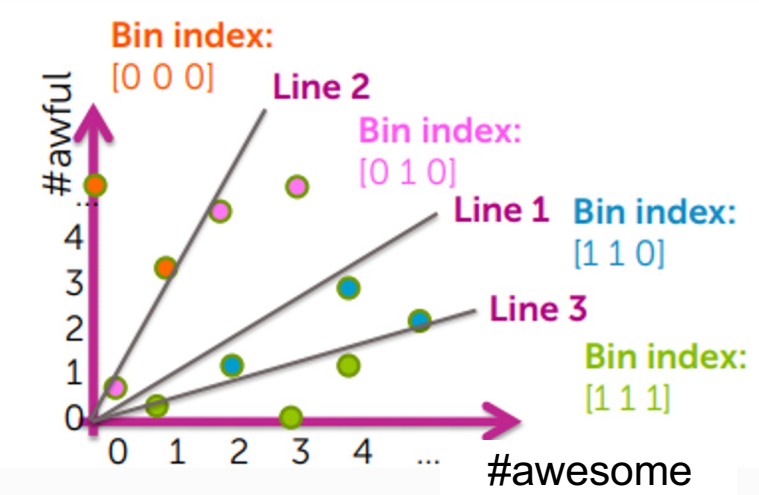
Improving search quality by searching neighboring bins

Bin	[0 0 0] = 0	[0 0 1] = 1	[0 1 0] = 2	[0 1 1] = 3	[1 0 0] = 4	[1 0 1] = 5	[1 1 0] = 6	[1 1 1] = 7
Data indices:	{1,2}	--	{4,8,11}	--	--	--	{7,9,10}	{3,5,6}

Quality of retrieved NN can only improve with searching more bins

Algorithm:

- Continue searching until computational budget is reached
- or quality of NN good enough



LSH recap

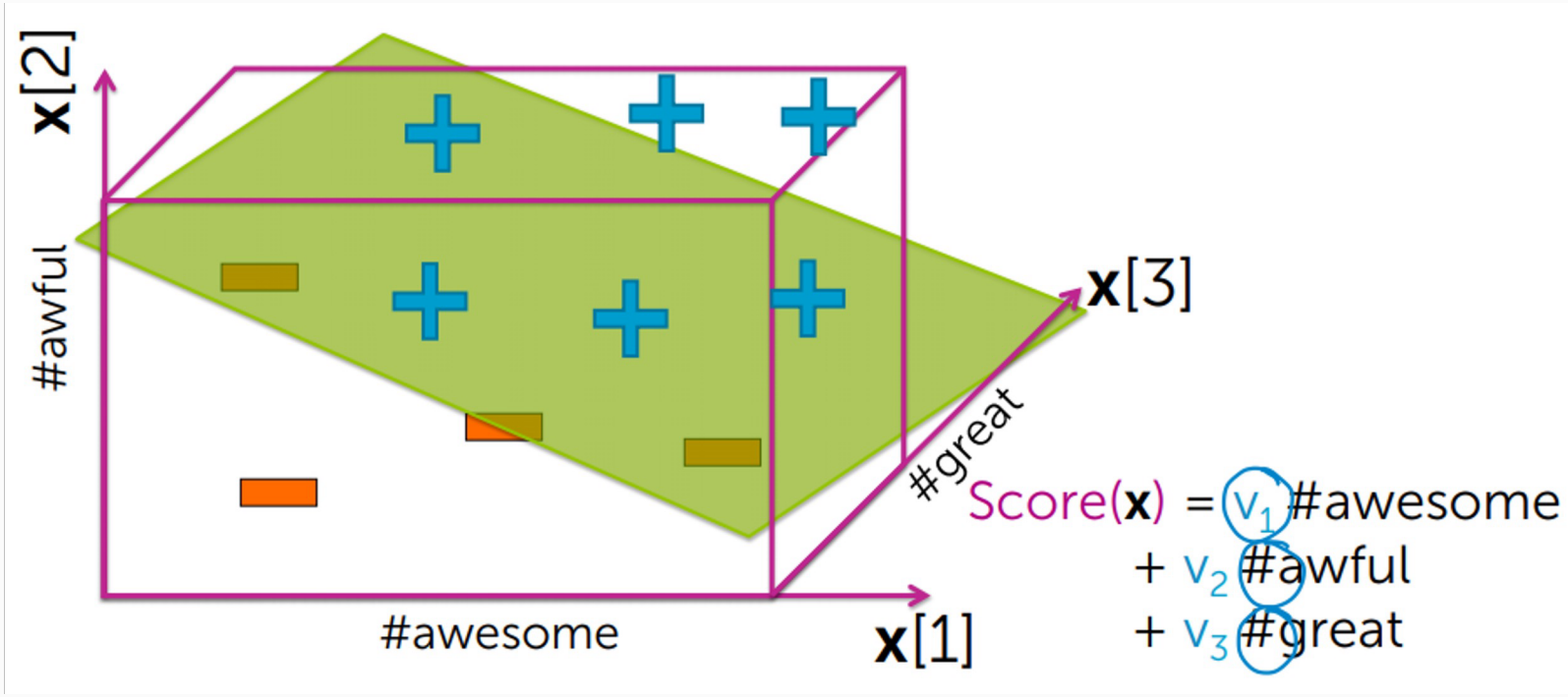
kd-tree competitor data structure

- Draw h random lines
- Compute "score" for each point under each line and translate to binary index
- Use h-bit binary vector per data point as bin index
- Create hash table

- For each query point x, search bin(x), then neighboring bins until time limit

Moving to higher dimensions d

Draw random *planes*



Cost of binning points in d-dim

$$\text{Score}(\mathbf{x}) = v_1^{(i)} \#\text{awesome} + v_2^{(i)} \#\text{awful} + v_3^{(i)} \#\text{great}$$

i^{th} hyperplane

Per data point, need d multiplies to determine bin index per plane

In high-dim, (and some applications) this is often a sparse mult.

One-time cost offset if many queries of fixed dataset

Using multiple tables for even greater efficiency in NN search

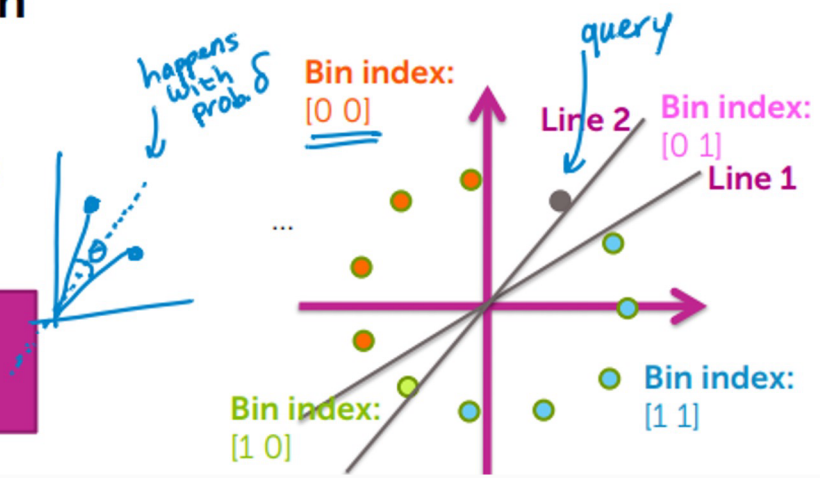
If I throw down 2 lines…

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

For simplicity, assume we **search bins 1 bit off** from query

Let δ be the probability of a line falling between points θ apart

Search 3 bins and do not find NN with probability δ^2

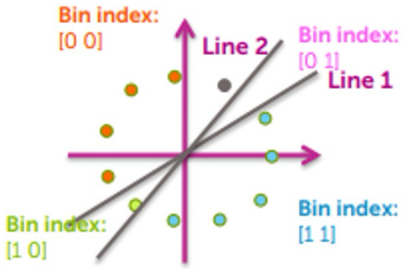


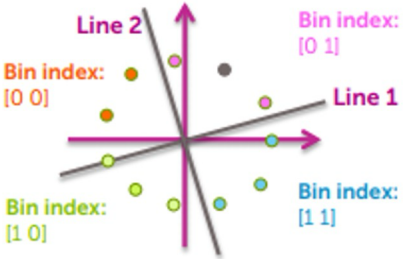
What if I repeat the 2-line binning?

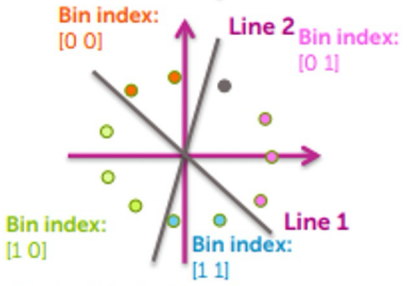
Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3







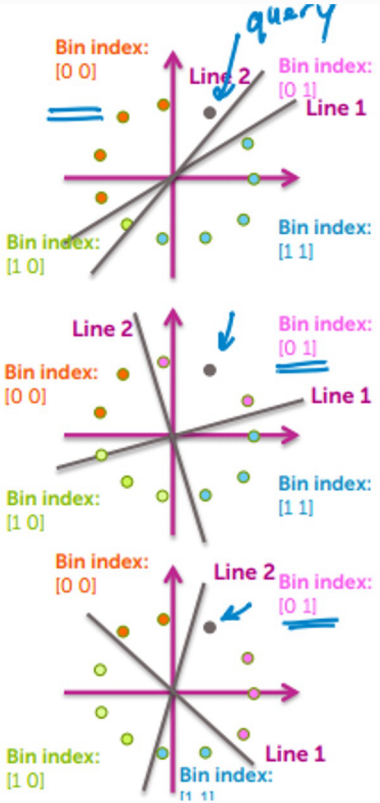
12

What if I repeat the 2-line binning?

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3



Now, search only query bin **per table**

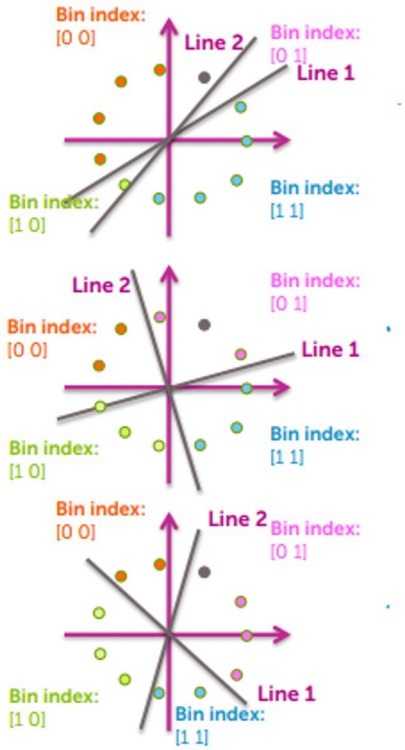
Still searching 3 bins, but what is chance of not finding NN?

What if I repeat the 2-line binning?

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3



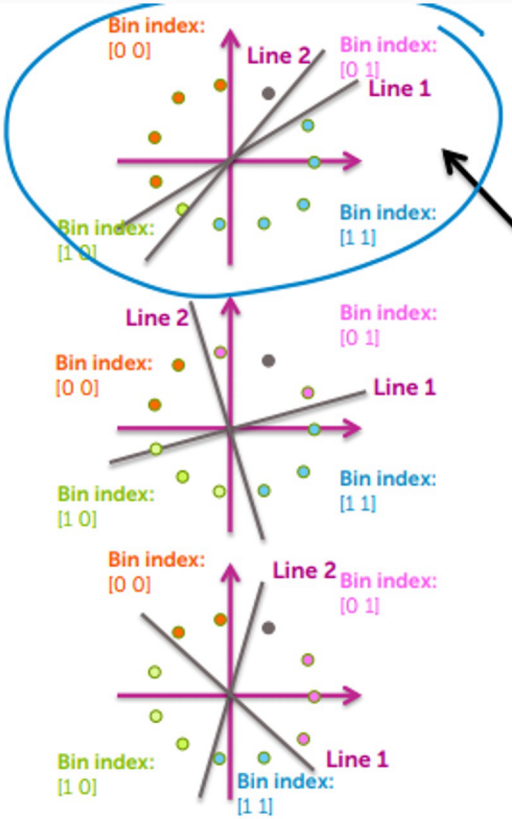
What is chance that query pt and NN are split in **all tables**?

Probability of splitting neighboring points many times

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3



Probability NN is in different bin:

Prob = 1-Pr(same bin)
= $1-(1-\delta)^2$
= $2\delta - \delta^2$

$1-\delta$ = prob. that 1 line does not split query + NN

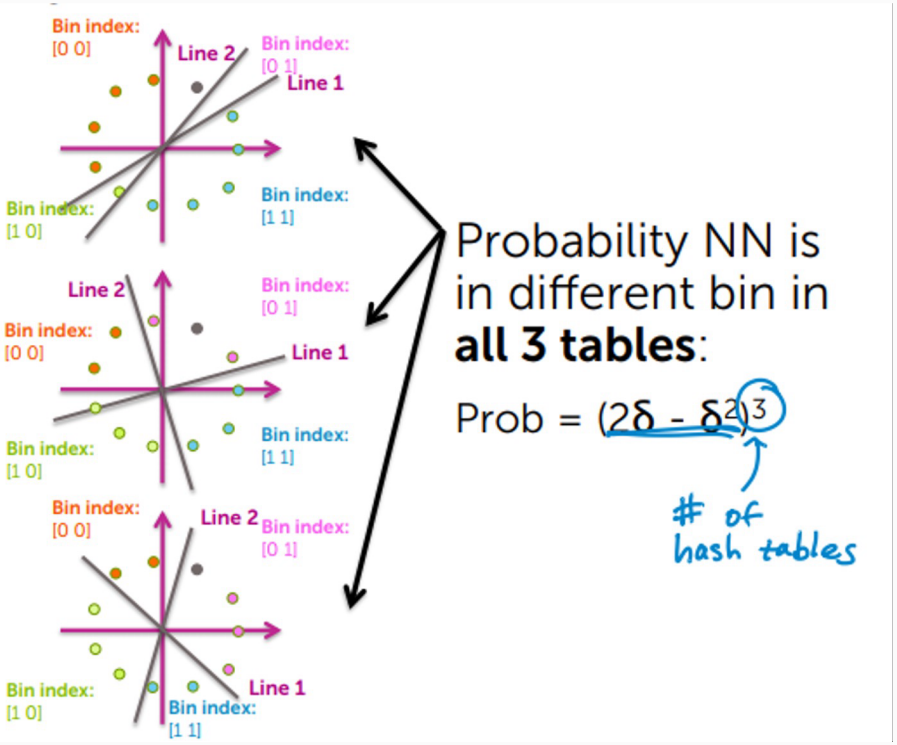
$(1-\delta)^2$ = prob. that 2 lines don't split pts

Probability of splitting neighboring points many times

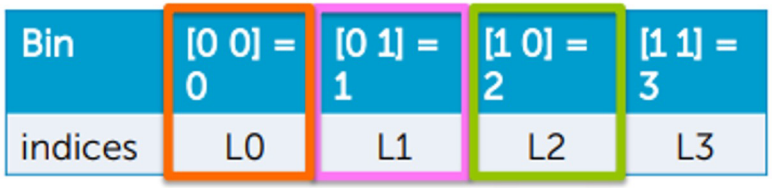
Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3

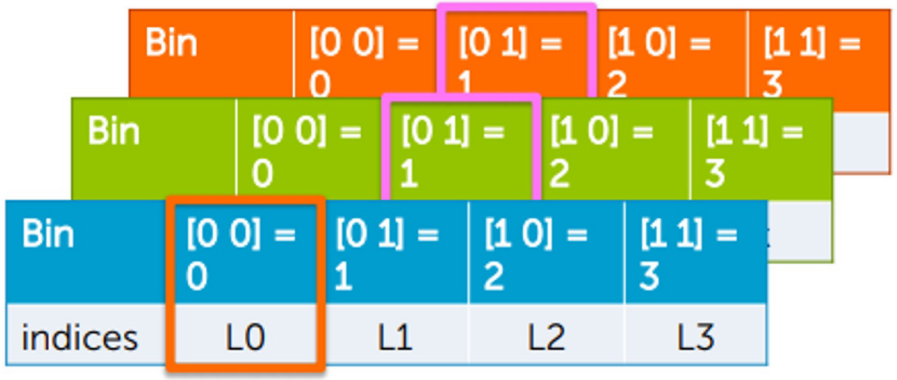
Bin	[0 0] = 0	[0 1] = 1	[1 0] = 2	[1 1] = 3
indices	L0	L1	L2	L3



Comparing approaches for 2-bit tables

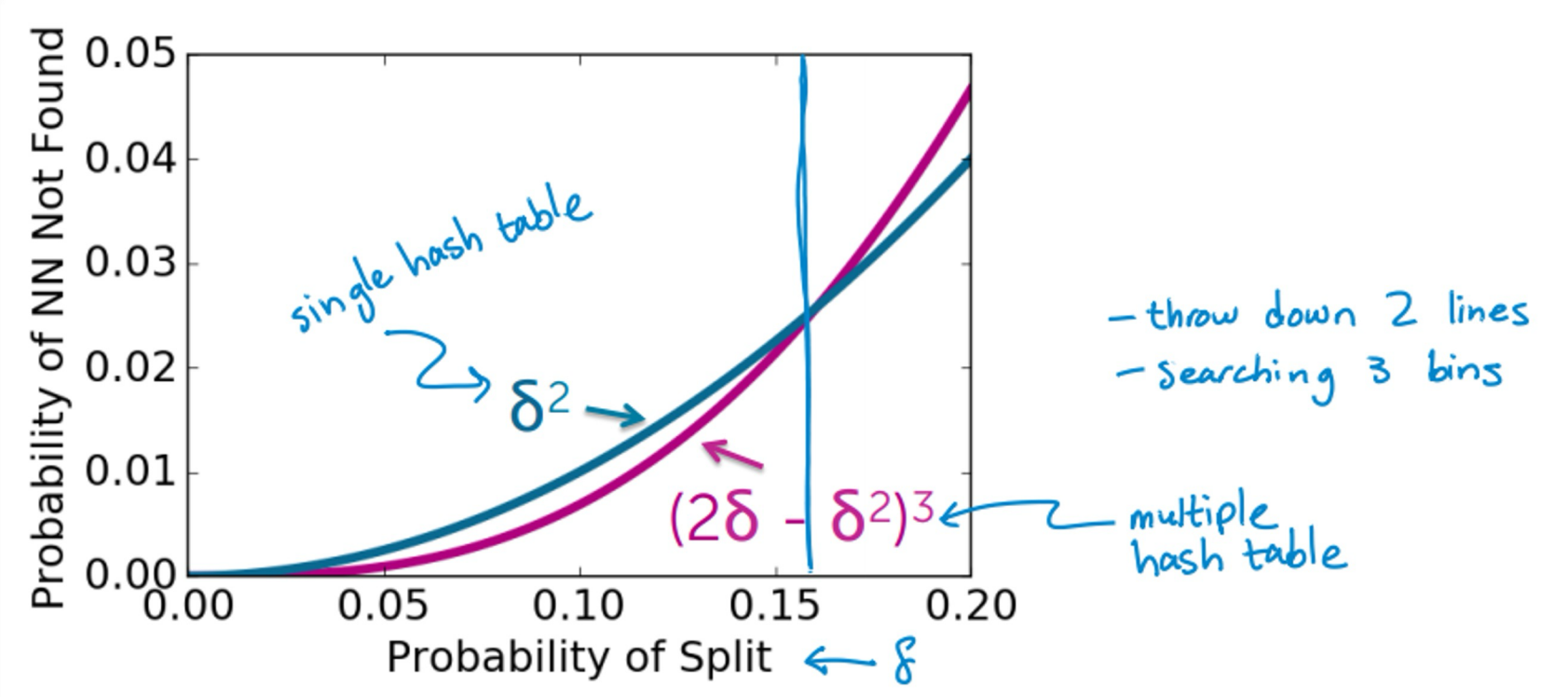


# bins searched	prob. of no NN
3	δ^2

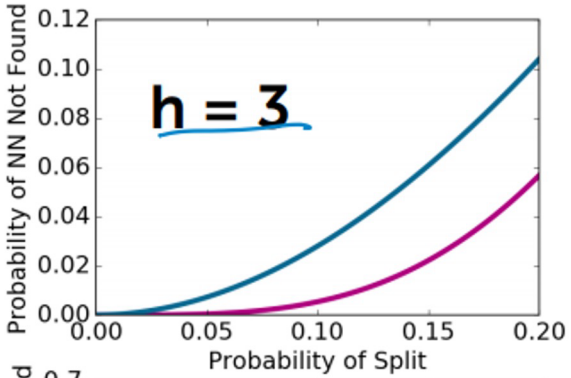


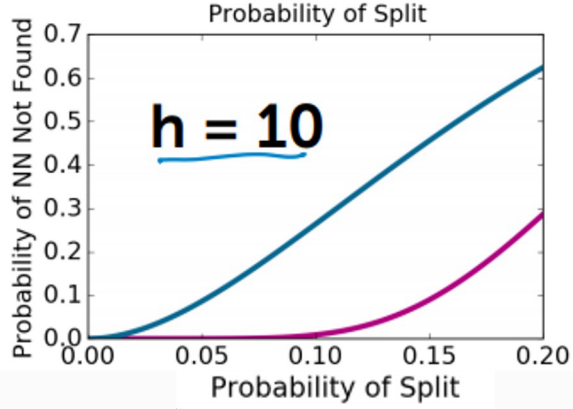
# bins searched	prob. of no NN
3	$(2\delta - \delta^2)^3$

Comparing probabilities



Comparing probabilities





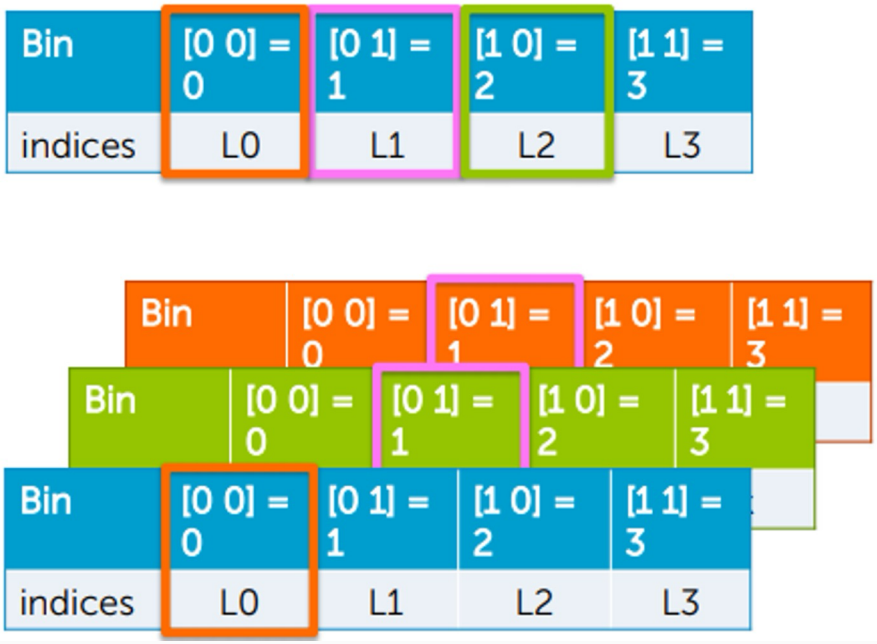
one hash table

$1-(1-\delta)^h-h\delta(1-\delta)^{h-1}$

$(1-(1-\delta)^h)^{h+1}$

multiple hash table

Summary of LSH approaches

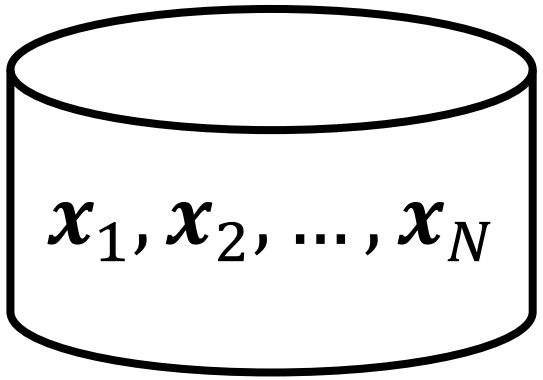


Cost of binning points is **lower**, but likely need to search **more bins** per query

Cost of binning points is **higher**, but likely need to search **fewer bins** per query

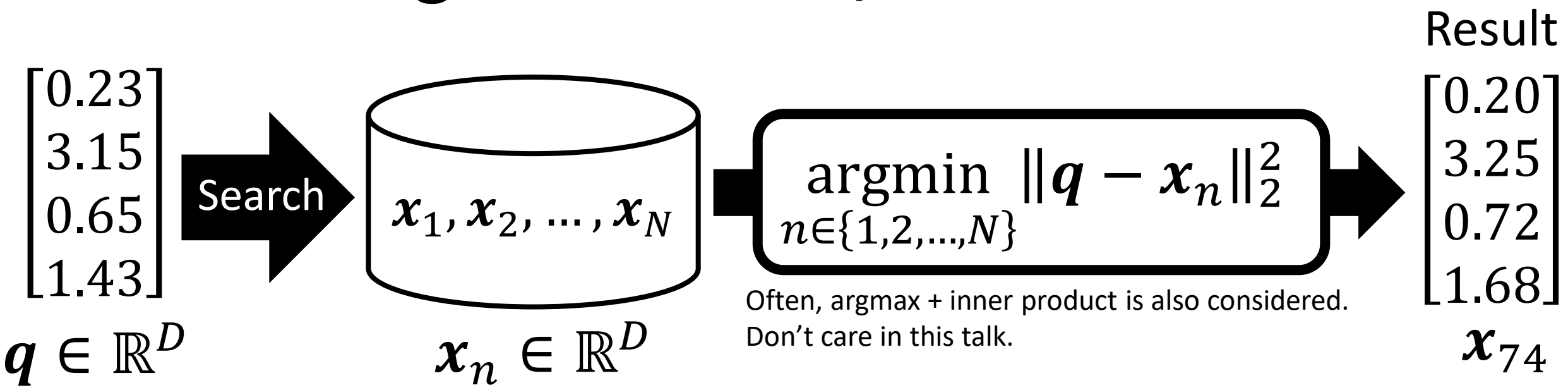
- **Background**
- **Graph-based search**
  - ✓ **Basic (construction and search)**
  - ✓ **Observation**
  - ✓ **Properties**
- **Representative works**
  - ✓ **HNSW, NSG, NGT, Vamana**
- **Discussion**

Nearest Neighbor Search; NN



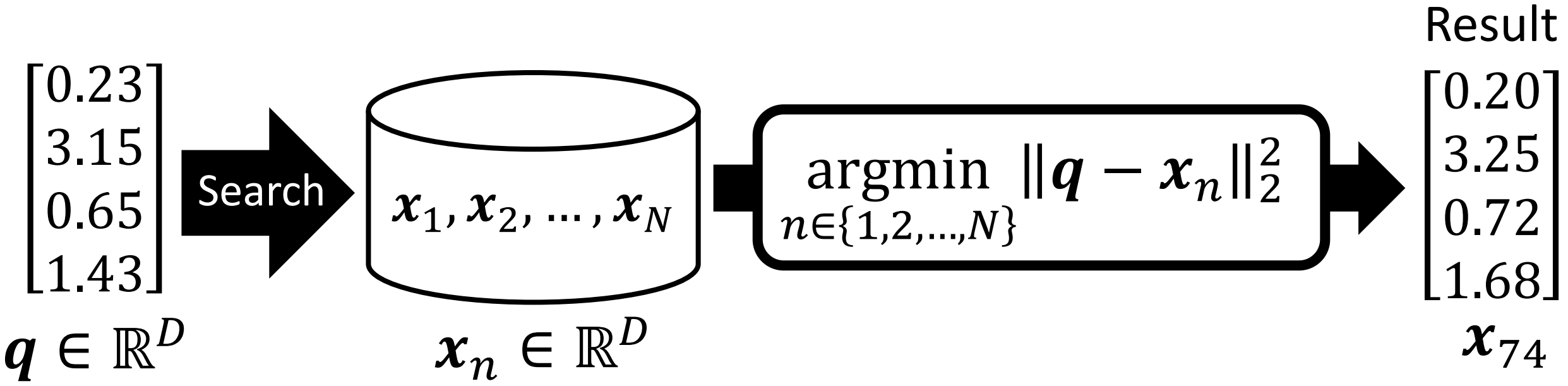
- N D-dim database vectors: $\{\boldsymbol{x}_n\}_{n=1}^{N}$

Nearest Neighbor Search; NN



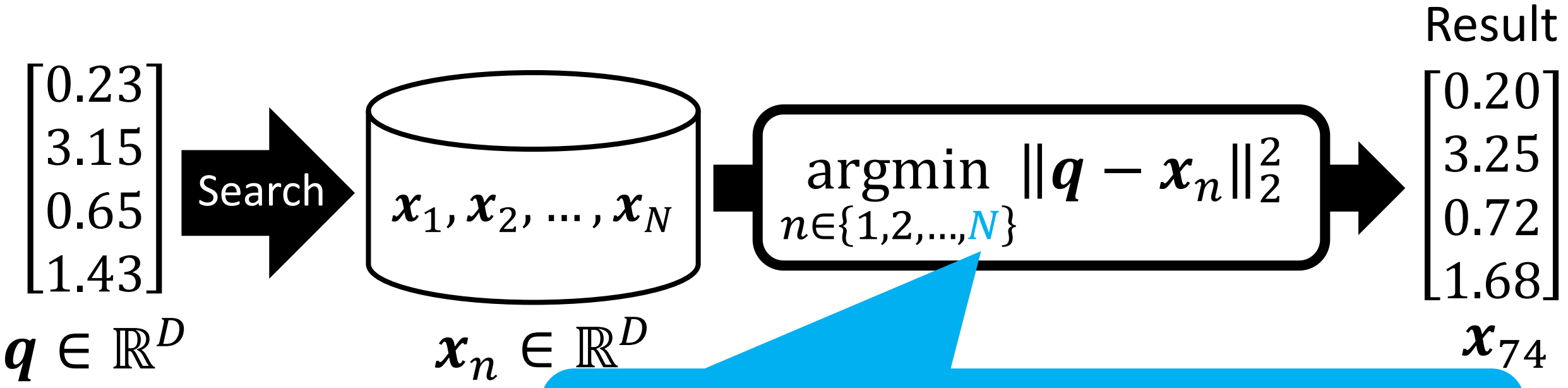
- N D-dim database vectors: $\{\boldsymbol{x}_n\}_{n=1}^N$
- Given a query $\boldsymbol{q}$, find the closest vector from the database
- One of the fundamental problems in computer science
- Solution: linear scan, $O(ND)$, slow ☹

Approximate Nearest Neighbor Search; ANN



- Faster search
- Don't necessarily have to be exact neighbors
- Trade off: runtime, accuracy, and memory-consumption

Approximate Nearest Neighbor Search; ANN



- In this talk, suppose: $N < 10^9$
- All data can be loaded on memory

- Faster search
- Don't necessarily have to be exact neighbors
- Trade off: runtime, accuracy, and memory-consumption

Real-world use cases 1: multimodal search

Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search

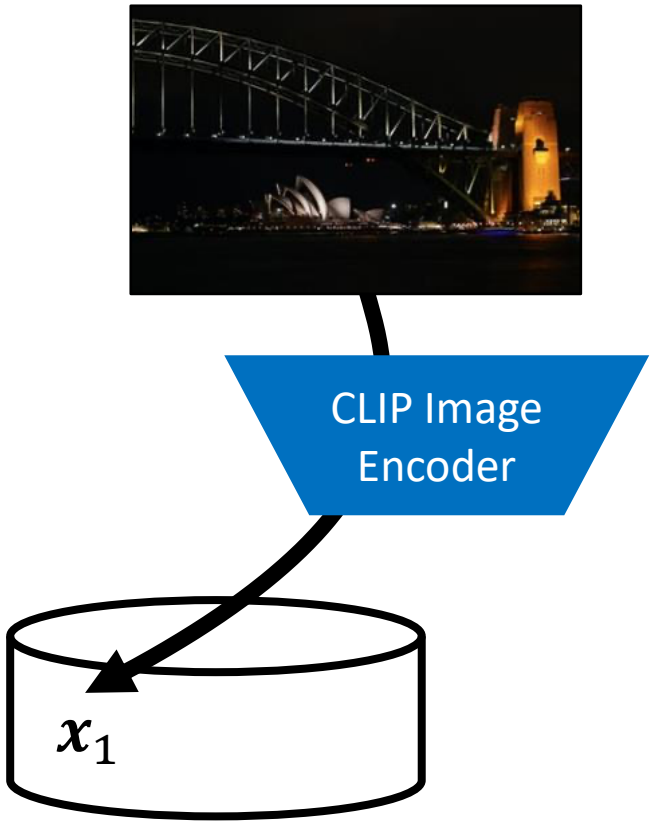


Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search

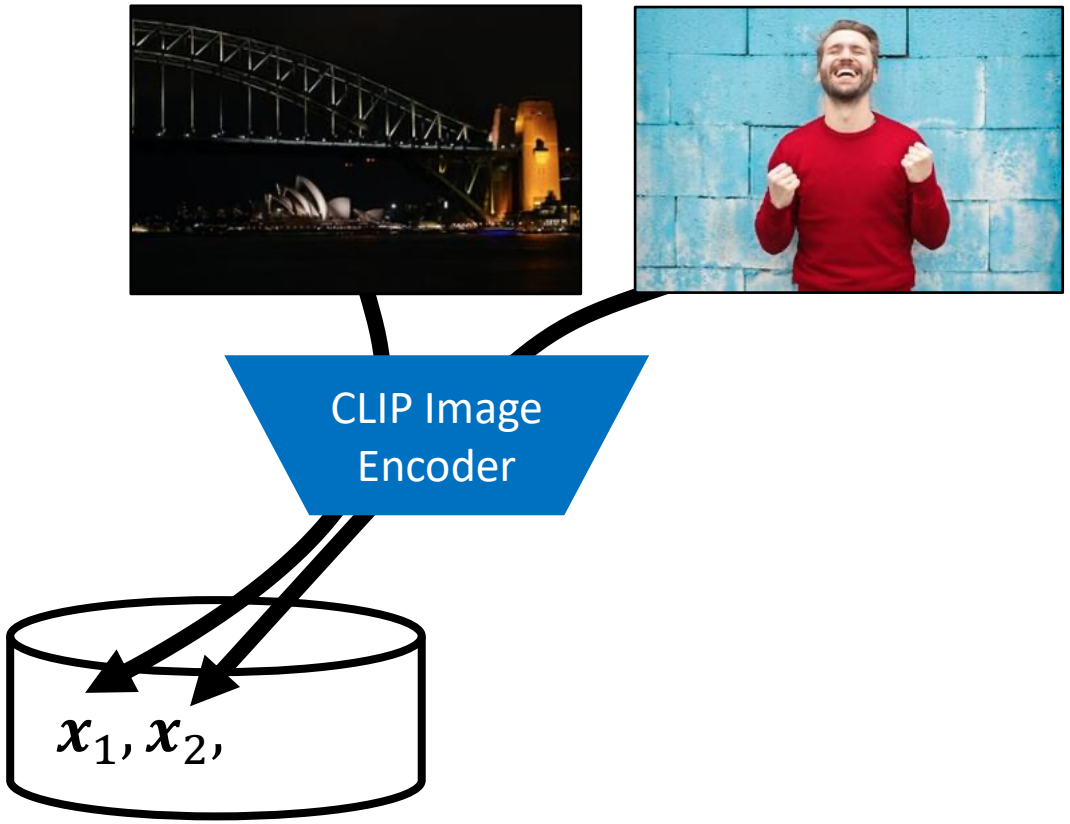


Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search

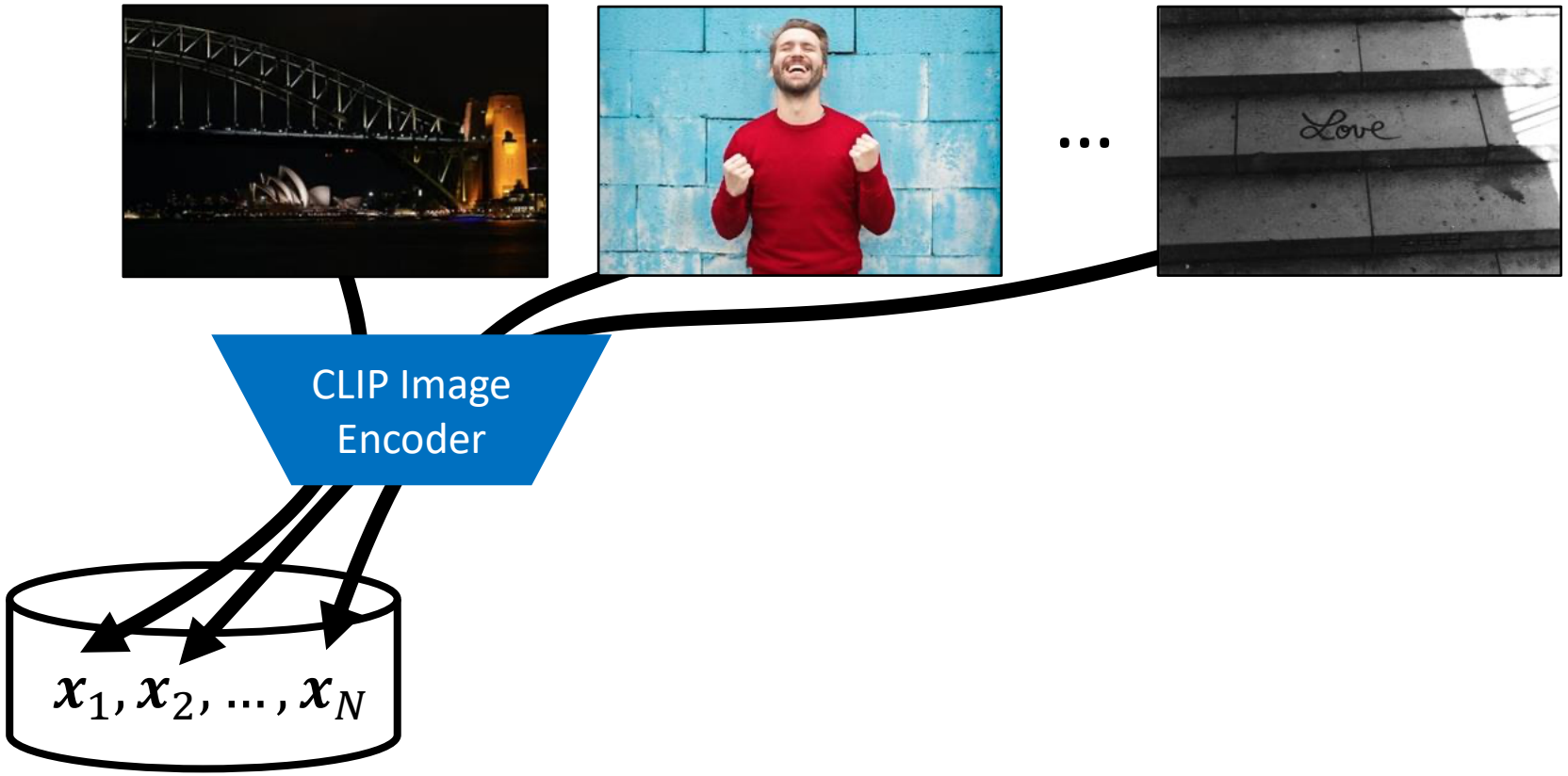


Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search

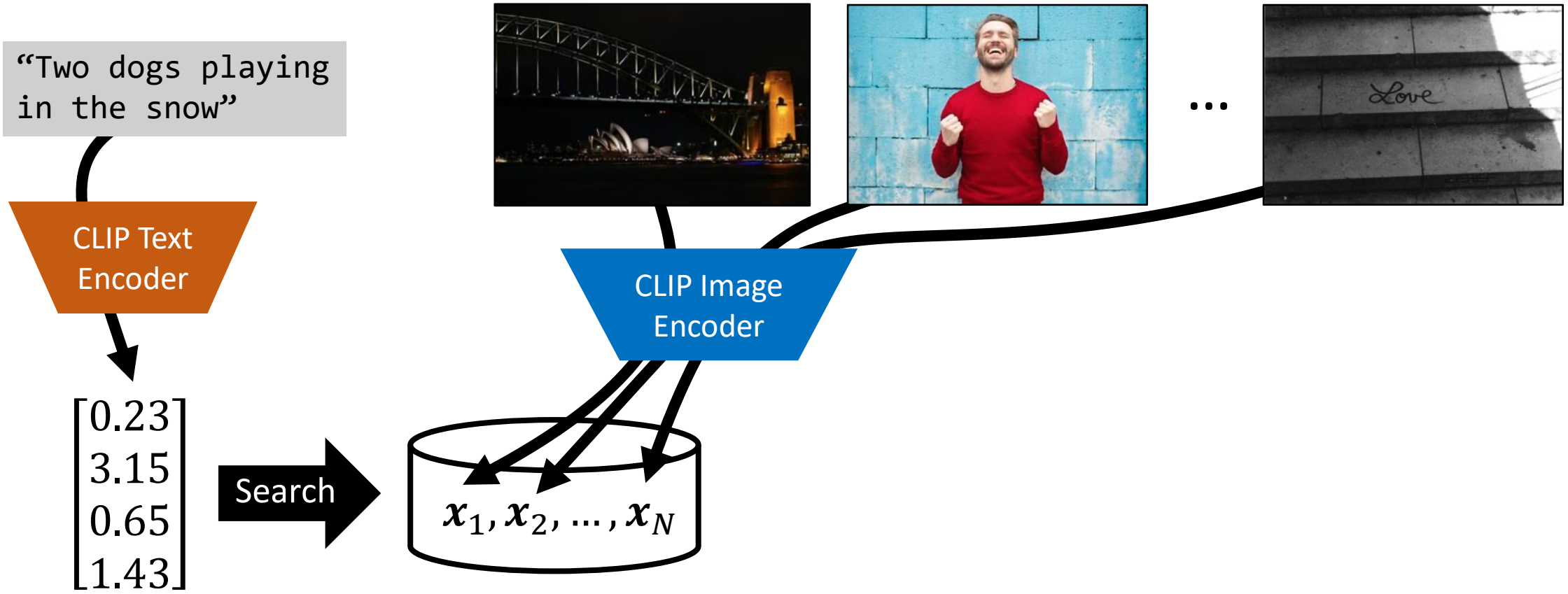


Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search

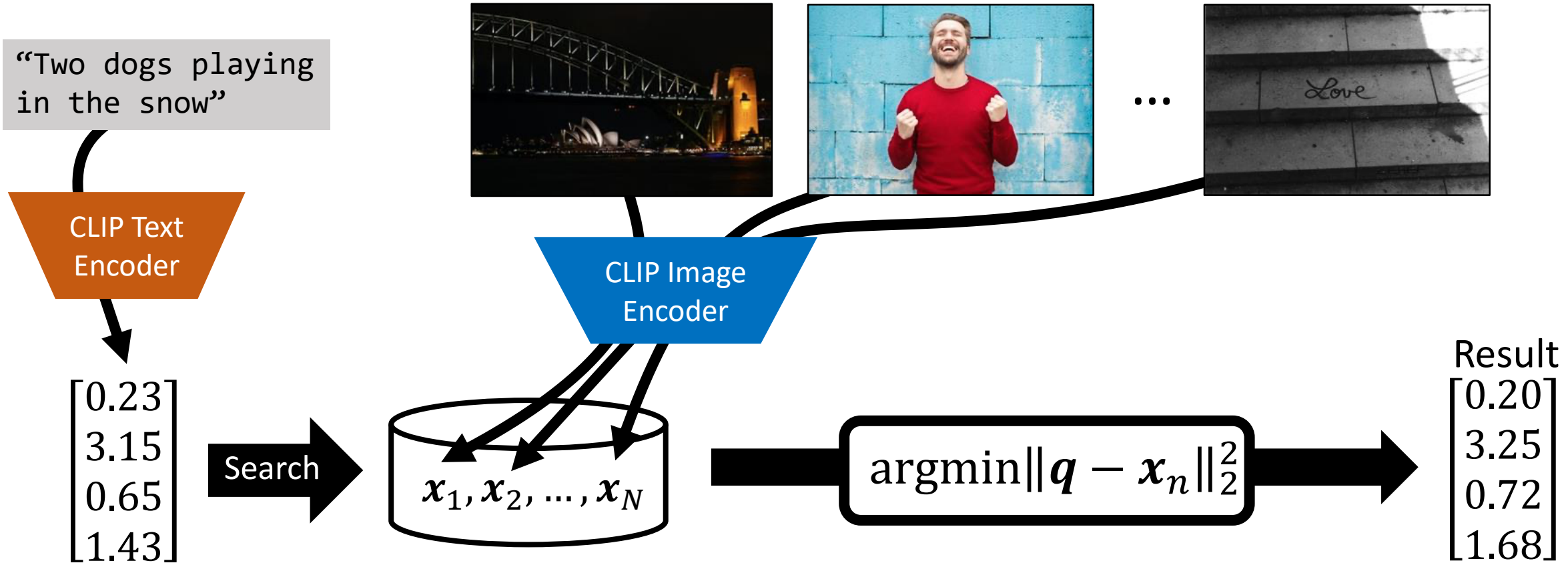
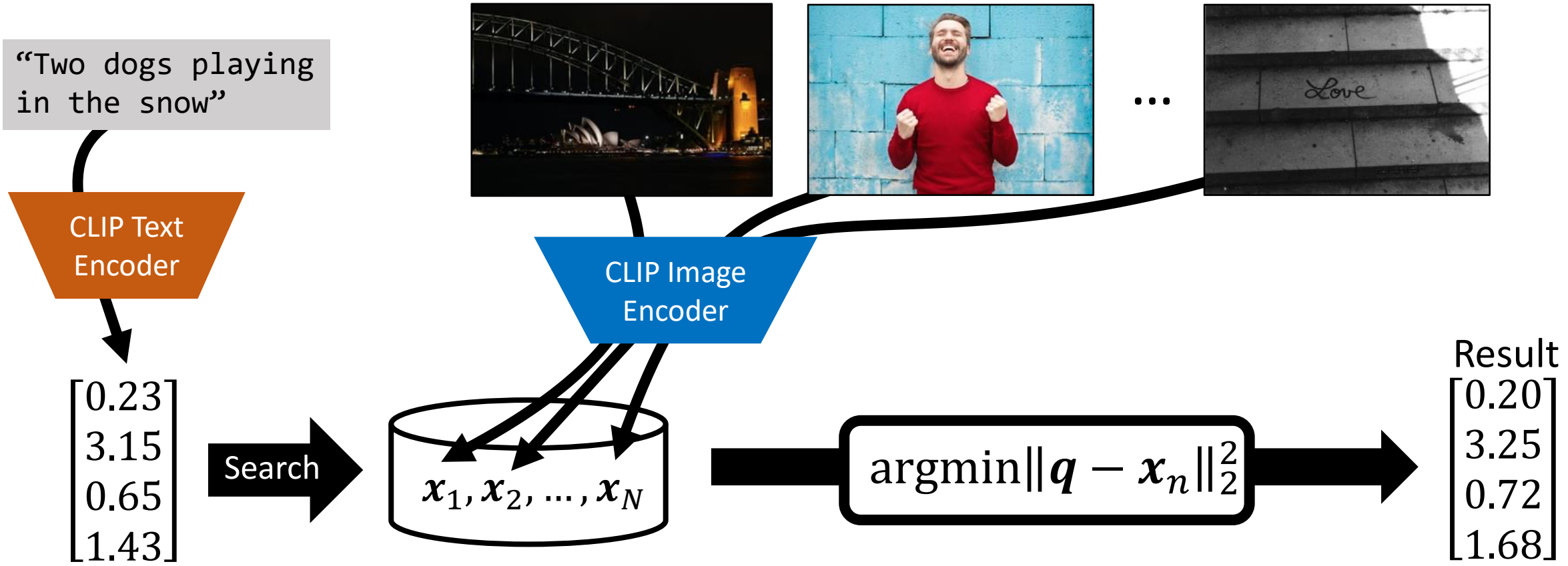


Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 1: multimodal search



- Encoder determines **the upper bound** of the accuracy of the system
- ANN determines a **trade-off** between accuracy, runtime, and memory

Image are from: https://github.com/haltakov/natural-language-image-search
Credit: Photos by Genton Damian, bruce mars, Dalal Nizam, and Richard Burlton on Unsplash

Real-world use cases 2: LLM + embedding

```
"Who won curling
gold at the 2022
Winter Olympics?"
```

ChatGPT 3.5
(trained in 2021)

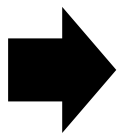
Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

ChatGPT 3.5
(trained in 2021)



"I'm sorry, but as an AI language model, I don't have information about the future events."

Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

```
"Who won curling
gold at the 2022
Winter Olympics?"
```

ChatGPT 3.5
(trained in 2021)

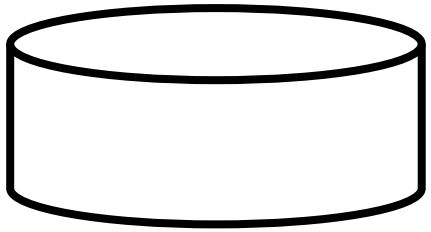
Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

ChatGPT 3.5
(trained in 2021)



“Chinami Yoshida¥n¥n==Personal…”

“Lviv bid for the 2022 Winter…”

⋮

“Damir Sharipzyanov¥n¥n=Career…”

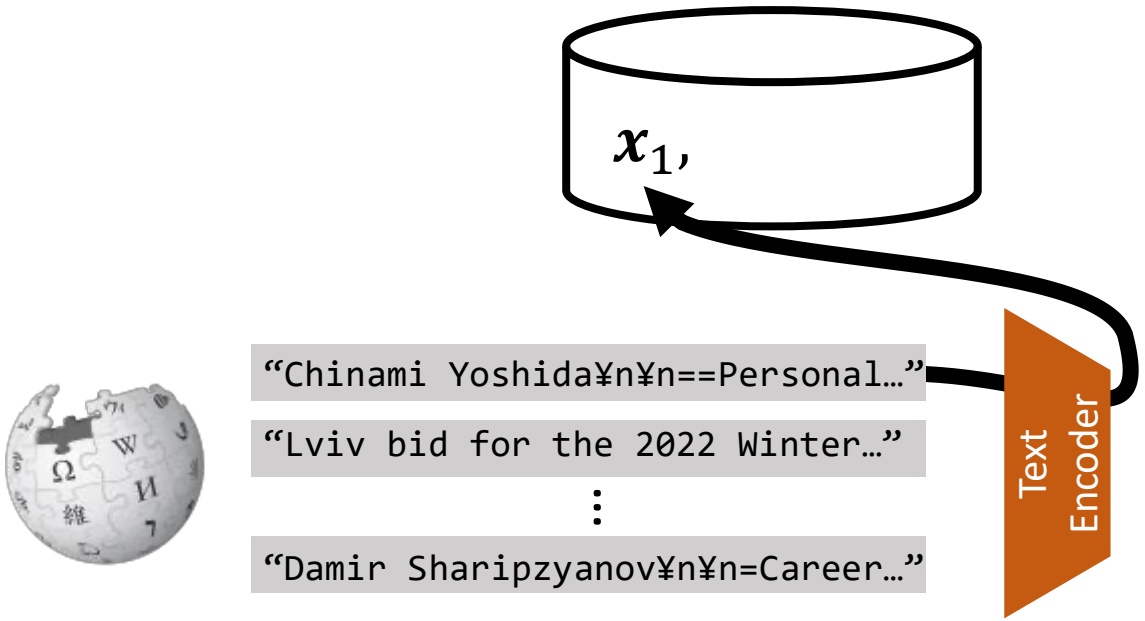
Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

ChatGPT 3.5
(trained in 2021)



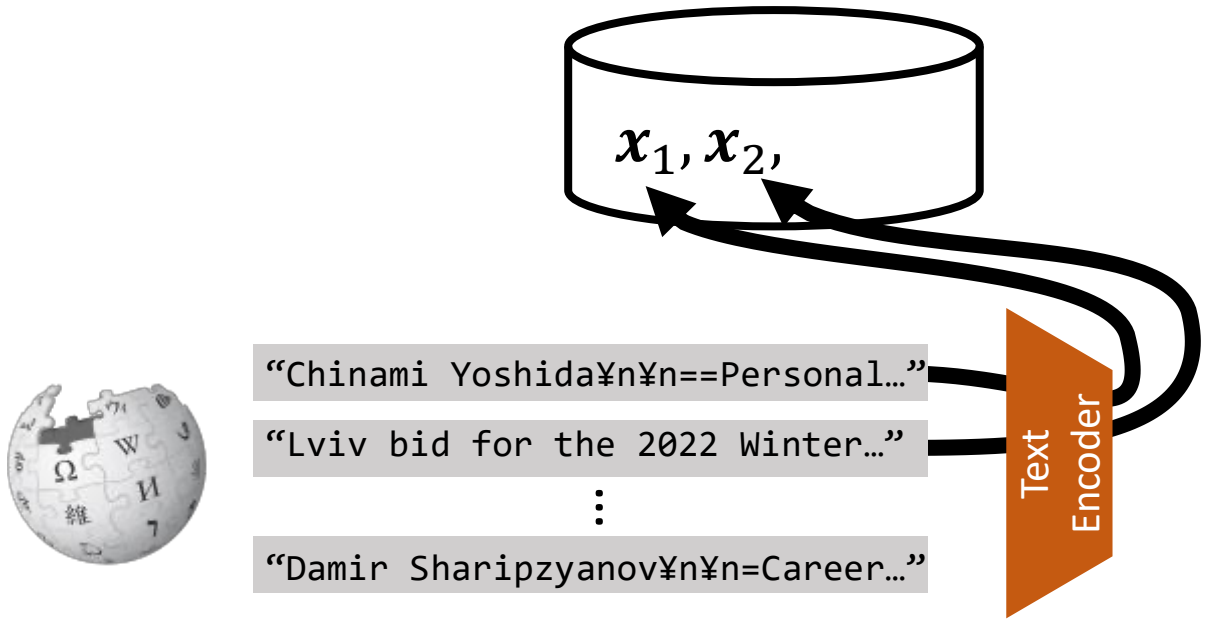
Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

ChatGPT 3.5
(trained in 2021)



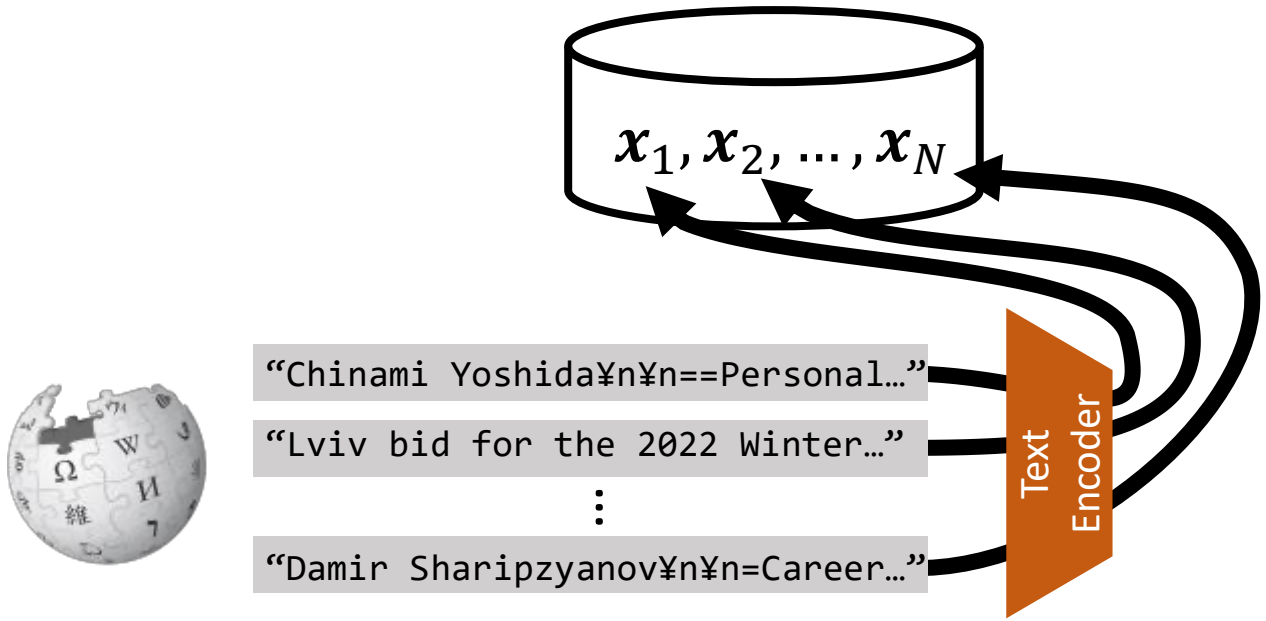
Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

ChatGPT 3.5
(trained in 2021)



Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding

"Who won curling gold at the 2022 Winter Olympics?"

Text Encoder

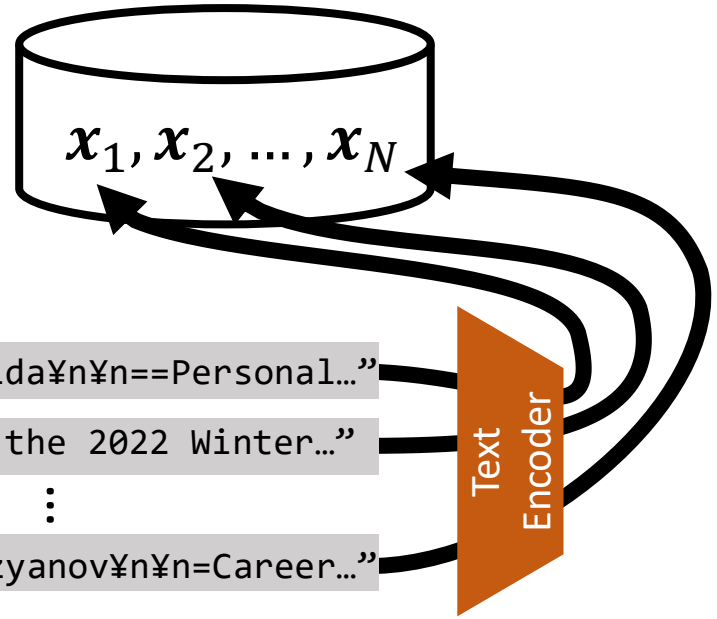
$$\begin{bmatrix} 0.23 \\ 3.15 \\ 0.65 \\ 1.43 \end{bmatrix}$$

"Chinami Yoshida¥n¥n==Personal…"

"Lviv bid for the 2022 Winter…"

⋮

"Damir Sharipzyanov¥n¥n=Career…"

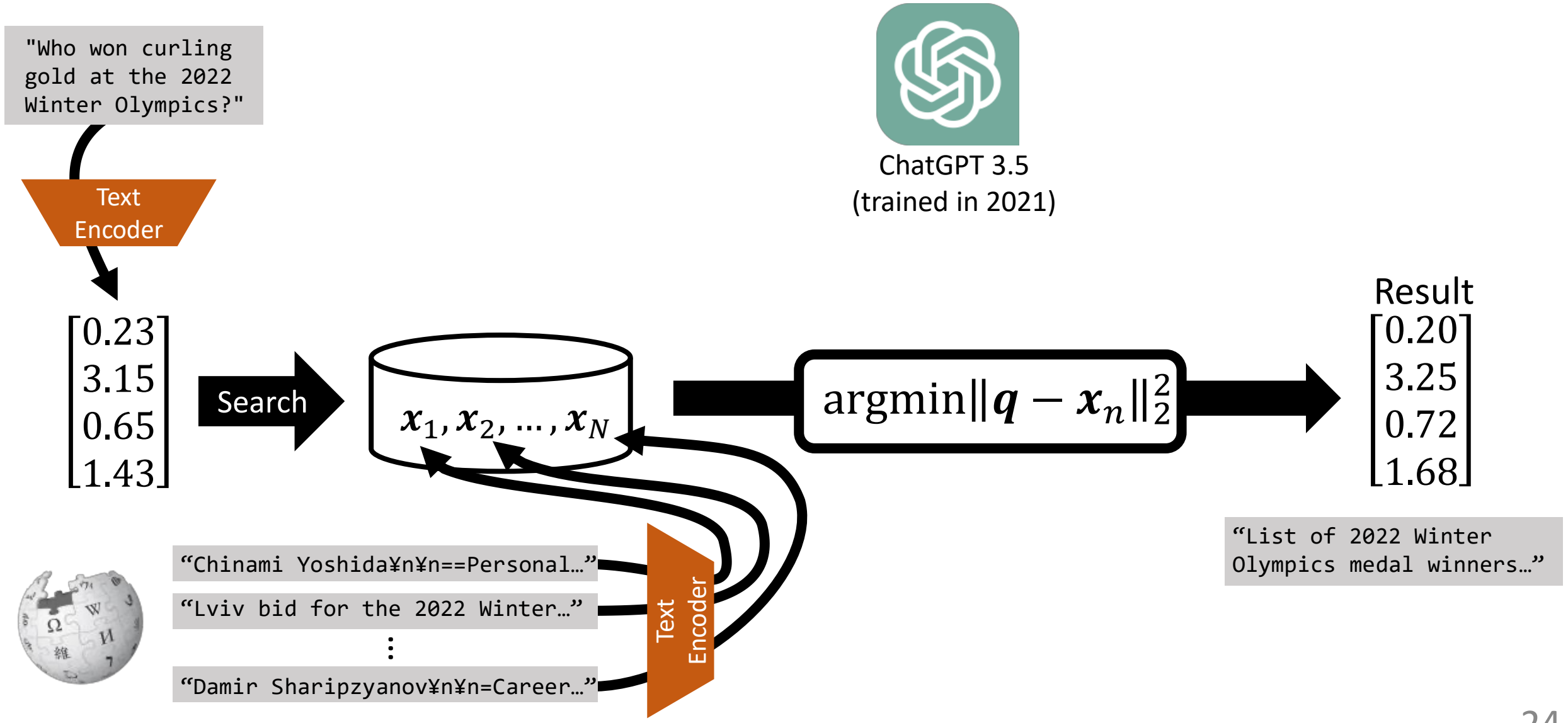


ChatGPT 3.5
(trained in 2021)

Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

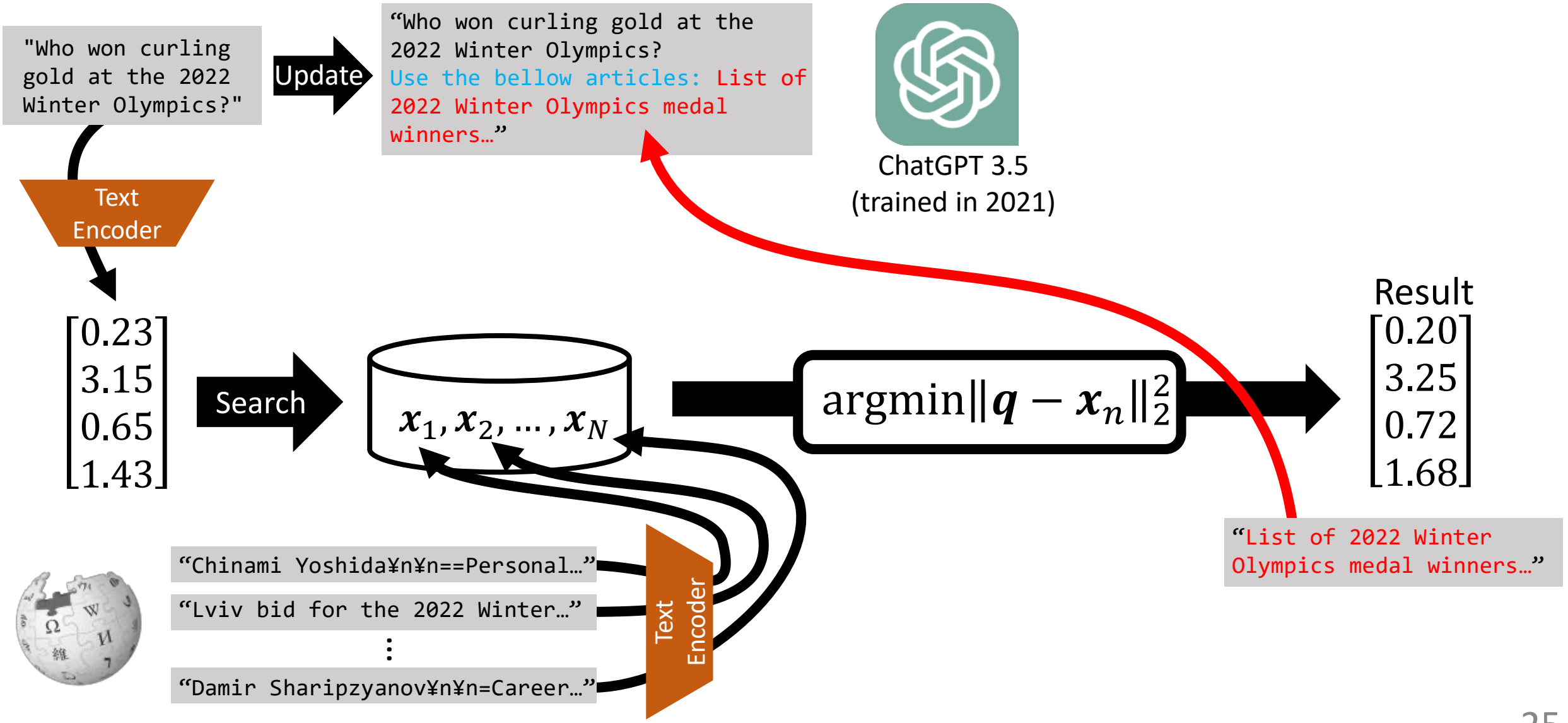
Real-world use cases 2: LLM + embedding



Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

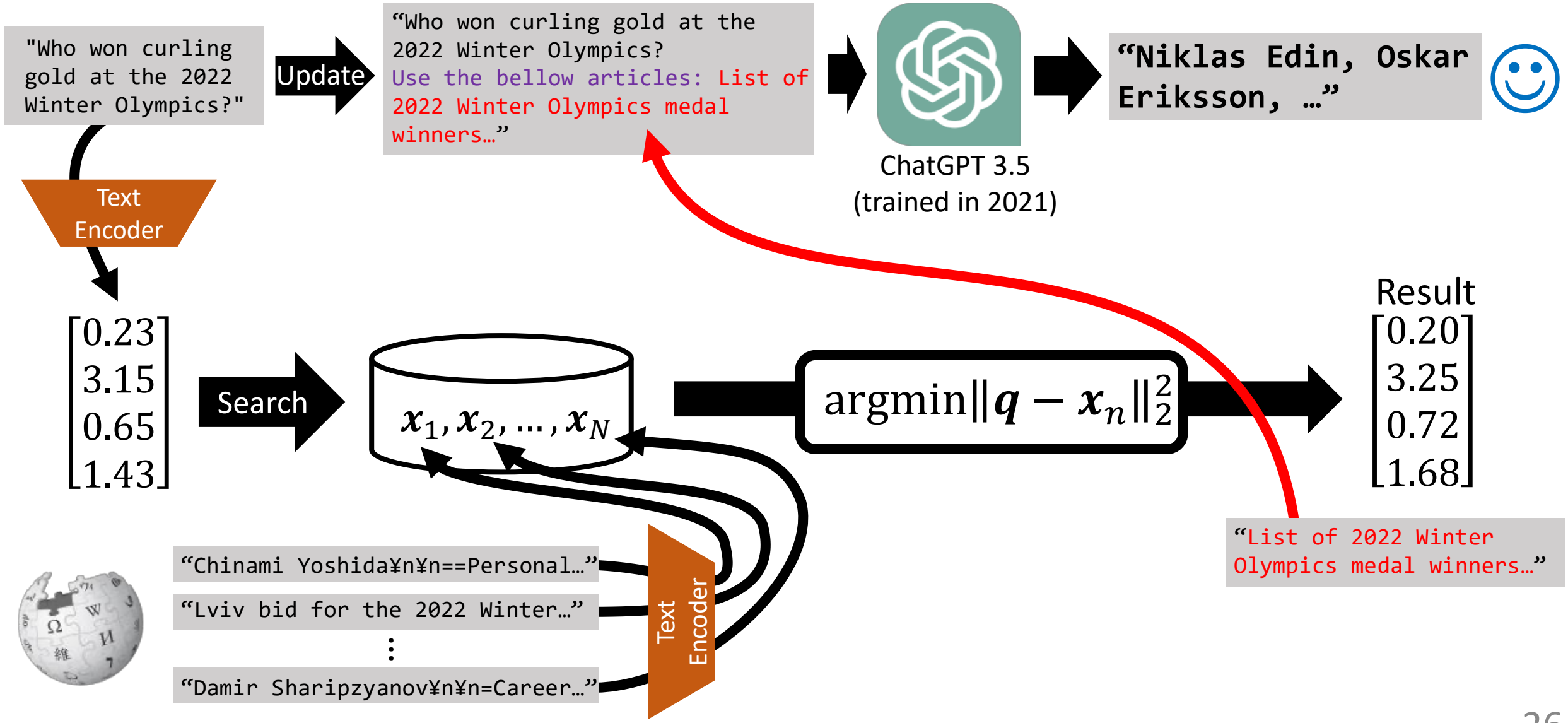
Real-world use cases 2: LLM + embedding



Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

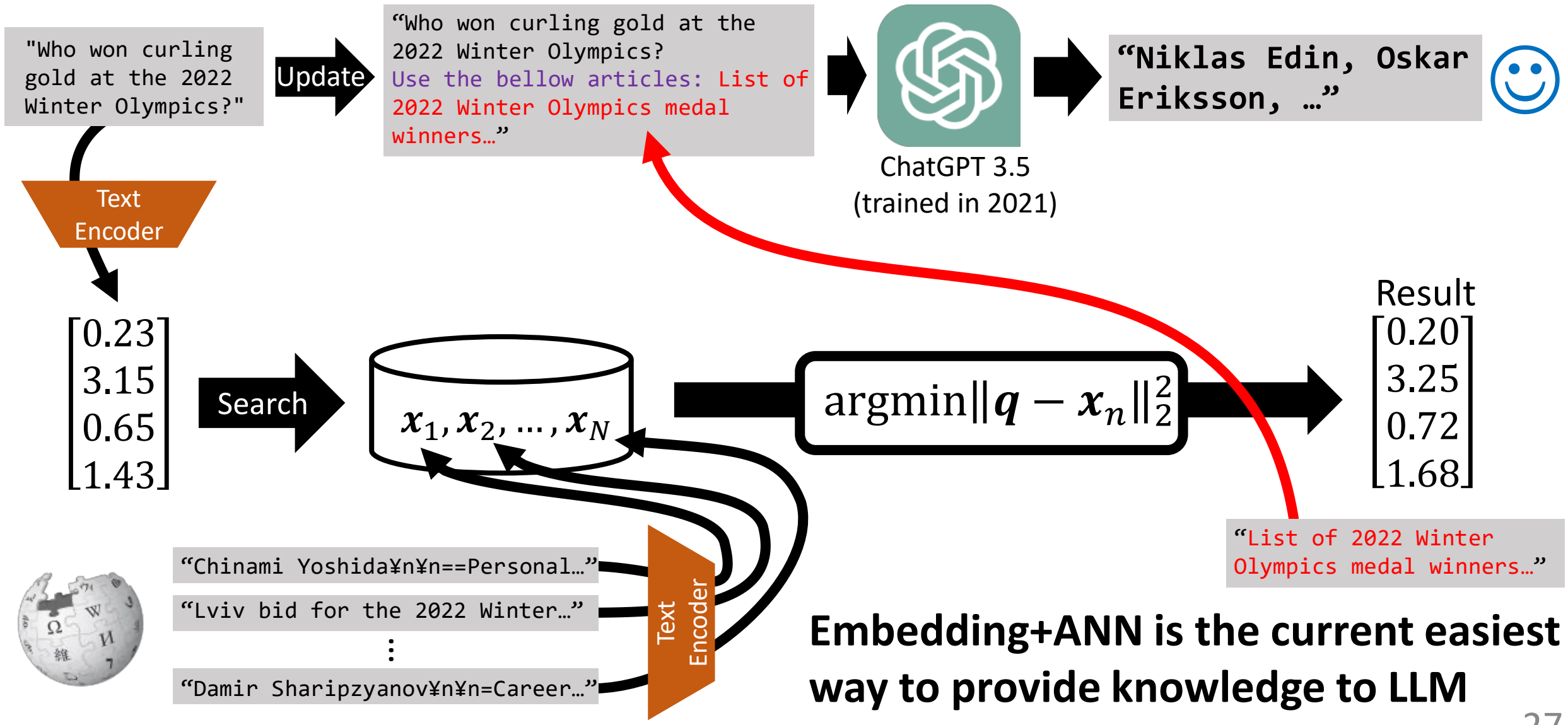
Real-world use cases 2: LLM + embedding



Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

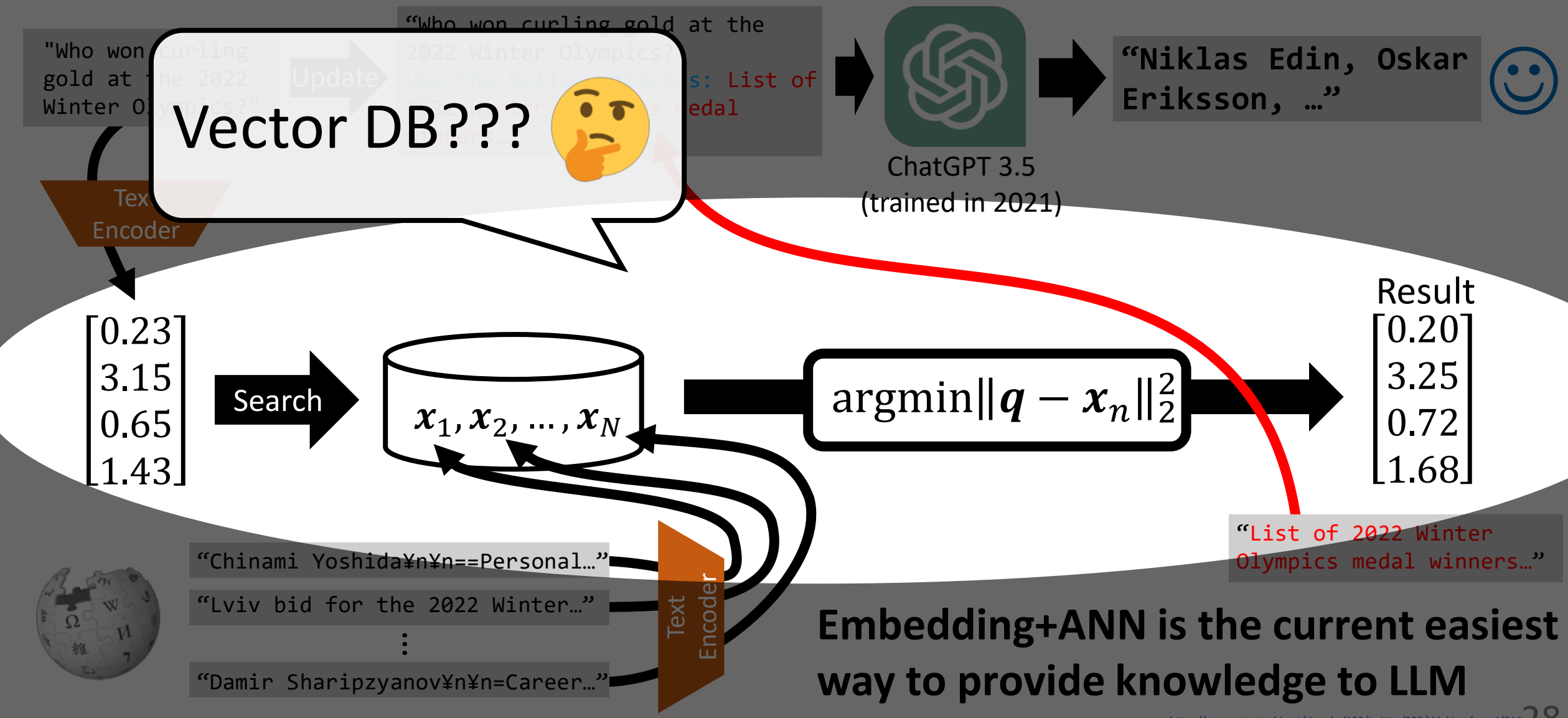
Real-world use cases 2: LLM + embedding



Embedding+ANN is the current easiest way to provide knowledge to LLM

Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

Real-world use cases 2: LLM + embedding



Embedding+ANN is the current easiest way to provide knowledge to LLM

Texts are from: https://github.com/openai/openaicookbook/blob/main/examples/Question_answering_using_embeddings.ipynb

Icon credit: https://ja.wikipedia.org/wiki/ChatGPT

https://em-content.zobj.net/thumbs/120/twitter/322/thinking-face_1f914.png

Three levels of technology

Algorithm

- Scientific paper
- Math
- Often, by researchers

Product Quantization + Inverted Index (PQ, IVFPQ) [Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

ScaNN (4-bit PQ) [Guo+, ICML 2020]

Library

- Implementations of algorithms
- Usually, a search function only
- By researchers, developers, etc

faiss

NMSLIB

hnswlib

ScaNN

Service (e.g., vector DB)

- Library + (handling metadata, serving, scaling, IO, CRUD, etc)
- Usually, by companies

Pinecone

Qdrant

Milvus

jina

Vald

Vertex AI Matching Engine

Weaviate

Three levels of technology

Algorithm
- Scientific paper
- Math
- Often, by researchers

Library
- Implementations of algorithms
- Usually, a search function only
- By researchers, developers, etc

Service (e.g., vector DB)
- Library + (handling metadata, serving, scaling, IO, CRUD, etc)
- Usually, by companies

Product Quantization + Inverted Index (PQ, IVFPQ) [Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

ScaNN (4-bit PQ) [Guo+, ICML 2020]

faiss

hnswlib

Pinecone

Qdrant

One library may implement multiple algorithms

☹ “I benchmarked faiss”

☺ “I benchmarked PQ in faiss”

Three levels of technology

One algorithm may be implemented in multiple libraries

- A
- Math
- Often, by re

Product Quantiza
Inverted Index (PQ, IV
[Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

ScaNN (4-bit PQ) [Guo+, ICML 2020]

- Usually, a search function only
- By researchers, developers, etc

faiss

NMSLIB

hnswlib

ScaNN

Service (e.g., vector DB)

- Library + (handling metadata, serving, scaling, IO, CRUD, etc)
- Usually, by companies

Pinecone

Qdrant

Milvus

jina

Vald

Vertex AI Matching Engine

Weaviate

Three levels of technology

Algorithm
- Scientific paper
- Math
- Often, by researchers

Library
- Implementations of algorithms
- Usually, a search function only
- By researchers, developers, etc

Service (e.g., vector DB)
- Library + (handling metadata, serving, scaling, IO, CRUD, etc)
- Usually, by companies

Product Quantization + Inverted Index (PQ, IVFPQ) [Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

ScaNN (4-bit PQ) [Guo+, ICML 2020]

faiss

ScaNN

Pinecone

Qdrant

Vald

Vertex AI Matching Engine

Weaviate

Often, one library = one algorithm

Three levels of technology

Algorithm

- Scientific paper
- Math
- Often, by researchers

, vector DB)

- ndling metadata,
- rving, scaling, IO, CRUD, etc)
- By researchers, developers, e… sually, by companies

One service may use some libraries

Product Quantization + Inverted Index (PQ, IVFPQ) [Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

… or re-implement algorithms from scratch (e.g., by Go)

faiss

NMSLIB

hnswlib

ScaNN

Pinecone

Milvus

Vald

Weaviate

Qdrant

jina

Vertex AI Matching Engine

Three levels of technology

Algorithm
- Scientific paper
- Math
- Often, by researchers

Library
- Implementations of algorithms
- Usually, a search function only
- By researchers, developers, etc

Service (e.g., vector DB)
- Library + (handling metadata, serving, scaling, IO, CRUD, etc)
- Usually, by companies

Product Quantization + Inverted Index (PQ, IVFPQ) [Jégou+, TPAMI 2011]

Hierarchical Navigable Small World (HNSW) [Malkov+, TPAMI 2019]

ScaNN (4-bit PQ) [Guo+, ICML 2020]

faiss

Pinecone

This talk mainly focuses algorithms

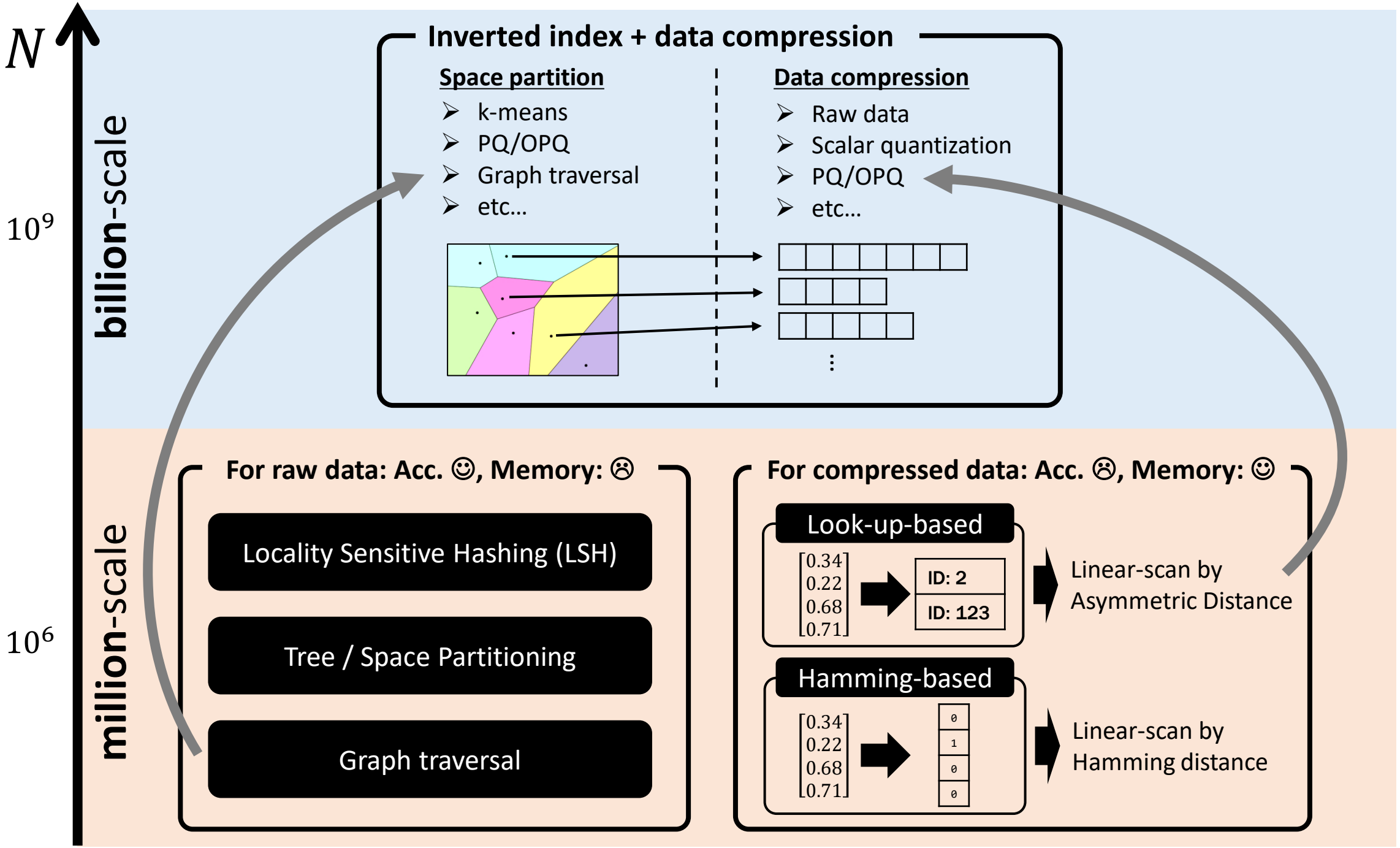
hnswlib

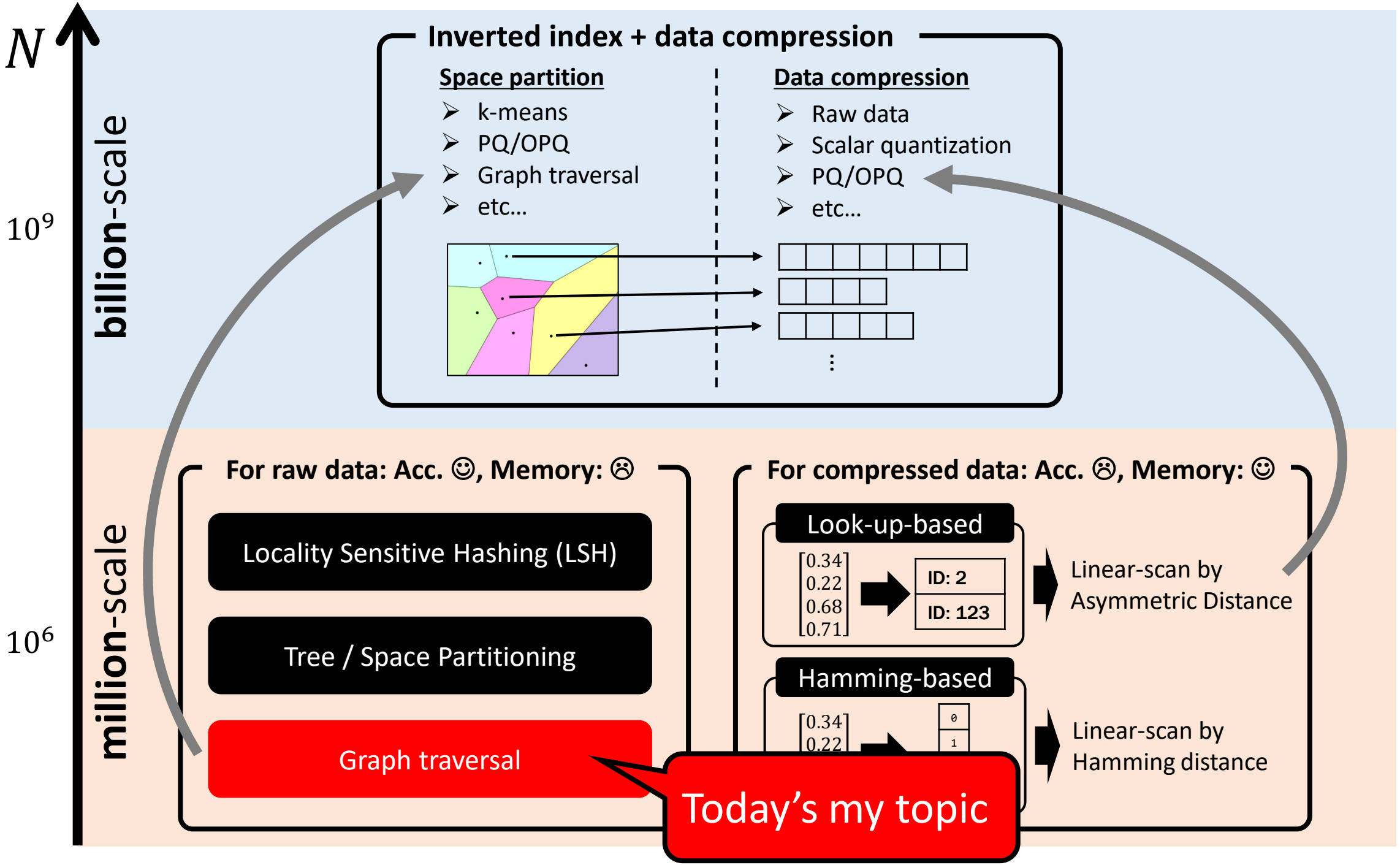
Vald

Vertex AI Matching Engine

ScaNN

Weaviate


N
billion-scale
10^9
million-scale
10^6
Inverted index + data compression
Space partition
k-means
PQ/OPQ
Graph traversal
etc…
Data compression
Raw data
Scalar quantization
PQ/OPQ
etc…
For raw data: Acc. ☺, Memory: ☹
Locality Sensitive Hashing (LSH)
Tree / Space Partitioning
Graph traversal
For compressed data: Acc. ☹, Memory: ☺
Look-up-based
0.34
0.22
0.68
0.71
ID: 2
ID: 123
Linear-scan by
Asymmetric Distance
Hamming-based
0.34
0.22
0.68
0.71
0
1
0
0
Linear-scan by
Hamming distance


N
billion-scale
10^9
million-scale
10^6
Inverted index + data compression
Space partition
k-means
PQ/OPQ
Graph traversal
etc…
Data compression
Raw data
Scalar quantization
PQ/OPQ
etc…
For raw data: Acc. ☺, Memory: ☹
Locality Sensitive Hashing (LSH)
Tree / Space Partitioning
Graph traversal
For compressed data: Acc. ☹, Memory: ☺
Look-up-based
0.34
0.22
0.68
0.71
ID: 2
ID: 123
Linear-scan by
Asymmetric Distance
Hamming-based
0.34
0.22
0
1
Linear-scan by
Hamming distance
Today's my topic

N

billion-scale

10^9

million-scale

10^6

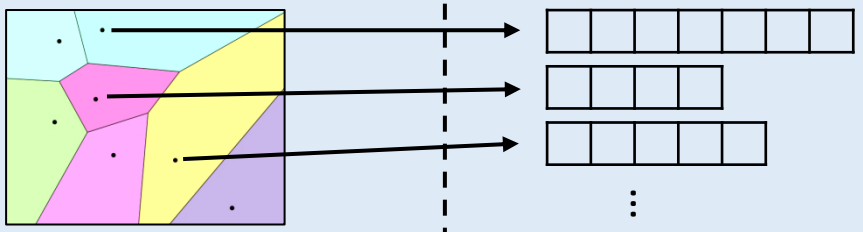
Inverted index + data compression

Space partition

- k-means
- PQ/OPQ
- Graph traversal
- etc…

Data compression

- Raw data
- Scalar quantization
- PQ/OPQ
- etc…



For raw data: Acc. ☺, Memory: ☹

- Locality Sensitive Hashing (LSH)
- Tree / Space Partitioning
- Graph traversal

For compressed data: Ac

Look-up-based

$\begin{bmatrix}0.34\\0.22\\0.68\\0.71\end{bmatrix}$ → ID: 2 | ID: 123

Hamming-based

$\begin{bmatrix}0.34\\0.22\end{bmatrix}$ → 0 | 1

Today's my topic

See my previous tutorial at CVPR20

https://speakerdeck.com/matsui_528/cvpr20-tutorial-billion-scale-approximate-nearest-neighbor-search

N

billion-scale

10^9

million-scale

10^6

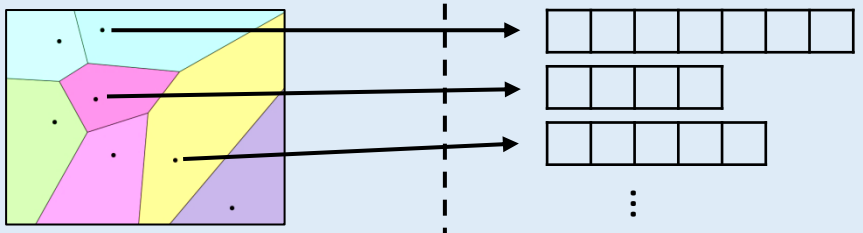
Inverted index + data compression

Space partition

- k-means
- PQ/OPQ
- Graph traversal
- etc…

Data compression

- Raw data
- Scalar quantization
- PQ/OPQ
- etc…



See Martin's next presentation!

For raw data: Acc. ☺, Memory: ☹

- Locality Sensitive Hashing (LSH)
- Tree / Space Partitioning
- Graph traversal

For compressed data: Ac

Look-up-based

$\begin{bmatrix}0.34\\0.22\\0.68\\0.71\end{bmatrix}$ → ID: 2 | ID: 123

Hamming-based

$\begin{bmatrix}0.34\\0.22\end{bmatrix}$ → 0 | 1

Today's my topic

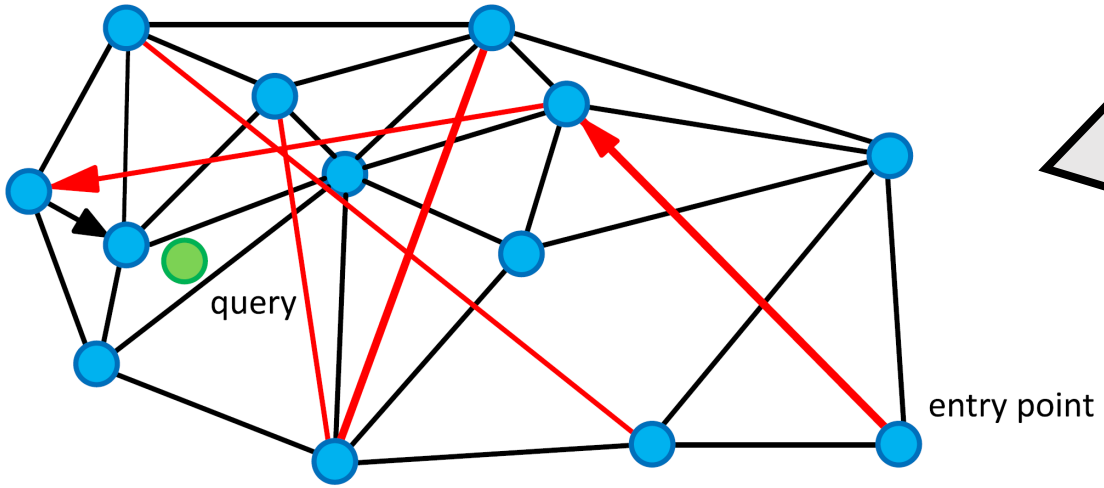
See my previous tutorial at CVPR20

https://speakerdeck.com/matsui_528/cvpr20-tutorial-billion-scale-approximate-nearest-neighbor-search

- Background
- Graph-based search
  - ✓ Basic (construction and search)
  - ✓ Observation
  - ✓ Properties
- Representative works
  - ✓ HNSW, NSG, NGT, Vamana
- Discussion

Graph search

- De facto standard if all data can be loaded on memory
- Fast and accurate for real-world data
- Important for billion-scale situation as well
  - ✓ Graph-search is a building block for billion-scale systems



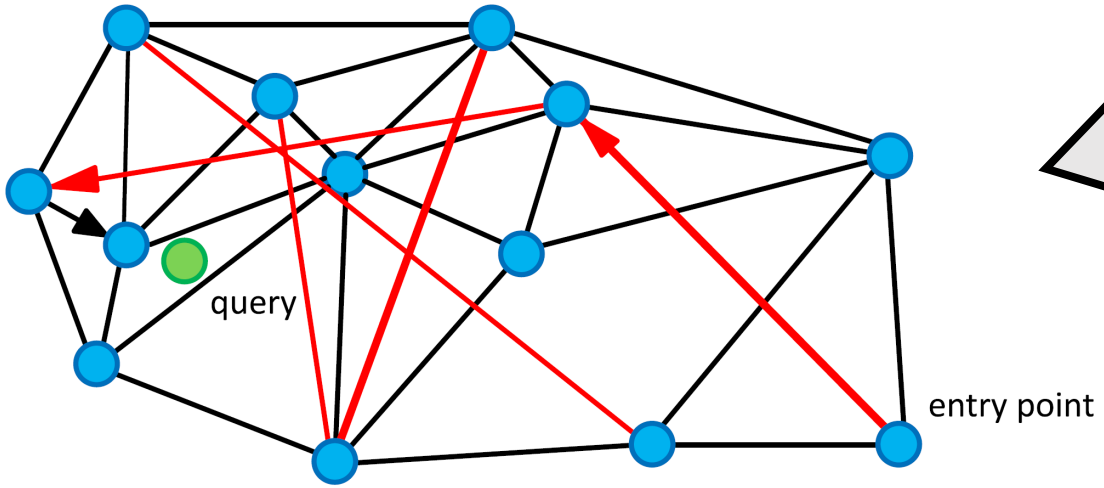
- Traverse graph towards the query
- Seems intuitive, but not so much easy to understand
- Review the algorithm carefully

Images are from [Malkov+, Information Systems, 2013]

Graph search

- De facto standard if all data can be loaded on memory
- Fast and accurate for real-world data
- Important for billion-scale situation as well
  - ✓ Graph-search is a building block for billion-scale systems

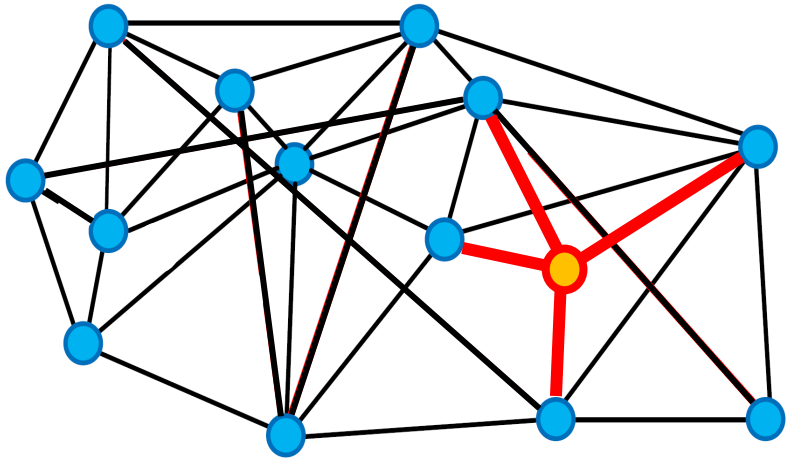
The purpose of this tutorial is to make graph search **not a black box**

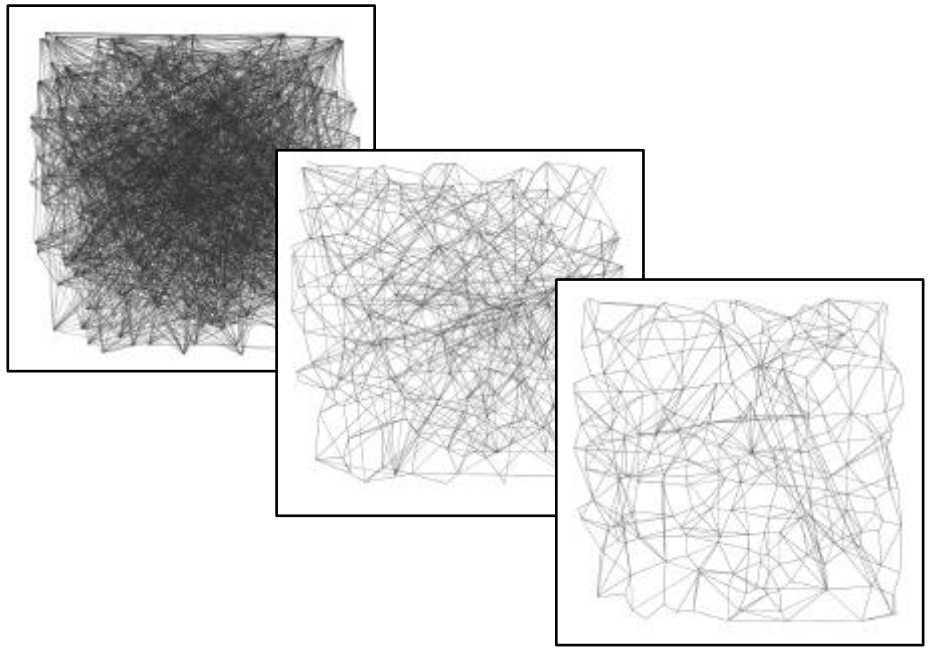


- Traverse graph towards the query
- Seems intuitive, but not so much easy to understand
- Review the algorithm carefully

Images are from [Malkov+, Information Systems, 2013]

Images are from [Malkov+, Information Systems, 2013] and [Subramanya+, NeruIPS 2019]





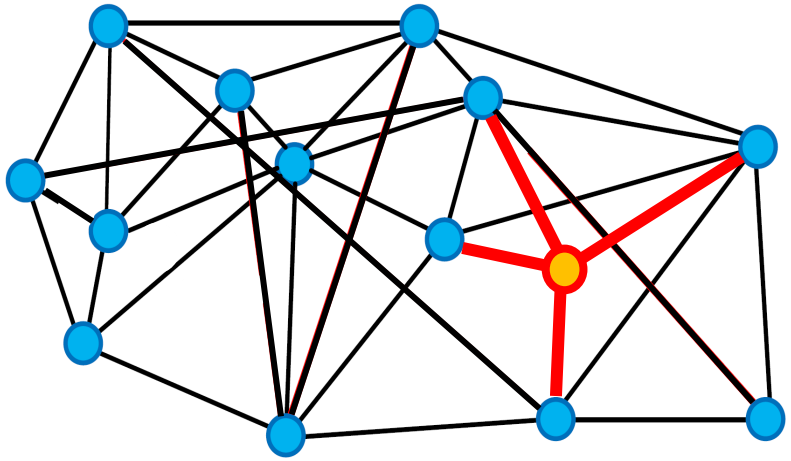
Increment approach

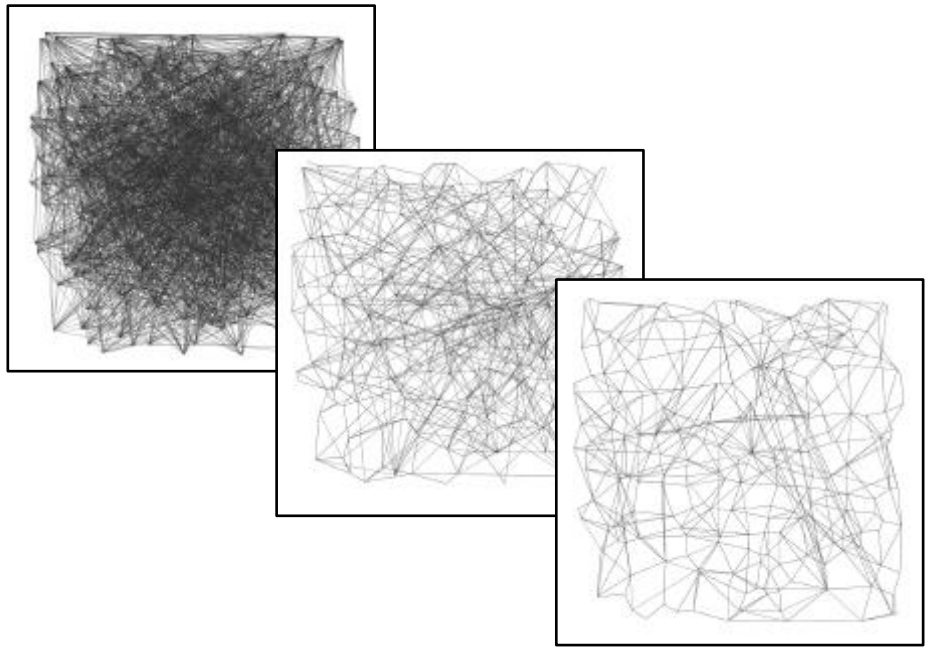
- Add a new item to the current graph incrementally

Refinement approach

- Iteratively refine an initial graph

Images are from [Malkov+, Information Systems, 2013] and [Subramanya+, NeruIPS 2019]





Increment approach

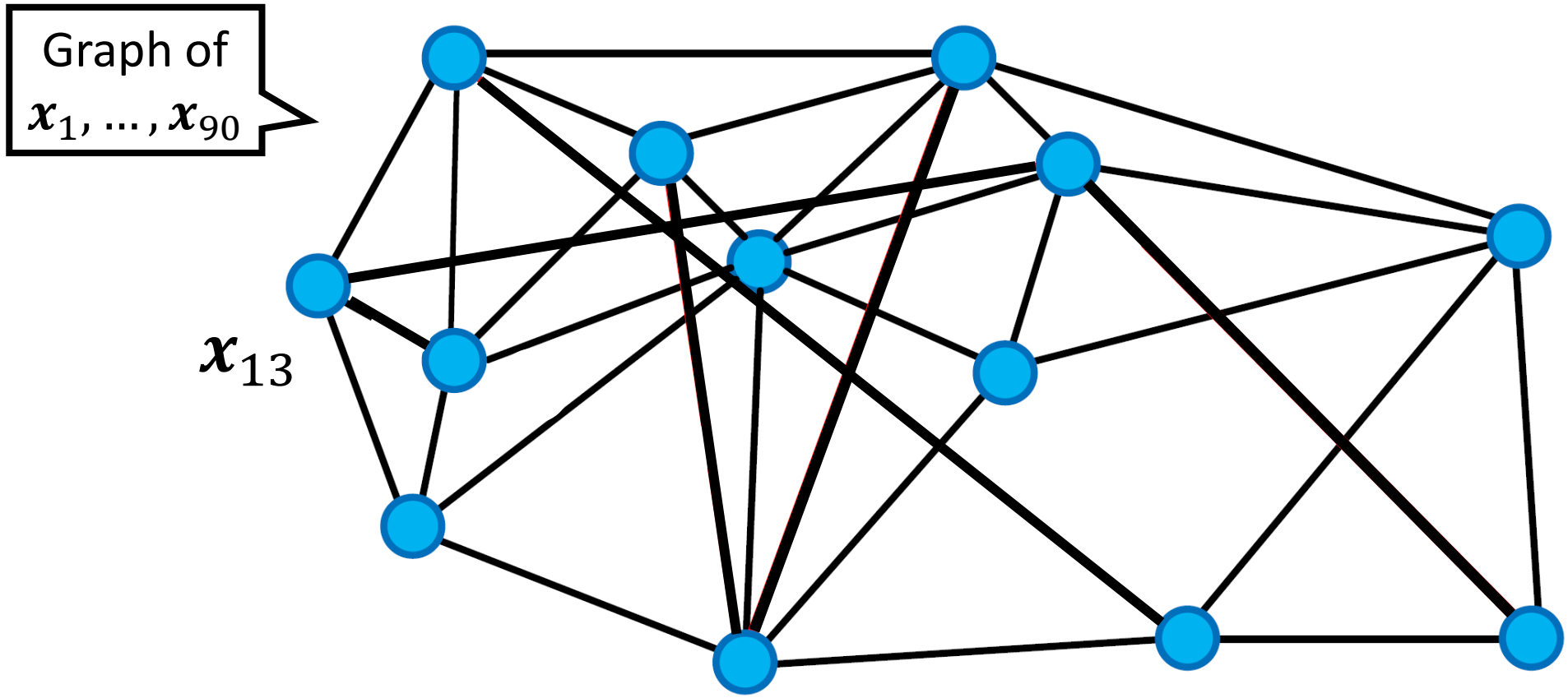
- Add a new item to the current graph incrementally

Refinement approach

- Iteratively refine an initial graph

Construction: incremental approach

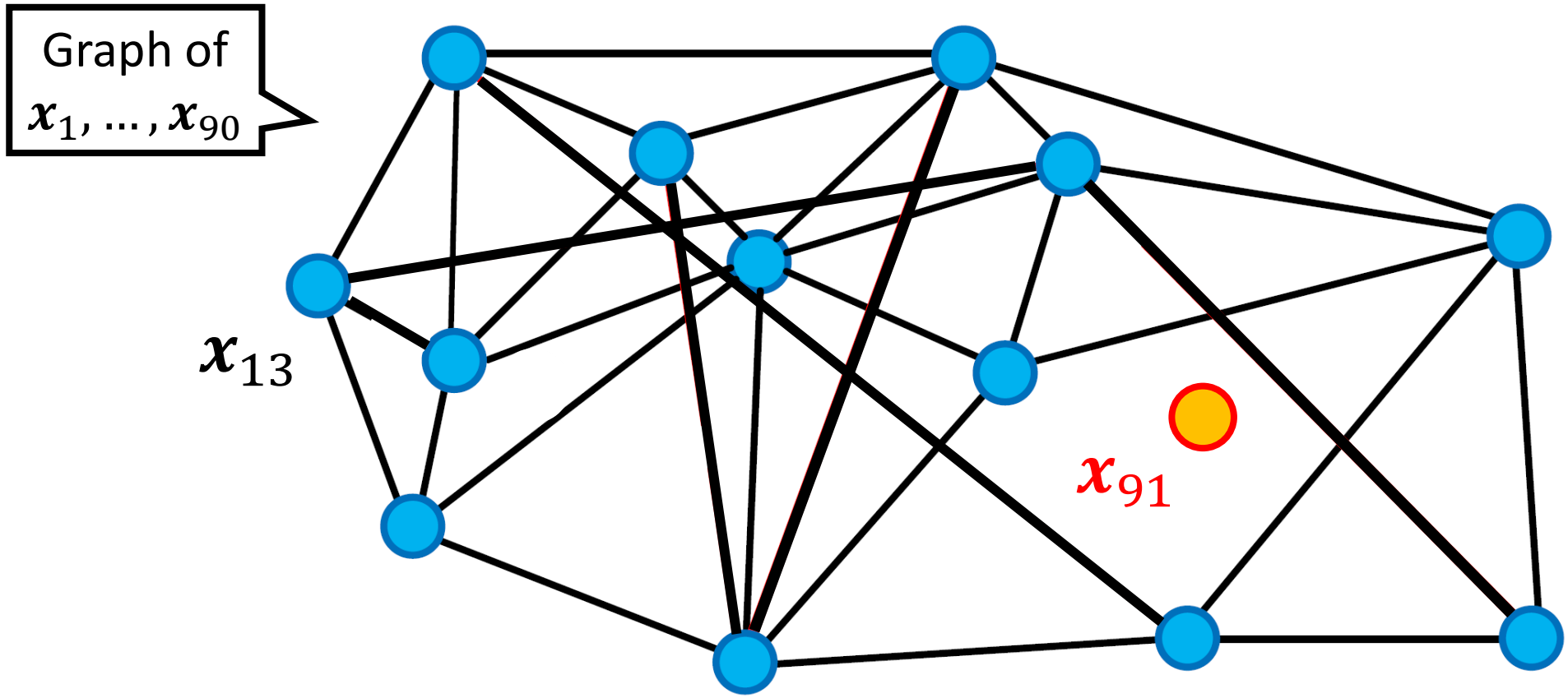
Images are from [Malkov+, Information Systems, 2013]



- Each node is a database vector

Construction: incremental approach

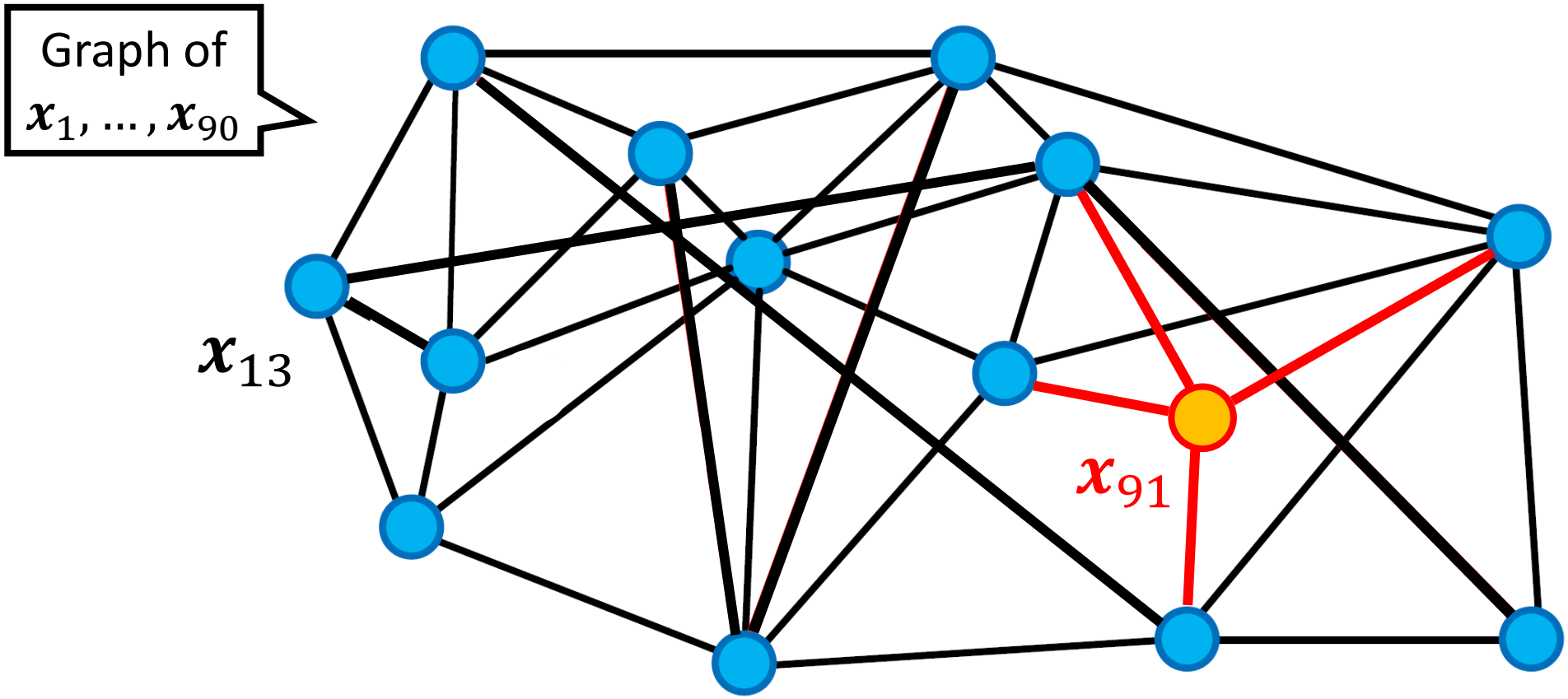
Images are from [Malkov+, Information Systems, 2013]



- Each node is a database vector
- Given a new database vector,

Construction: incremental approach

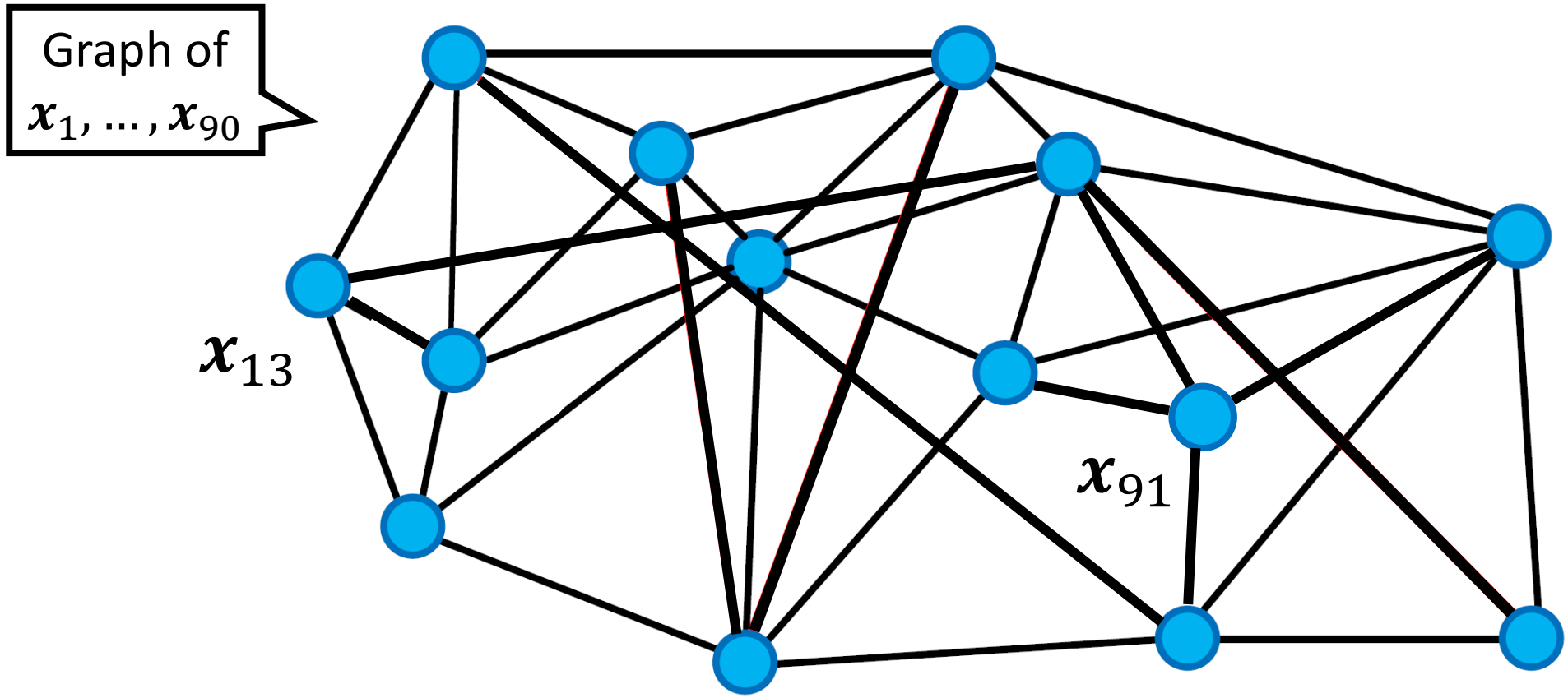
Images are from [Malkov+, Information Systems, 2013]



- Each node is a database vector
- Given a new database vector, create new edges to neighbors

Construction: incremental approach

Images are from [Malkov+, Information Systems, 2013]



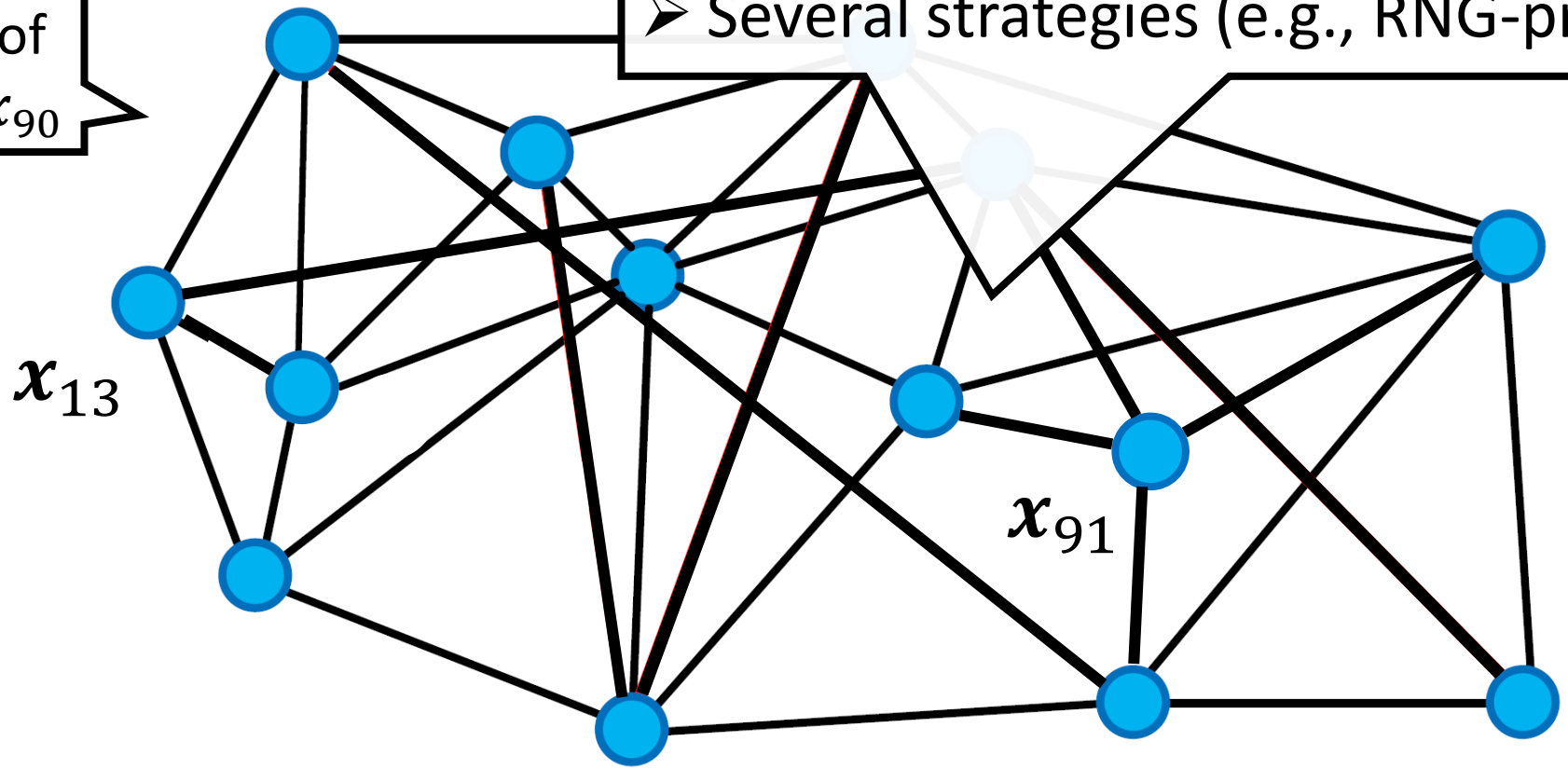
- Each node is a database vector
- Given a new database vector, create new edges to neighbors

Construction: incremental approach

Images are from [Malkov+, Information Systems, 2013]

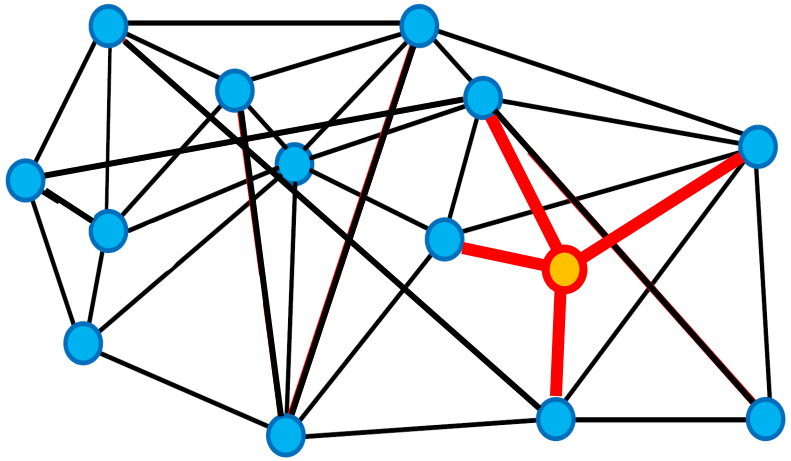
- Prune edges if some node have too many edges
- Several strategies (e.g., RNG-pruning)

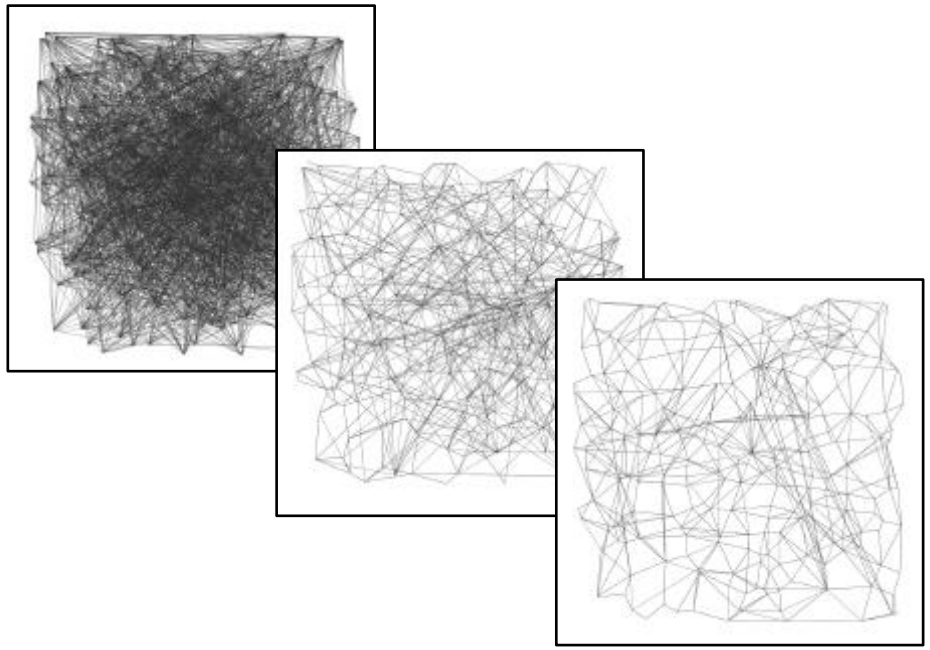
Graph of $x_1, \ldots, x_{90}$



- Each node is a database vector
- Given a new database vector, create new edges to neighbors

Images are from [Malkov+, Information Systems, 2013] and [Subramanya+, NeruIPS 2019]





Increment approach

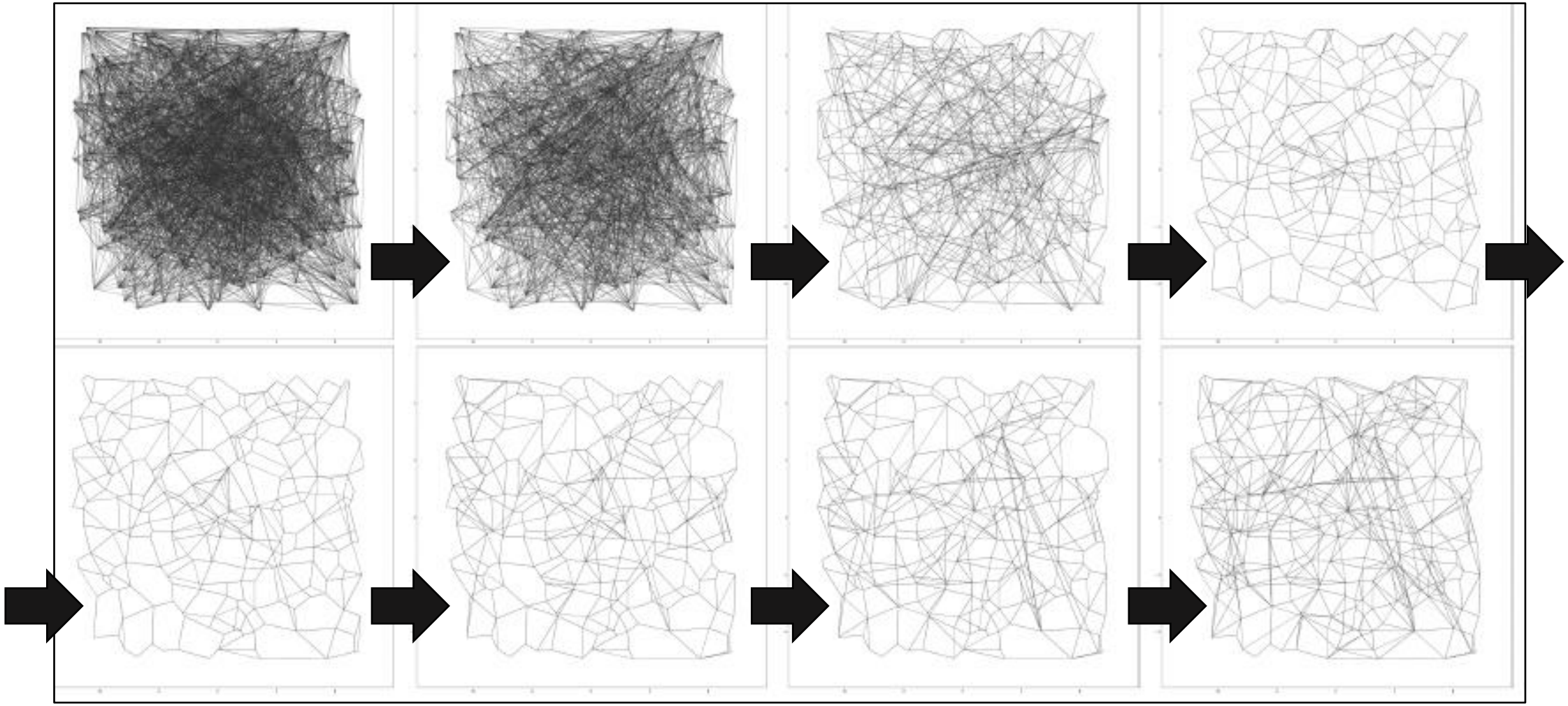
- Add a new item to the current graph incrementally

Refinement approach

- Iteratively refine an initial graph

Construction: refinement approach

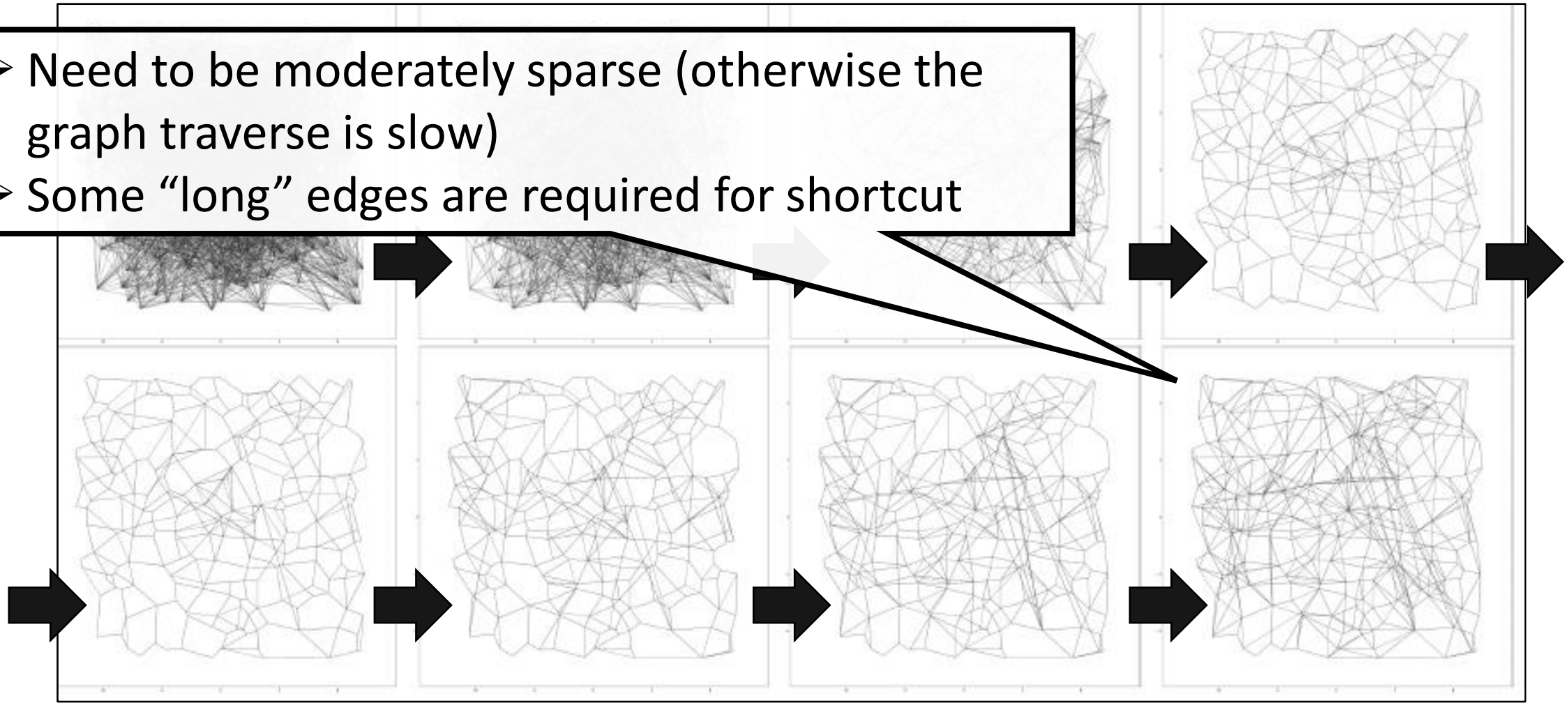
Images are from [Subramanya+, NeruIPS 2019]



- Create an initial graph (e.g., random graph or approx. kNN graph)
- Refine it iteratively (pruning/adding edges)

Construction: refinement approach

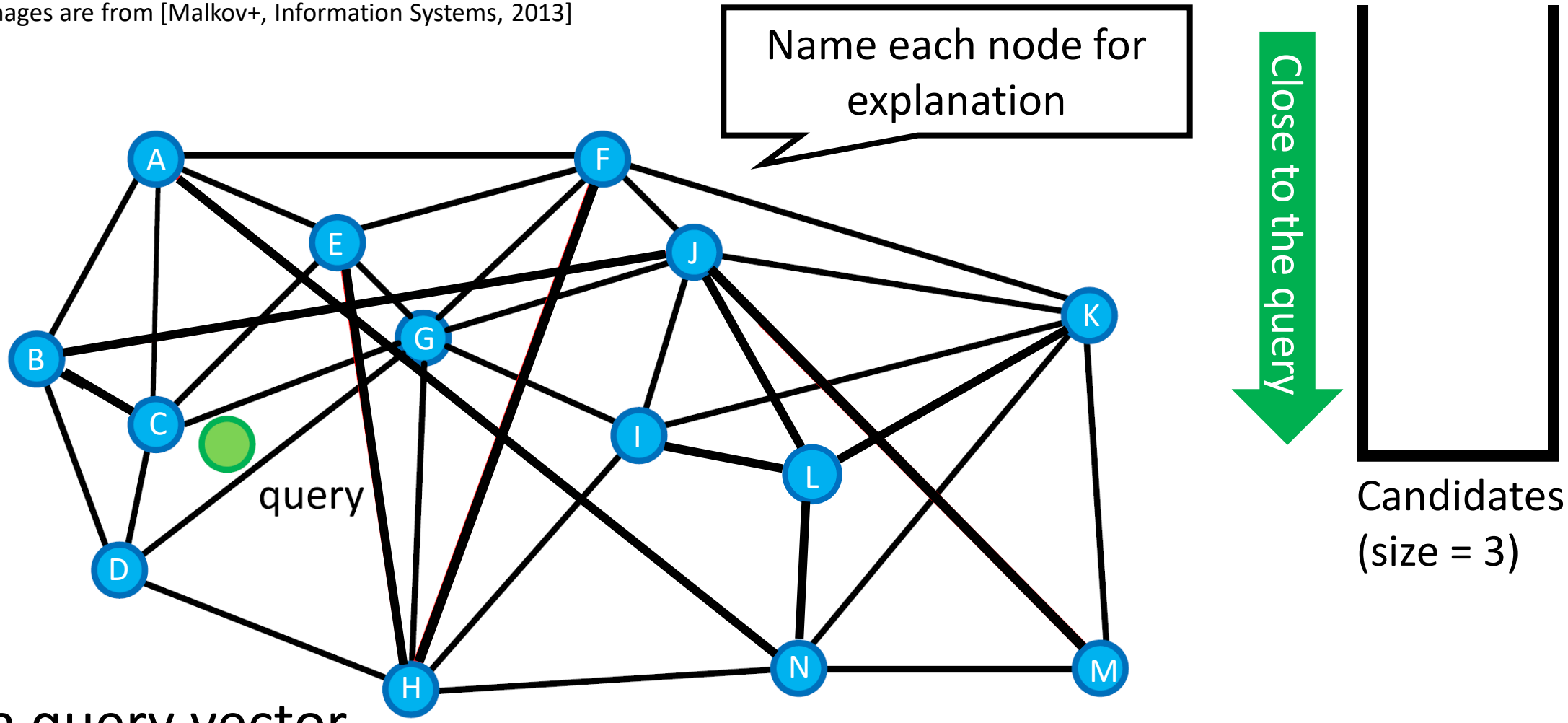
Images are from [Subramanya+, NeruIPS 2019]



- Create an initial graph (e.g., random graph or approx. kNN graph)
- Refine it iteratively (pruning/adding edges)

Search

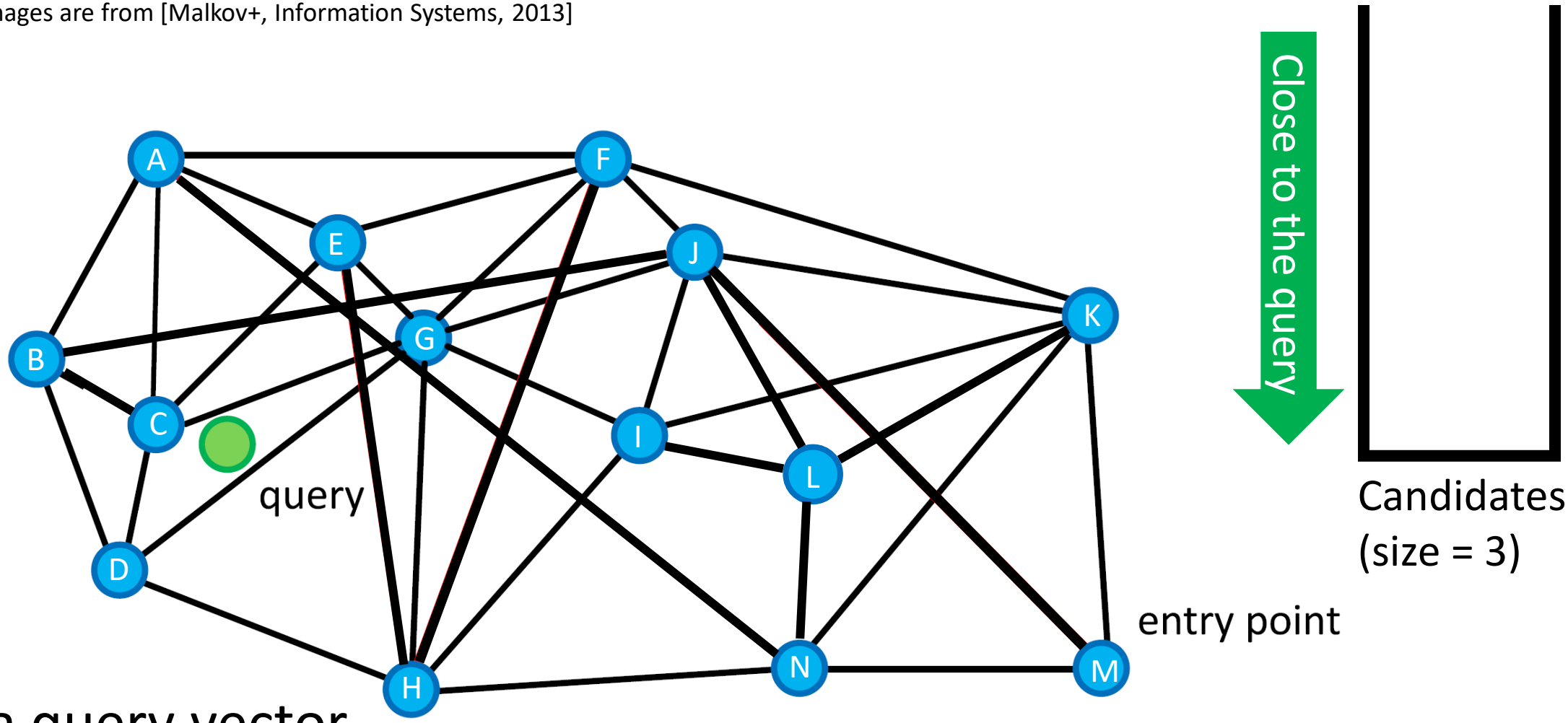
Images are from [Malkov+, Information Systems, 2013]



- Given a query vector

Search

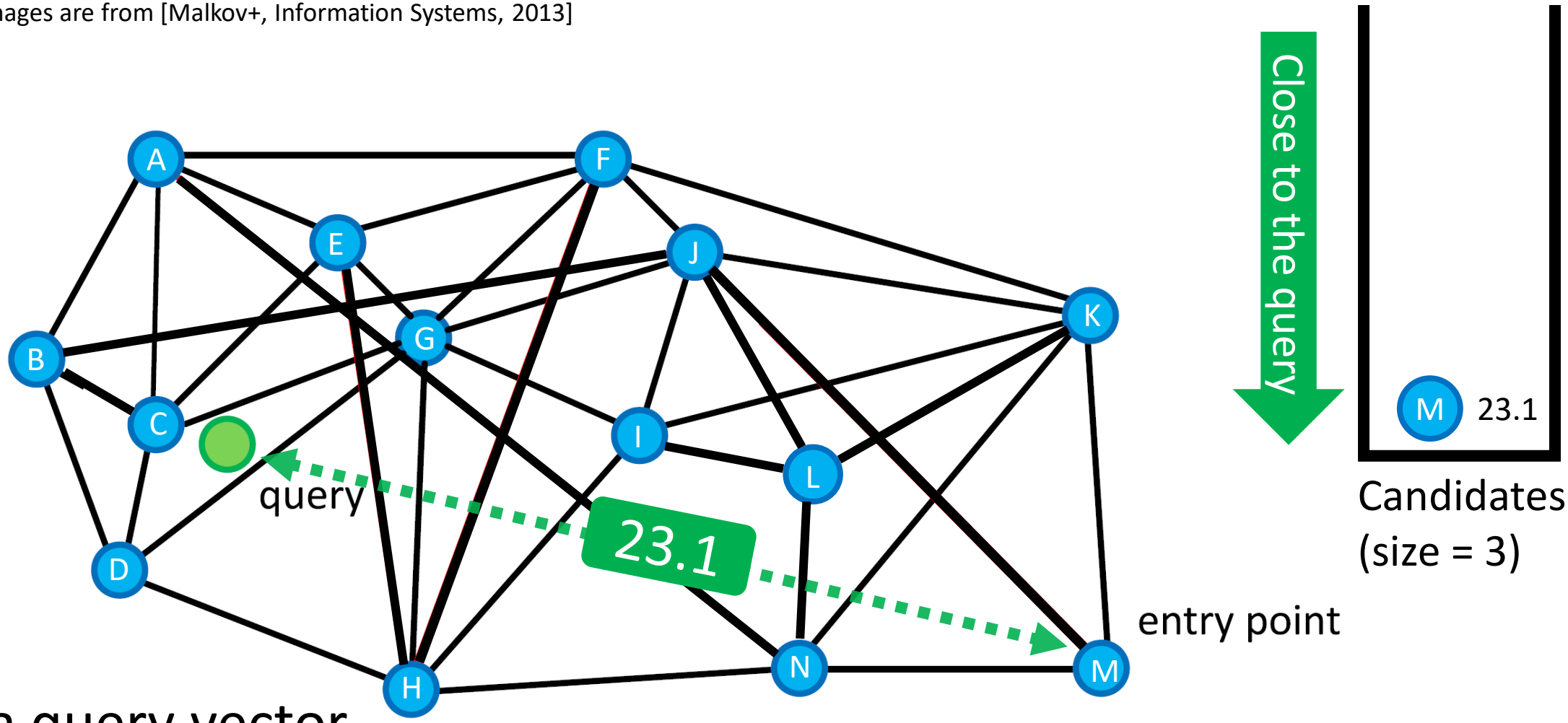
Images are from [Malkov+, Information Systems, 2013]



- Given a query vector
- Start from an entry point (e.g., M)

Search

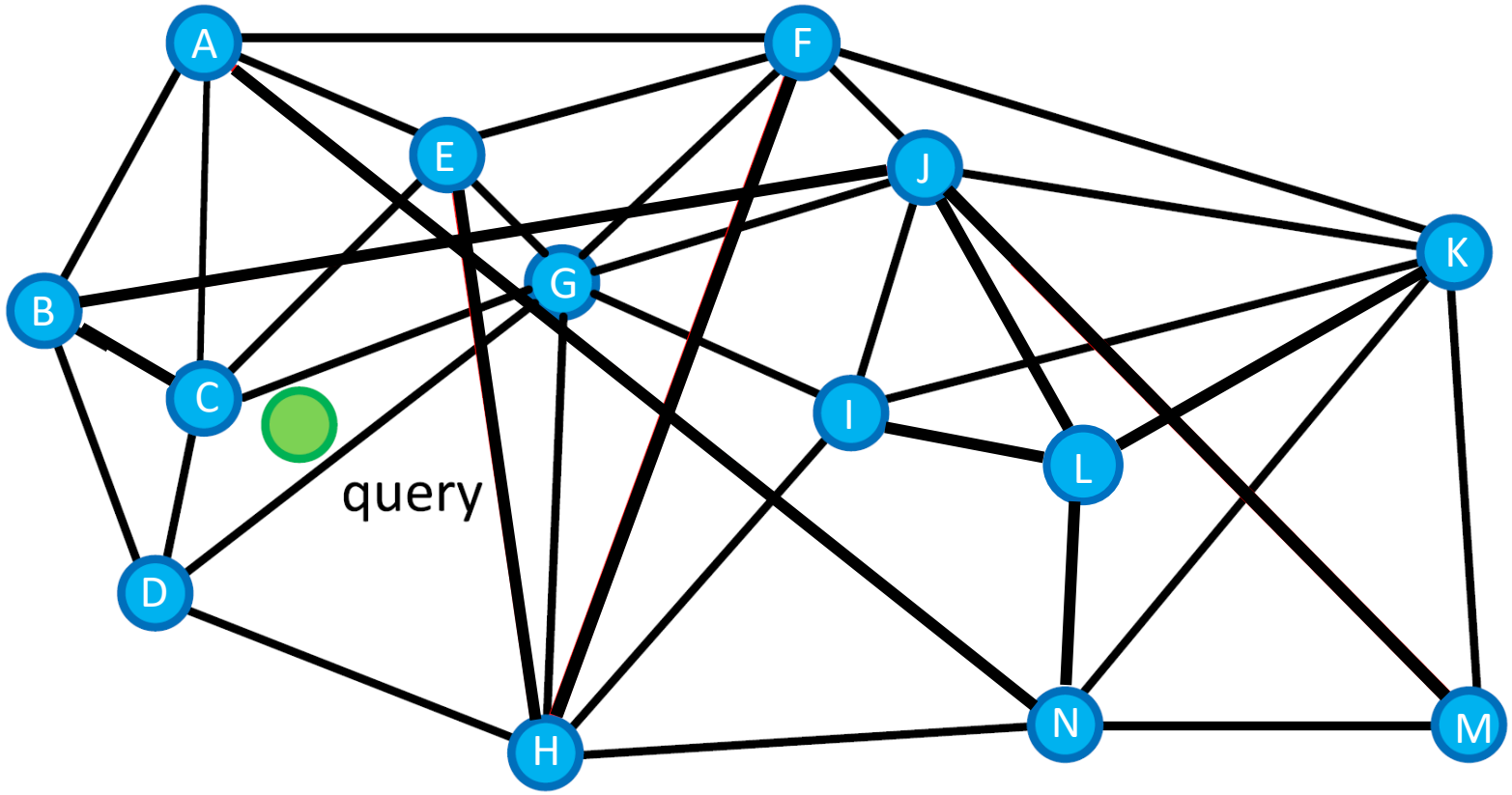
Images are from [Malkov+, Information Systems, 2013]

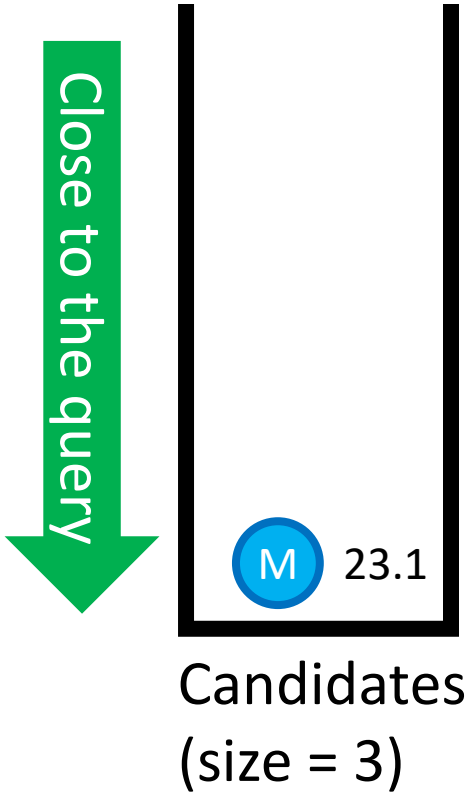


- Given a query vector
- Start from an entry point (e.g., M). Record the distance to q.

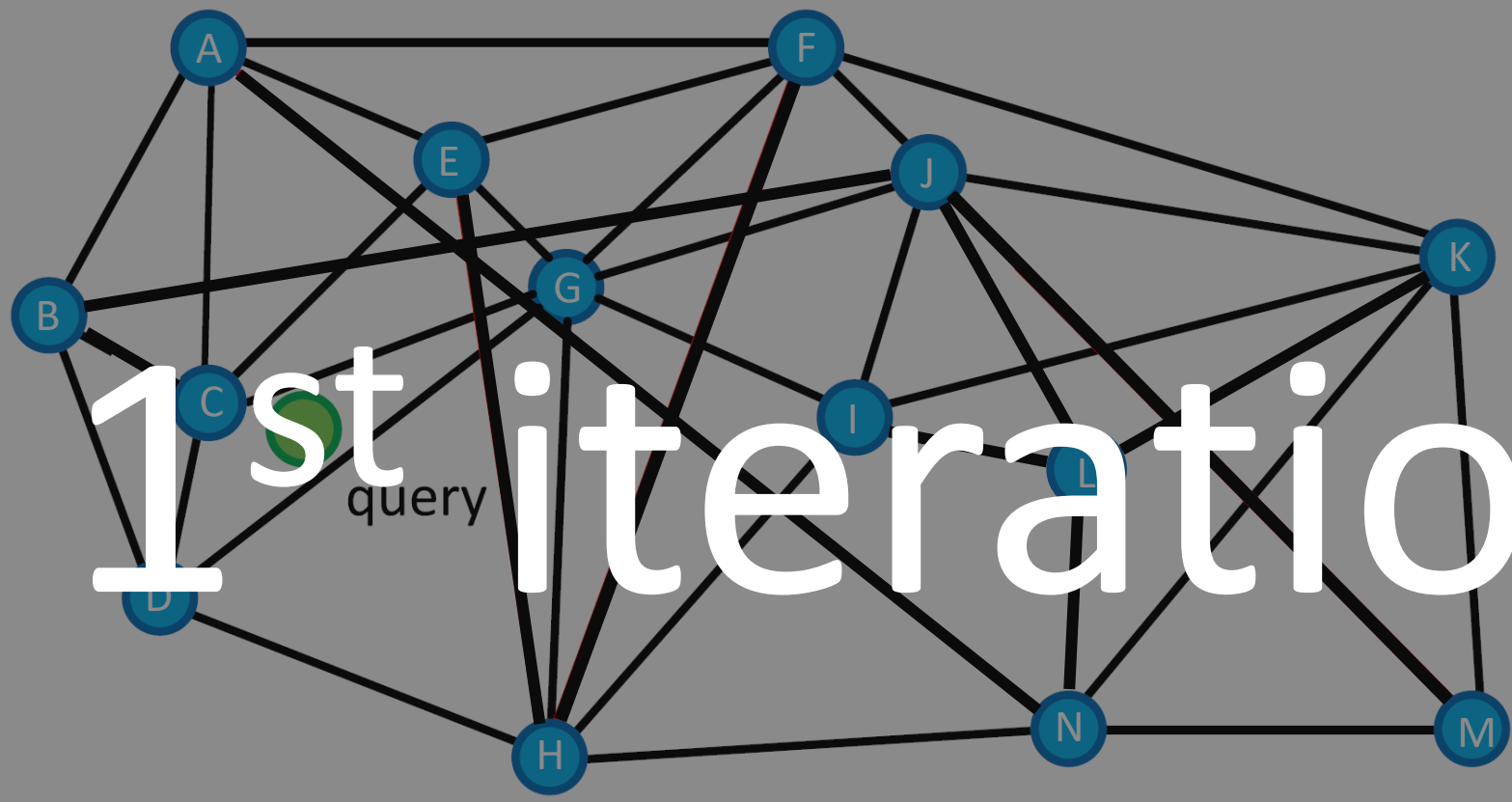
Search

Images are from [Malkov+, Information Systems, 2013]



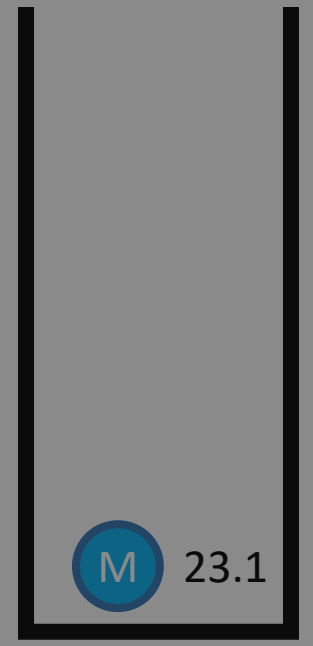


Images are from [Malkov+, Information Systems, 2013]



1st iteration

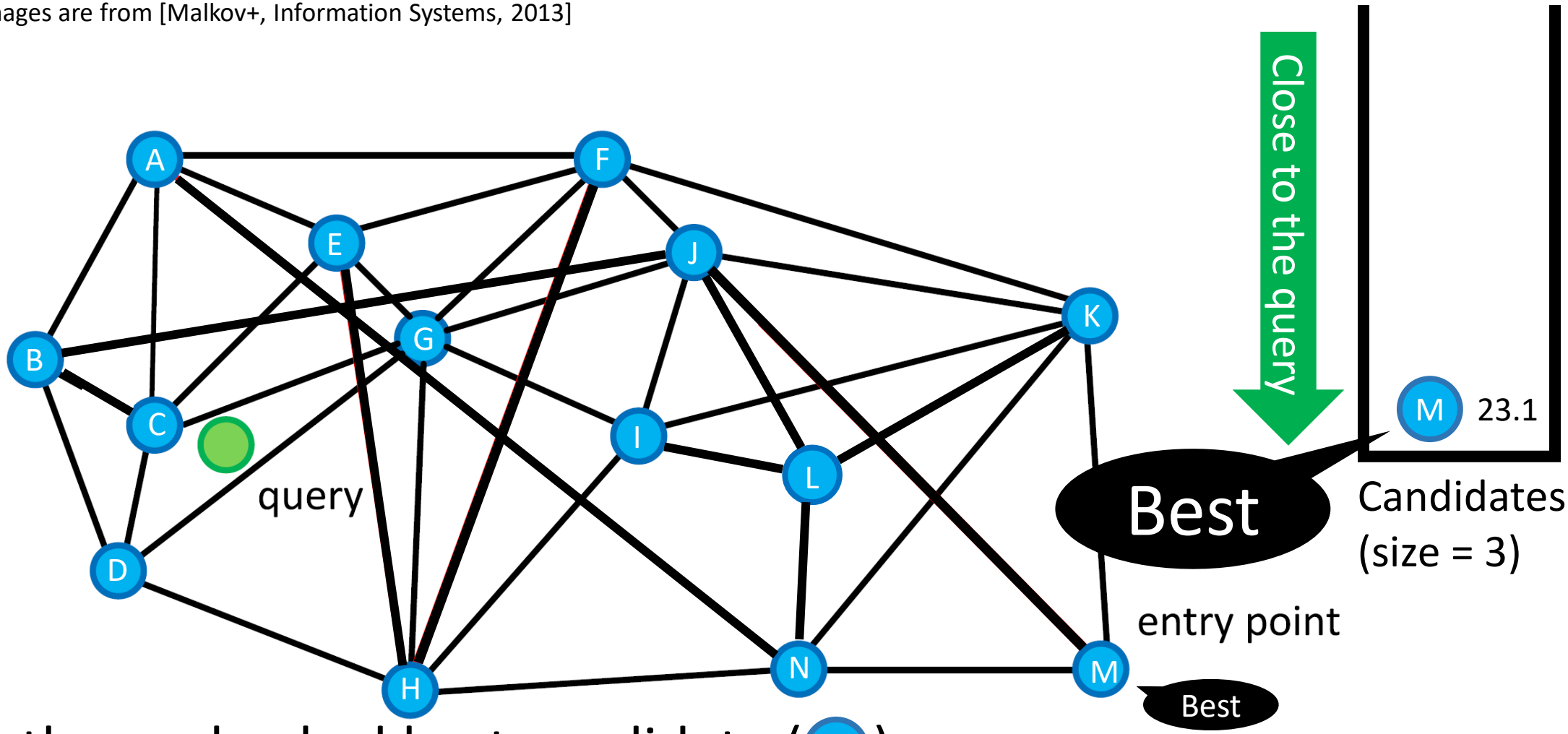
Close to the query



Candidates
(size = 3)

Search

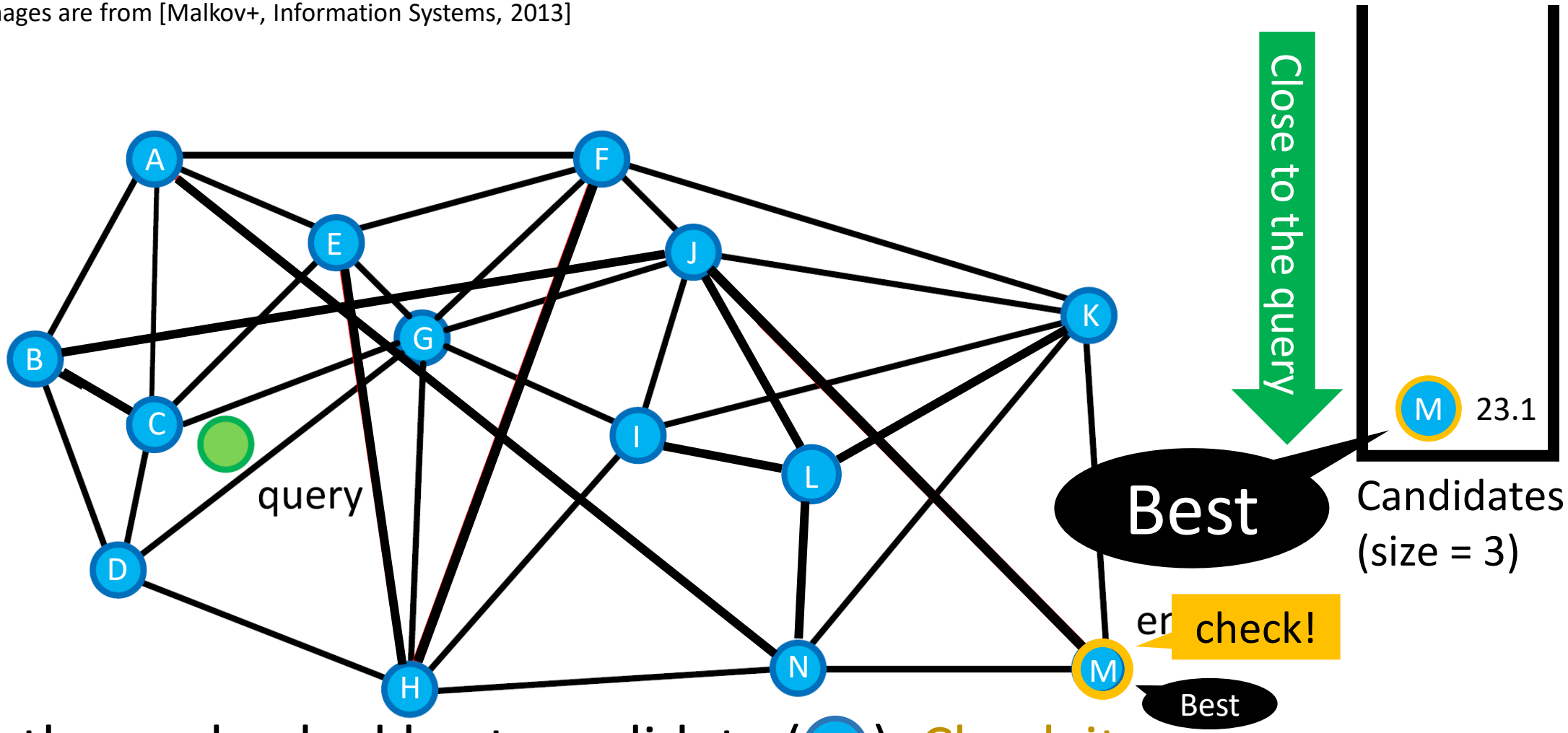
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (M)

Search

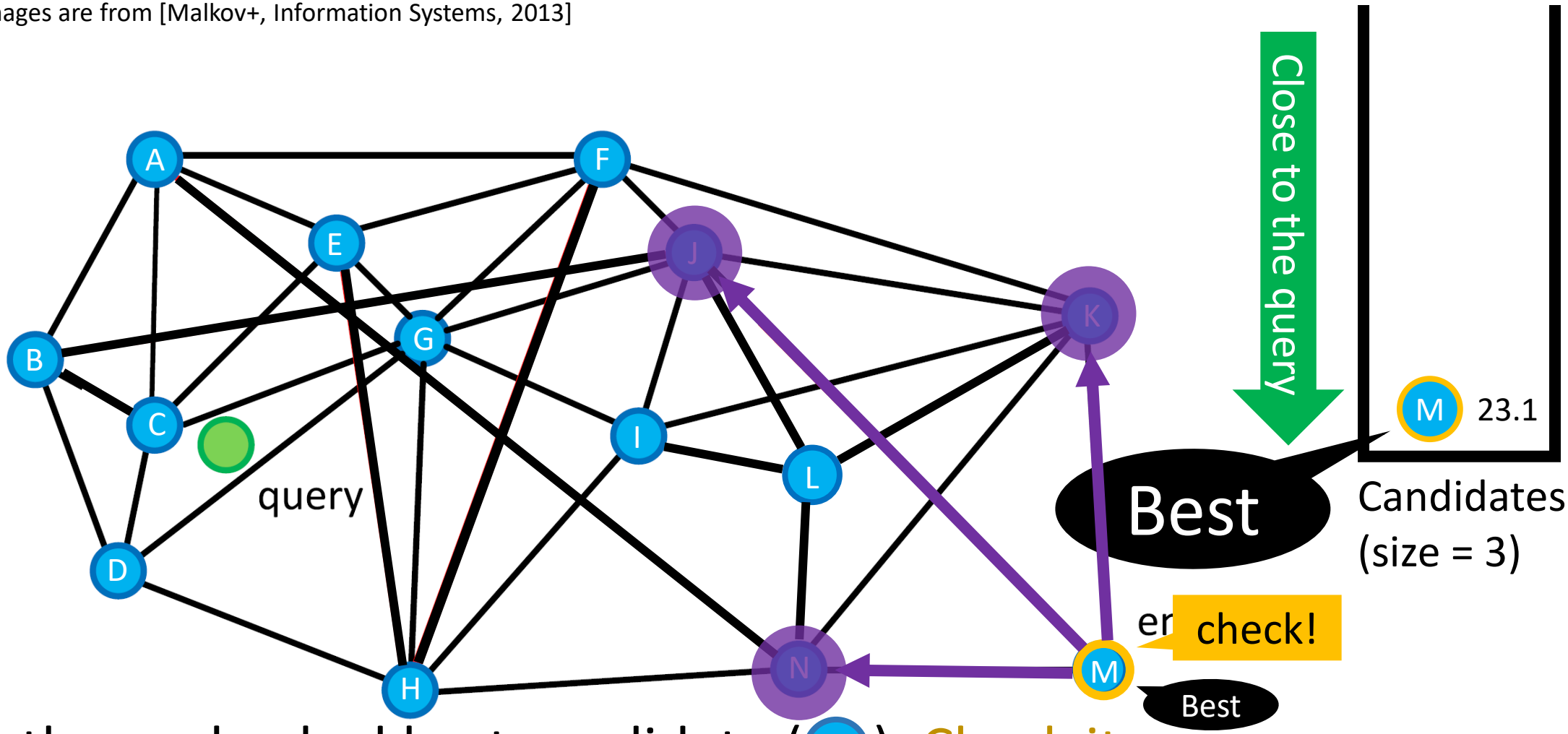
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (M). Check it.

Search

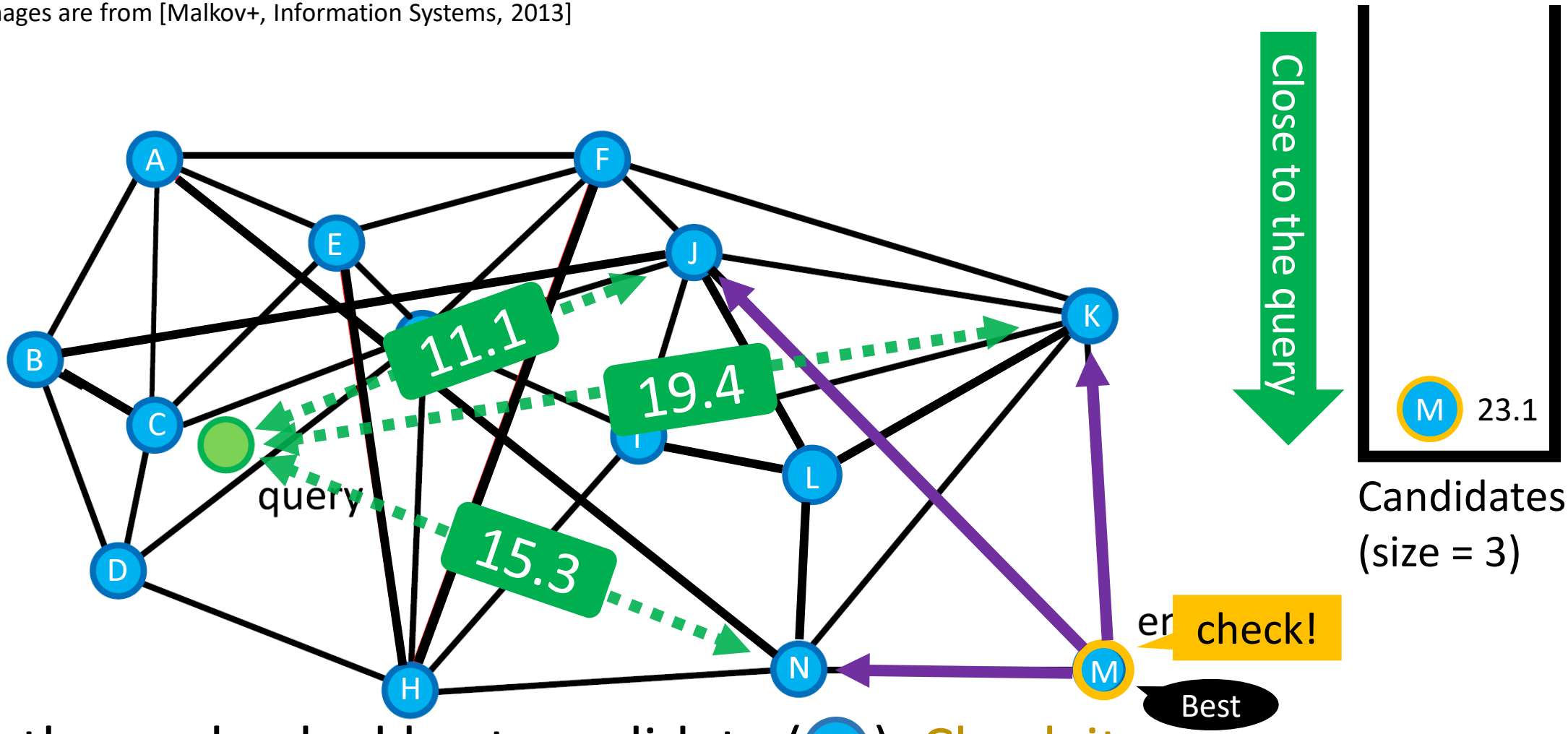
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.

Search

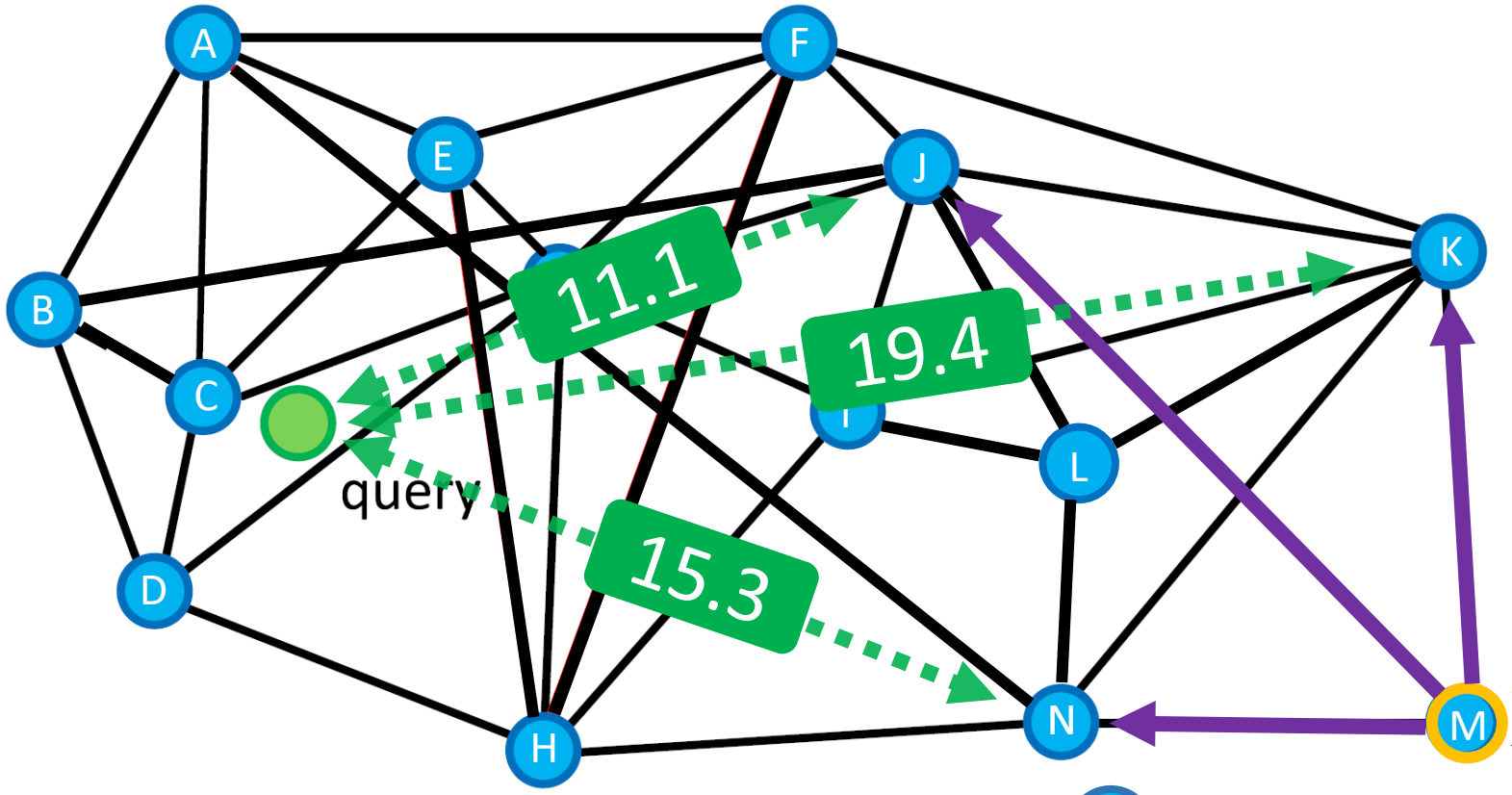
Images are from [Malkov+, Information Systems, 2013]

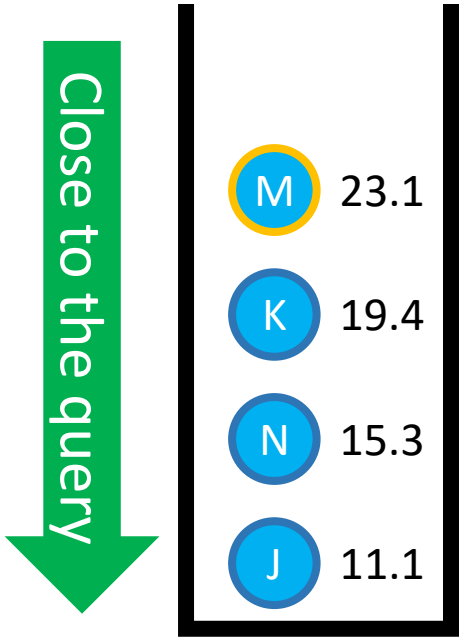


- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.
- Record the distances to q.

Search

Images are from [Malkov+, Information Systems, 2013]

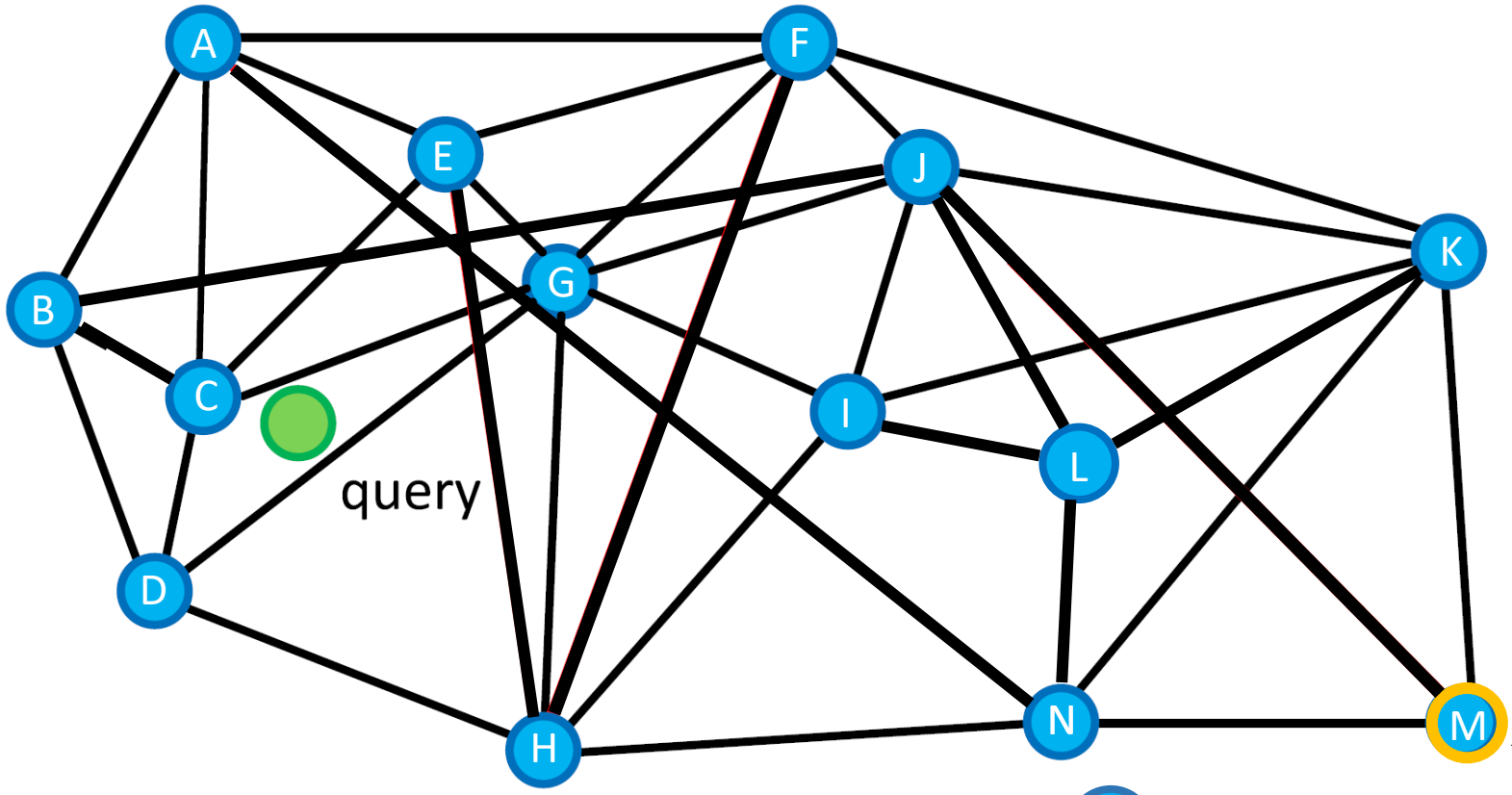


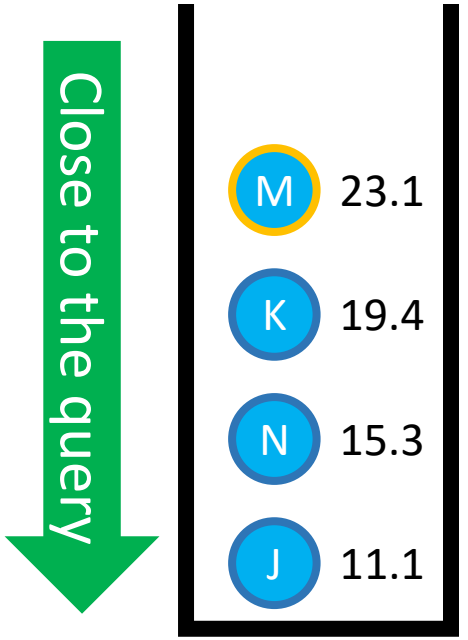


- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.
- Record the distances to q.

Search

Images are from [Malkov+, Information Systems, 2013]

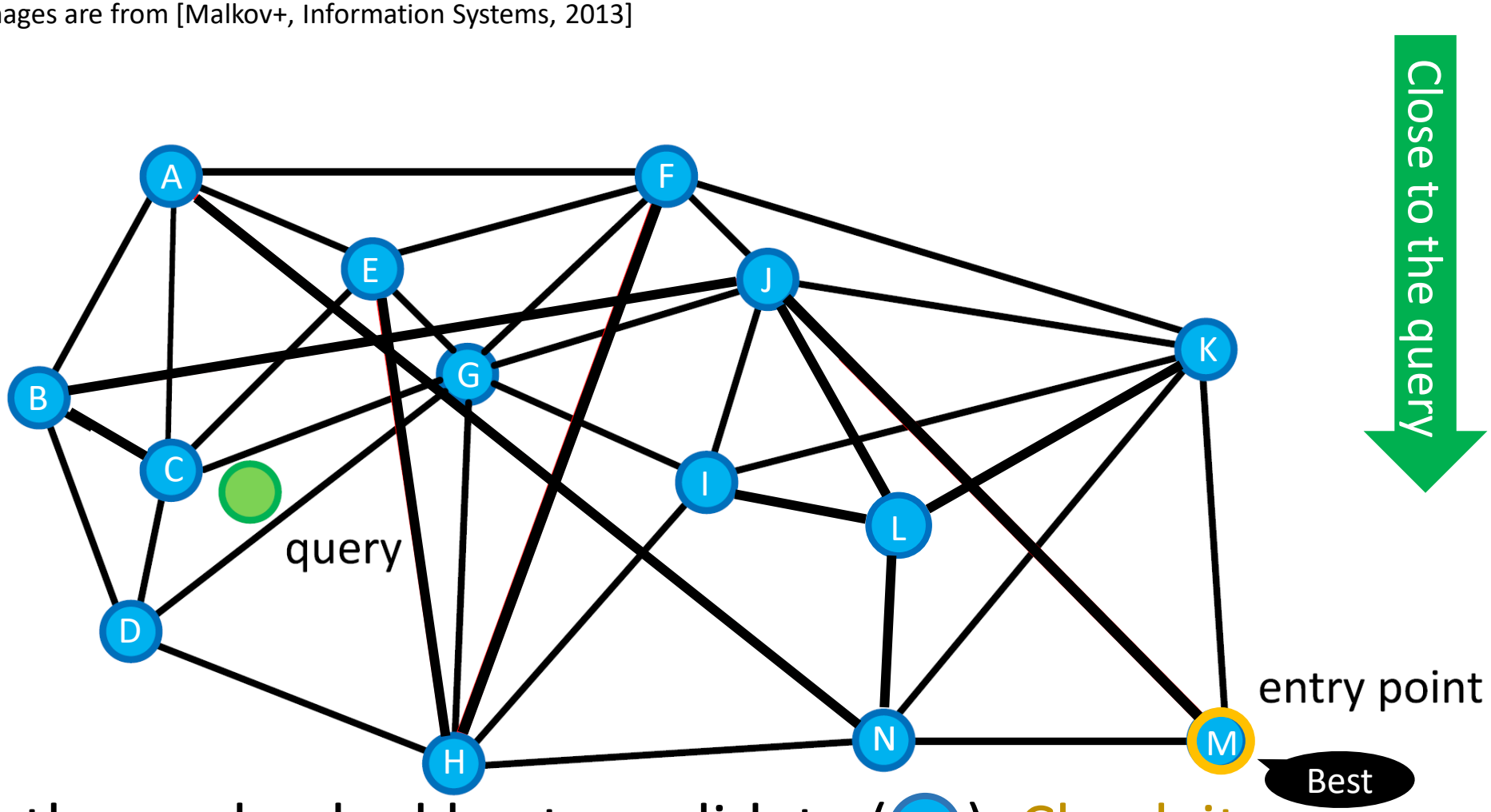




- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.
- Record the distances to q.

Search

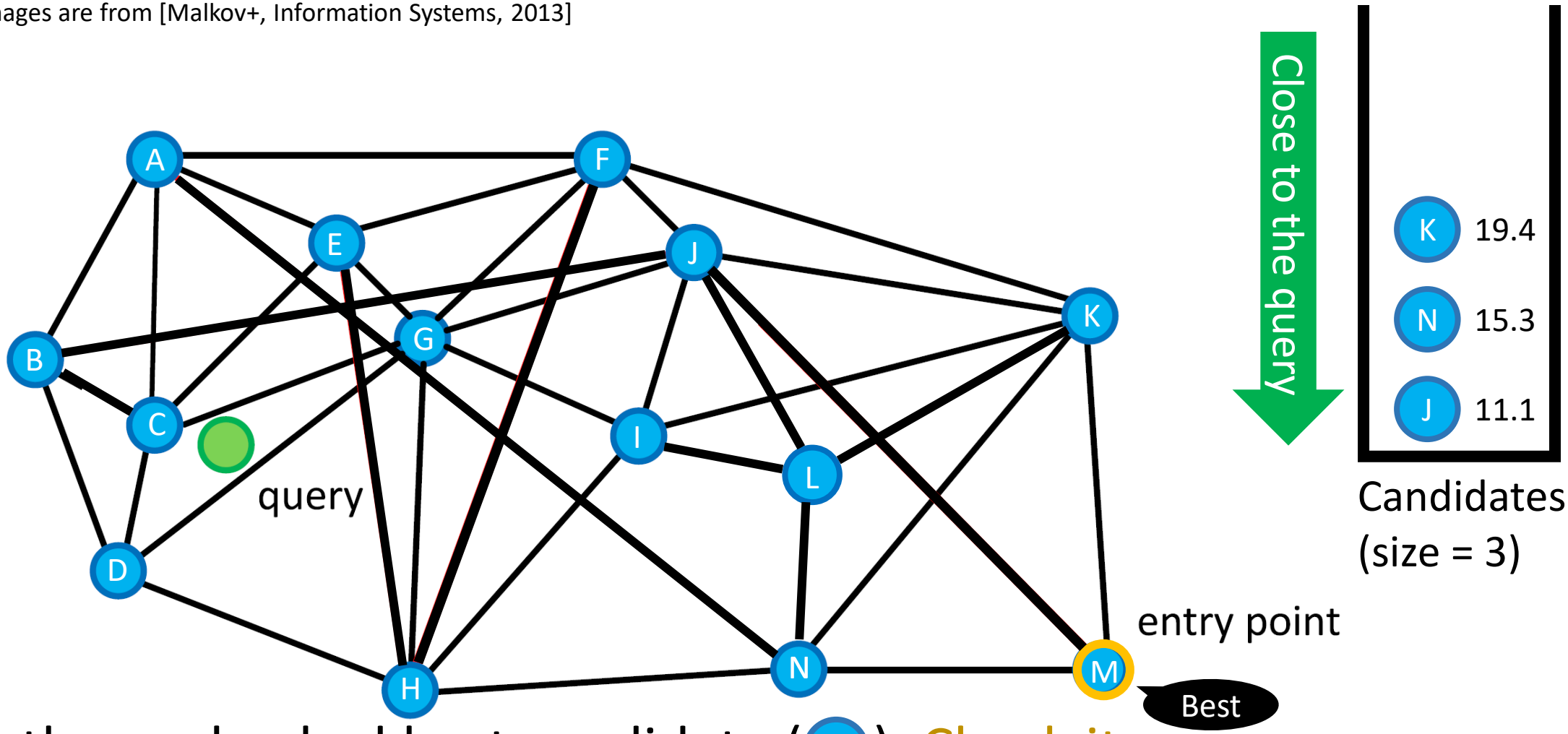
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

Search

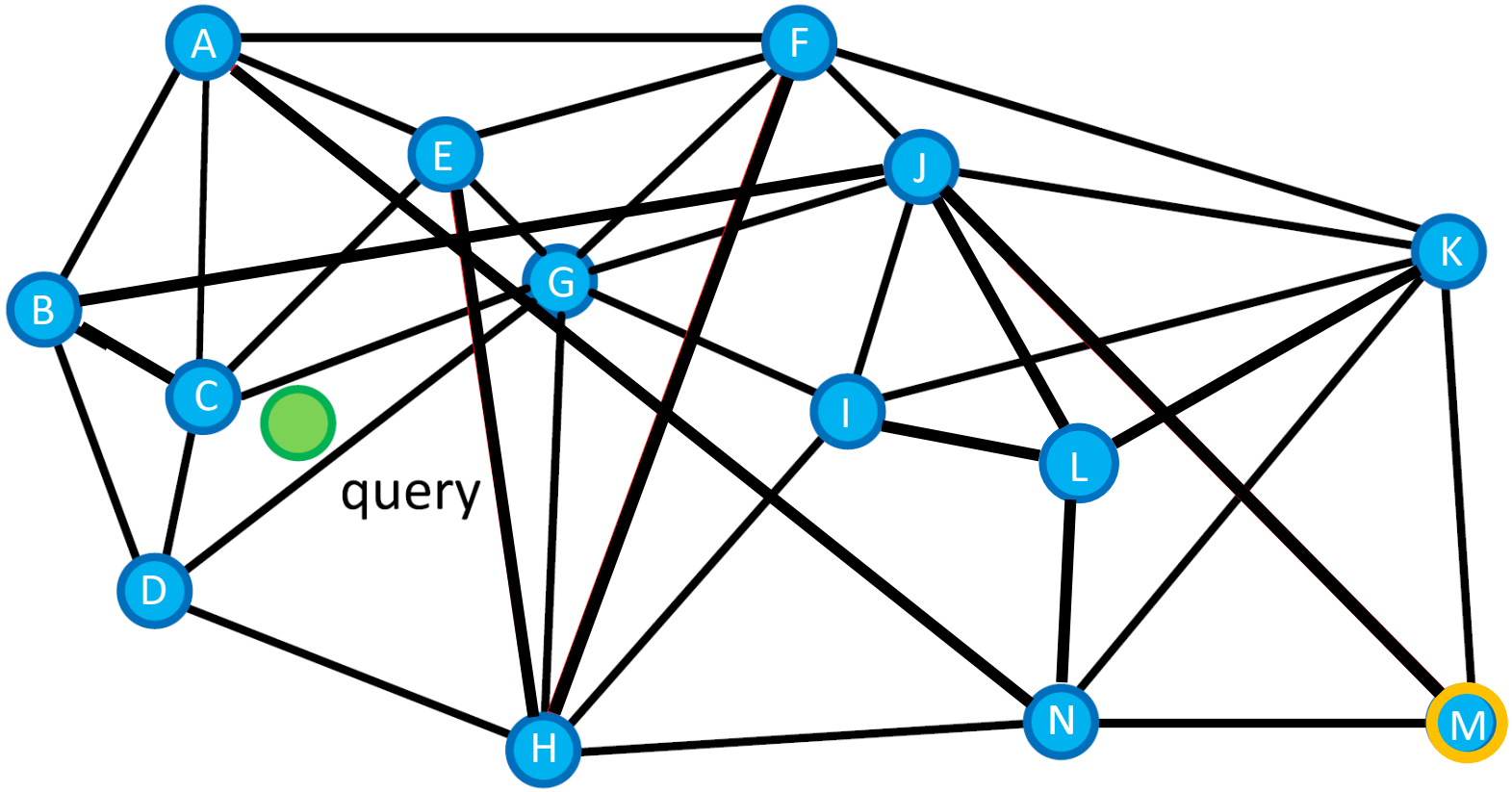
Images are from [Malkov+, Information Systems, 2013]

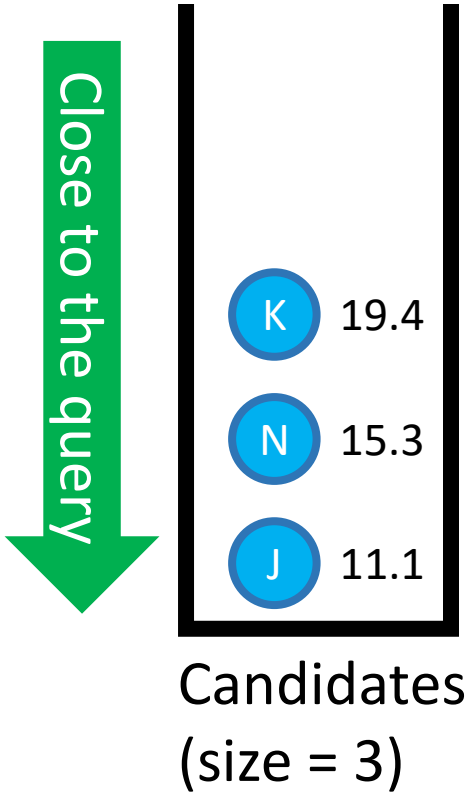


- Pick up the unchecked best candidate (M). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

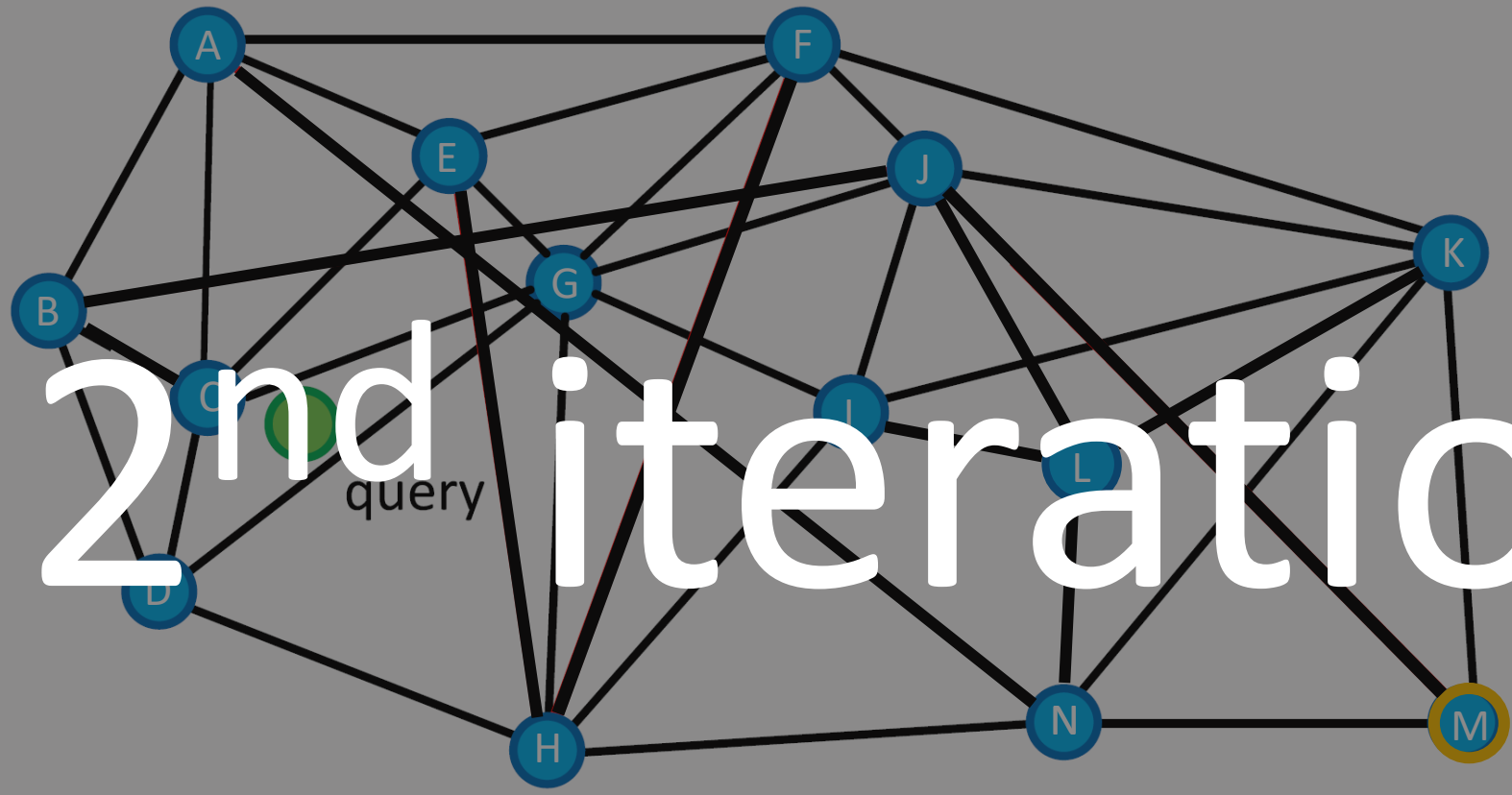
Search

Images are from [Malkov+, Information Systems, 2013]



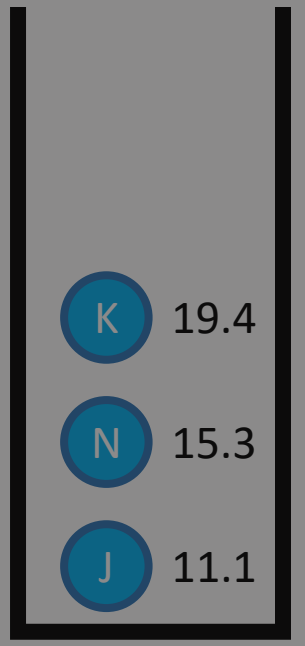


Images are from [Malkov+, Information Systems, 2013]



2nd iteration

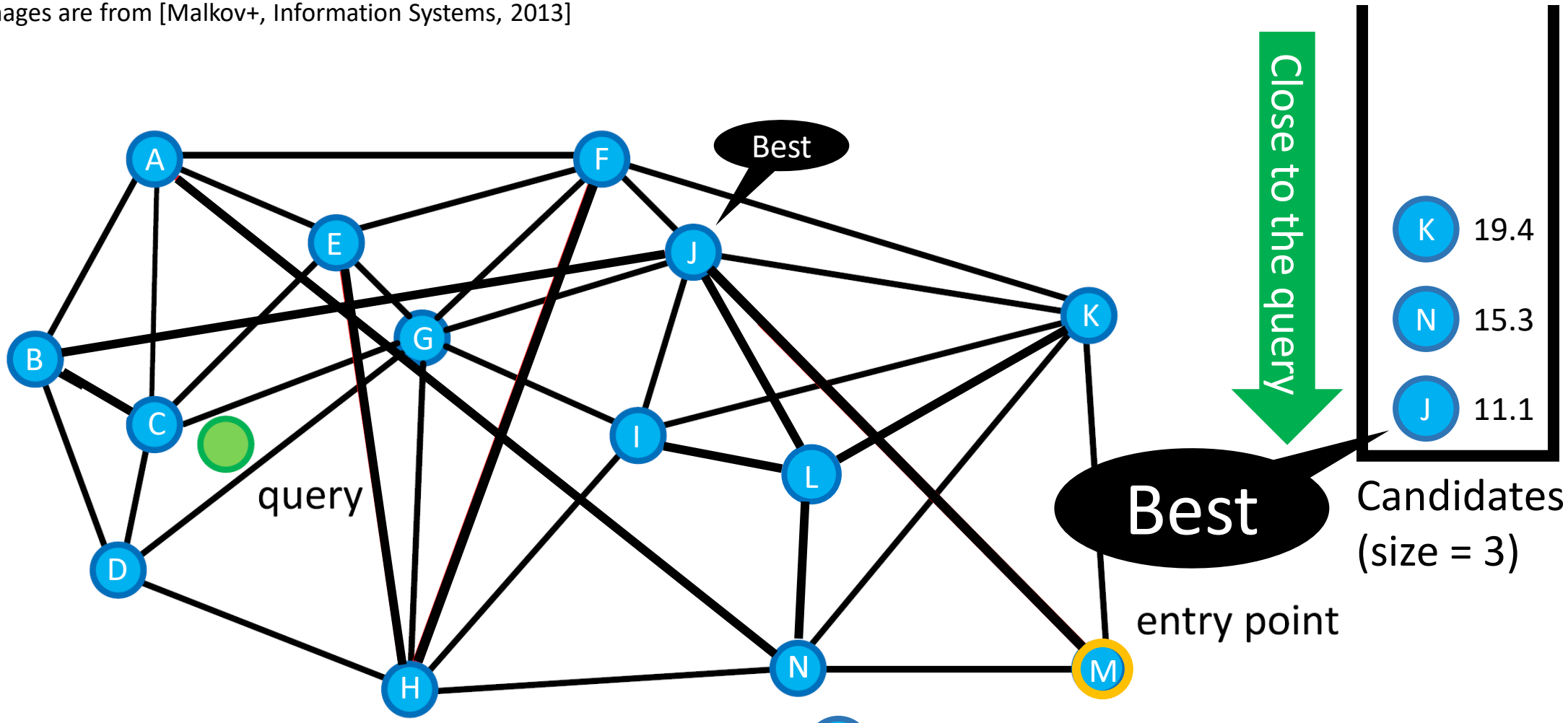
Close to the query



Candidates
(size = 3)

Search

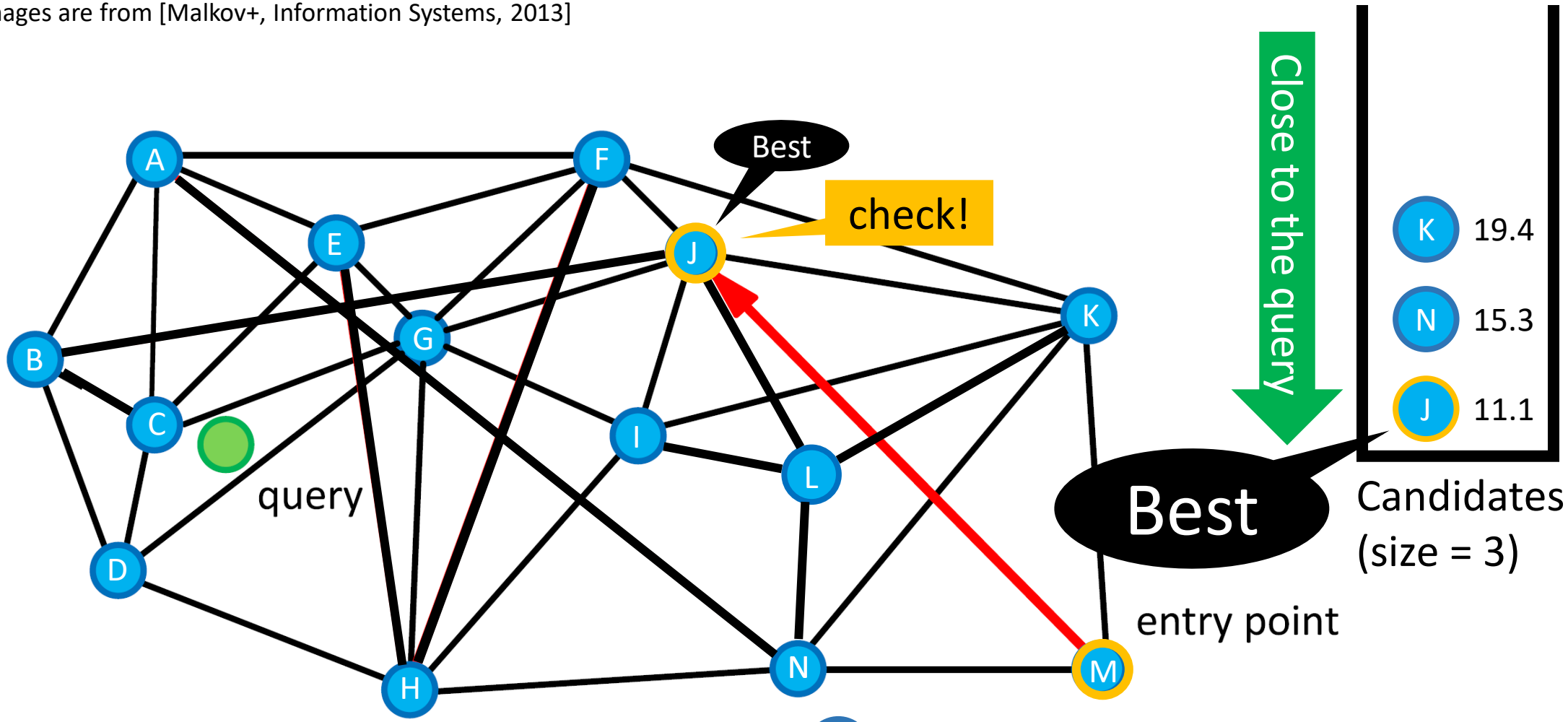
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (J)

Search

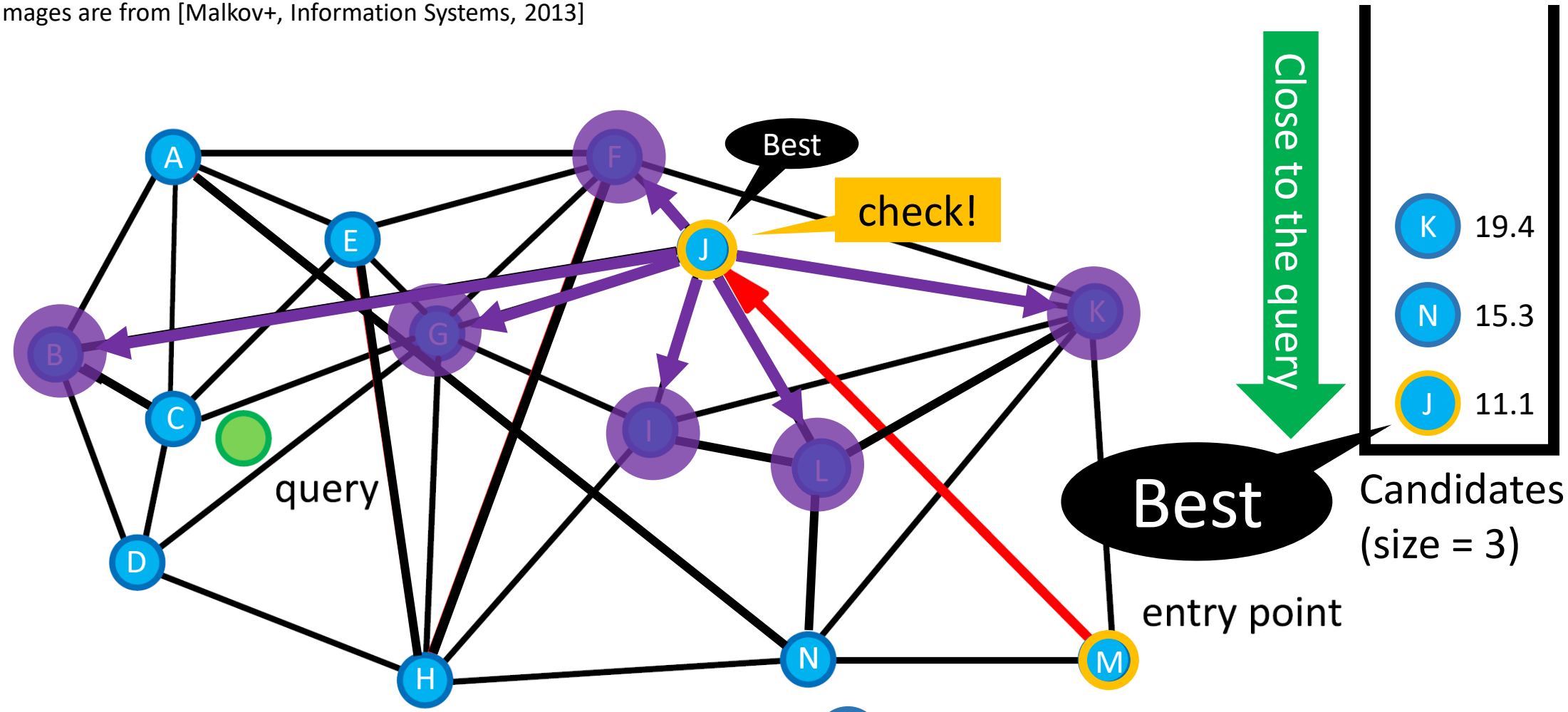
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (J). Check it.

Search

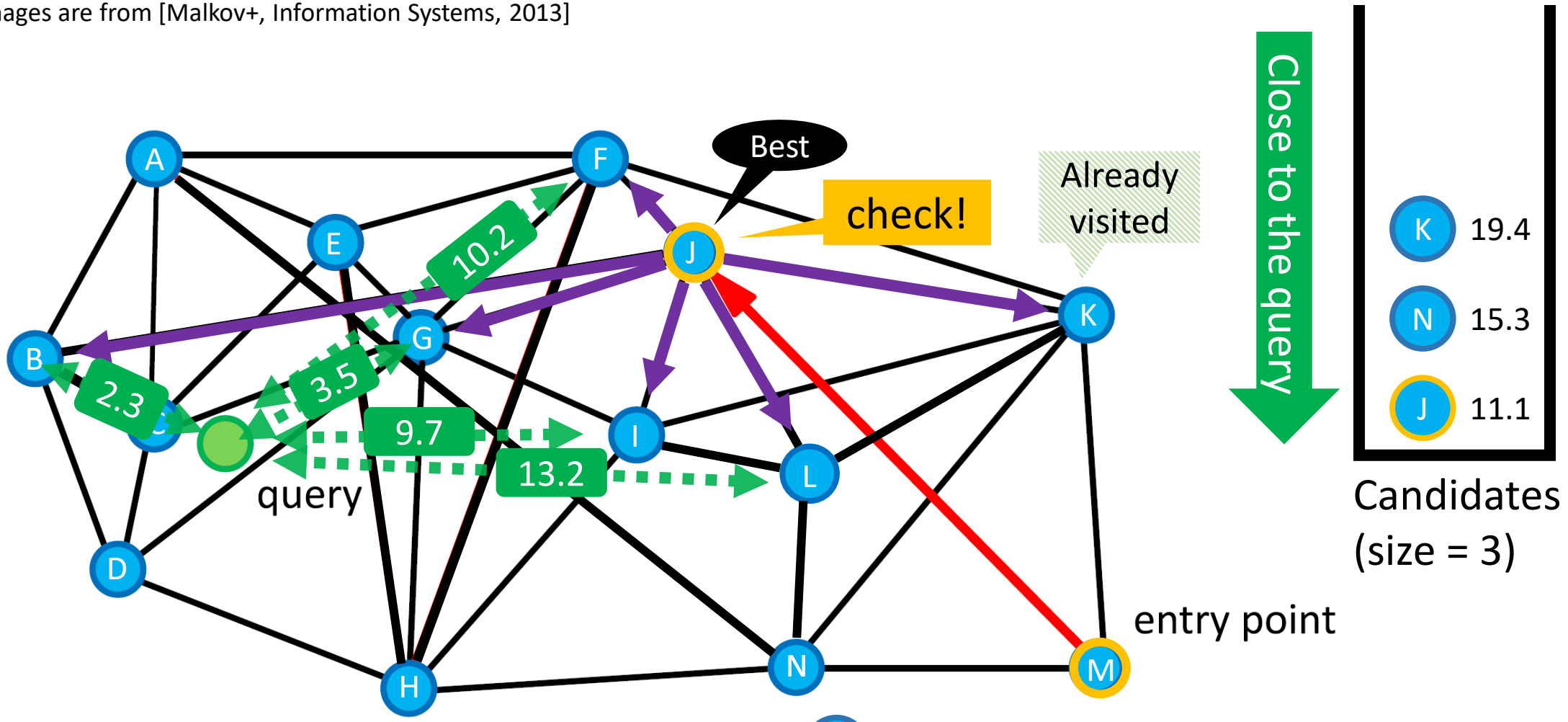
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.

Search

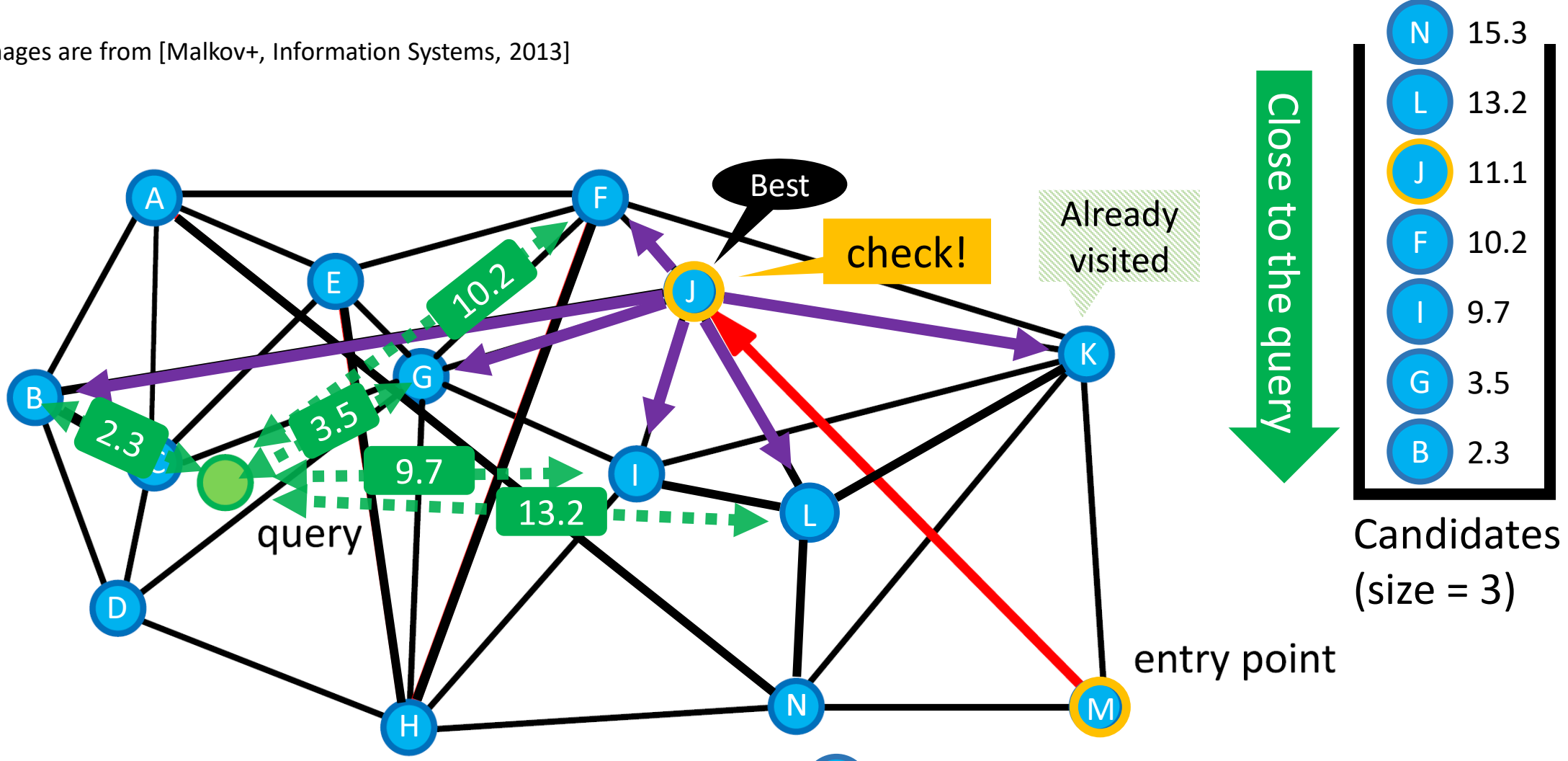
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.
- Record the distances to q.

Search

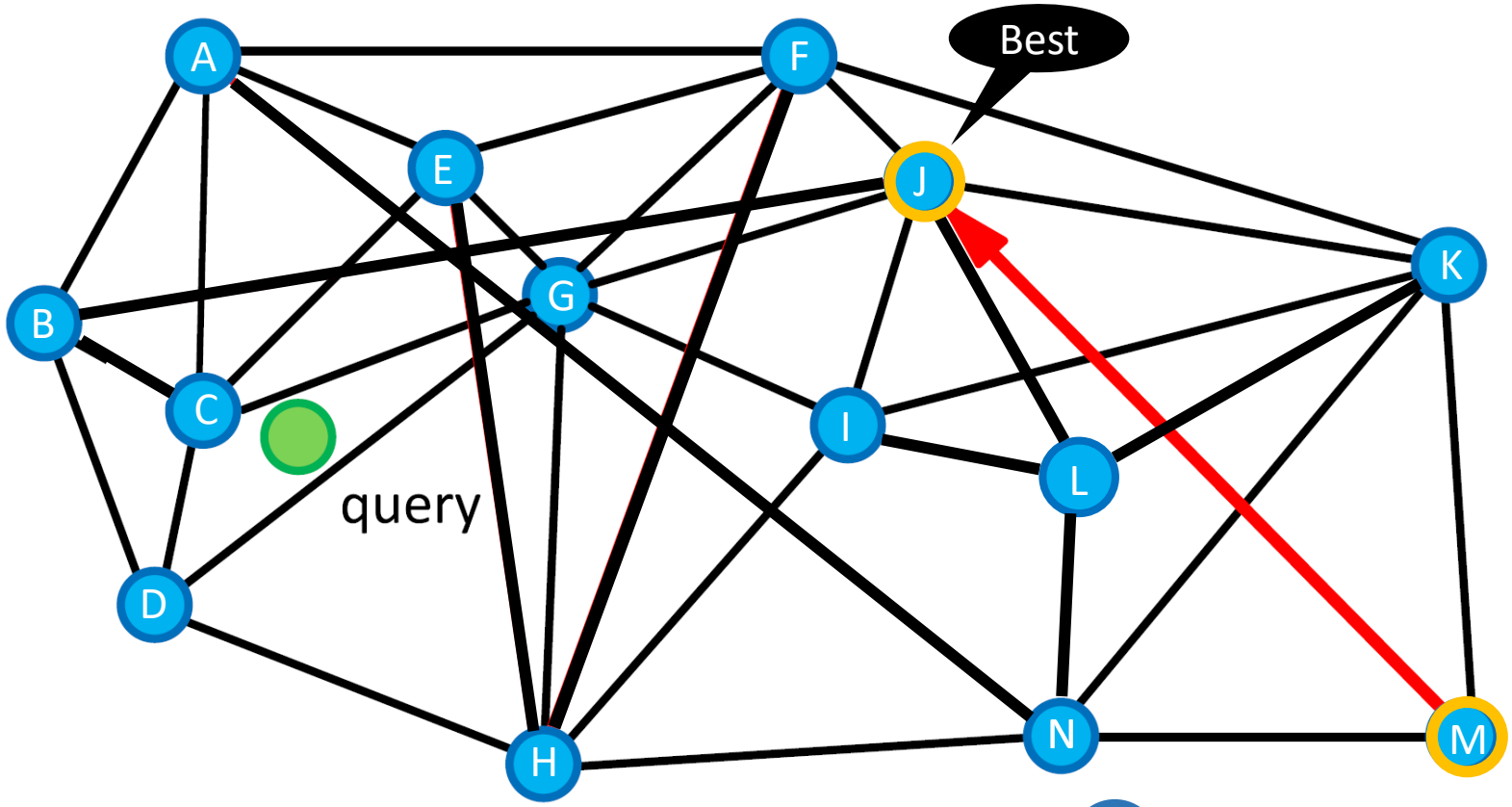
Images are from [Malkov+, Information Systems, 2013]

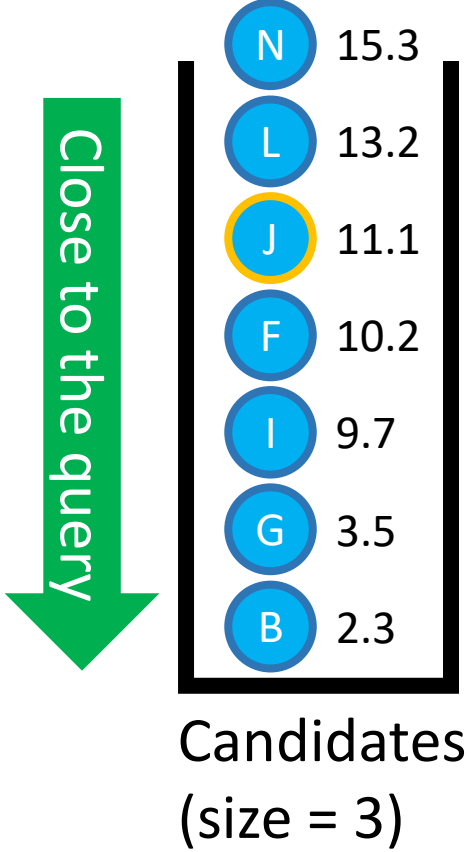


- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.
- Record the distances to q.

Search

Images are from [Malkov+, Information Systems, 2013]

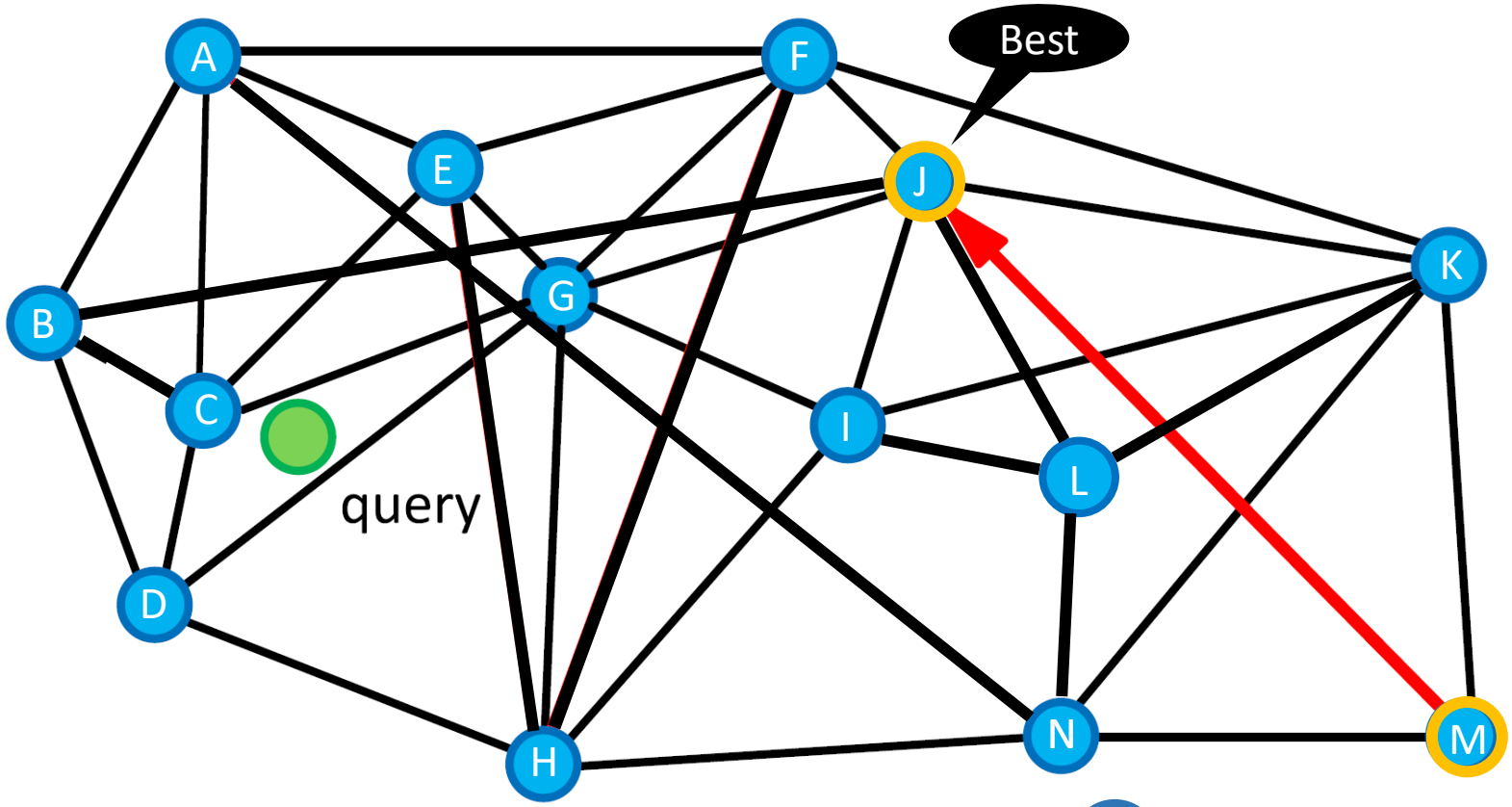


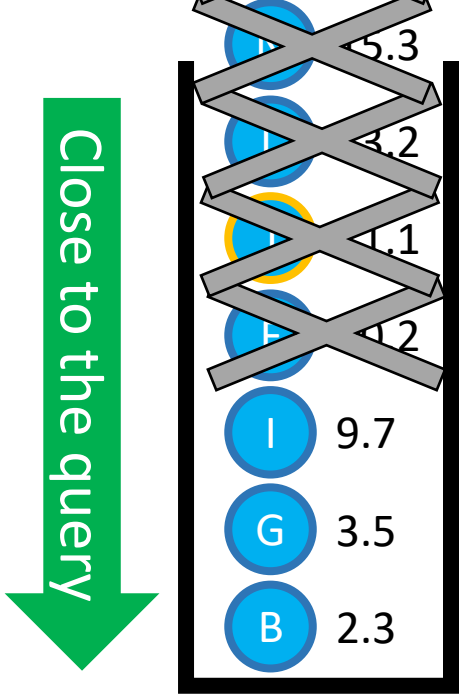


- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.
- Record the distances to q.

Search

Images are from [Malkov+, Information Systems, 2013]



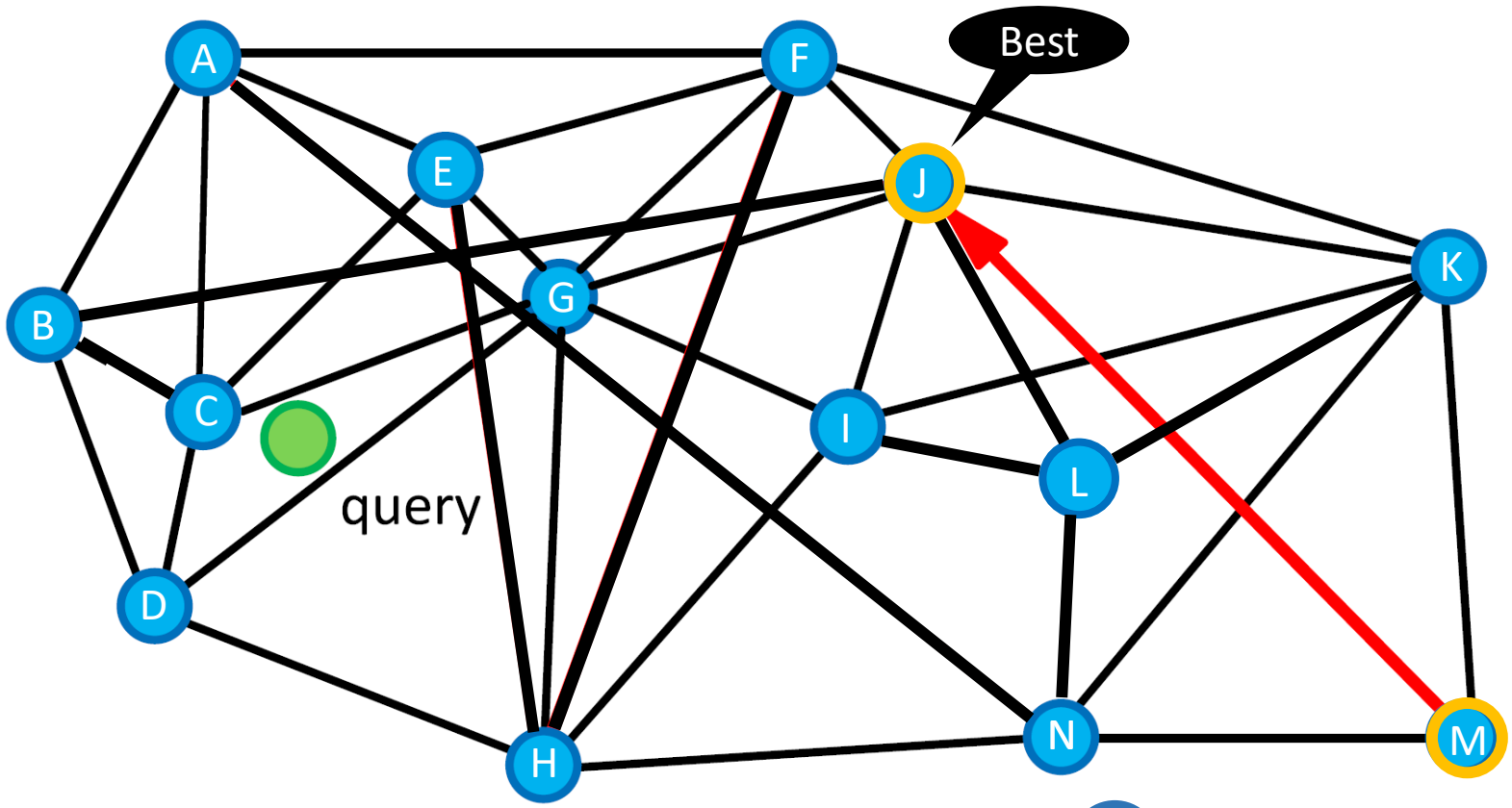


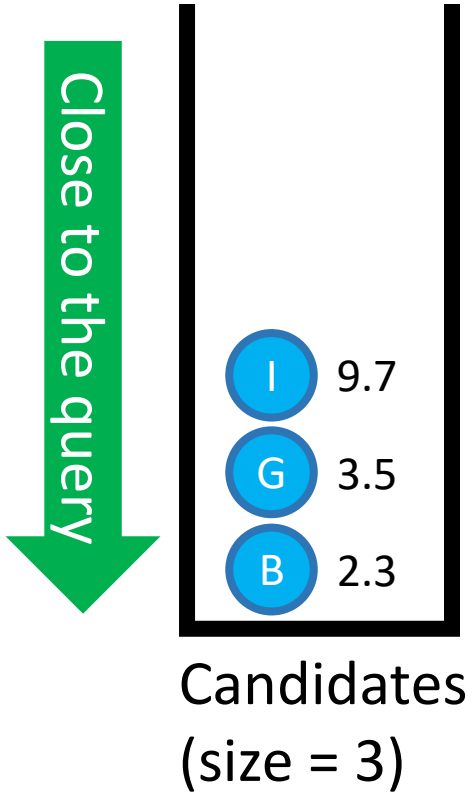
Candidates (size = 3)

- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

Search

Images are from [Malkov+, Information Systems, 2013]

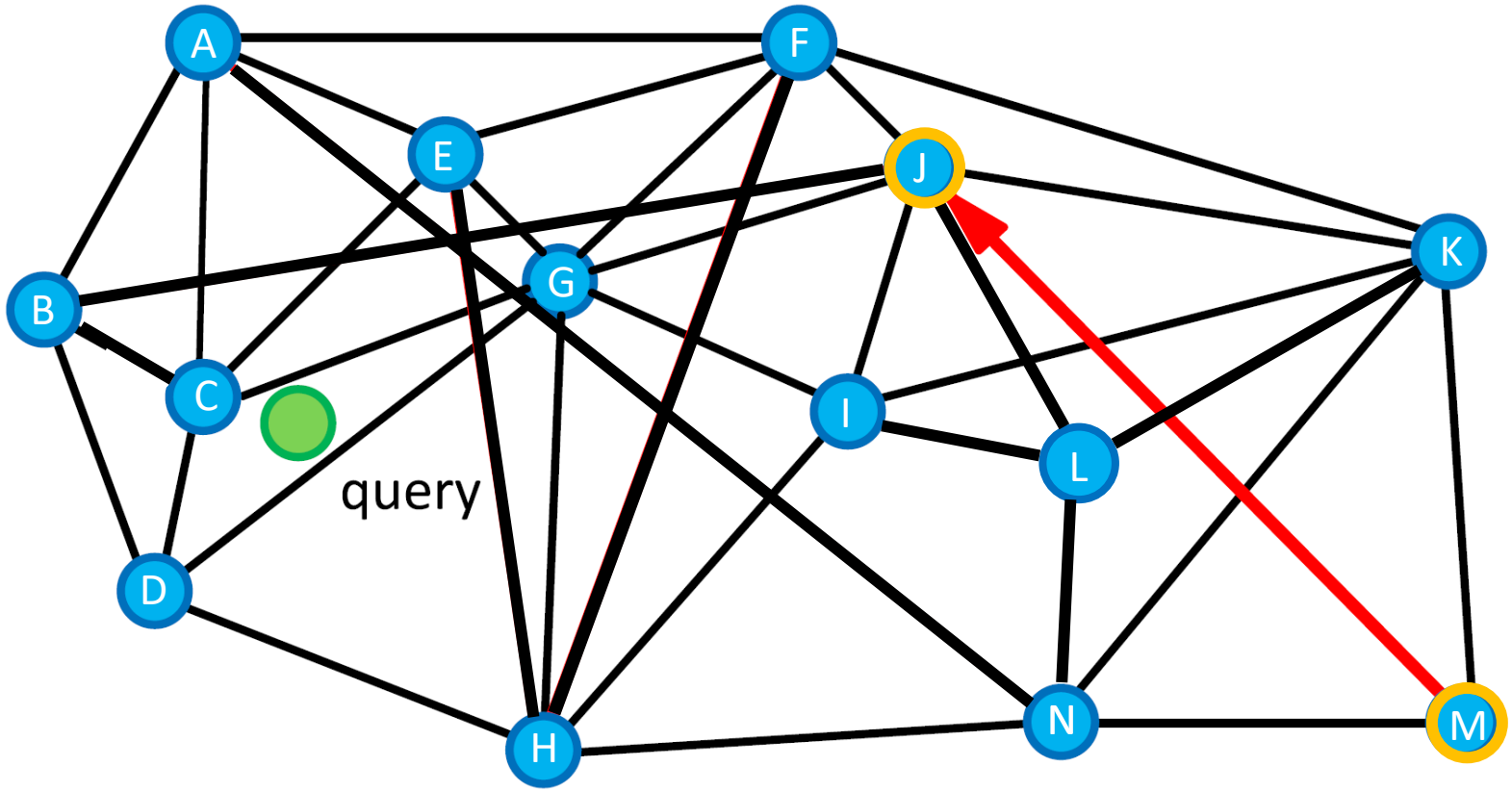


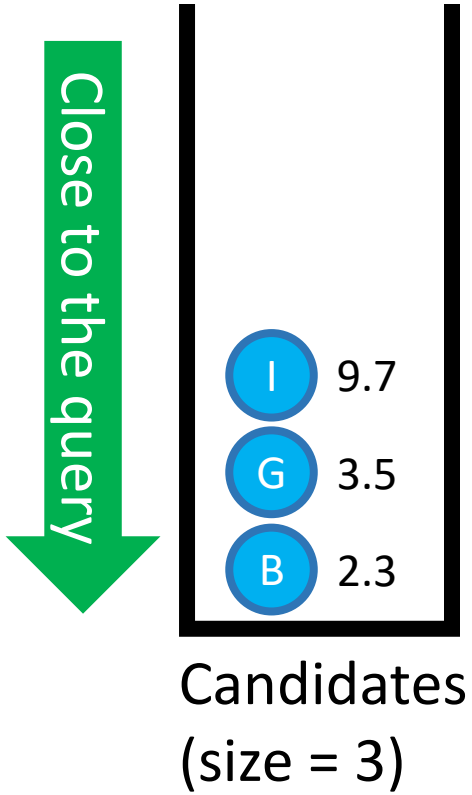


- Pick up the unchecked best candidate (J). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

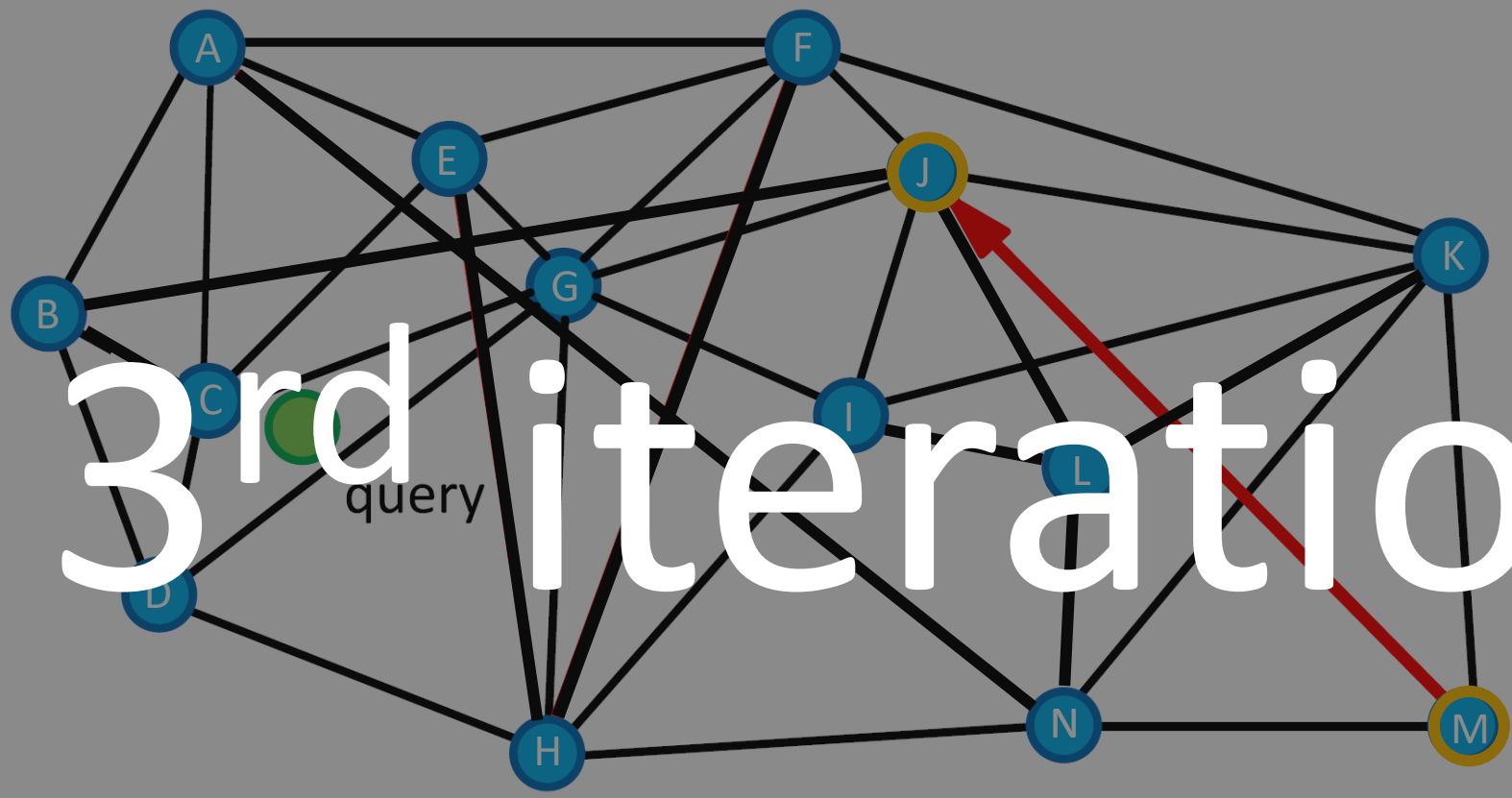
Search

Images are from [Malkov+, Information Systems, 2013]



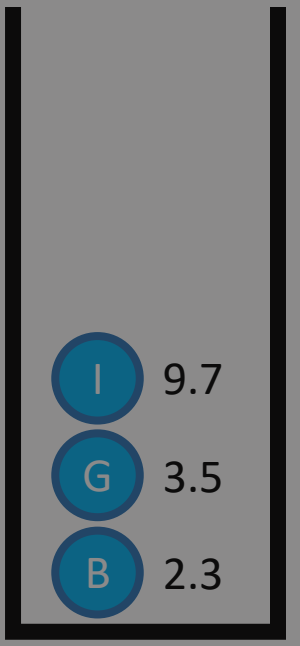


Images are from [Malkov+, Information Systems, 2013]



3rd iteration

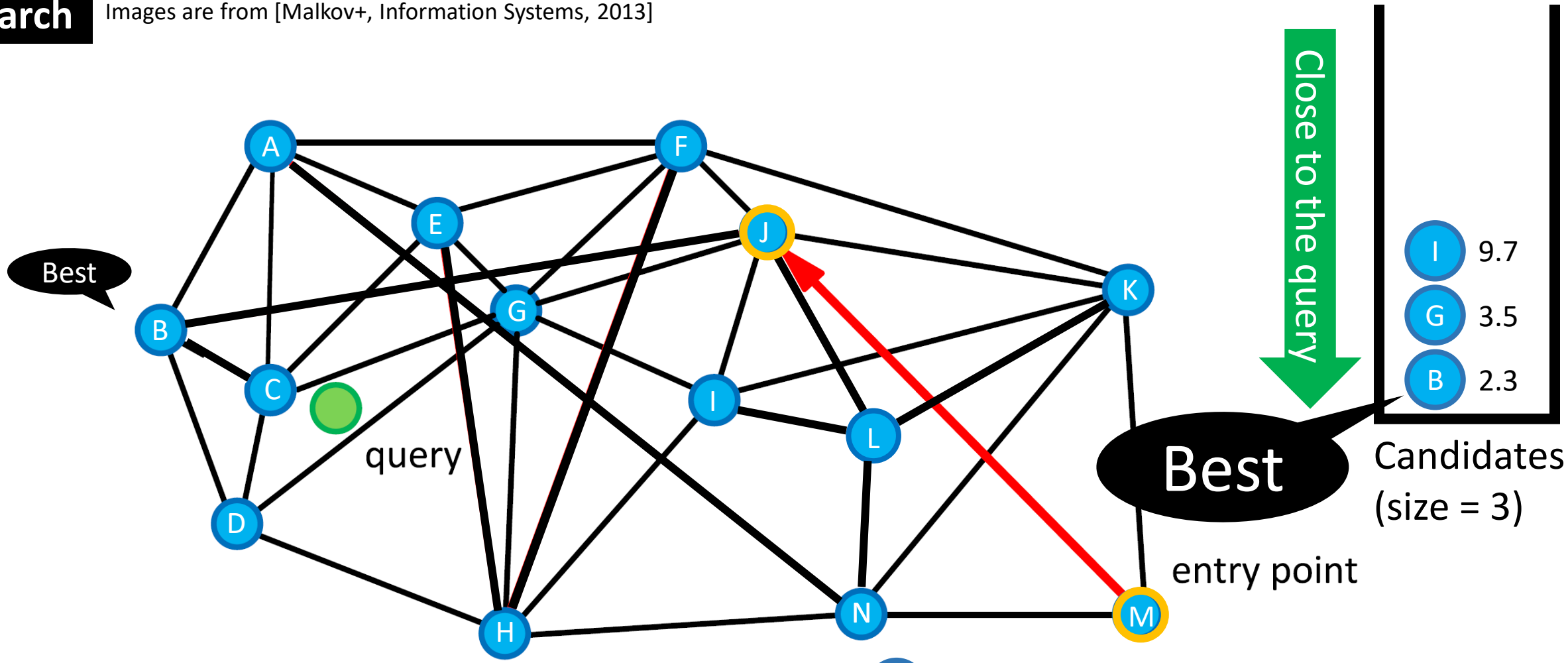
Close to the query



Candidates (size = 3)

Search

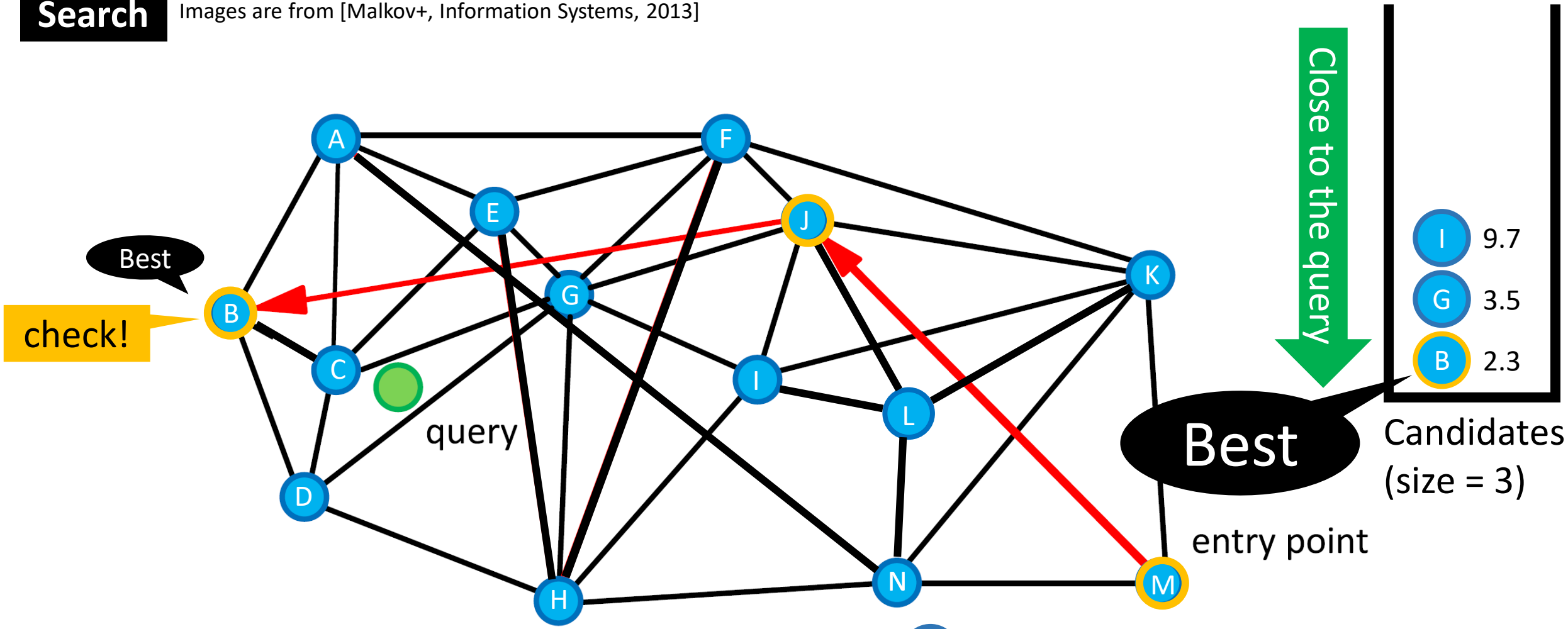
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B)

Search

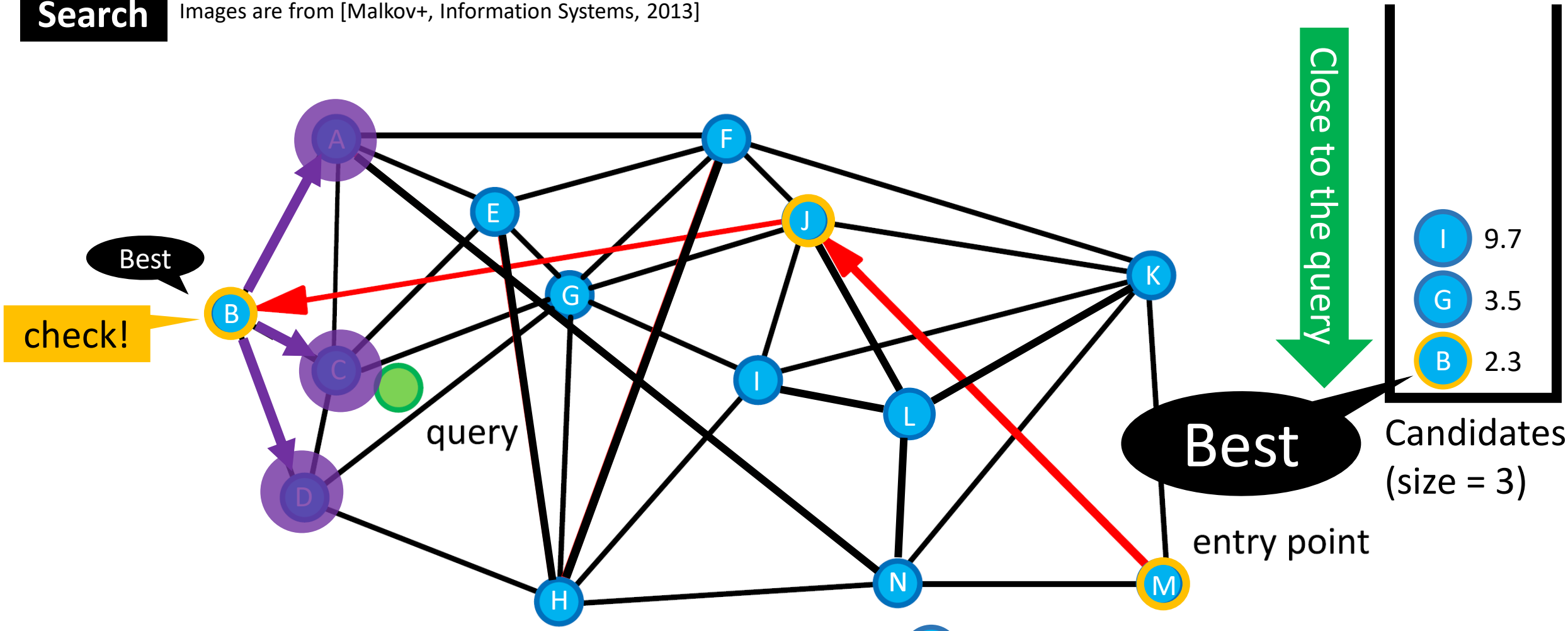
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.

Search

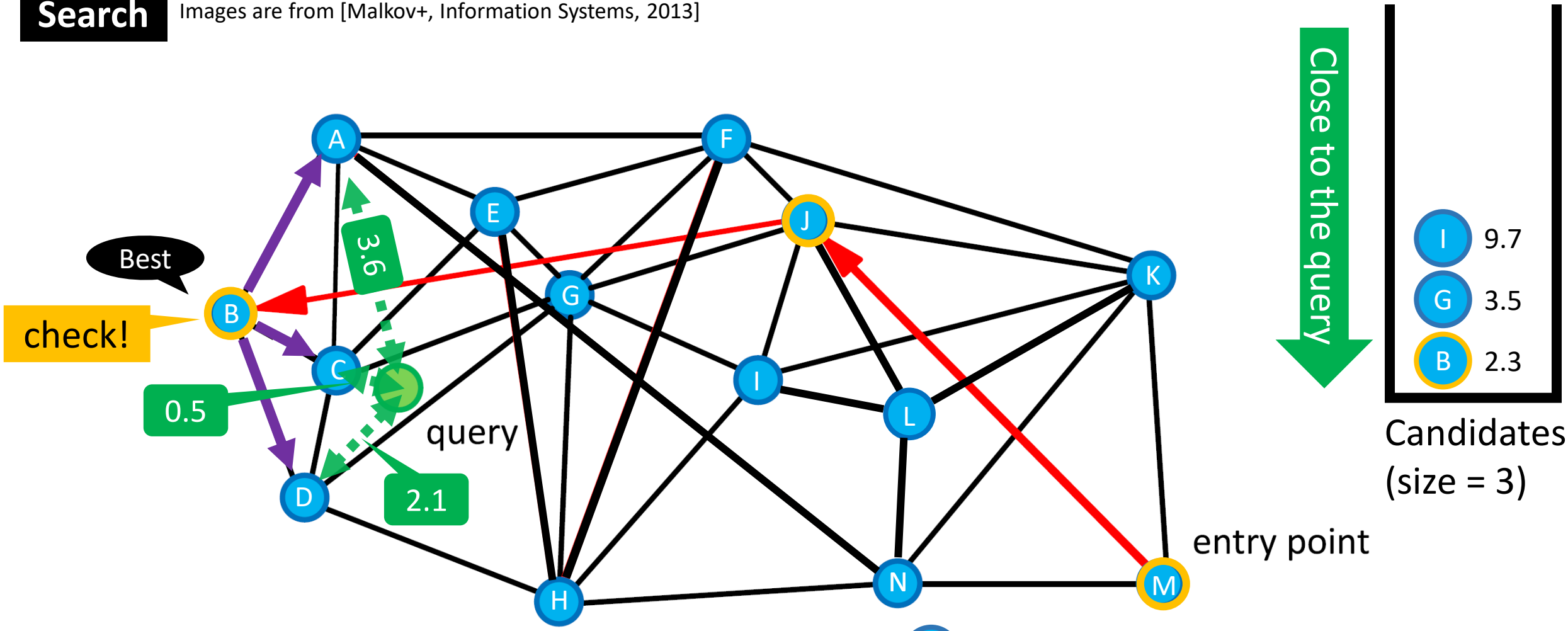
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.

Search

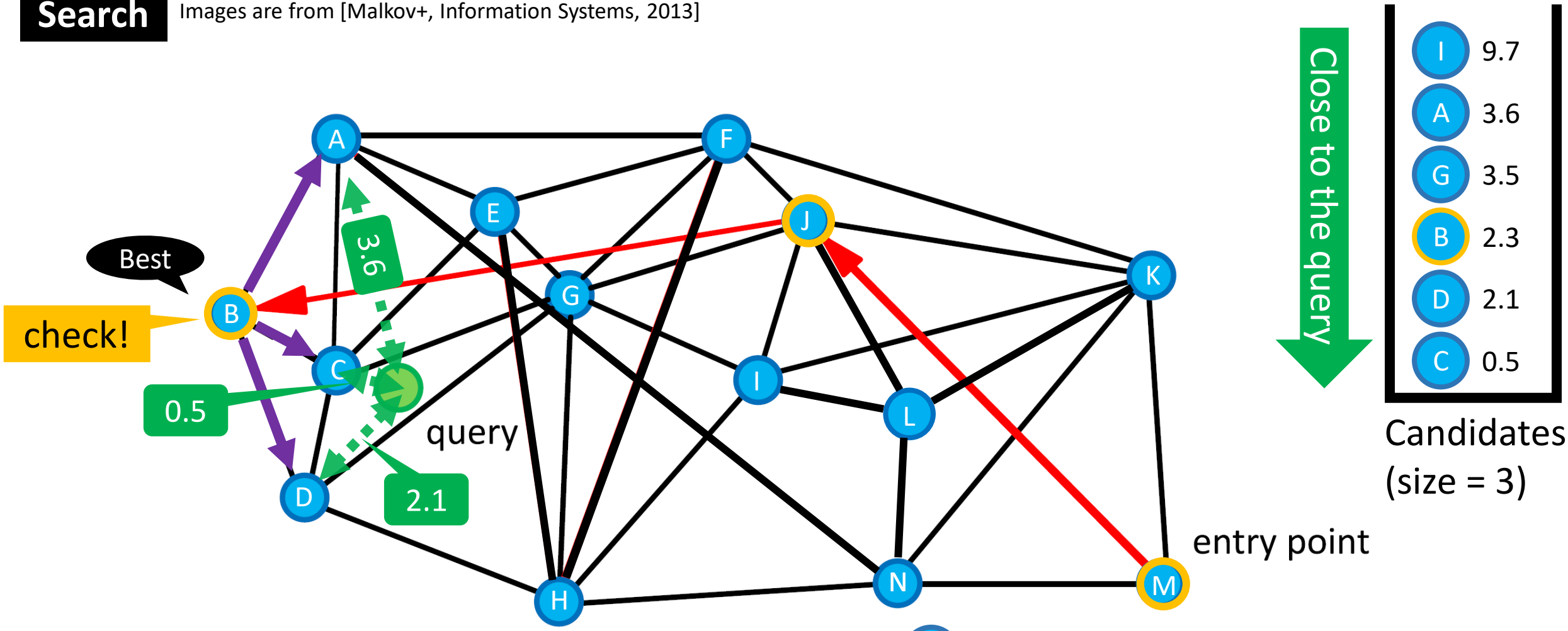
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.
- Record the distances to q.

Search

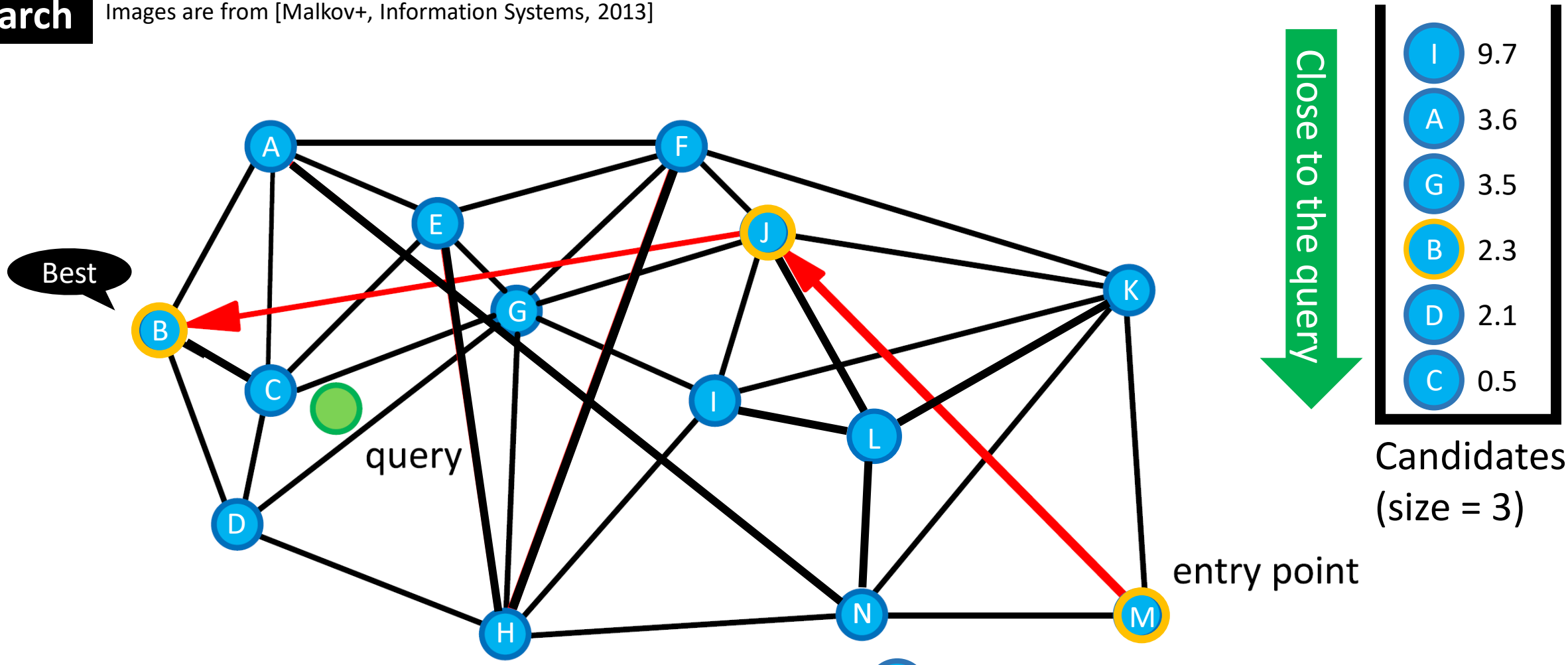
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.
- Record the distances to q.

Search

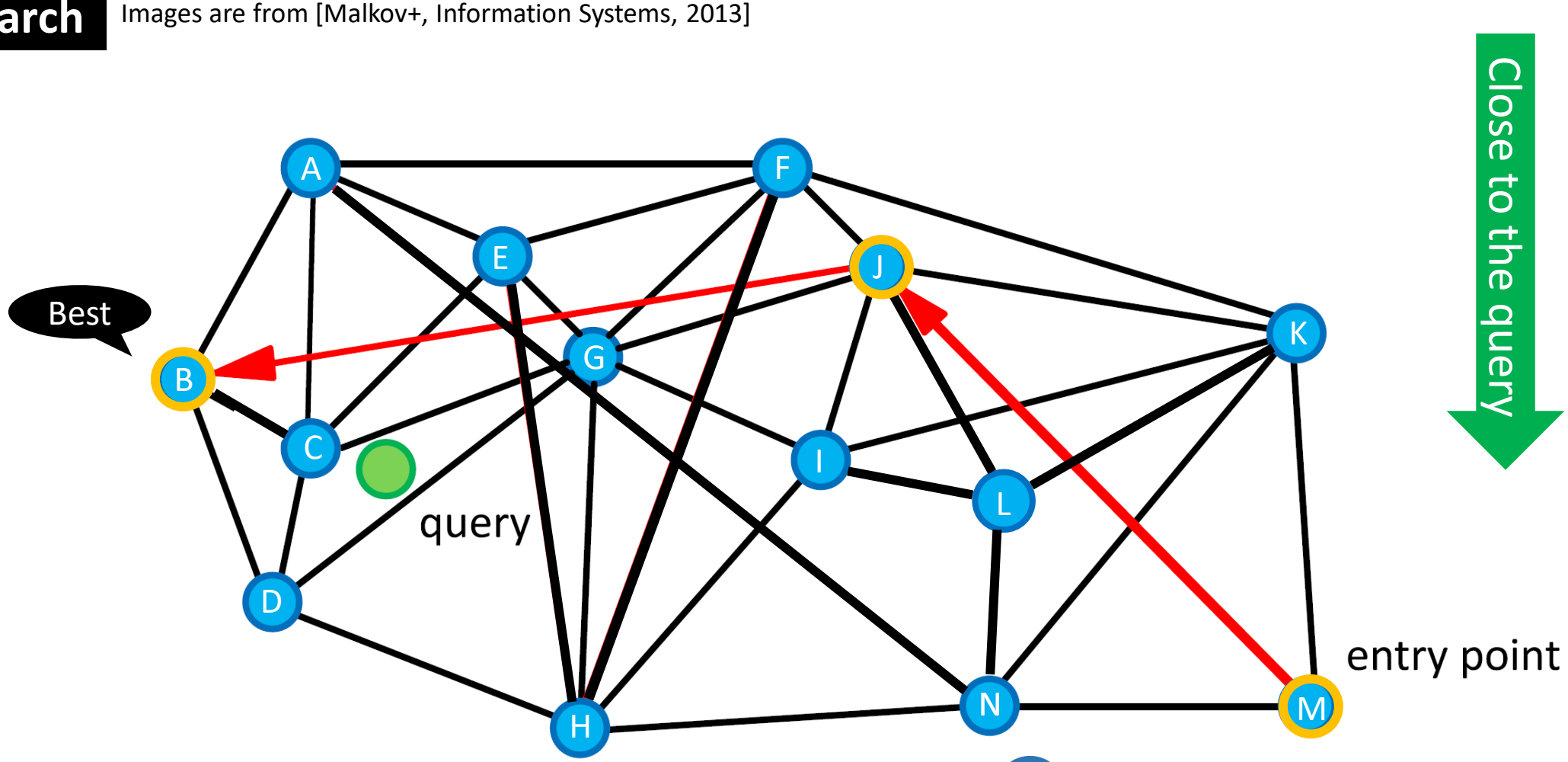
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.
- Record the distances to q.

Search

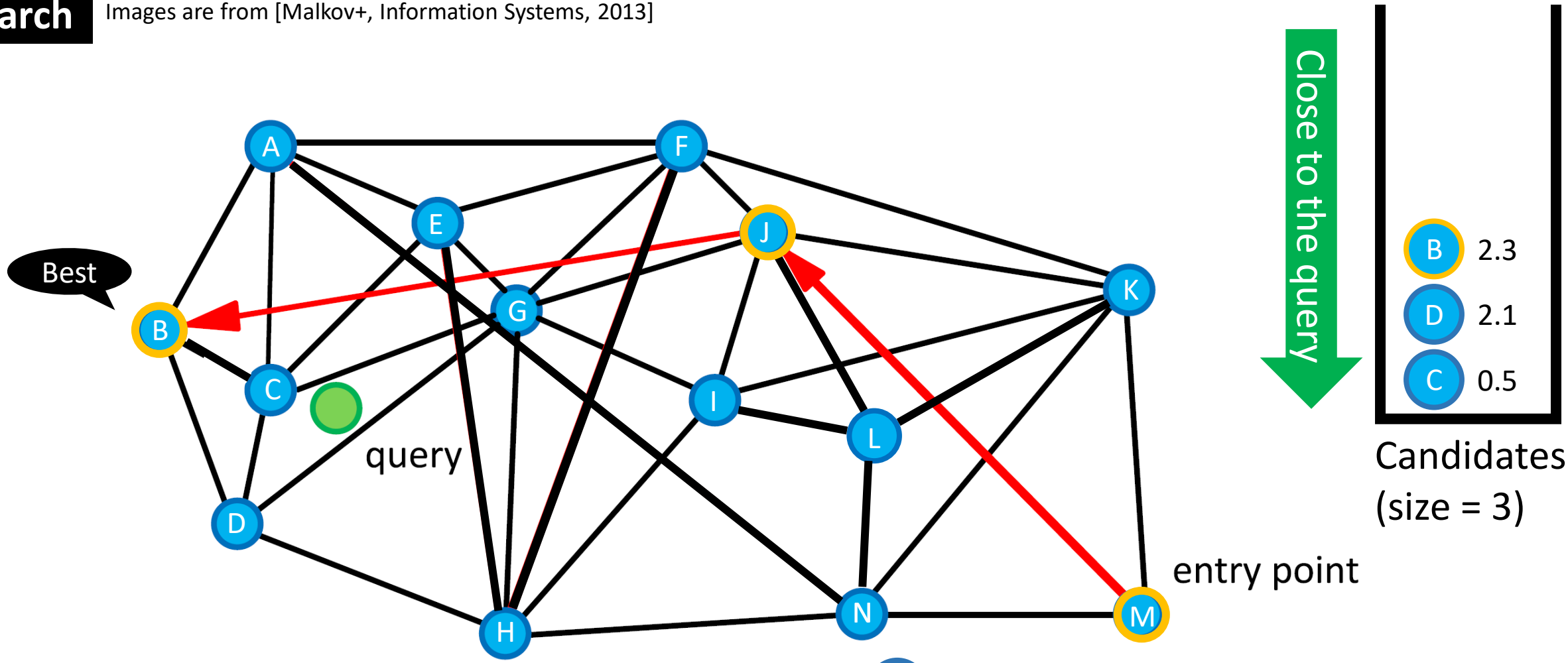
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

Search

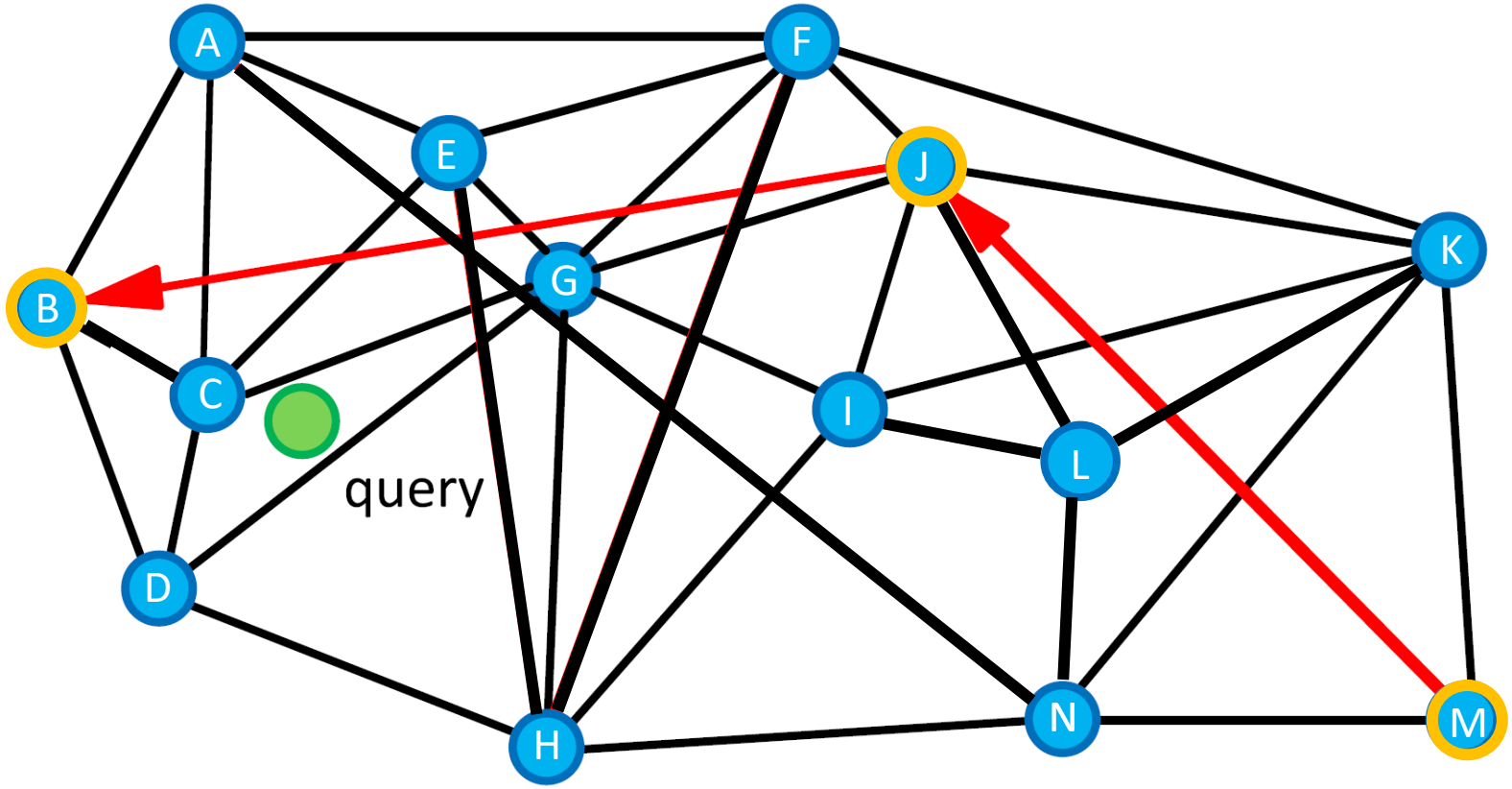
Images are from [Malkov+, Information Systems, 2013]

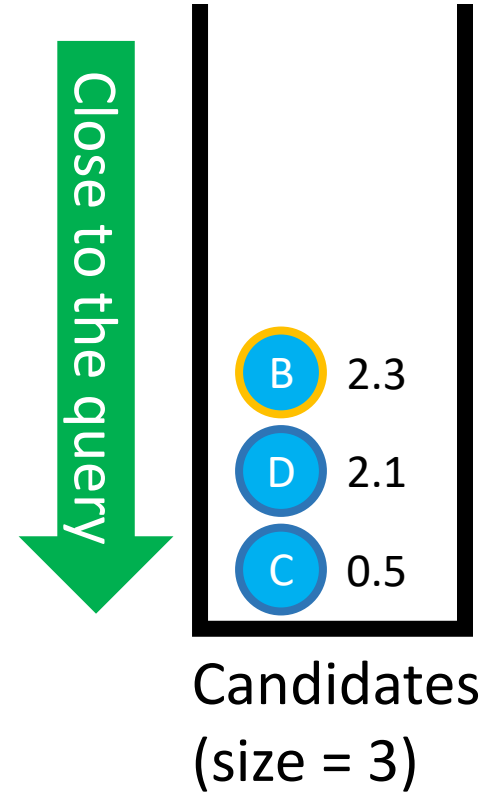


- Pick up the unchecked best candidate (B). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

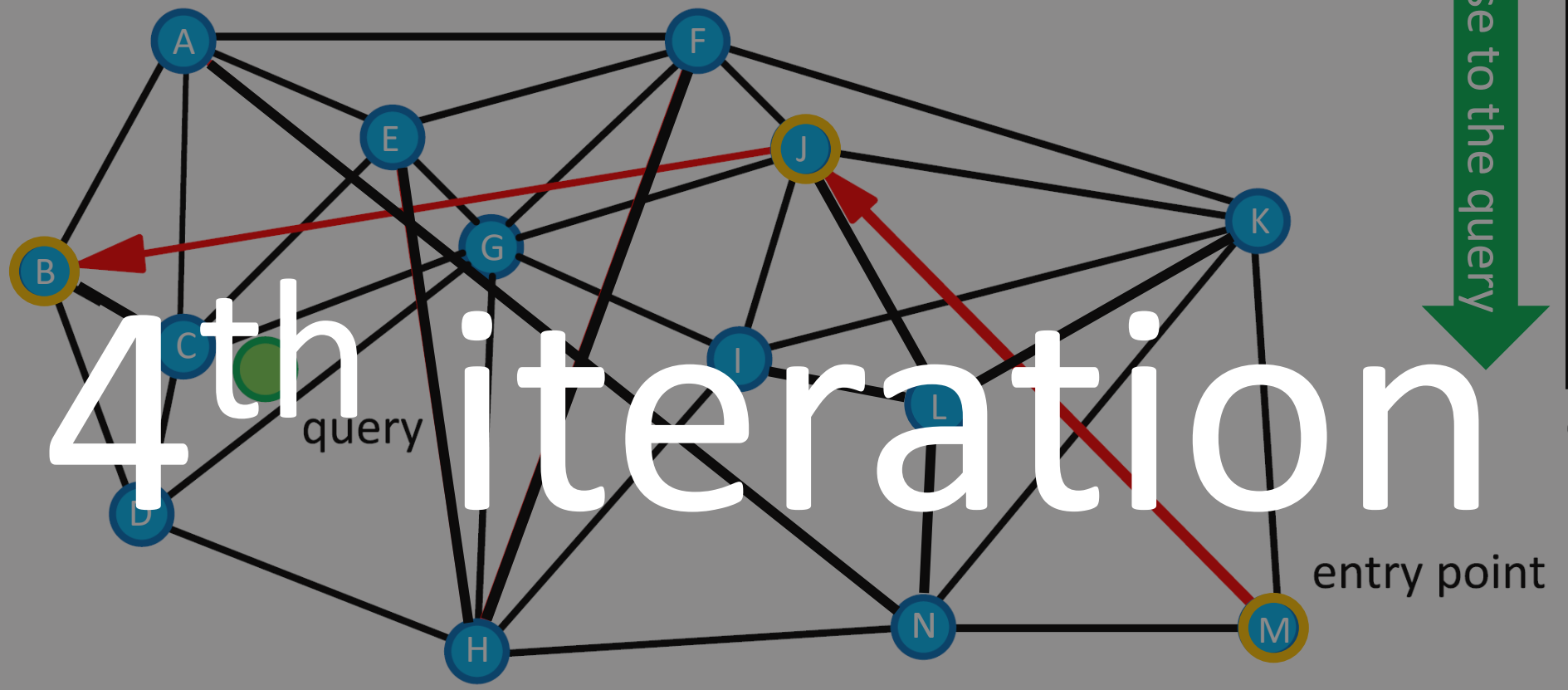
Search

Images are from [Malkov+, Information Systems, 2013]





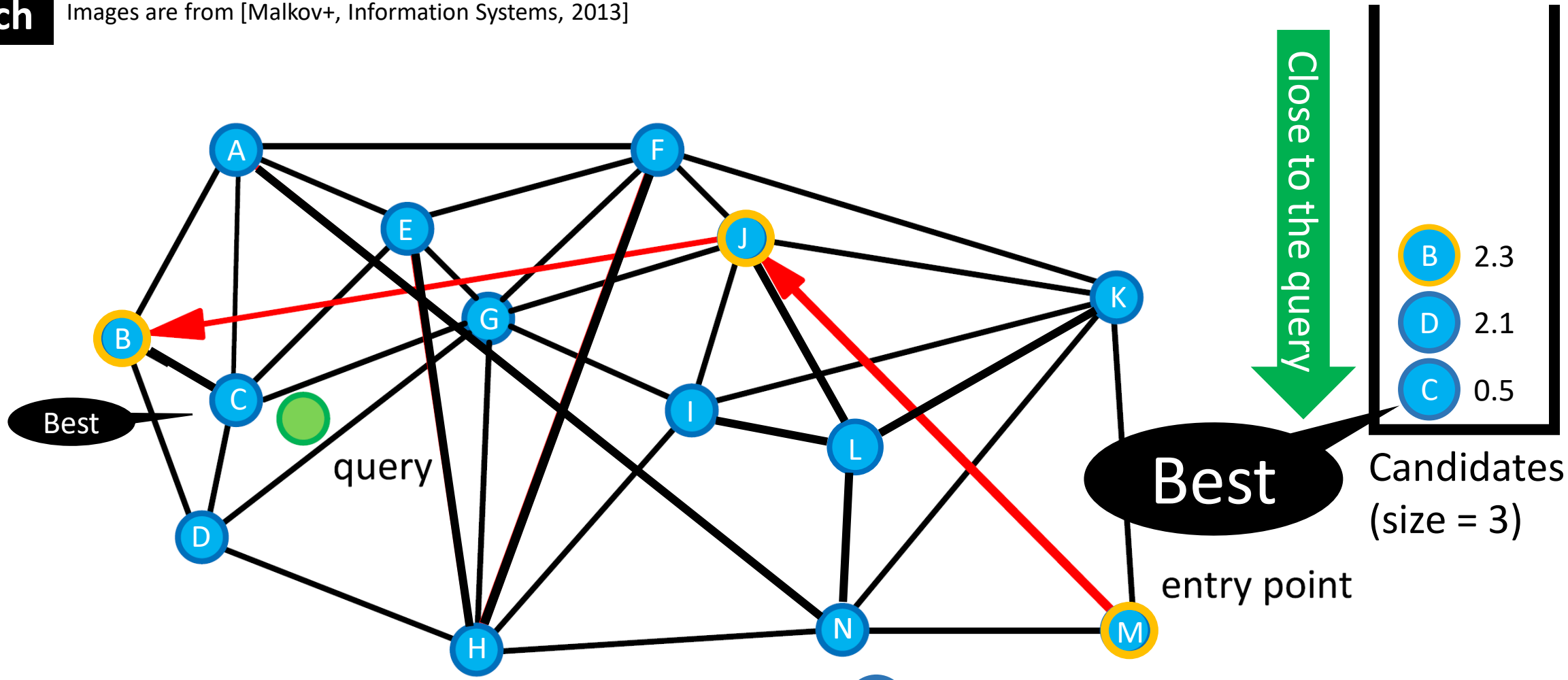
Images are from [Malkov+, Information Systems, 2013]



4th iteration

Search

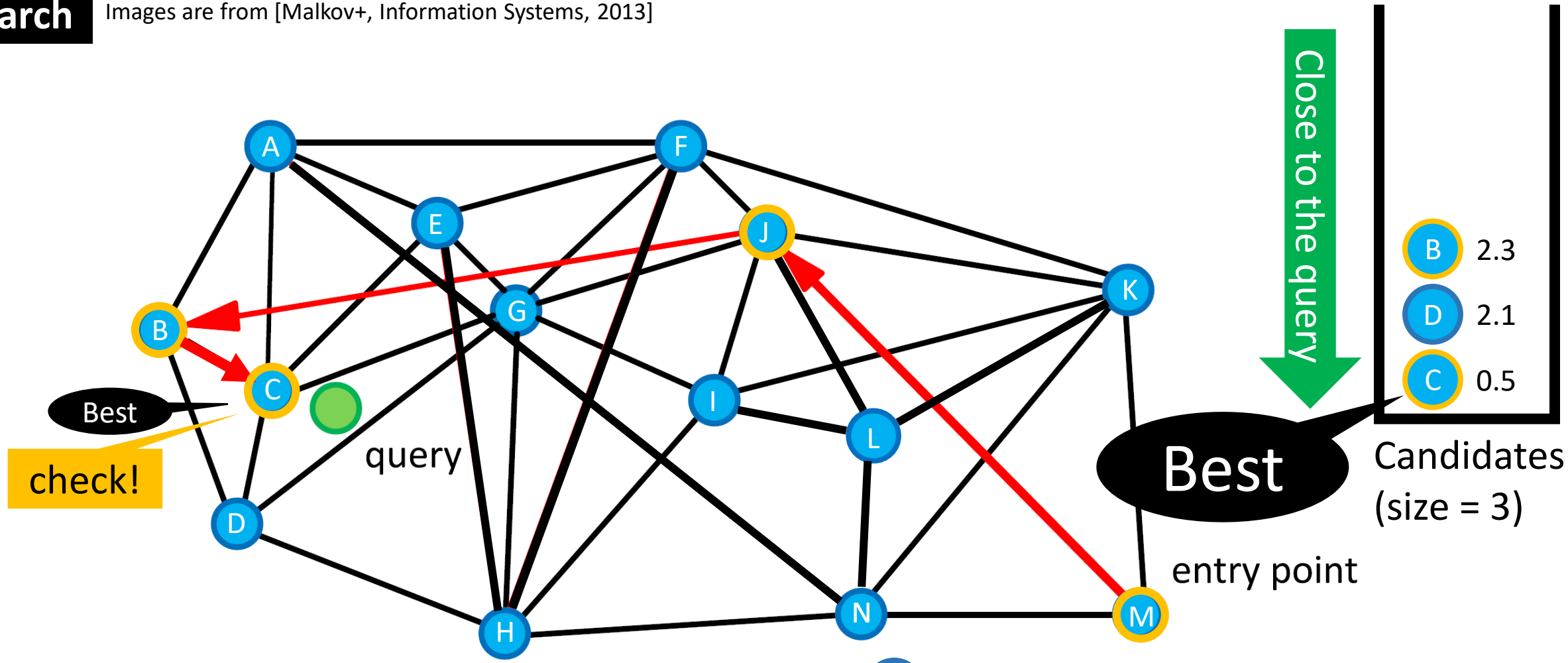
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (C).

Search

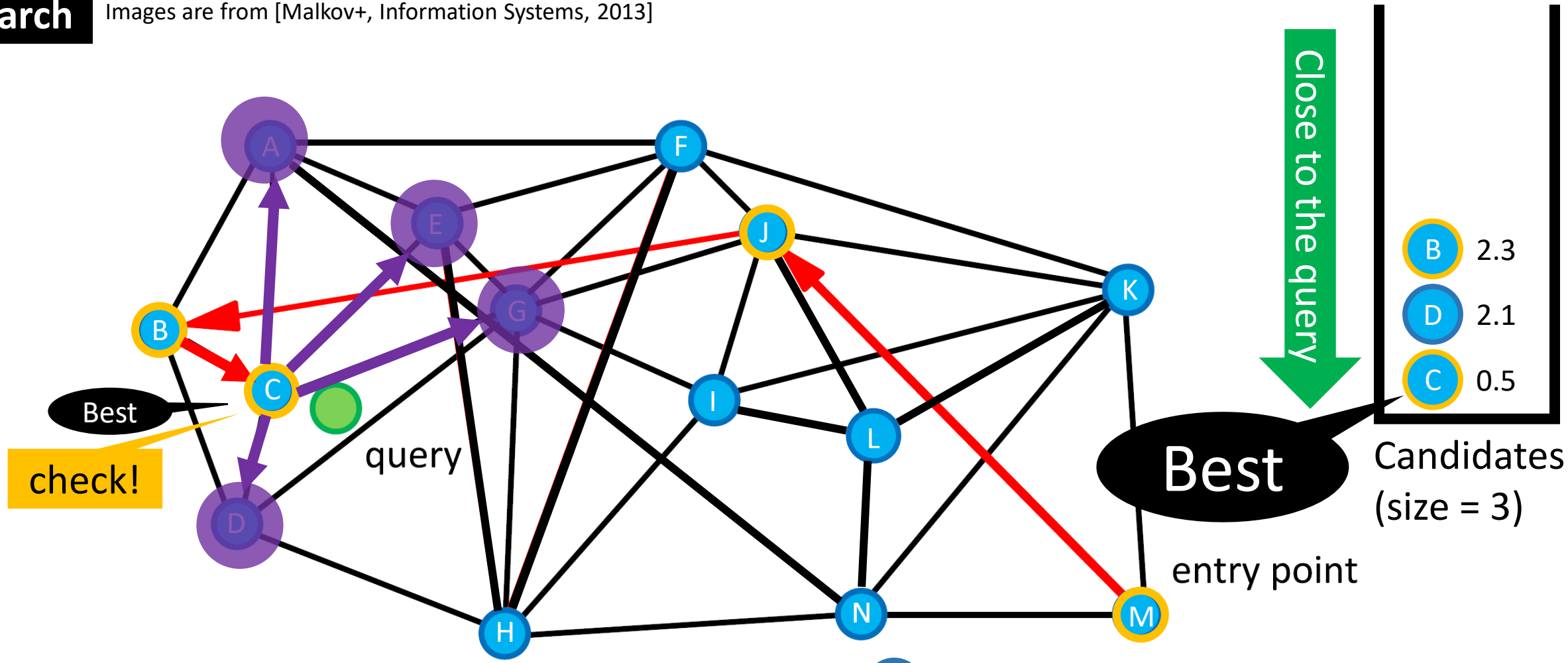
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (C). Check it.

Search

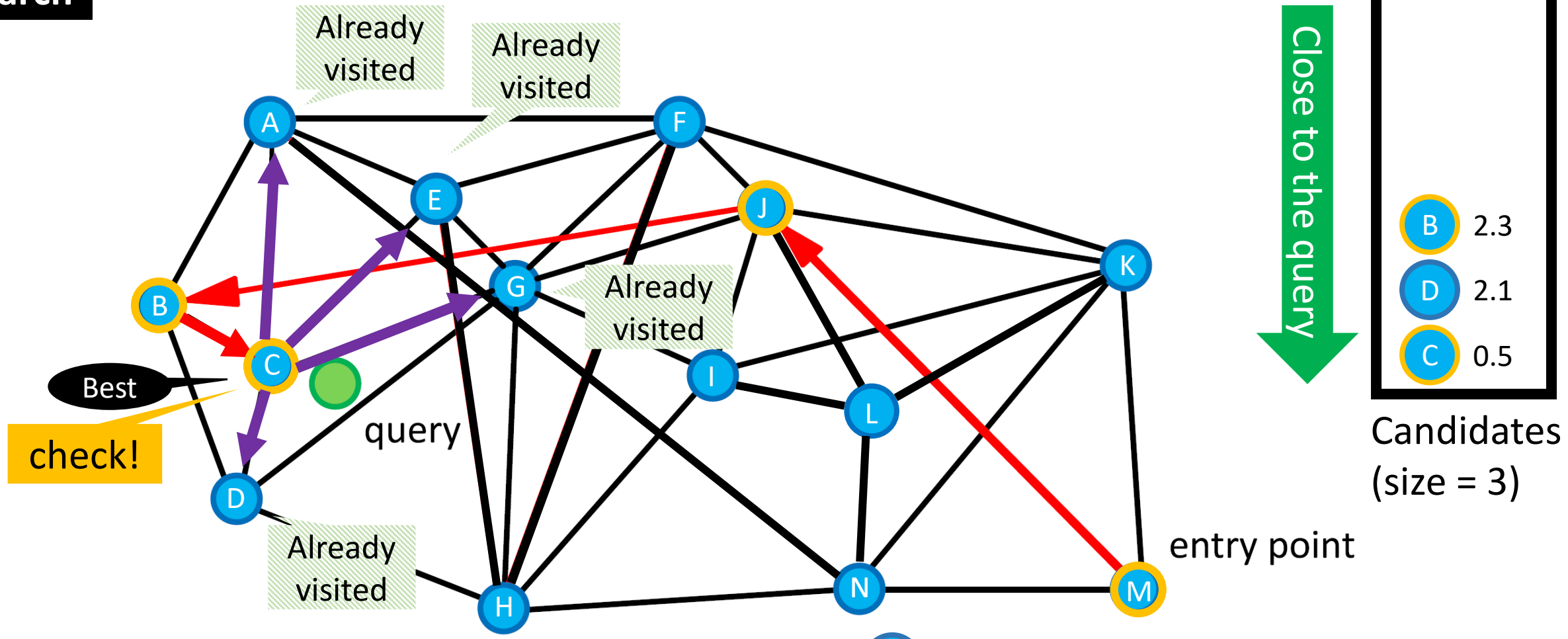
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (C). Check it.
- Find the connected points.

Search

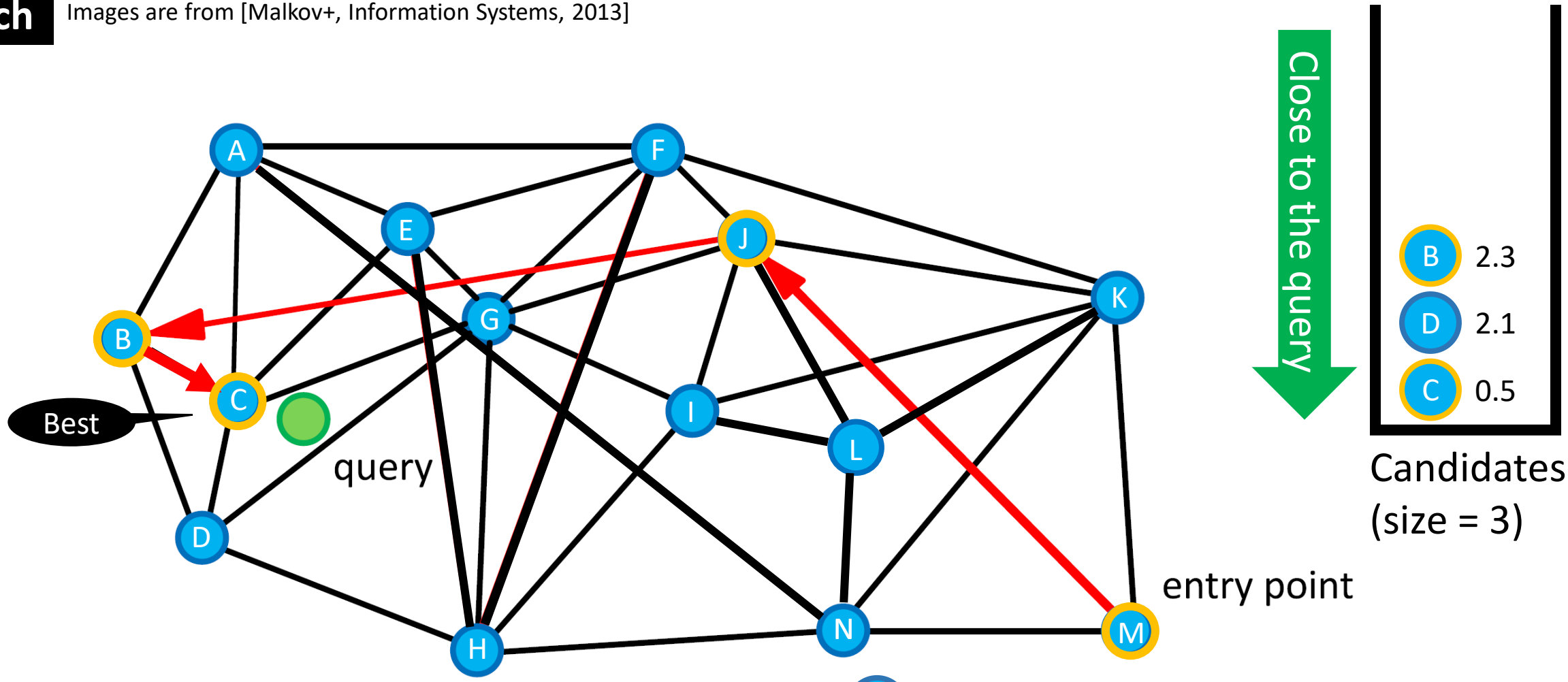
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (C). Check it.
- Find the connected points.
- Record the distances to q.

Search

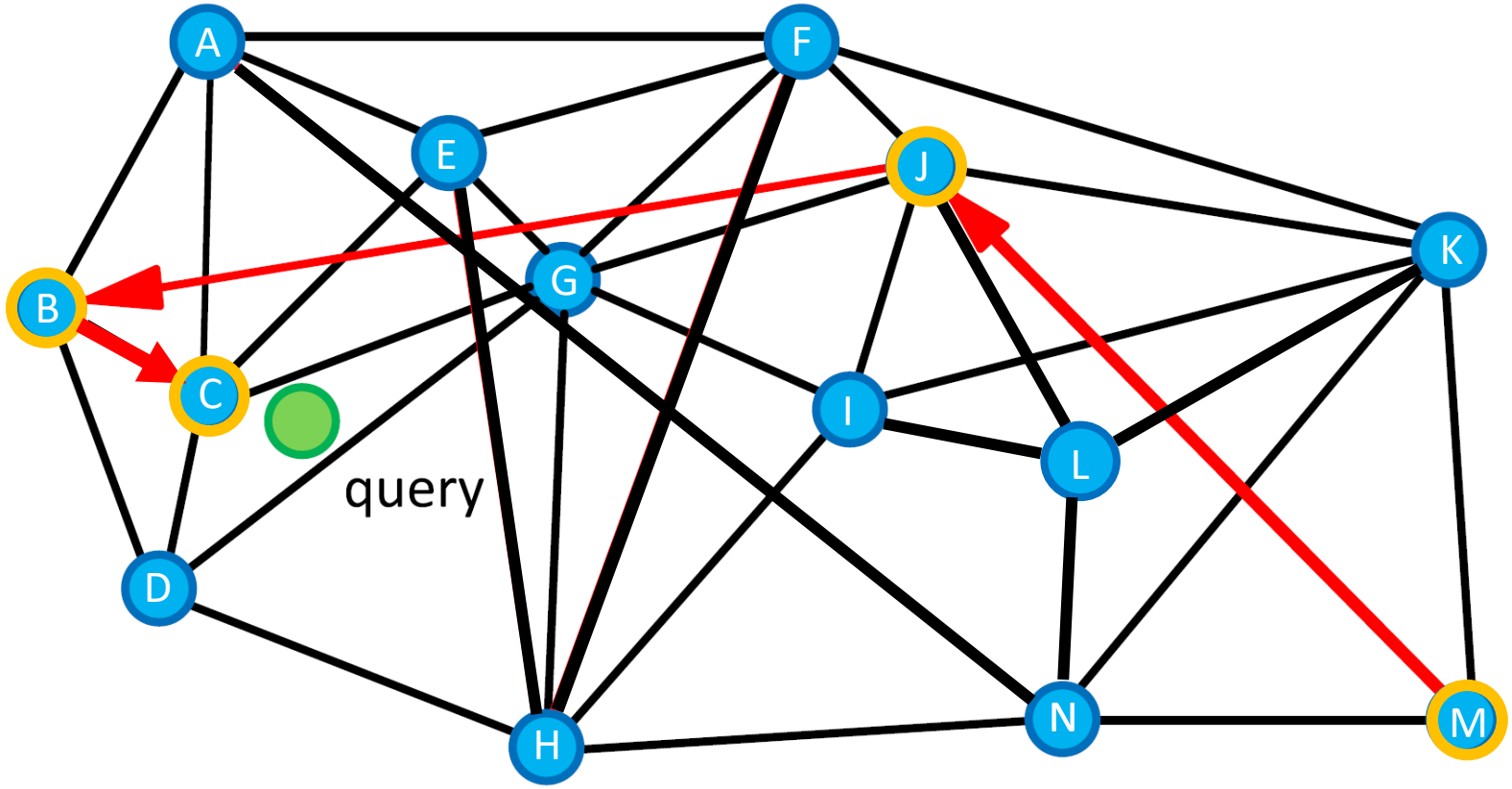
Images are from [Malkov+, Information Systems, 2013]

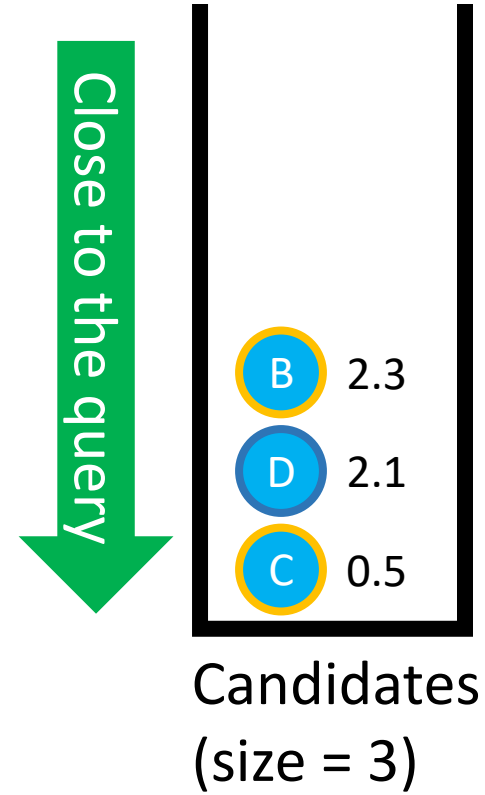


- Pick up the unchecked best candidate (C). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

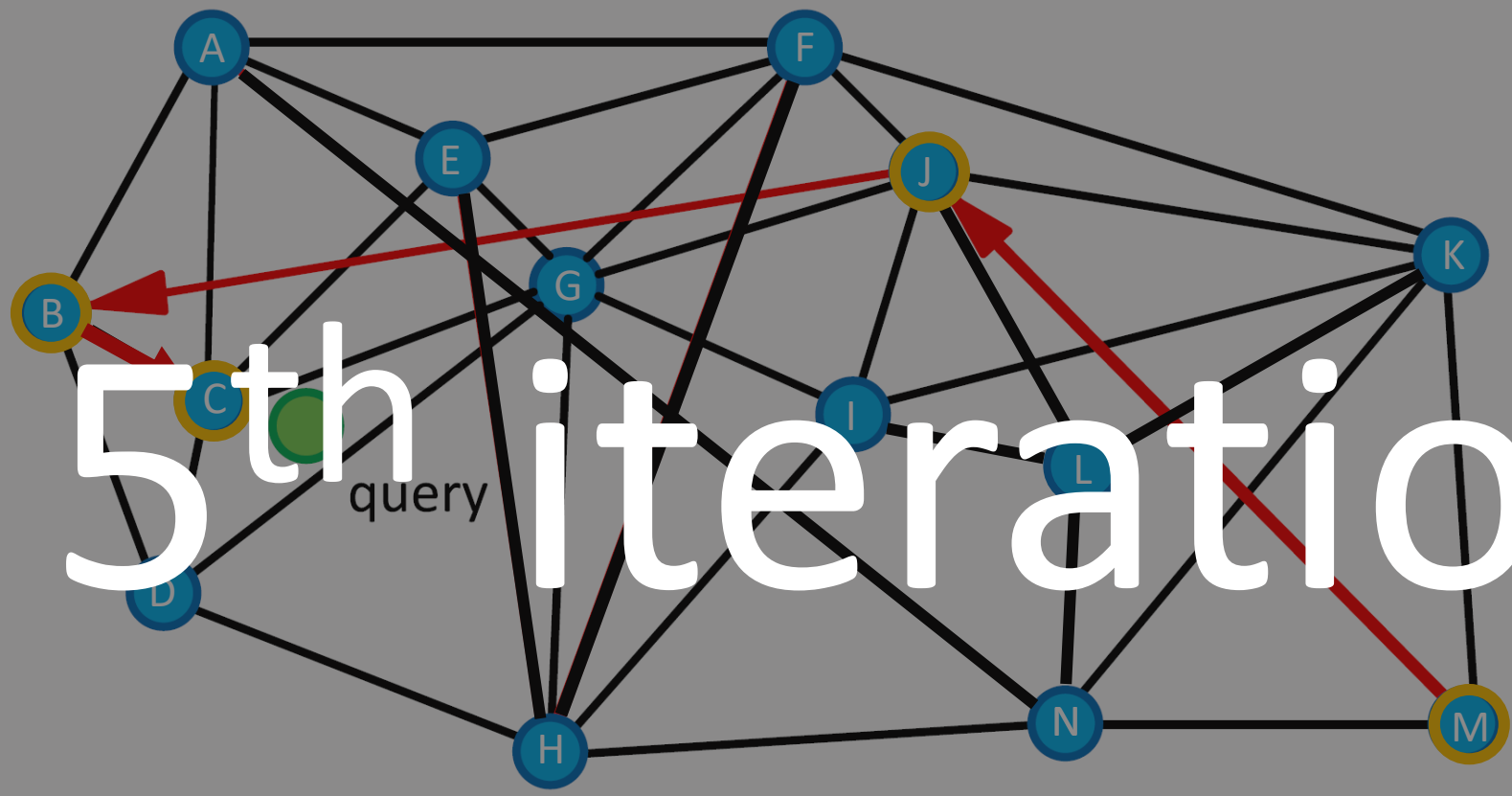
Search

Images are from [Malkov+, Information Systems, 2013]



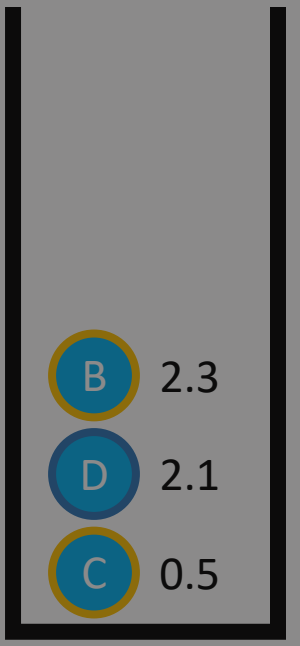


Images are from [Malkov+, Information Systems, 2013]



5th iteration

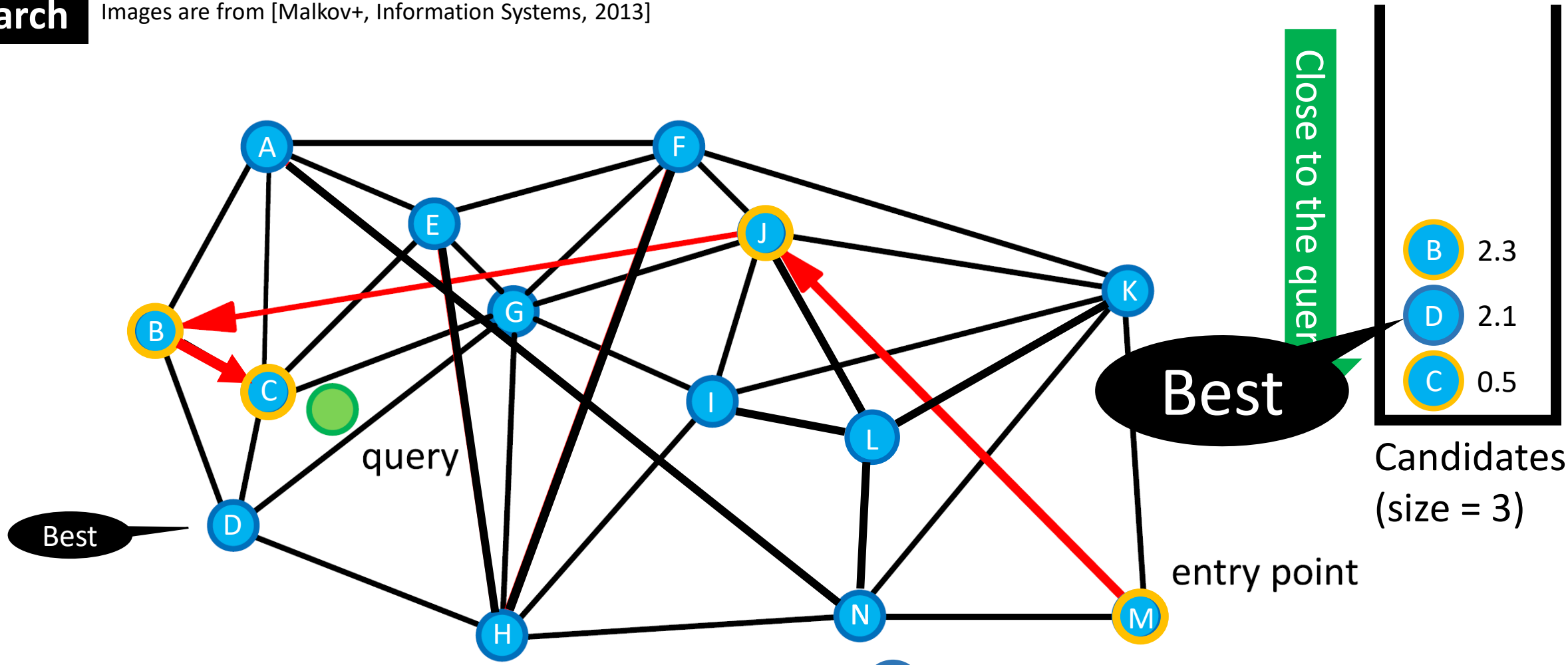
Close to the query



Candidates (size = 3)

Search

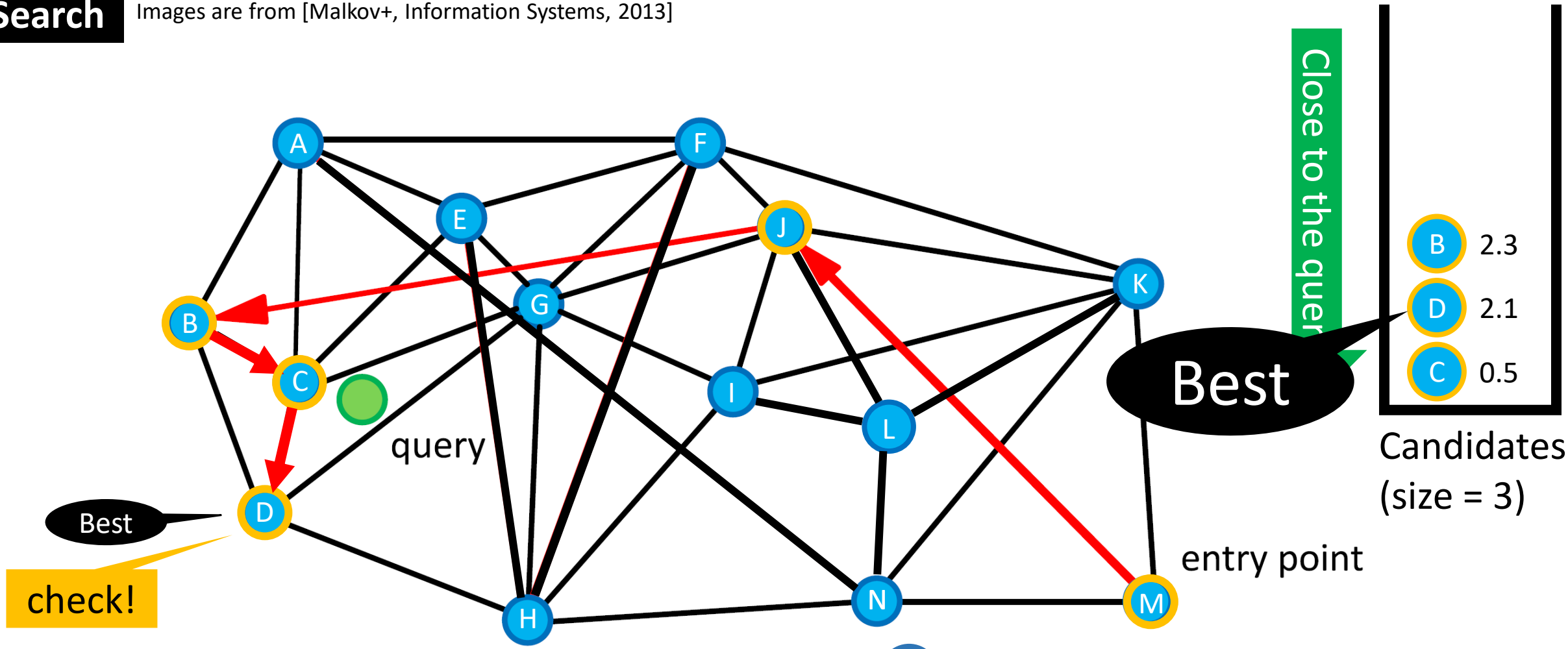
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D).

Search

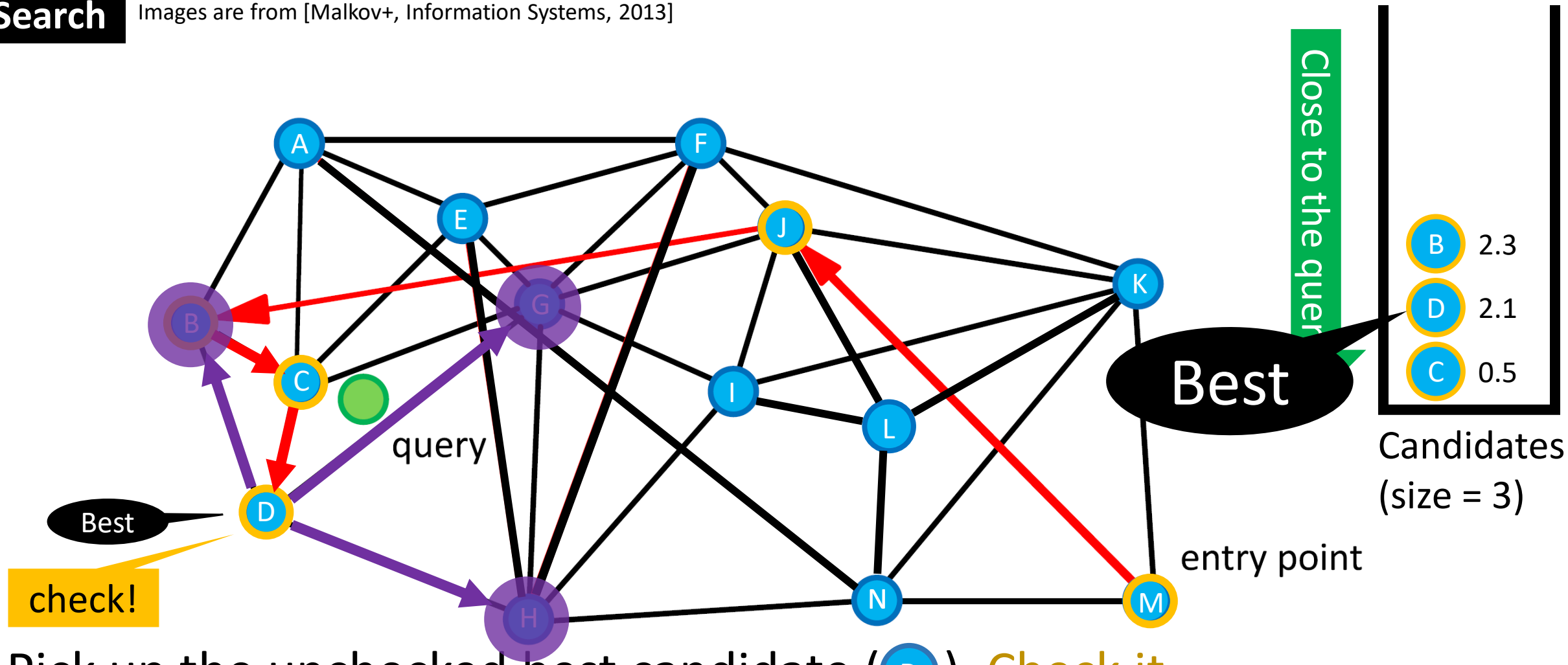
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.

Search

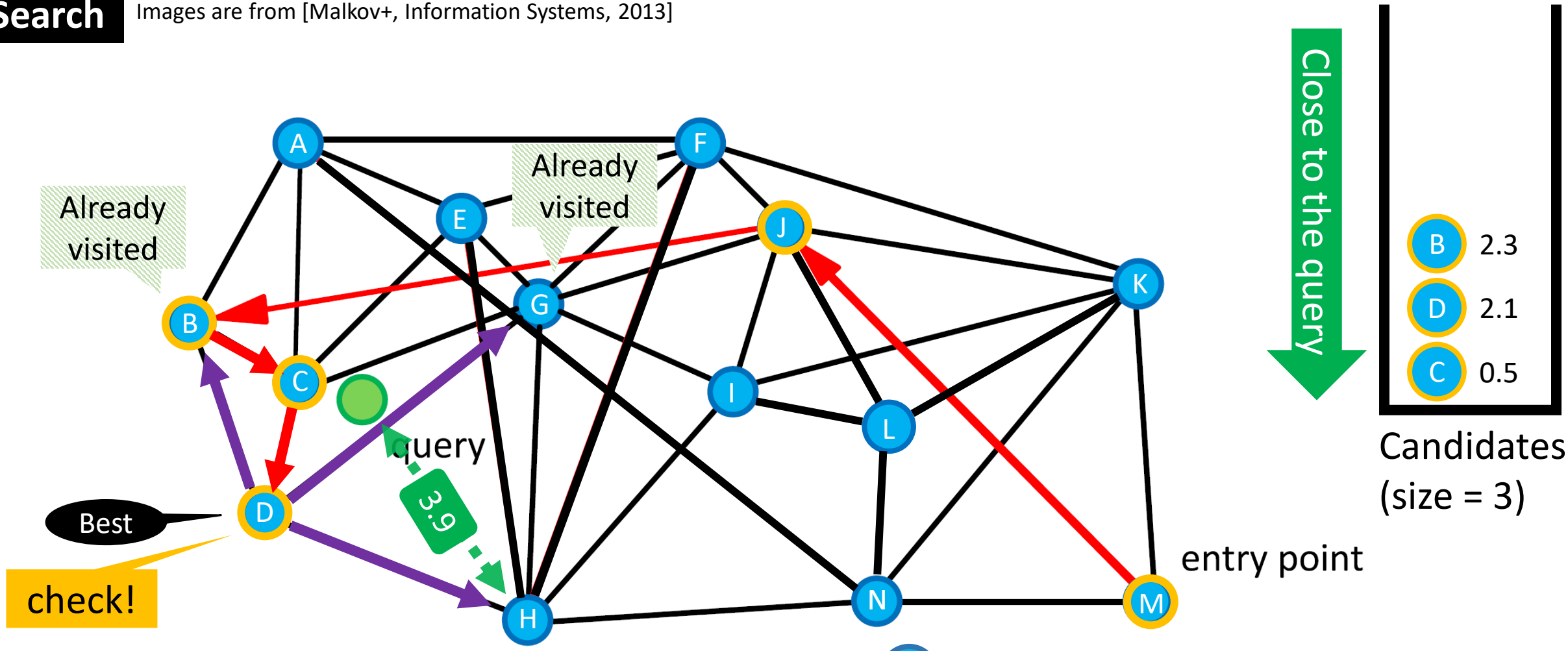
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.

Search

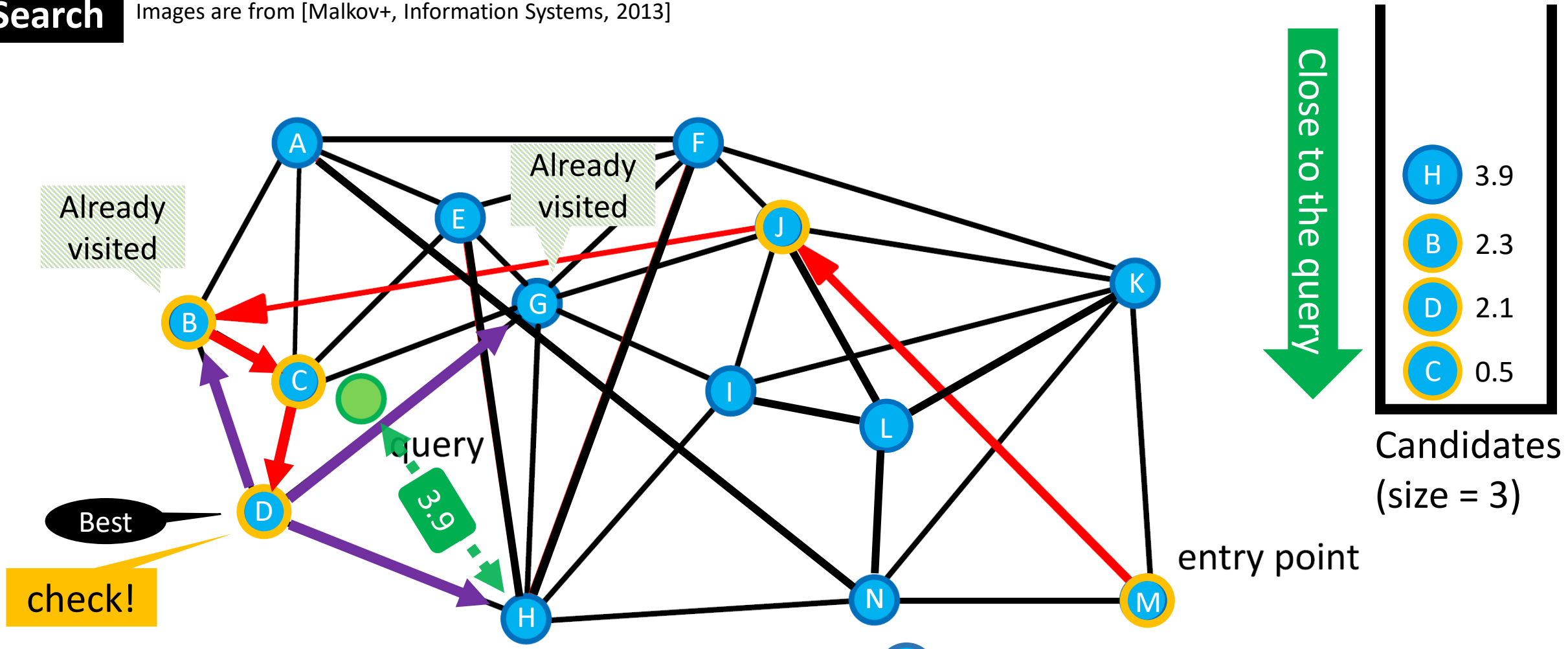
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.
- Record the distances to q.

Search

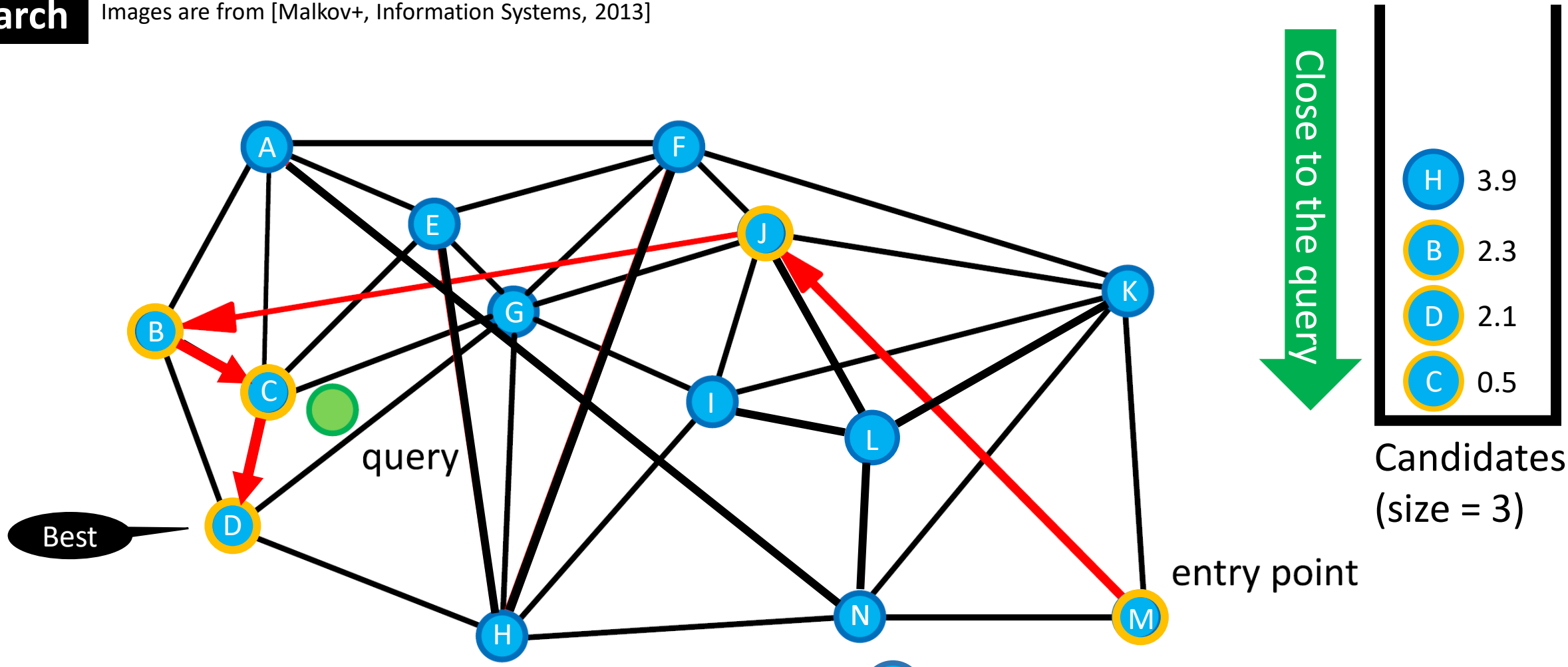
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.
- Record the distances to q.

Search

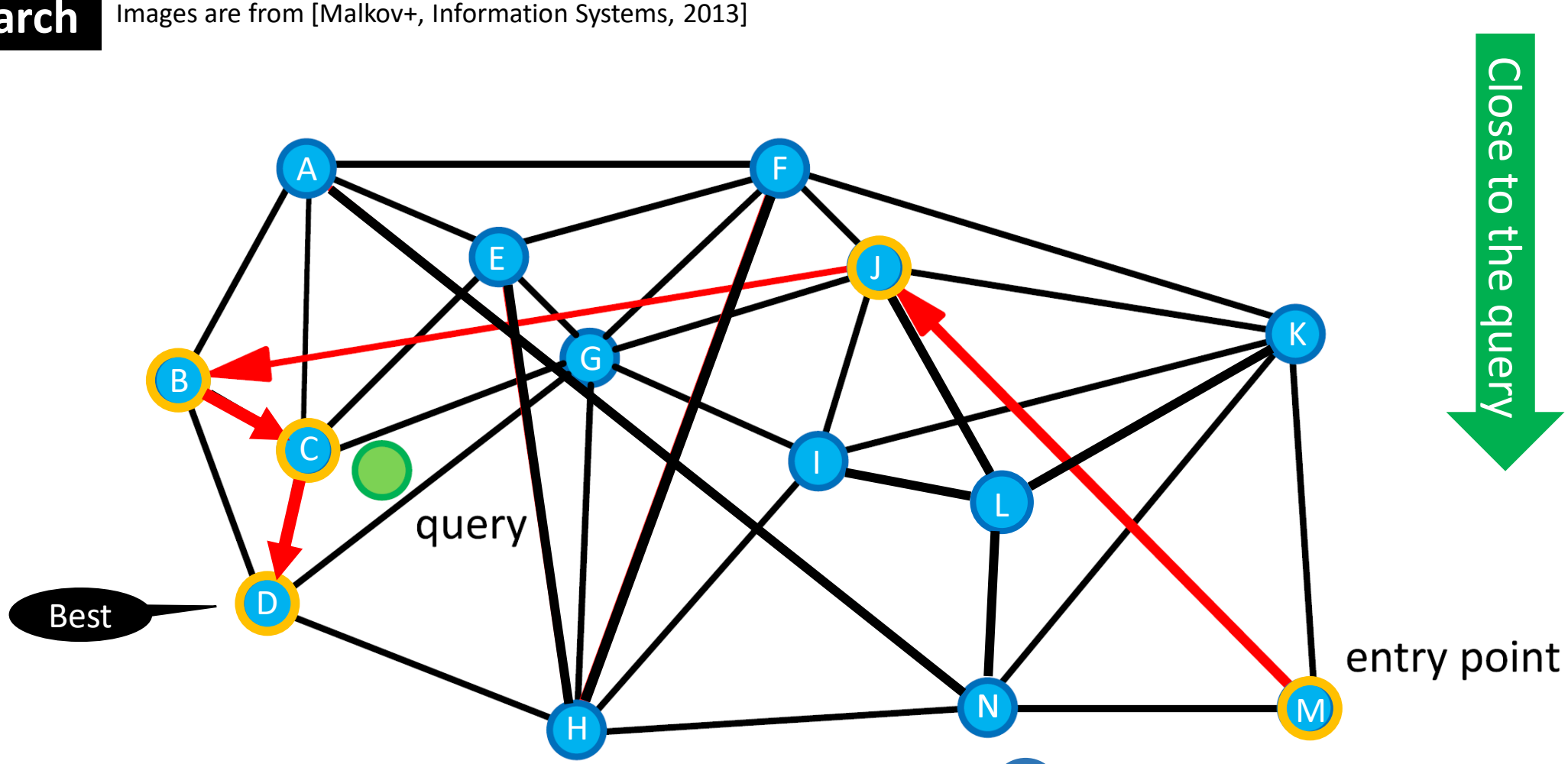
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.
- Record the distances to q.

Search

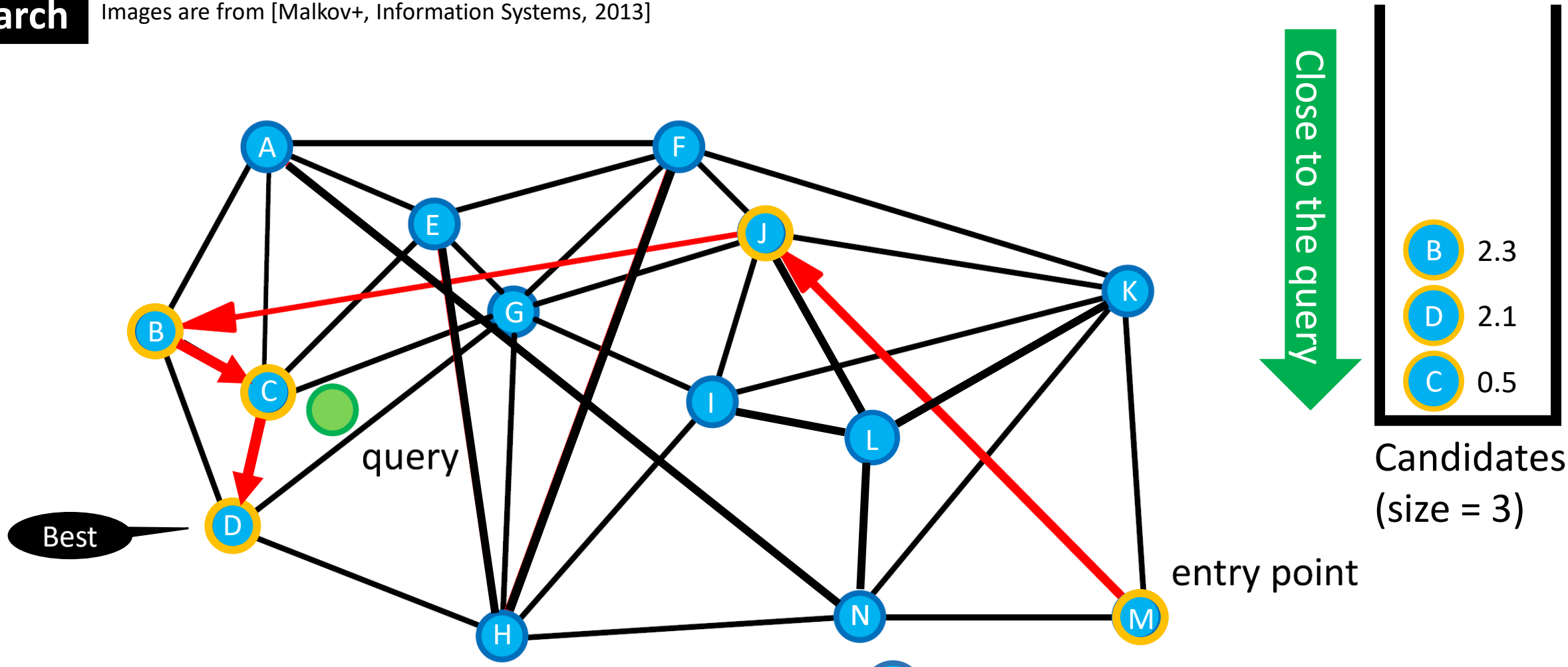
Images are from [Malkov+, Information Systems, 2013]



- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

Search

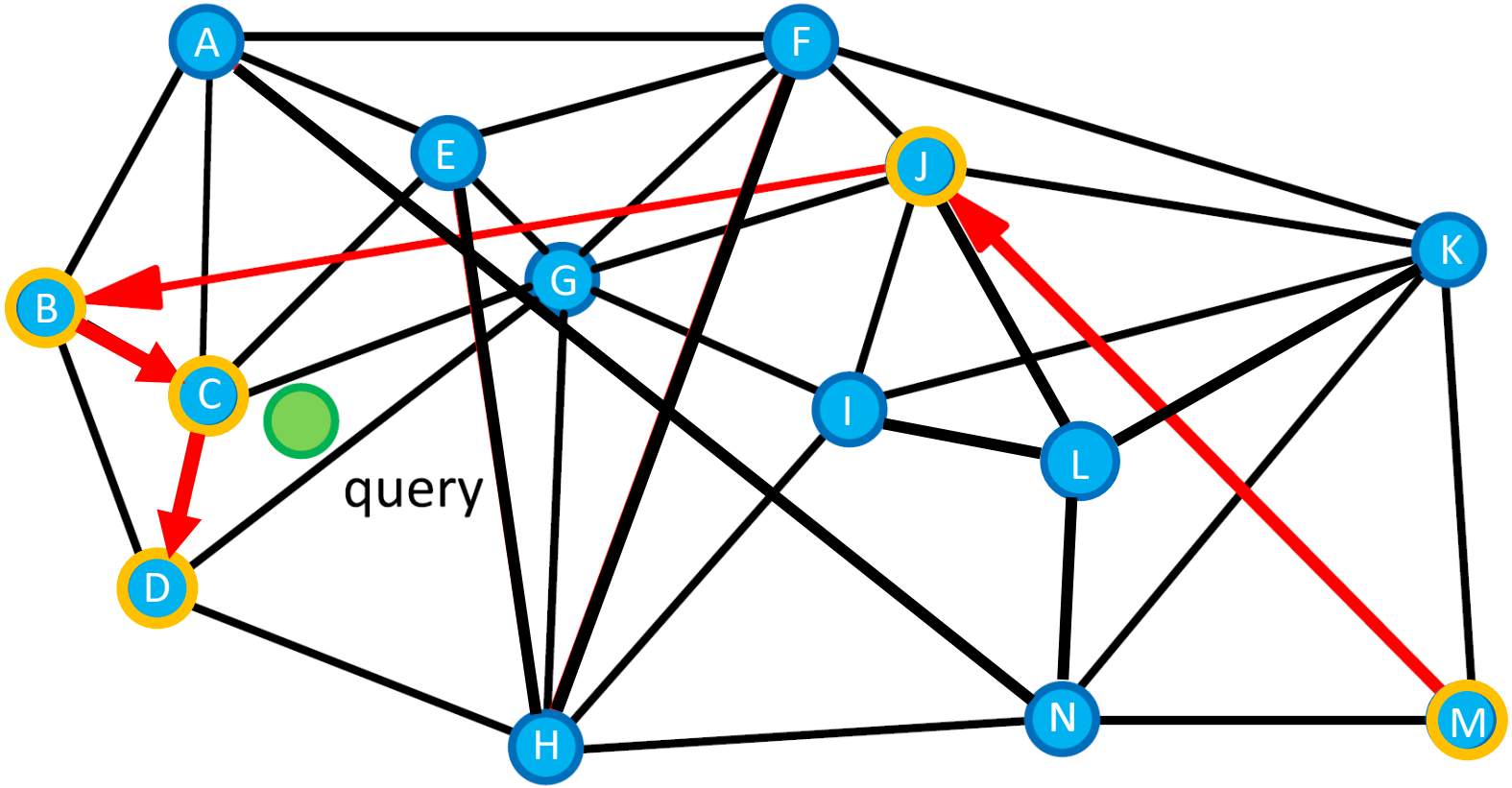
Images are from [Malkov+, Information Systems, 2013]

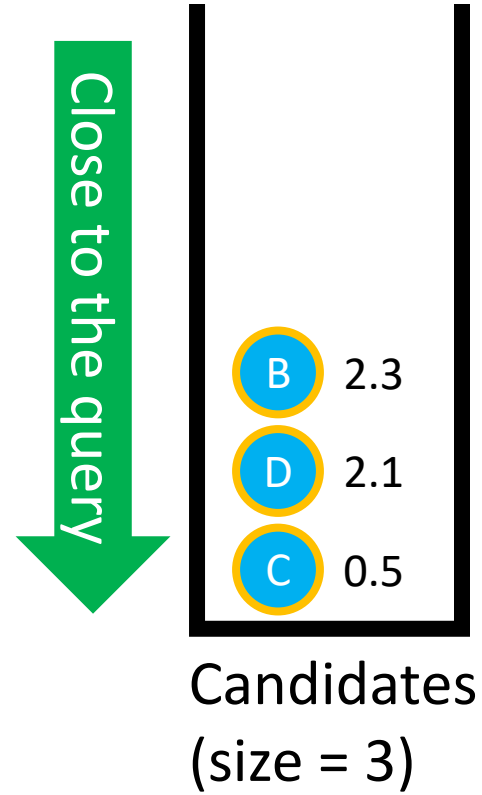


- Pick up the unchecked best candidate (D). Check it.
- Find the connected points.
- Record the distances to q.
- Maintain the candidates (size=3)

Search

Images are from [Malkov+, Information Systems, 2013]



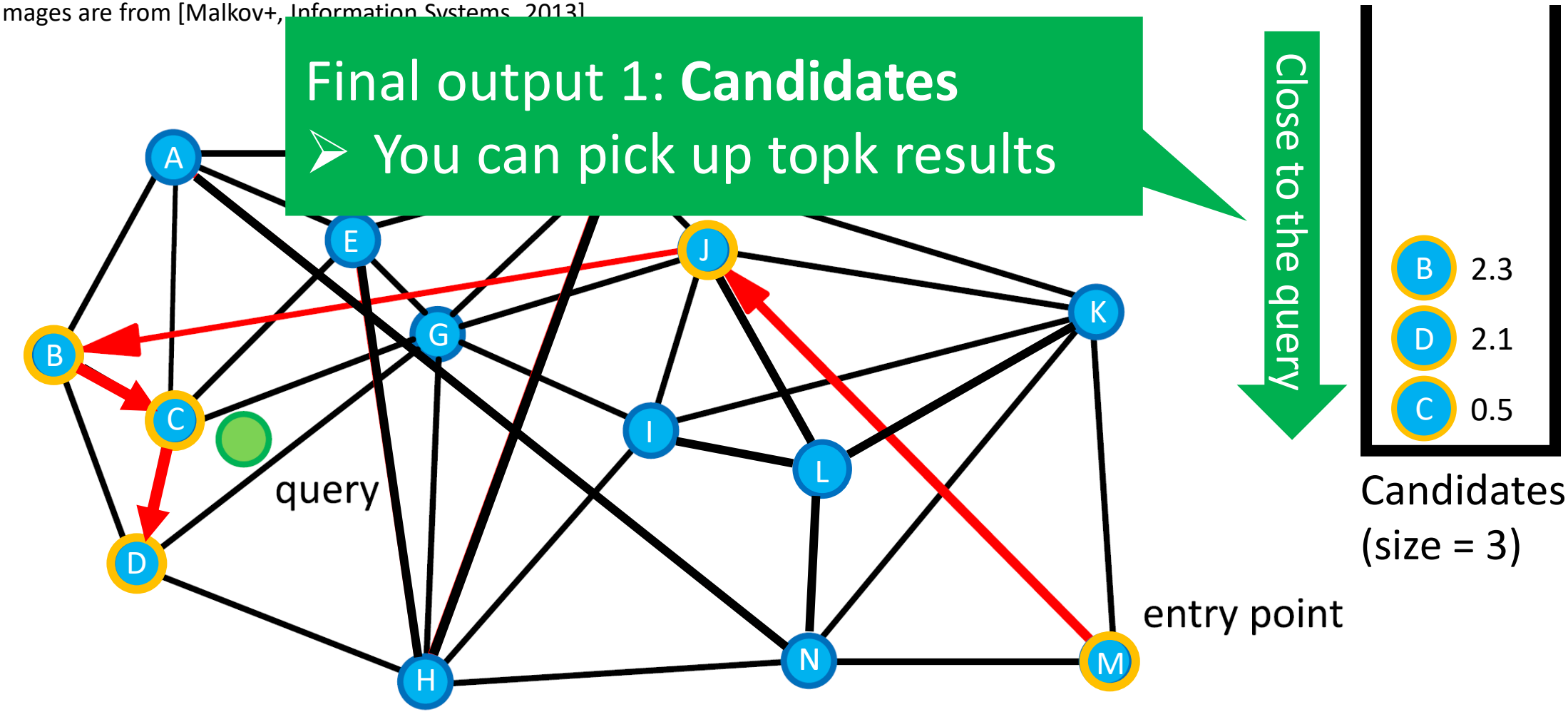


- All candidates are checked. Finish.
- Here, C is the closet to the query ( )

Images are from [Malkov+, Information Systems, 2013]

Final output 1: **Candidates**

- You can pick up topk results

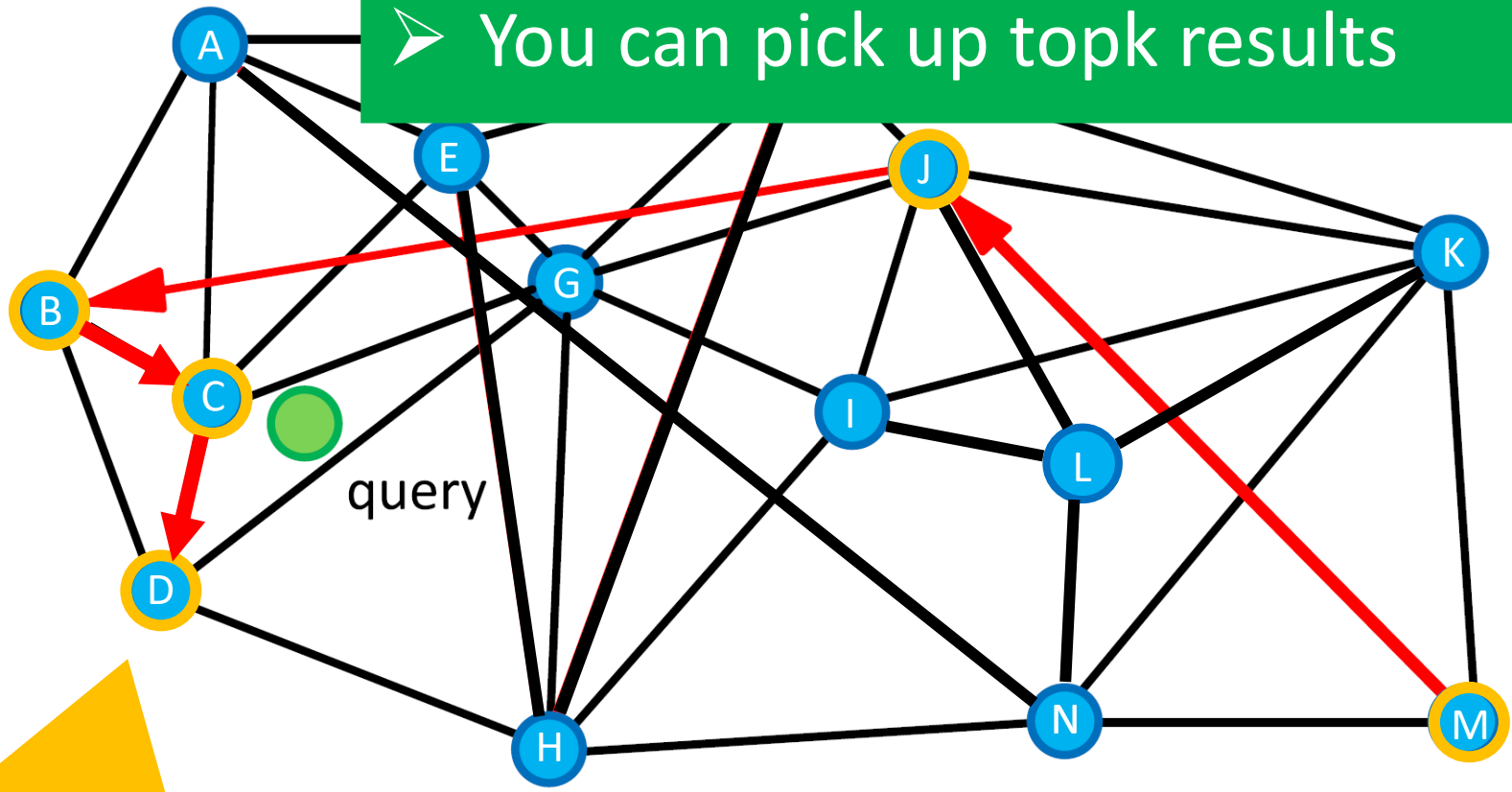


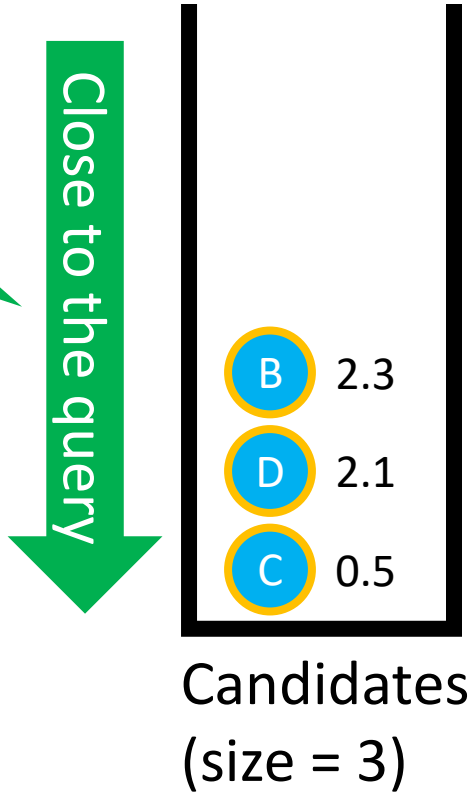
- All candidates are checked. Finish.
- Here, C is the closet to the query ( )

Images are from [Malkov+, Information Systems, 2013]

Final output 1: **Candidates**

- You can pick up topk results





- All ca es are checked. Finish.
- ery ( )

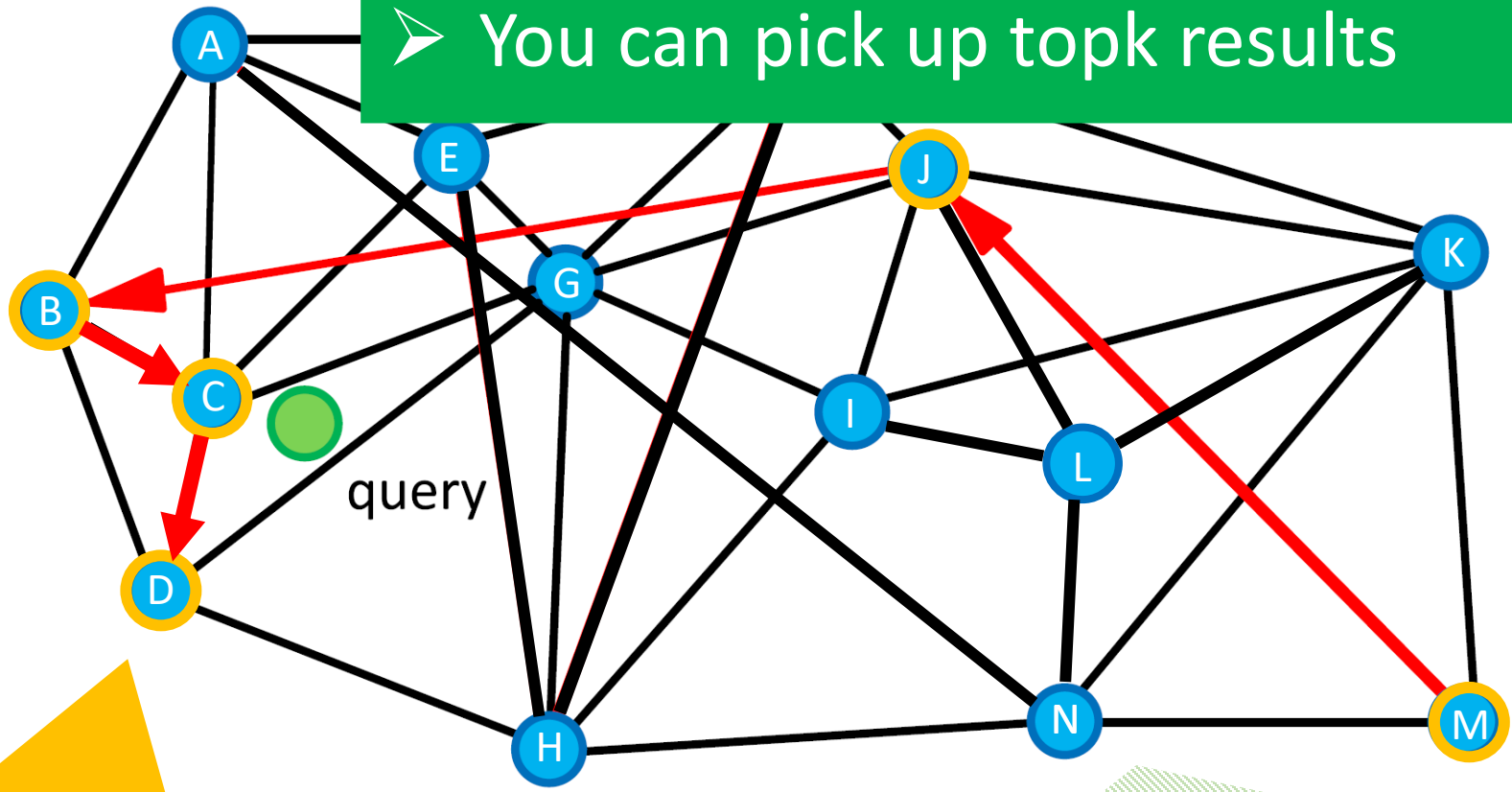
Final output 2: **Checked items**

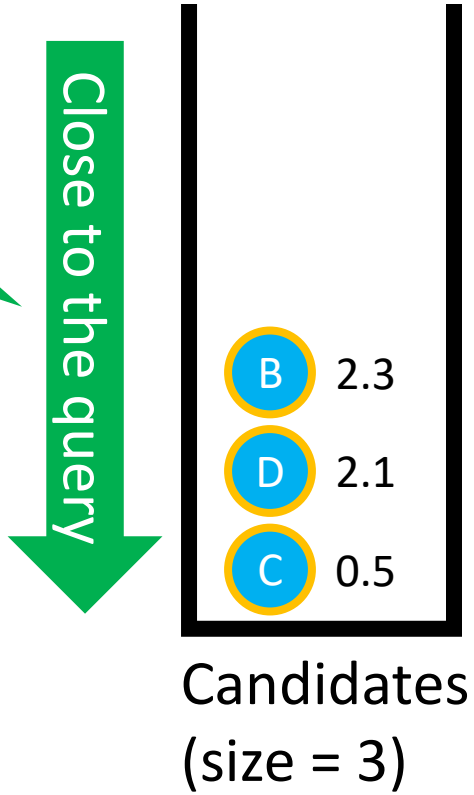
- i.e., search path

Images are from [Malkov+, Information Systems, 2013]

Final output 1: **Candidates**

- You can pick up topk results





- All ca es are checked. Finish
- ery

Final output 2: **Checked items**

- i.e., search path

Final output 3: **Visit flag**

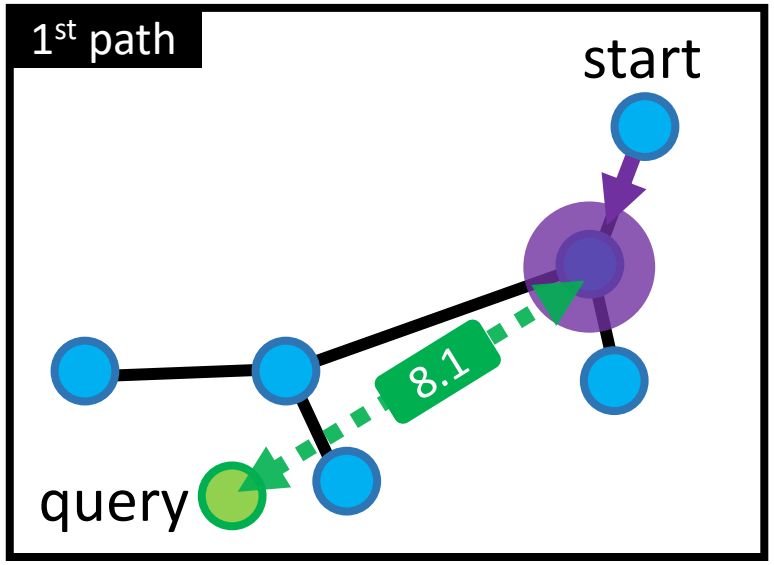
- For each item, visited or not

- Background
- Graph-based search
  - ✓ Basic (construction and search)
  - ✓ Observation
  - ✓ Properties
- Representative works
  - ✓ HNSW, NSG, NGT, Vamana
- Discussion

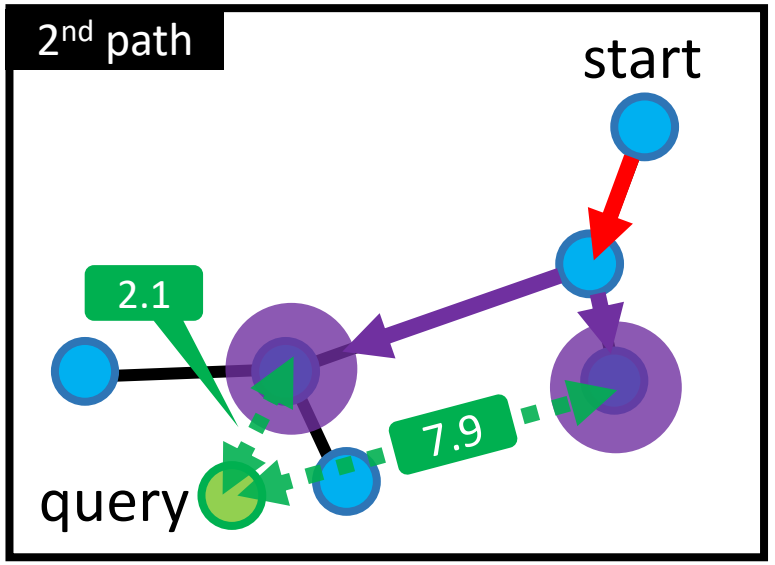
Observation: runtime

- Item comparison takes time; $O(D)$

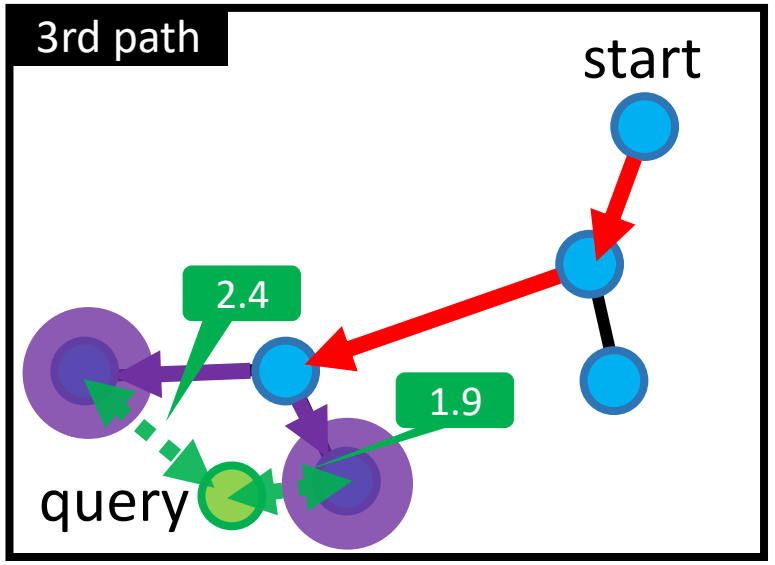
- The overall runtime ~ `#item_comparison`
  ~ `length_of_search_path` * `average_outdegree`



`outdegree = 1`



`outdegree = 2`



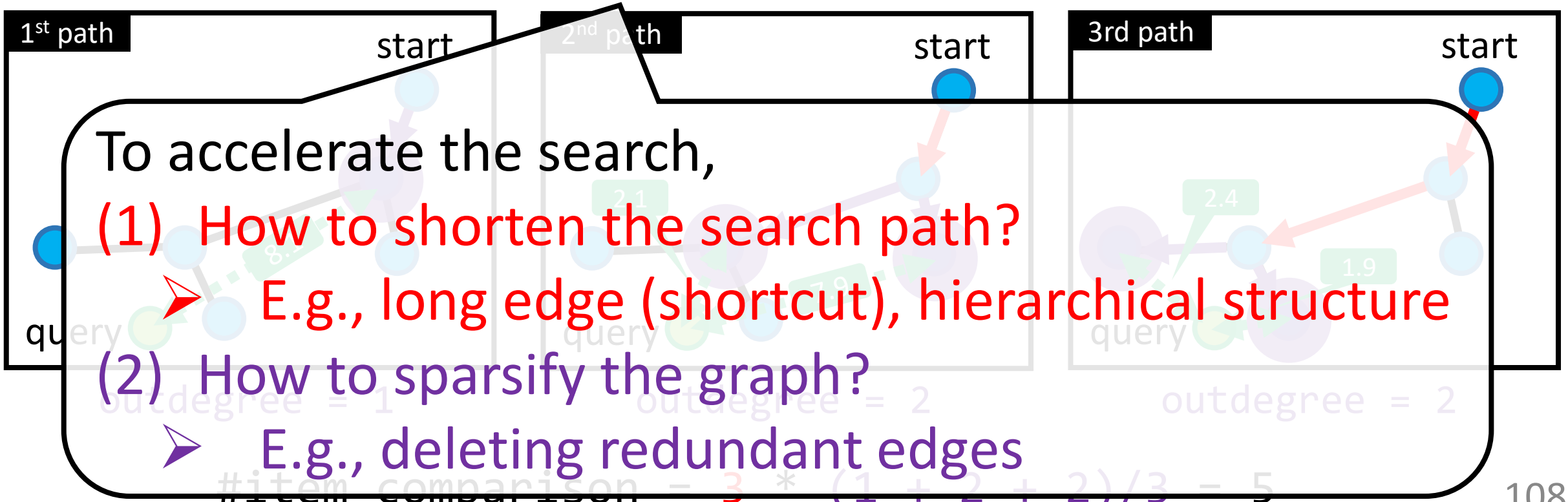
`outdegree = 2`

`#item_comparison = 3 * (1 + 2 + 2)/3 = 5`

Observation: runtime

- Item comparison takes time; $O(D)$

- The overall runtime ~ `#item_comparison`
  ~ `length_of_search_path * average_outdegree`

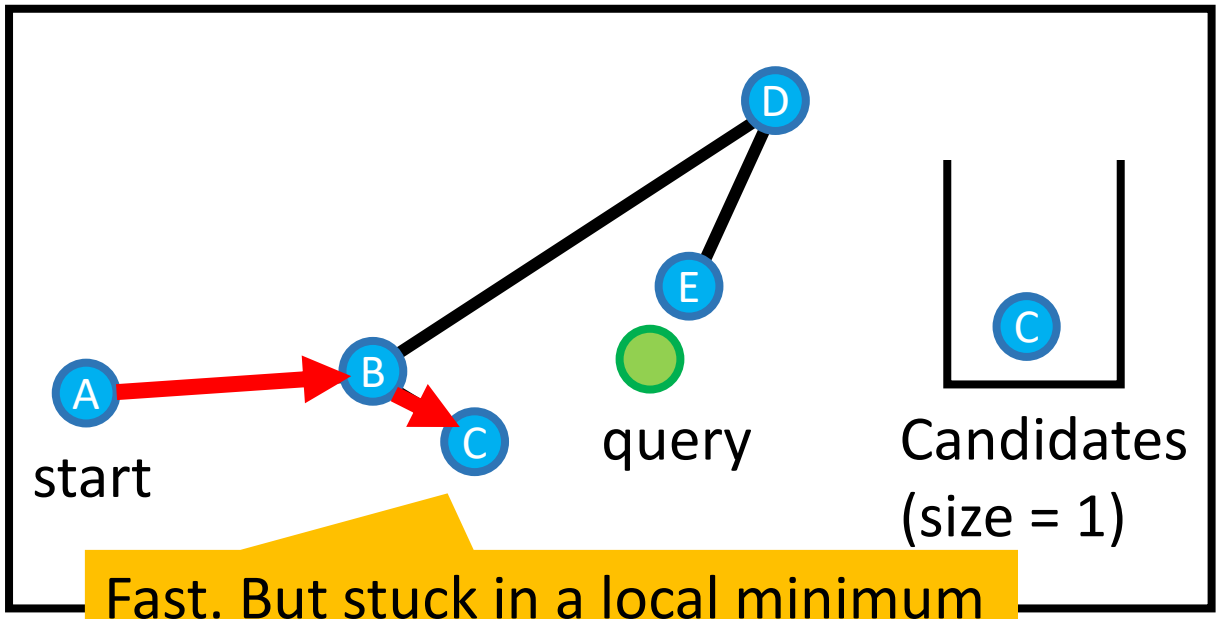


To accelerate the search,

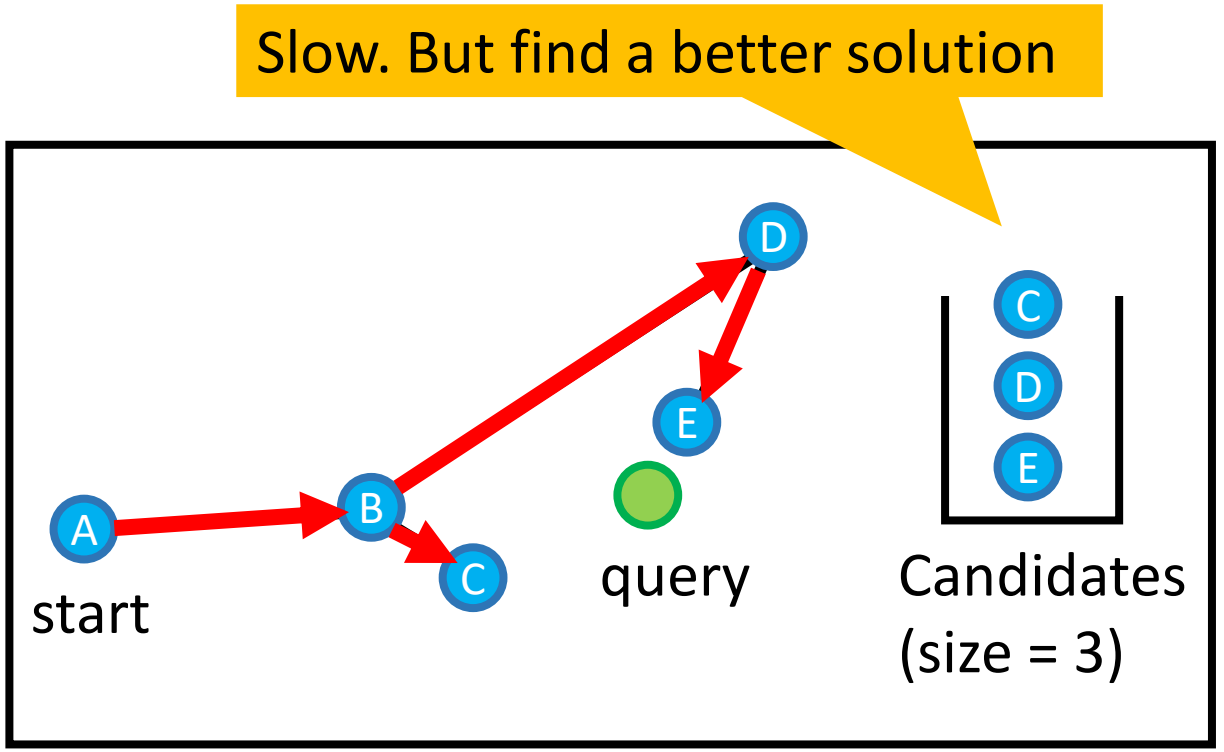
(1) How to shorten the search path?
- E.g., long edge (shortcut), hierarchical structure

(2) How to sparsify the graph?
- E.g., deleting redundant edges

Observation: candidate size



size = 1: Greedy search



size > 1: Beam search

- Larger candidate size, better but slower results
- Online parameter to control the trade-off
- Called “ef” in HNSW

Pseudo code

Algorithm 1. Search-on-Graph(G, **p**, **q**, l)

Require: graph G, start node **p**, query point **q**, candidate pool size l
Ensure: k nearest neighbors of **q**
1: $i = 0$, candidate pool $S = \emptyset$
2: S.add(**p**)
3: **while** $i < l$ **do**
4: i = the id of the first unchecked node p_i in S
5: mark $\mathbf{p}_i$ as checked
6: **for all** neighbor **n** of $\mathbf{p}_i$ in G **do**
7: **if** **n** has not been visited **then**
8: S.add(**n**)
9: **end if**
10: **end for**
11: sort S in ascending order of the distance to **q**
12: **if** S.size() $> l$ **then**
13: S.resize(l) // remove nodes from back of S to keep its
14: size no larger than l
15: **end if**
16: **end while**
17: return the first k nodes in S

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes
begin
initialize sets $\mathcal{L} \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \emptyset$
while $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$ **do**
let $p* \leftarrow \arg\min_{p \in \mathcal{L} \setminus \mathcal{V}} ||x_p - x_q||$
update $\mathcal{L} \leftarrow \mathcal{L} \cup N_{\text{out}}(p^*)$ and $\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$
if $|\mathcal{L}| > L$ **then**
update $\mathcal{L}$ to retain closest L points to x_q
return [closest k points from $\mathcal{L}$; $\mathcal{V}$]

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:
V = $\{v_0\}$ // a set of visited vertices
H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates
while *has runtime budget* **do**
v_i = SelectNearest(H, q)
for $\hat{v} \in Expand(v_i, G)$ **do**
if $\hat{v} \notin V$ **then**
$V := Add(V, \hat{v})$
$H := Insert(H, \hat{v}, d(\hat{v}, q))$
end
end
end
return TopK(V, q, k)

Learning to route [Baranchuk+, ICML 19]

- All papers have totally different pseudo code ☹
- Principles are the same. But small details are different.
- Hint: Explicitly state the data structure or not

Pseudo code

Candidates are stored in an array

Algorithm 1. Search-on-Graph(G, p, q, l)

Require: graph G, start node p, query point q, candidate pool size l

Ensure: k nearest neighbors of q

```
1: i = 0, candidate pool S = ∅
2: S.add(p)
3: while i < l do
4:     i = the id of the first unchecked node p_i in S
5:     mark p_i as checked
6:     for all neighbor n of p_i in G do
7:         if n has not been visited then
8:             S.add(n)
9:         end if
10:    end for
11:    sort S in ascending order of the distance to q
12:    if S.size() > l then
13:        S.resize(l) // remove nodes from back of S to keep its
14:        size no larger than l
15:    end if
16: end while
17: return the first k nodes in S
```

NSG [Cong+, VLDB 19]

Candidates are stored in a set

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$

Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes

```
begin
    initialize sets: L ← {s} and V ← ∅
    while L \ V ≠ ∅ do
        let p* ← arg min_{p∈L\V} ||x_p − x_q||
        update L ← L ∪ N_out(p*) and
          V ← V ∪ {p*}
        if |L| > L then
            update L to retain closest L
              points to x_q
    return [closest k points from L; V]
```

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k

Initialization:

V = $\{v_0\}$ // a set of visited vertices

H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates

```
while has runtime budget do
    v_i = SelectNearest(H, q)
    for v̂ ∈ Expand(v_i, G) do
        if v̂ ∉ V then
            V := Add(V, v̂)
            H := Insert(H, v̂, d(v̂, q))
        end
    end
end
return TopK(V, q, k)
```

Learning to route [Baranchuk+, ICML 19]

- Sort the array explicitly
- When need to sort, say "closest L points"
- Candidates are stored in a heap; automatically sorted

- Although they look totally different pseudo code ☹
- Principles are the same. But small details are different.
- Hint: Explicitly state the data structure or not

Pseudo code

Algorithm 1. Search-on-Graph(G, $\mathbf{p}$, $\mathbf{q}$, l)

Require: graph G, start node $\mathbf{p}$, query point $\mathbf{q}$, candidate pool size l
Ensure: k nearest neighbors of $\mathbf{q}$

```
1: i = 0, candidate pool S = ∅
2: S.add(p)
3: while i < l do
4:     i = the id of the first unchecked node p_i in S
5:     mark p_i as checked
6:     for all neighbor n of p_i in G do
7:         if n has not been visited then
8:             S.add(n)
9:         end if
10:    end for
11:    sort S in ascending order of the distance to q
12:    if S.size() > l then
13:        S.resize(l) // remove nodes from back of S to keep its
14:        size no larger than l
15:    end if
16: end while
17: return the first k nodes in S
```

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes

```
begin
    initialize sets L ← {s} and V ← ∅
    while L \ V ≠ ∅ do
        let p* ← arg min_{p∈L\V} ||x_p − x_q||
        update L ← L ∪ N_out(p*) and
        V ← V ∪ {p*}
        if |L| > L then
            update L to retain closest L
            points to x_q
    return [closest k points from L; V]
```

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:

```
V = {v_0} // a set of visited vertices
H = {v_0 : d(v_0, q)} // a heap of candidates
while has runtime budget do
    v_i = SelectNearest(H, q)
    for v̂ ∈ Expand(v_i, G) do
        if v̂ ∉ V then
            V := Add(V, v̂)
            H := Insert(H, v̂, d(v̂, q))
        end
    end
end
return TopK(V, q, k)
```

Learning to route [Baranchuk+, ICML 19]

- All pseudo codes have different pseudo code ☹
- Principles are the same. But small details are different.
- Just "check" ave totally
- Hint: Explicitly state the data structure or not

Checked items are stored in a set ("visit" in this code means "check" in our notation)

Pseudo code

Algorithm 1. Search-on-Graph(G, **p**, **q**, l)

Require: graph G, start node **p**, query point **q**, candidate pool size l
Ensure: k nearest neighbors of **q**
1: $i = 0$, candidate pool $S = \emptyset$
2: S.add(**p**)
3: **while** $i < l$ **do**
4: i = the id of the first unchecked node p_i in S
5: mark $\mathbf{p}_i$ as checked
6: **for all** neighbor **n** of $\mathbf{p}_i$ in G **do**
7: **if** **n** has not been visited **then**
8: S.add(**n**)
9: **end if**
10: **end for**
11: sort S in ascending order of the distance to **q**
12: **if** S.size() $> l$ **then**
13: S.resize(l) // remove nodes from back of S to keep its
14: size no larger than l
15: **end if**
16: **end while**
17: return the first k nodes in S

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes
begin
initialize sets $\mathcal{L} \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \emptyset$
while $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$ **do**
let $p* \leftarrow \arg\min_{p \in \mathcal{L} \setminus \mathcal{V}} ||x_p - x_q||$
update $\mathcal{L} \leftarrow \mathcal{L} \cup N_{\text{out}}(p^*)$ and $\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$
if $|\mathcal{L}| > L$ **then**
update $\mathcal{L}$ to retain closest L points to x_q
return [closest k points from $\mathcal{L}$; $\mathcal{V}$]

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:
V = $\{v_0\}$ // a set of visited vertices
H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates
while *has runtime budget* **do**
v_i = SelectNearest(H, q)
for $\hat{v} \in Expand(v_i, G)$ **do**
if $\hat{v} \notin V$ **then**
$V := Add(V, \hat{v})$
$H := Insert(H, \hat{v}, d(\hat{v}, q))$
end
end
end
return TopK(V, q, k)

Learning to route [Baranchuk+, ICML 19]

Visited item are simply said to be "visited"; implying an additional hidden data structure (array)

Visited items are stored in a set

- All papers have totally different pseudo code ☹
- Principles are the same. But small details are different.
- Hint: Explicitly state the data structure or not

Pseudo code

Algorithm 1. Search-on-Graph(G, **p**, **q**, l)

Require: graph G, start node **p**, query point **q**, candidate pool size l
Ensure: k nearest neighbors of **q**
1: $i = 0$, candidate pool $S = \emptyset$
2: S.add(**p**)
3: **while** $i < l$ **do**
4: i = the id of the first unchecked node p_i in S
5: mark $\mathbf{p}_i$ as checked
6: **for all** neighbor **n** of $\mathbf{p}_i$ in G **do**
7: **if** **n** has not been visited **then**
8: S.add(**n**)
9: **end if**
10: **end for**
11: sort S in ascending order of the distance to **q**
12: **if** S.size() $> l$ **then**
13: S.resize(l) // remove nodes from back of
14: size no larger than l
15: **end if**
16: **end while**
17: return the first k nodes in S

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes

begin
initialize sets $\mathcal{L} \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \emptyset$
while $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$ **do**
let $p* \leftarrow \arg\min_{p \in \mathcal{L} \setminus \mathcal{V}} ||\mathsf{x}_p - \mathsf{x}_q||$
update $\mathcal{L} \leftarrow \mathcal{L} \cup N_{\text{out}}(p^*)$ and
$\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$
if $|\mathcal{L}| > L$ **then**
update $\mathcal{L}$ to retain closest L
; $\mathcal{V}$]

IPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:
V = $\{v_0\}$ // a set of visited vertices
H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates
while *has runtime budget* **do**
v_i = SelectNearest(H, q)
for $\hat{v} \in Expand(v_i, G)$ **do**
if $\hat{v} \notin V$ **then**
$V := Add(V, \hat{v})$
$H := Insert(H, \hat{v}, d(\hat{v}, q))$
end
end
end
return TopK(V, q, k)

Learning to route [Baranchuk+, ICML 19]

Termination condition??

- All papers have totally different pseudo code ☹
- Principles are the same. But small details are different.
- Hint: Explicitly state the data structure or not

Pseudo code

Algorithm 1. Search-on-Graph(G, **p**, **q**, l)

Require: graph G, start node **p**, query point **q**, candidate pool size l
Ensure: k nearest neighbors of **q**
1: $i = 0$, candidate pool $S = \emptyset$
2: S.add(**p**)
3: **while** $i < l$ **do**
4: i = the id of the first unchecked node p_i in S
5: mark $\mathbf{p}_i$ as checked
6: **for all** neighbor **n** of $\mathbf{p}_i$ in G **do**
7: **if** **n** has not been visited **then**
8: S.add(**n**)
9: **end if**
10: **end for**
11: sort S in ascending order of the distance to **q**
12: **if** S.size() $> l$ **then**
13: S.resize(l) // remove nodes from back of S to keep its size no larger than l
14: **end if**
15: **end while**
16: return the first k nodes in S

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes
begin
initialize sets $\mathcal{L} \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \emptyset$
while $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$ **do**
let $p* \leftarrow \arg\min_{p \in \mathcal{L} \setminus \mathcal{V}} ||\mathrm{x}_p - \mathrm{x}_q||$
update $\mathcal{L} \leftarrow \mathcal{L} \cup N_{\text{out}}(p^*)$ and $\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$
if $|\mathcal{L}| > L$ **then**
update $\mathcal{L}$ to retain closest L points to x_q
return [closest k points from $\mathcal{L}$; $\mathcal{V}$]

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:
V = $\{v_0\}$ // a set of visited vertices
H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates
while *has runtime budget* **do**
v_i = SelectNearest(H, q)
for $\hat{v} \in Expand(v_i, G)$ **do**
if $\hat{v} \notin V$ **then**
$V := Add(V, \hat{v})$
$H := Insert(H, \hat{v}, d(\hat{v}, q))$
end
end
end
return TopK(V, q, k)

Learning to route [Baranchuk+, ICML 19]

- All papers have totally different pseudo code ☹
- Principle: the same. Details: slightly different
- Hint: Explicitly state the data structure or not
- My explanation was based on NSG, but with slight modifications for simplicity:
  - Candidates are stored in an automatically-sorted array
  - Termination condition is “all candidates are checked”

Pseudo code

Algorithm 1. Search-on-Graph(G, **p**, **q**, l)

Require: graph G, start node **p**, query point **q**, candidate pool size l
Ensure: k nearest neighbors of **q**
1: $i = 0$, candidate pool $S = \emptyset$
2: S.add(**p**)
3: **while** $i < l$ **do**
4: i = the id of the first unchecked node p_i in S
5: mark $\mathbf{p}_i$ as checked
6: **for all** neighbor **n** of $\mathbf{p}_i$ in G **do**
7: **if** **n** has not been visited **then**
8: S.add(**n**)
9: **end if**
10: **end for**
11: sort S in ascending order of the distance to **q**
12: **if** S.size() $> l$ **then**
13: S.resize(l) // remove nodes from back of S to keep its size no larger than l
14: **end if**
15: **end while**
16: **return** the first k nodes in S

NSG [Cong+, VLDB 19]

Algorithm 1: GreedySearch(s, x_q, k, L)

Data: Graph G with start node s, query x_q, result size k, search list size $L \geq k$
Result: Result set $\mathcal{L}$ containing k-approx NNs, and a set $\mathcal{V}$ containing all the visited nodes
begin
initialize sets $\mathcal{L} \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \emptyset$
while $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$ **do**
let $p* \leftarrow \arg\min_{p \in \mathcal{L} \setminus \mathcal{V}} ||x_p - x_q||$
update $\mathcal{L} \leftarrow \mathcal{L} \cup N_{out}(p^*)$ and $\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$
update $\mathcal{L}$ to retain closest L points to x_q
return [closest k points from $\mathcal{L}$; $\mathcal{V}$]

DiskANN [Subramanya+, NeurIPS 19]

Algorithm 1 Beam search

Data: graph G, query q, initial vertex v_0, output size k
Initialization:
V = $\{v_0\}$ // a set of visited vertices
H = $\{v_0 : d(v_0, q)\}$ // a heap of candidates
while *has runtime budget* **do**
v_i = SelectNearest(H, q)
for $\hat{v} \in Expand(v_i, G)$ **do**
if $\hat{v} \notin V$ **then**
$V := Add(V, \hat{v})$
$H := Insert(H, \hat{v}, d(\hat{v}, q))$
end
end
return TopK(V, q, k)

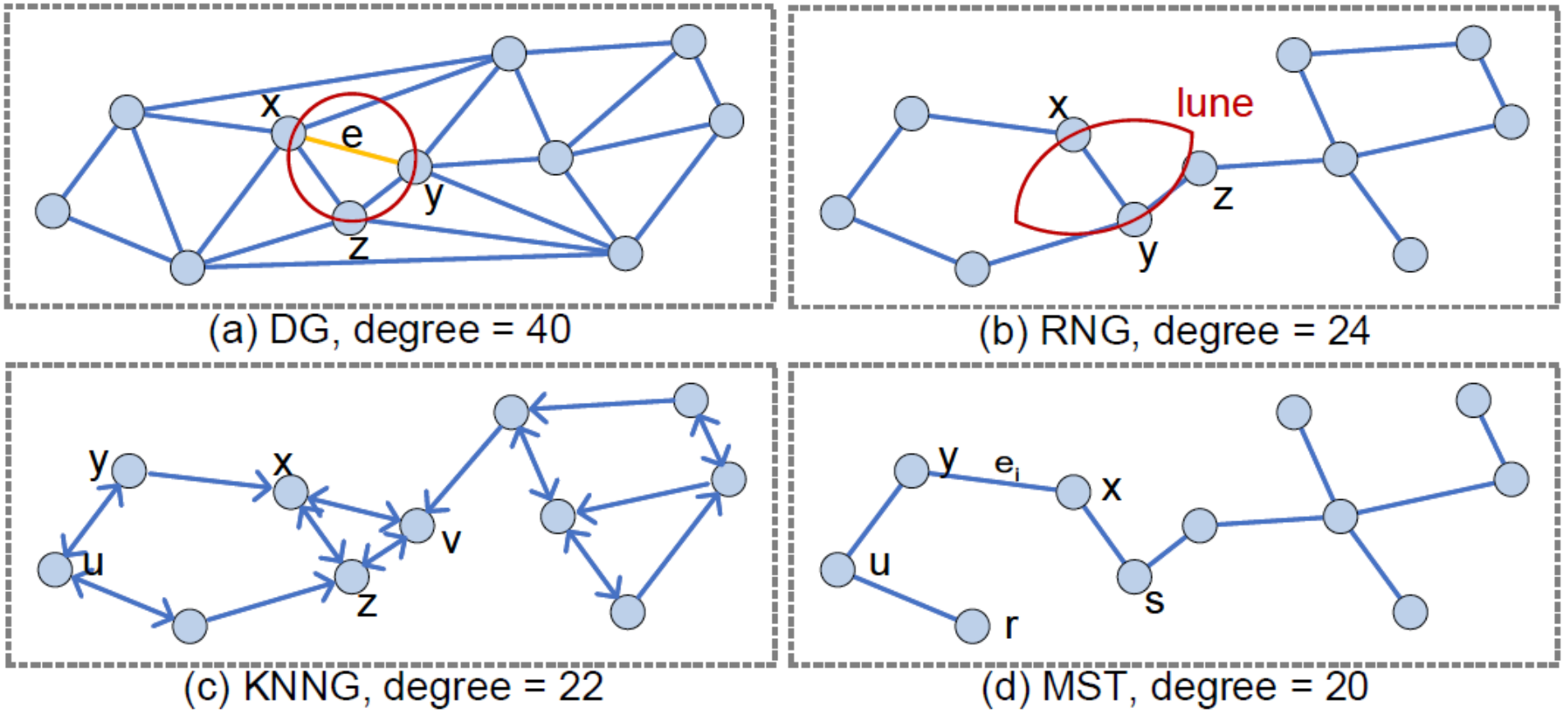
Learning to route [Baranchuk+, ICML 19]

Formal (?) definition would be helpful for everyone

- All papers have totally different pseudo code ☹
- Principles are the same. But small parts are very different
- Hint: Explicitly state the data structure or not

- Background
- Graph-based search
  - ✓ Basic (construction and search)
  - ✓ Observation
  - ✓ Properties
- Representative works
  - ✓ HNSW, NSG, NGT, Vamana
- Discussion

Base graph

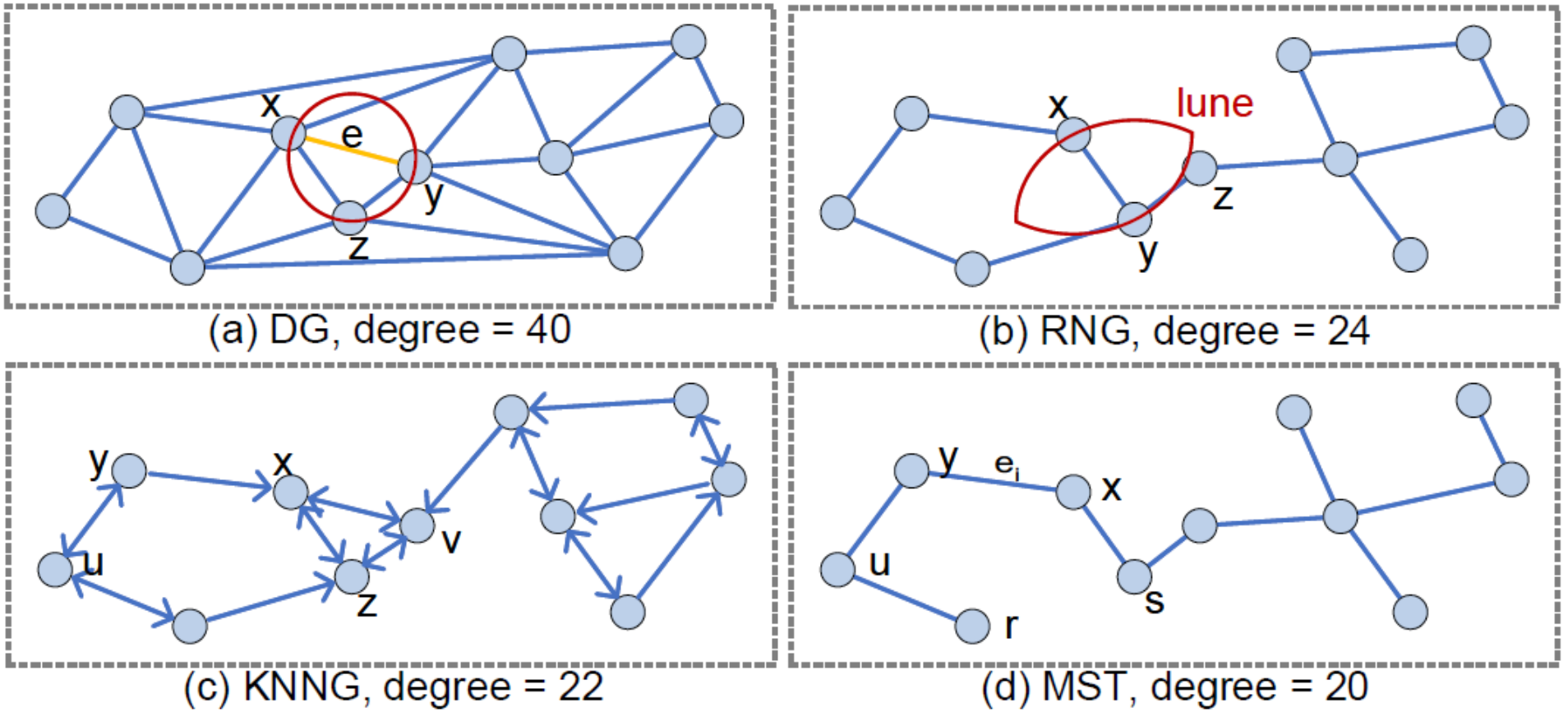


- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Base graph

Principal:
- Not too dense: Search is slow for dense graph
- But moderately dense: Each points should be reachable

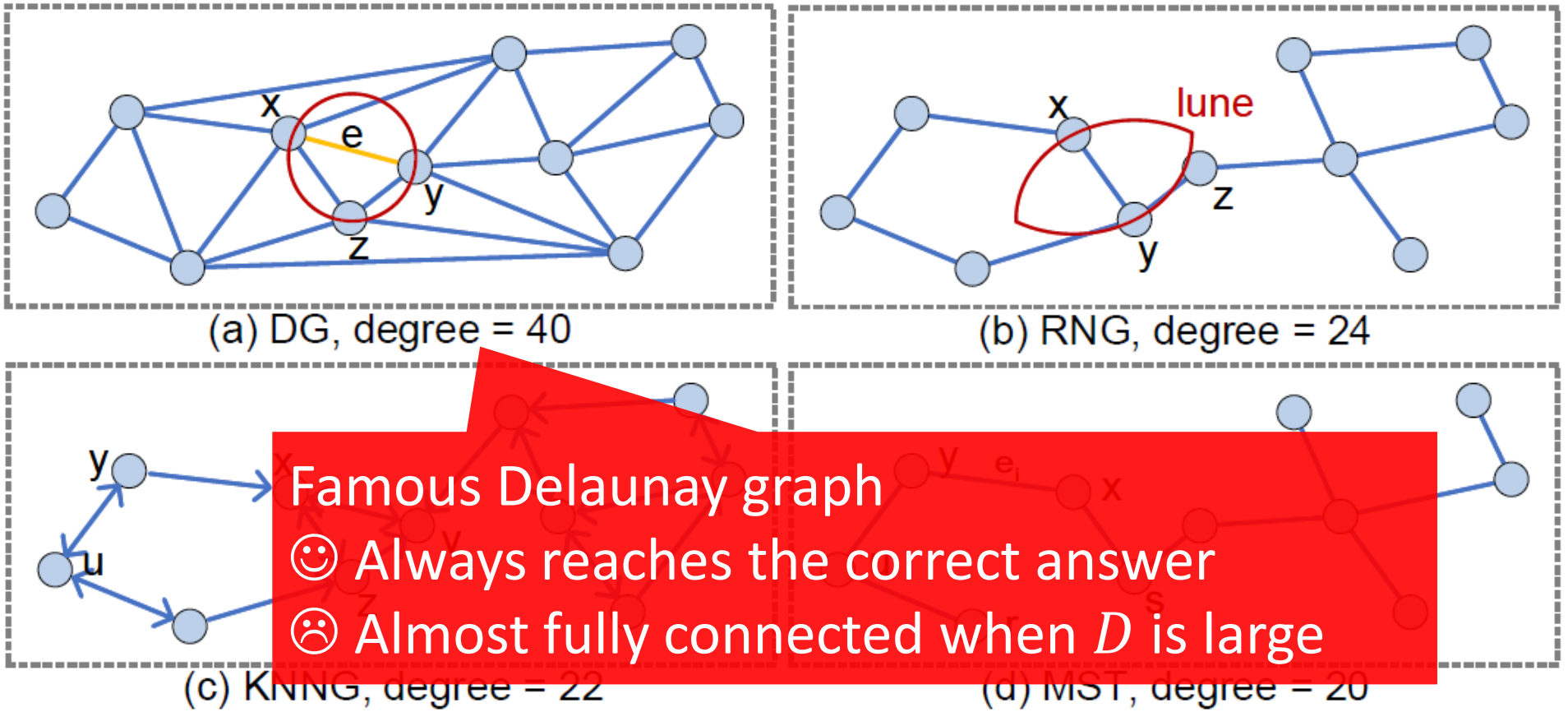


- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Base graph

Principal:
- Not too dense: Search is slow for dense graph
- But moderately dense: Each points should be reachable



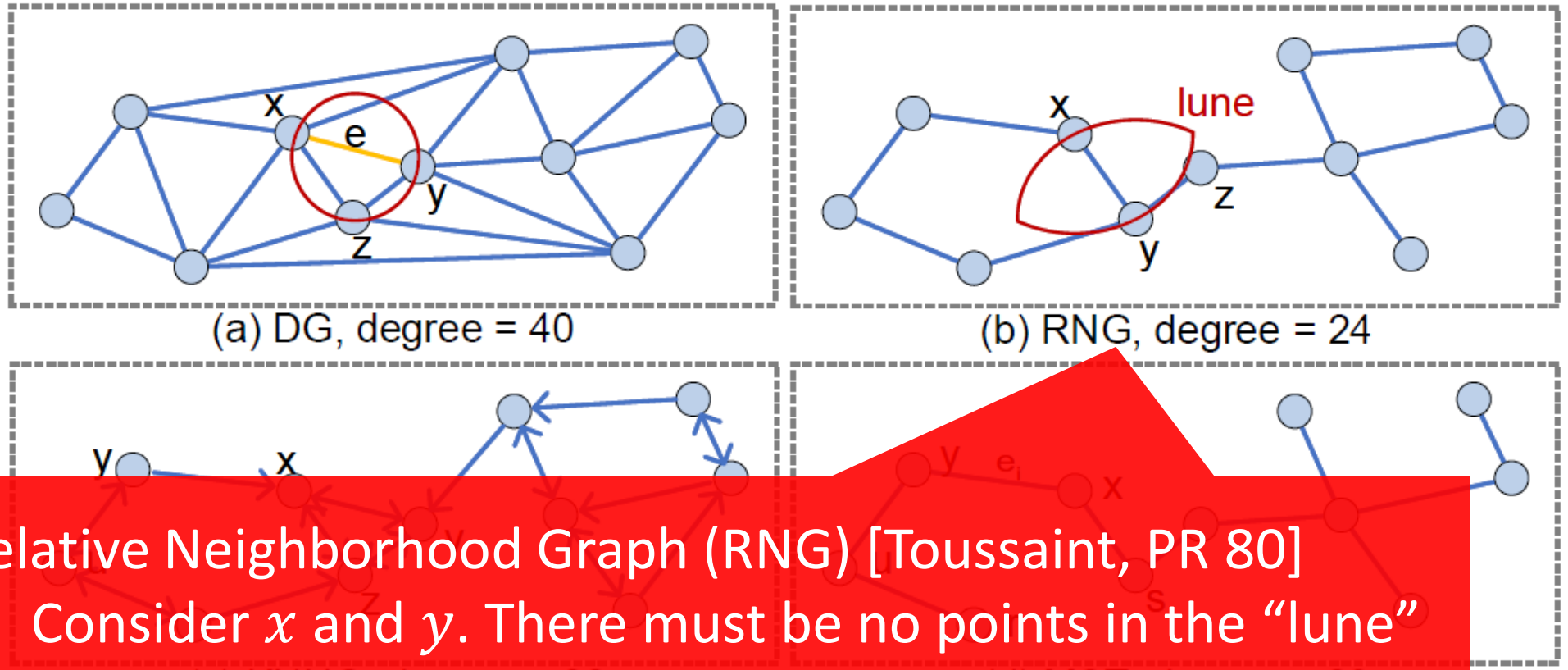
Famous Delaunay graph
☺ Always reaches the correct answer
☹ Almost fully connected when D is large

- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Base graph

Principal:
- Not too dense: Search is slow for dense graph
- But moderately dense: Each points should be reachable



Relative Neighborhood Graph (RNG) [Toussaint, PR 80]
- Consider x and y. There must be no points in the "lune"
- Can cut off redundant edges
- Not famous in general, but widely used in ANN
- Will review again later

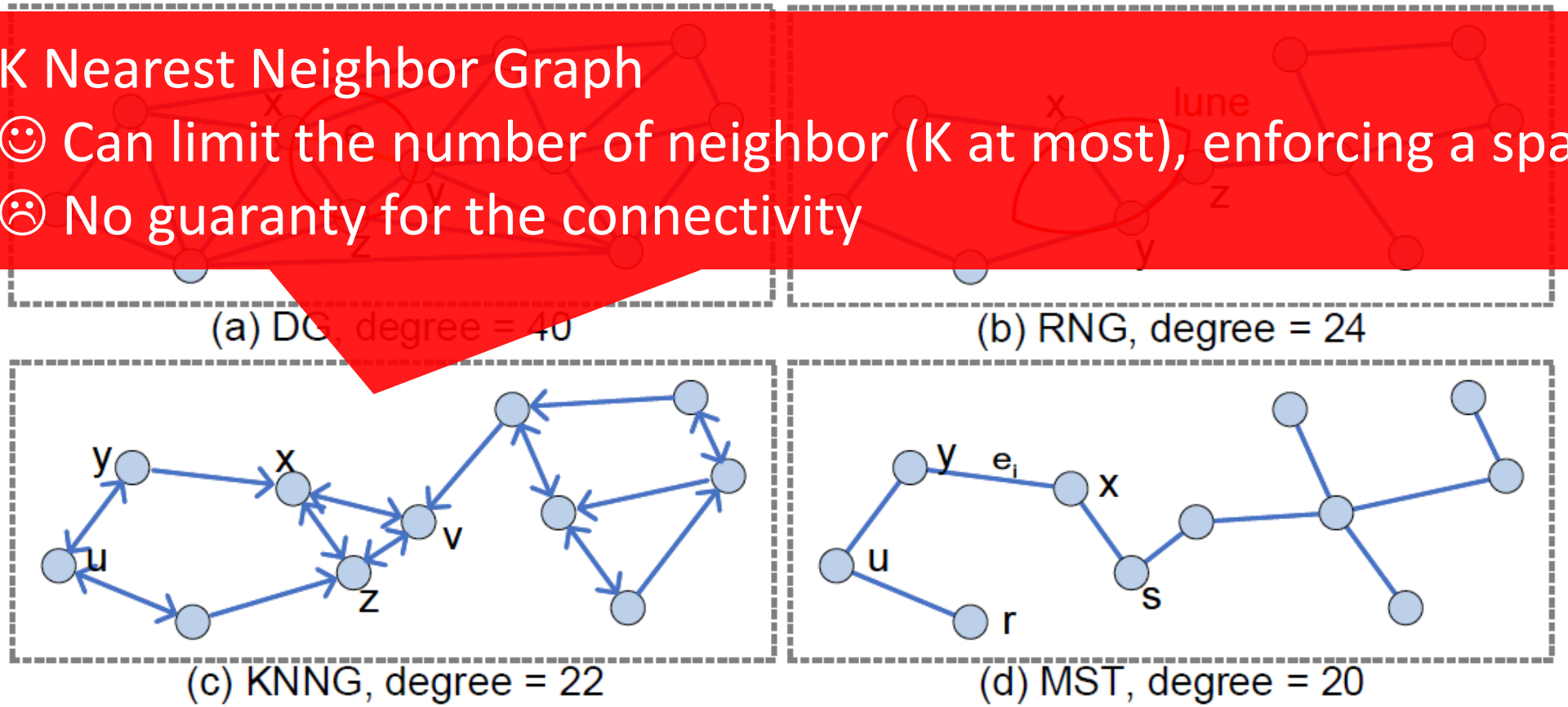
- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Base graph

Principal:
- Not too dense: Search is slow for dense graph
- But moderately dense: Each points should be reachable

K Nearest Neighbor Graph
☺ Can limit the number of neighbor (K at most), enforcing a sparsity
☹ No guaranty for the connectivity



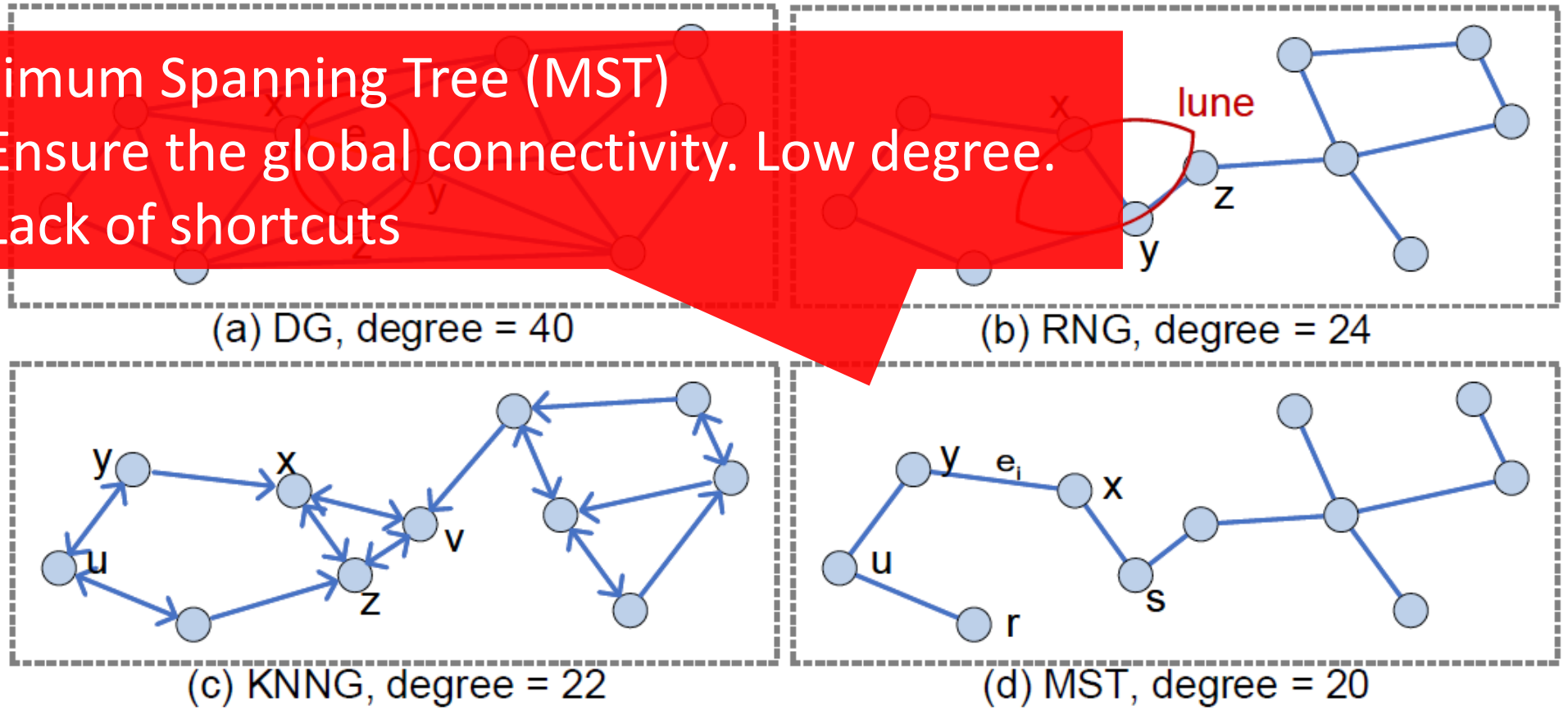
- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Base graph

Principal:
- Not too dense: Search is slow for dense graph
- But moderately dense: Each points should be reachable

Minimum Spanning Tree (MST)
☺ Ensure the global connectivity. Low degree.
☹ Lack of shortcuts



(a) DG, degree = 40
(b) RNG, degree = 24
(c) KNNG, degree = 22
(d) MST, degree = 20

- Although there are many graph algorithms, there exists four base graphs.
- These base graphs are (1) slow to be constructed, and (2) often too dense
- Each algorithm often improves one of the base graphs

Images are from an excellent survey paper [Wang+, VLDB 2021]

Graph search algorithms

Table 2: Summary of important representative graph-based ANNS algorithms

Algorithm	Base Graph	Edge	Build Complexity	Search Complexity
KGraph [31]	KNNG	directed	$O(\|S\|^{1.14})$	$O(\|S\|^{0.54})^{\ddagger}$
NGT [46]	KNNG+DG+RNG	directed	$O(\|S\|^{1.14})^{\ddagger}$	$O(\|S\|^{0.59})^{\ddagger}$
SPTAG [27]	KNNG+RNG	directed	$O(\|S\| \cdot \log(\|S\|^{c} + t^{t}))^{\dagger}$	$O(\|S\|^{0.68})^{\ddagger}$
NSW [65]	DG	undirected	$O(\|S\| \cdot \log^{2}(\|S\|))^{\ddagger}$	$O(\log^{2}(\|S\|))^{\dagger}$
IEH [54]	KNNG	directed	$O(\|S\|^{2} \cdot \log(\|S\|) + \|S\|^{2})^{\ddagger}$	$O(\|S\|^{0.52})^{\ddagger}$
FANNG [43]	RNG	directed	$O(\|S\|^{2} \cdot \log(\|S\|))$	$O(\|S\|^{0.2})$
HNSW [67]	DG+RNG	directed	$O(\|S\| \cdot \log(\|S\|))$	$O(\log(\|S\|))$
EFANNA [36]	KNNG	directed	$O(\|S\|^{1.13})^{\ddagger}$	$O(\|S\|^{0.55})^{\ddagger}$
DPG [61]	KNNG+RNG	undirected	$O(\|S\|^{1.14} + \|S\|)^{\ddagger}$	$O(\|S\|^{0.28})^{\ddagger}$
NSG [38]	KNNG+RNG	directed	$O(\|S\|^{\frac{1+c}{c}} \cdot \log(\|S\|) + \|S\|^{1.14})^{\dagger}$	$O(\log(\|S\|))$
HCNNG [72]	MST	directed	$O(\|S\| \cdot \log(\|S\|))$	$O(\|S\|^{0.4})^{\ddagger}$
Vamana [88]	RNG	directed	$O(\|S\|^{1.16})^{\ddagger}$	$O(\|S\|^{0.75})^{\ddagger}$
NSSG [37]	KNNG+RNG	directed	$O(\|S\| + \|S\|^{1.14})$	$O(\log(\|S\|))$

† c, t are the constants. ‡ Complexity is not informed by the authors; we derive it based on the related papers' descriptions and experimental estimates. See Appendix D for deatils.

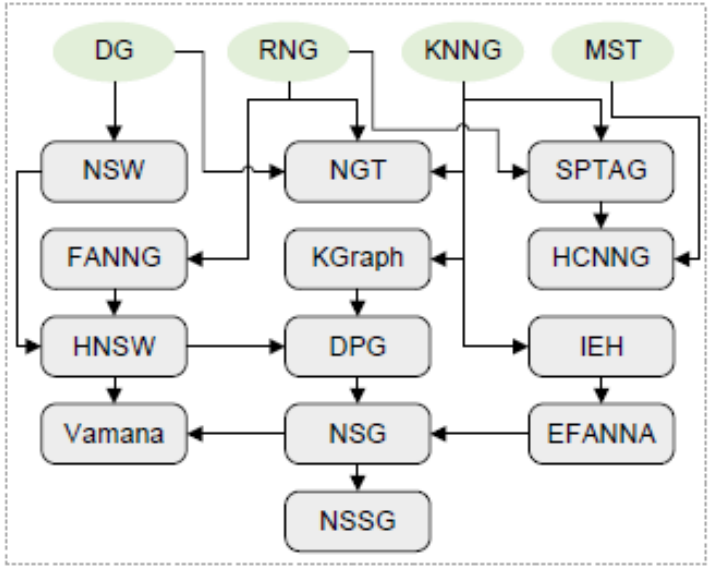
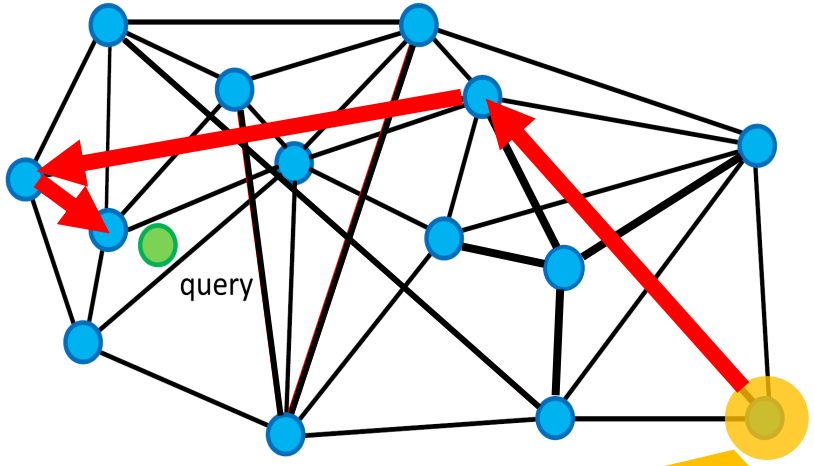


Figure 3: Roadmaps of graph-based ANNS algorithms. The arrows from a base graph (green shading) to an algorithm (gray shading) and from one algorithm to another indicate the dependence and development relationships.

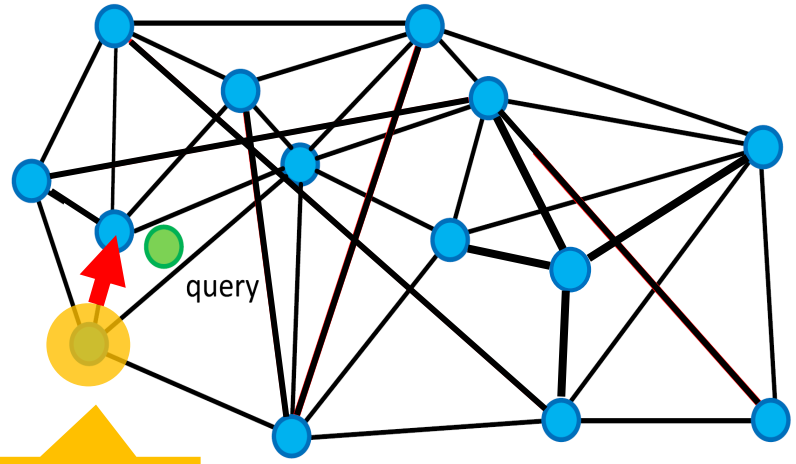
- Lots of algorithms 😂
- The basic structure is same: (1) designing a good graph + (2) beam search

Images are from an excellent survey paper [Wang+, VLDB 2021]

The initial seed matters

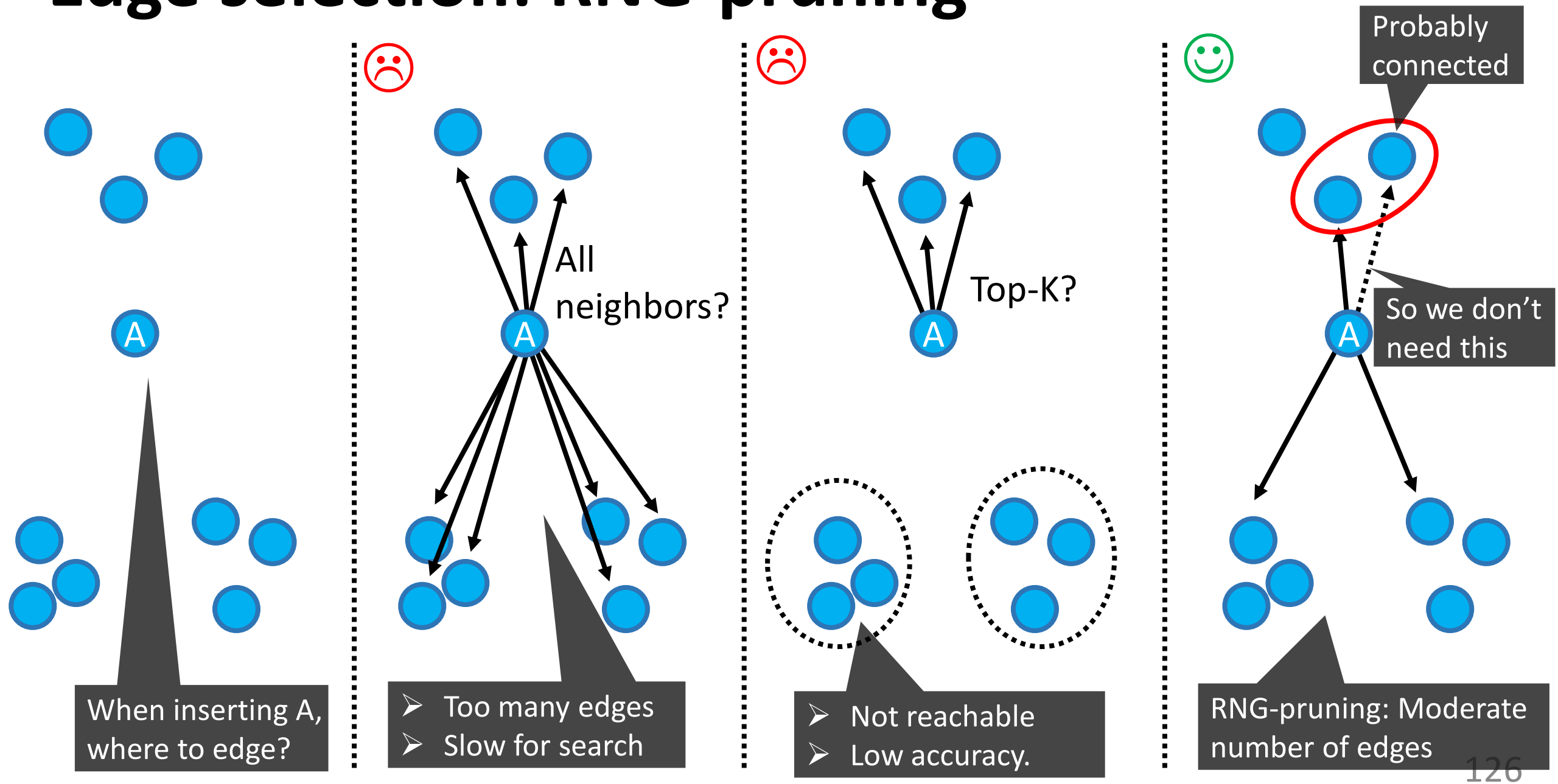


v.s.

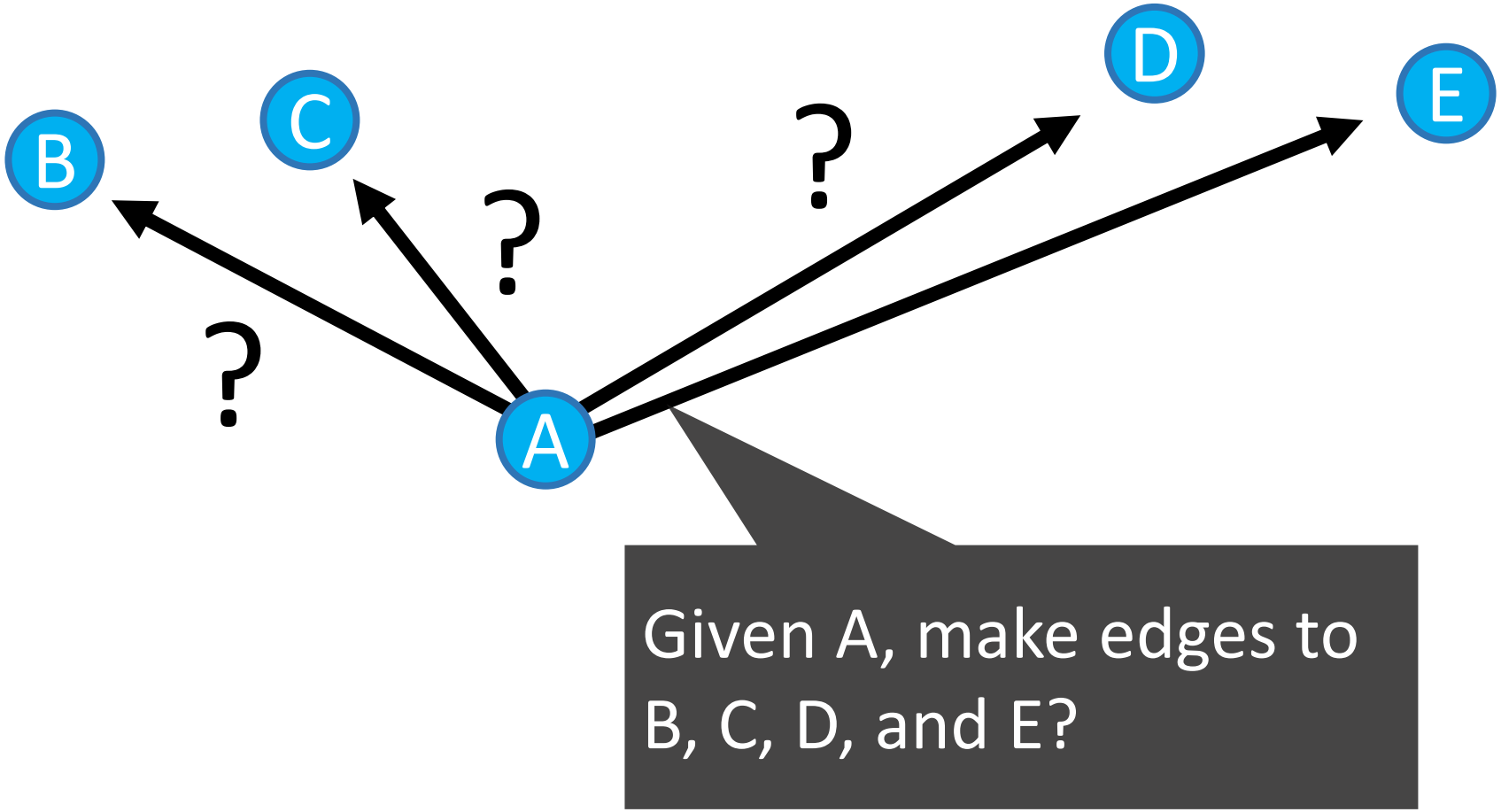


- Starting from a good seed ➡ Shorter path ➡ Faster search
- Finding a good seed is also an ANN problem
- Solve a small ANN problem by tree [NST; Iwasaki+, arXiv 18], hash [Effana; Fu+, arXiv 16] or LSH [LGTM; Arai+, DEXA 21]

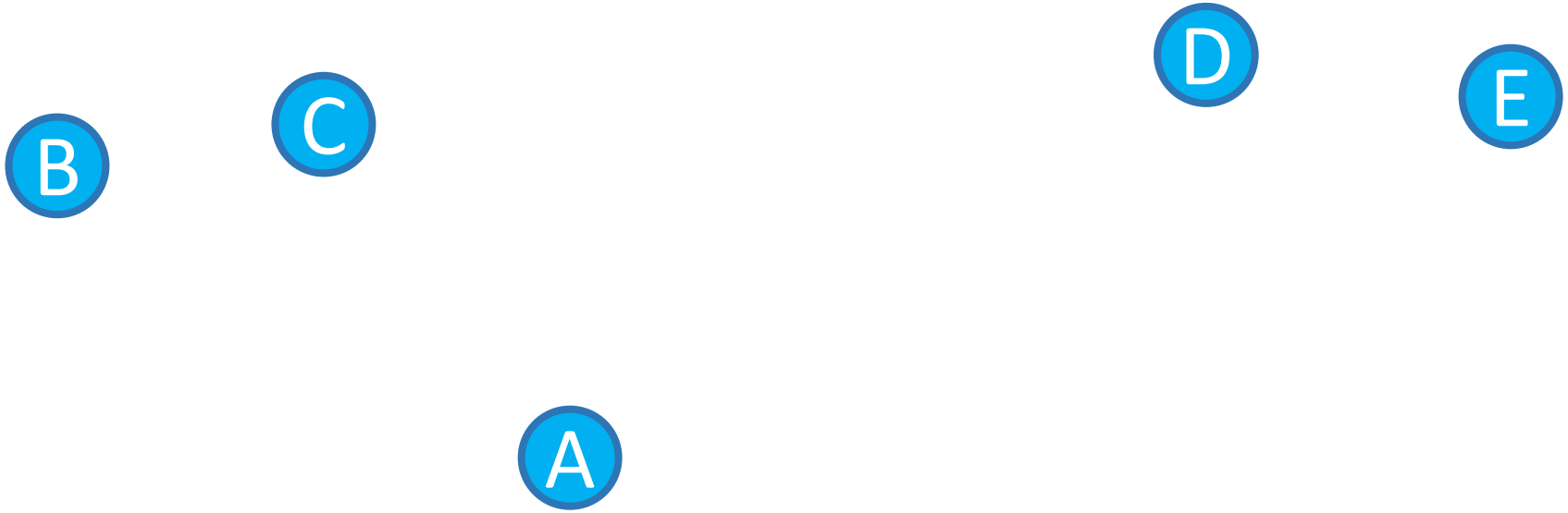
Edge selection: RNG-pruning



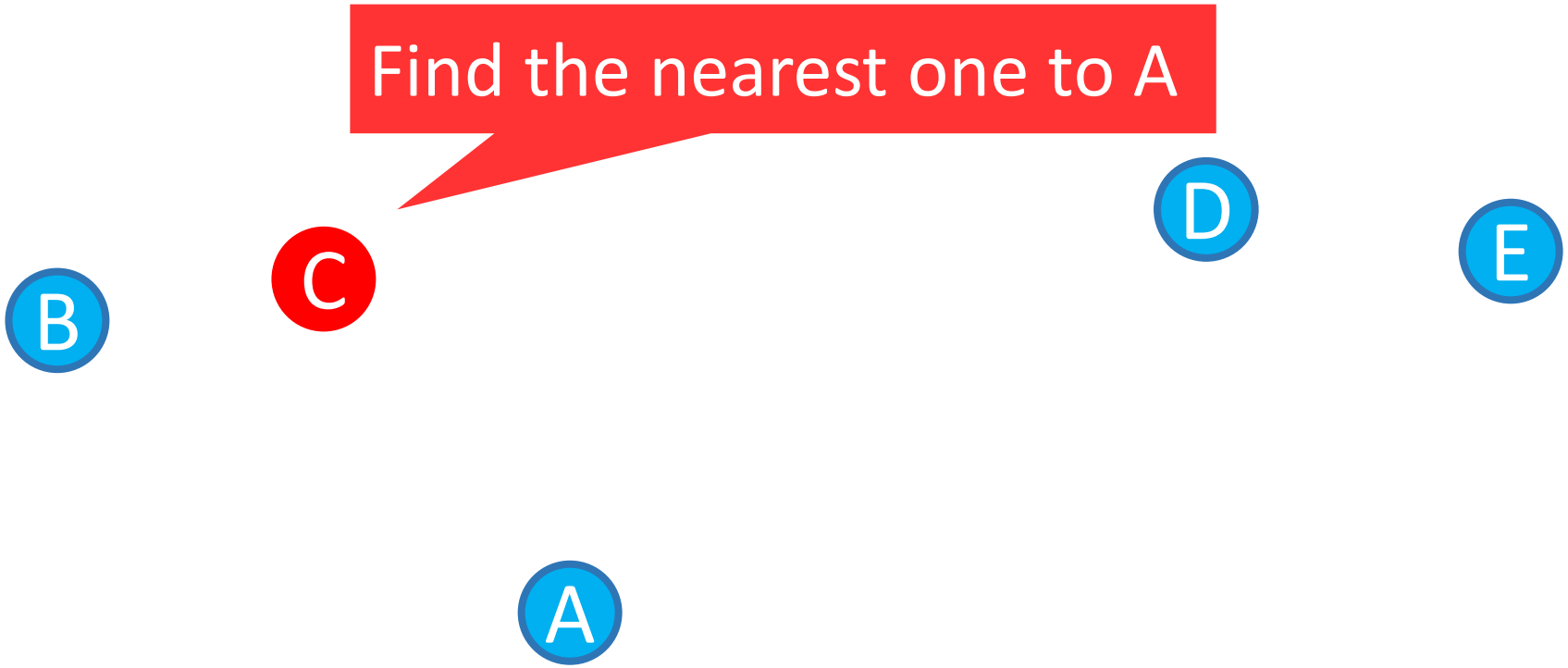
Edge selection: RNG-pruning



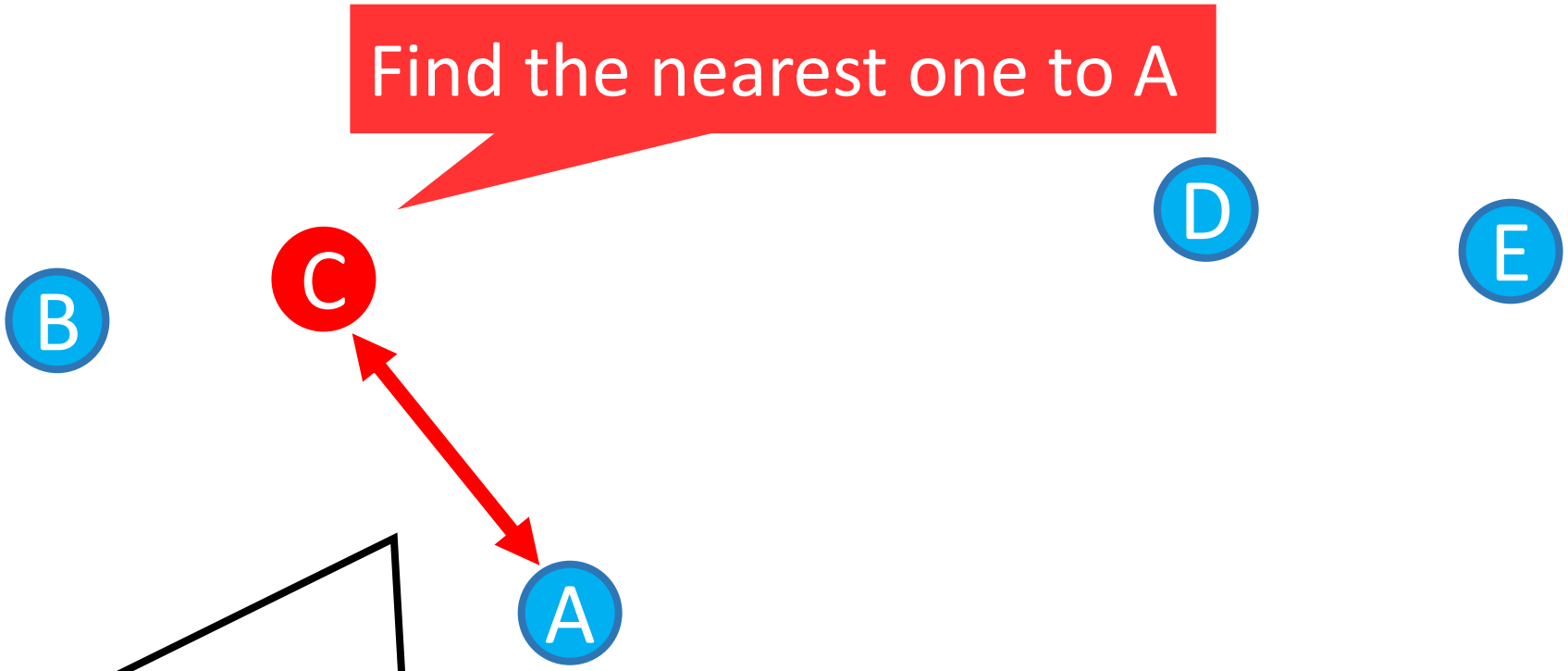
Edge selection: RNG-pruning



Edge selection: RNG-pruning

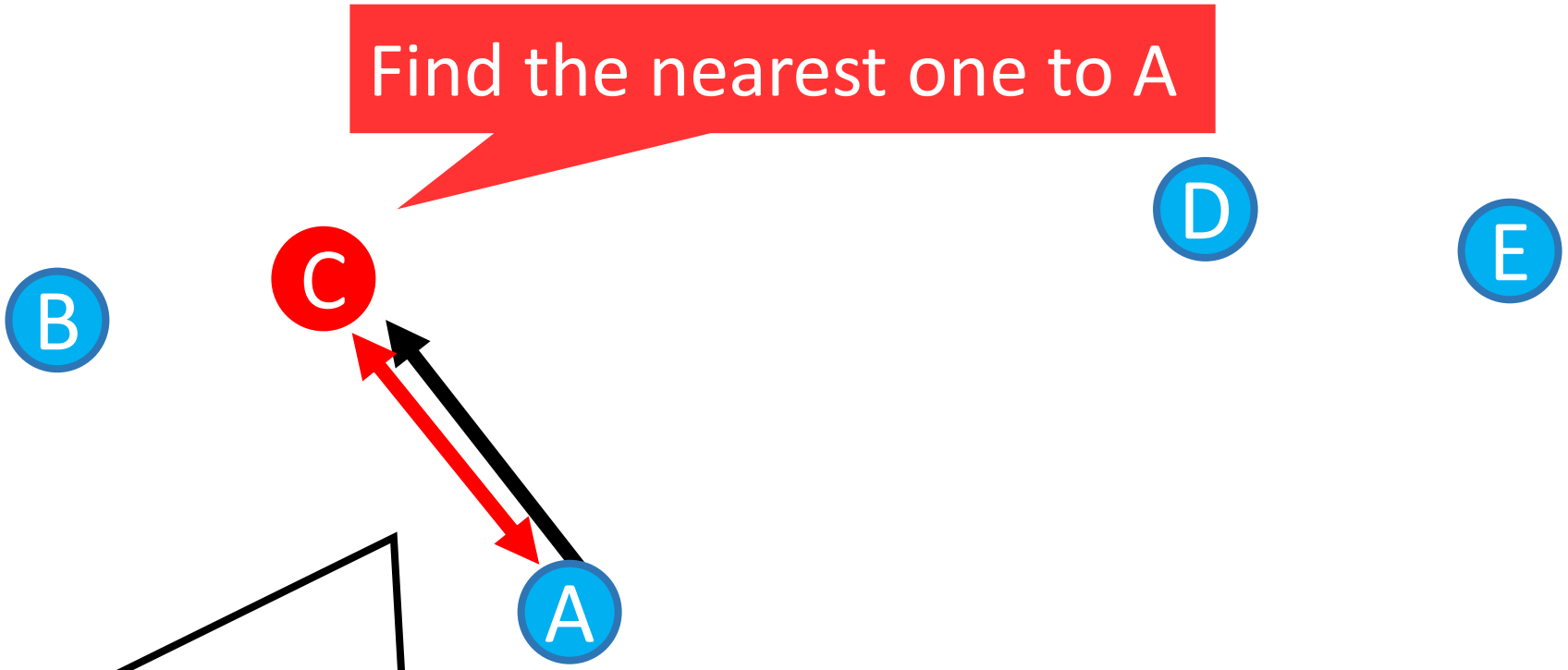


Edge selection: RNG-pruning



- For all neighbors of A, compare and
- If is the shortest, make an edge

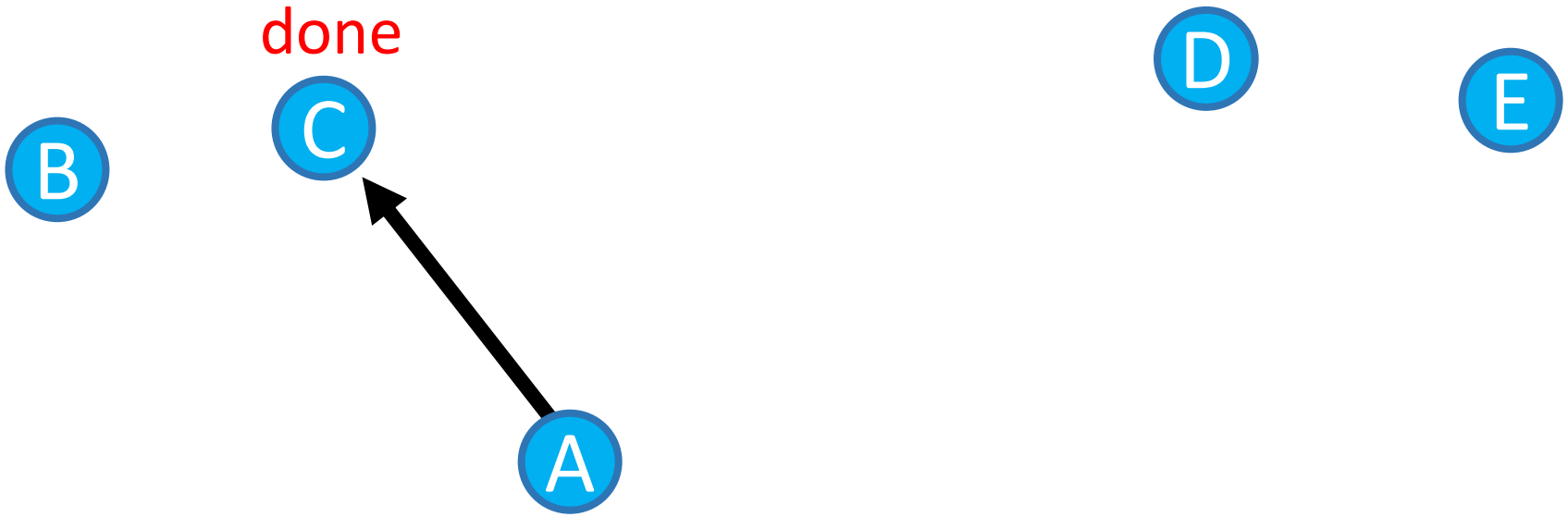
Edge selection: RNG-pruning



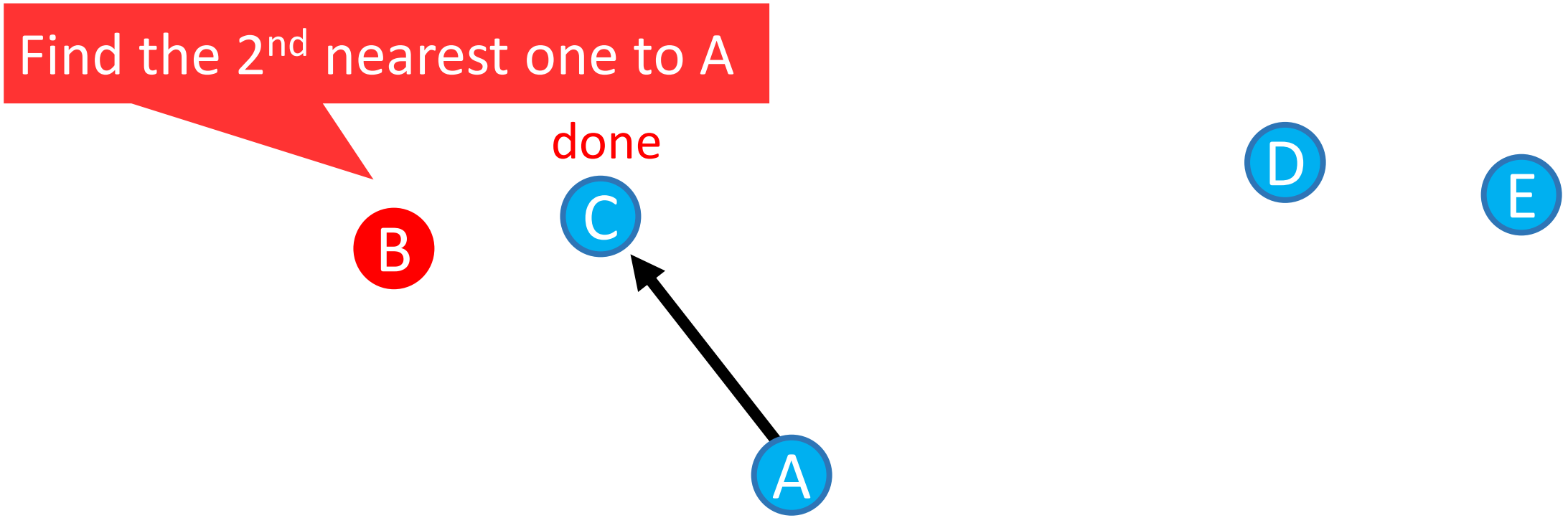
- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

This time, there are no neighbors. So let's make an edge

Edge selection: RNG-pruning

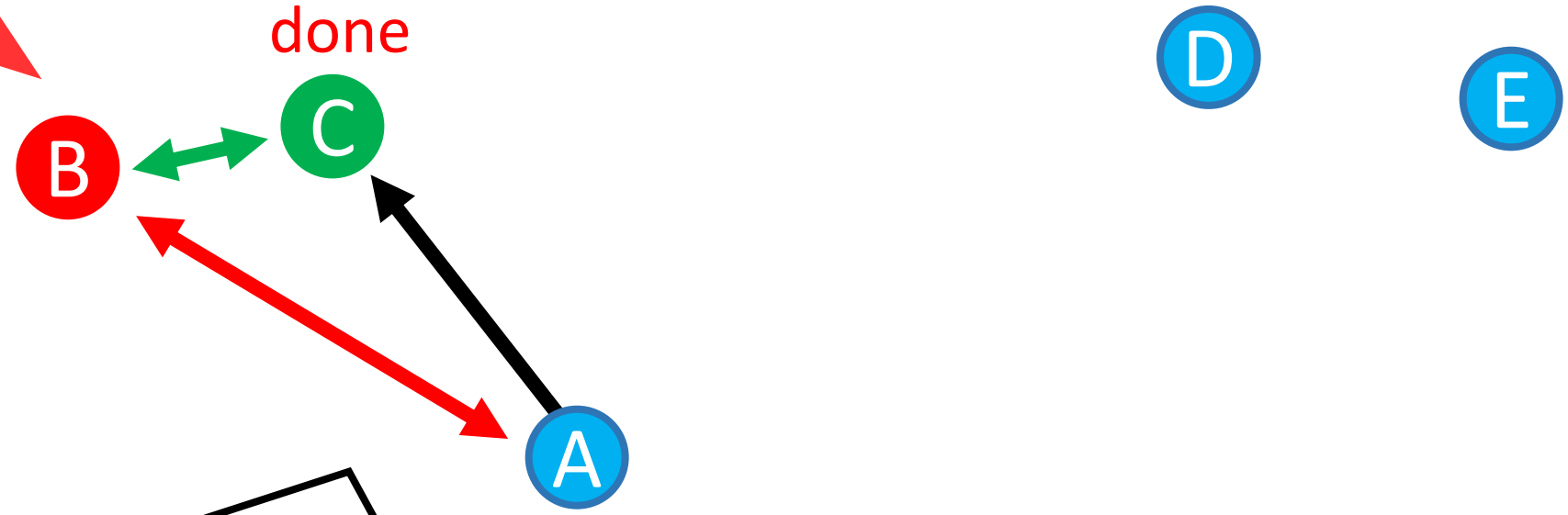


Edge selection: RNG-pruning



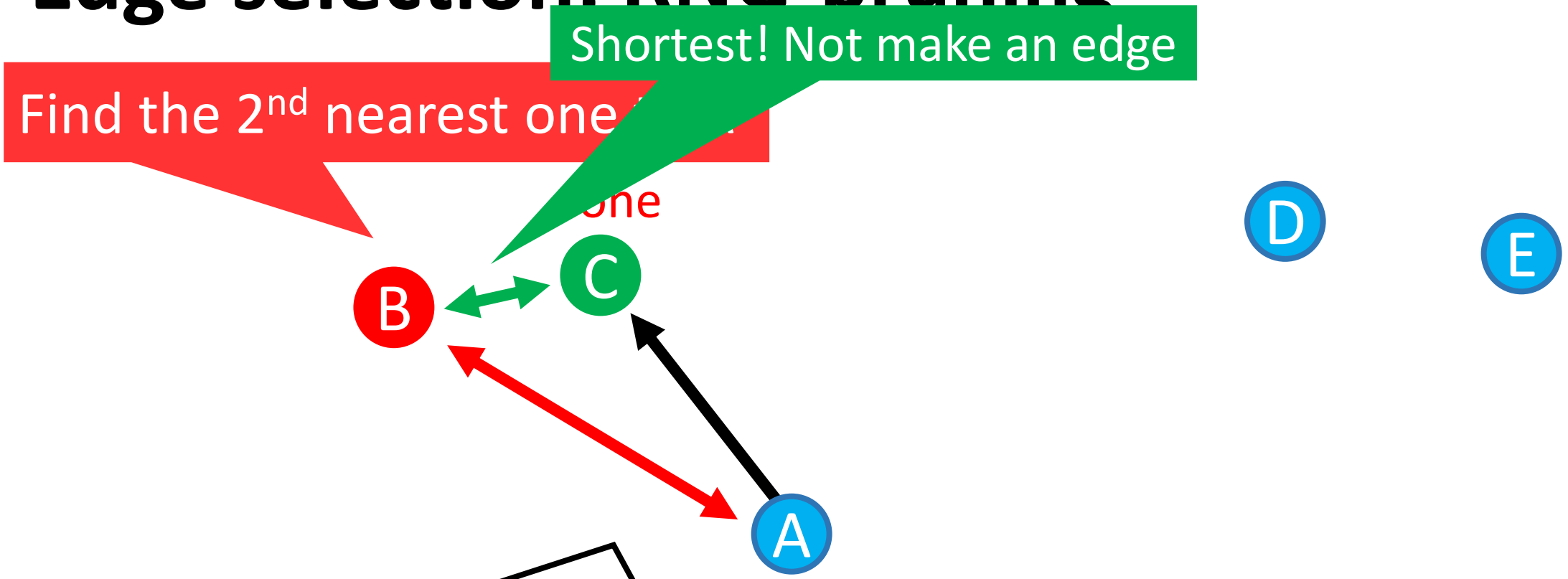
Edge selection: RNG-pruning

Find the 2nd nearest one to A



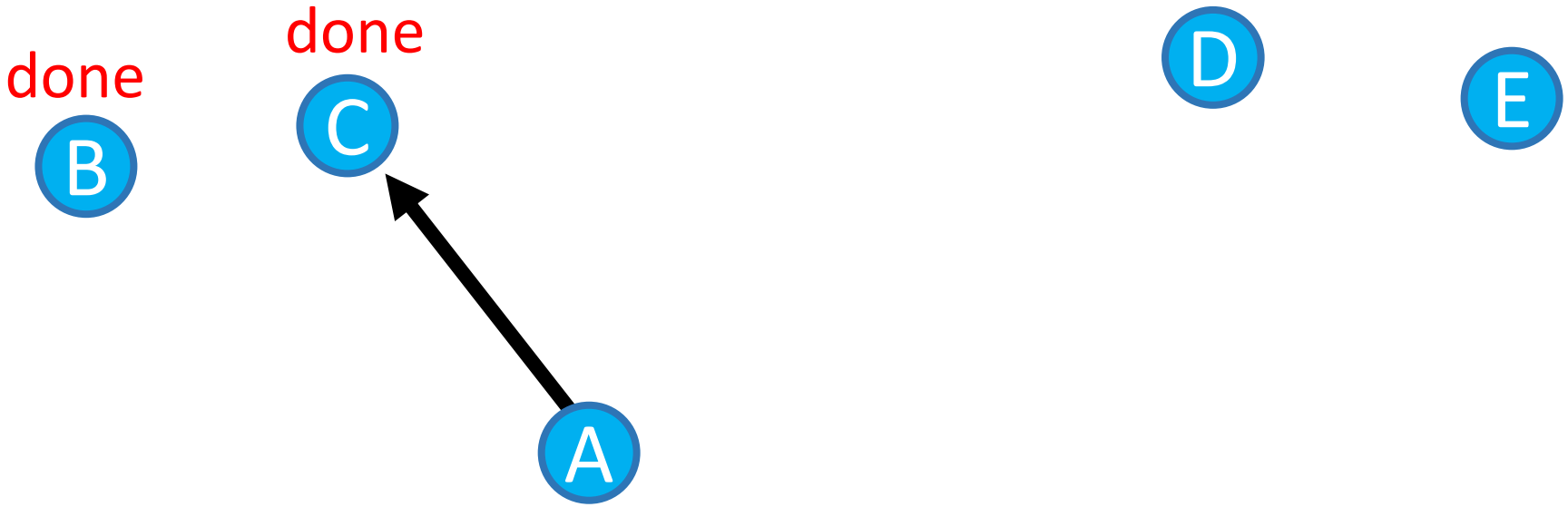
- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

Edge selection: RNG-pruning



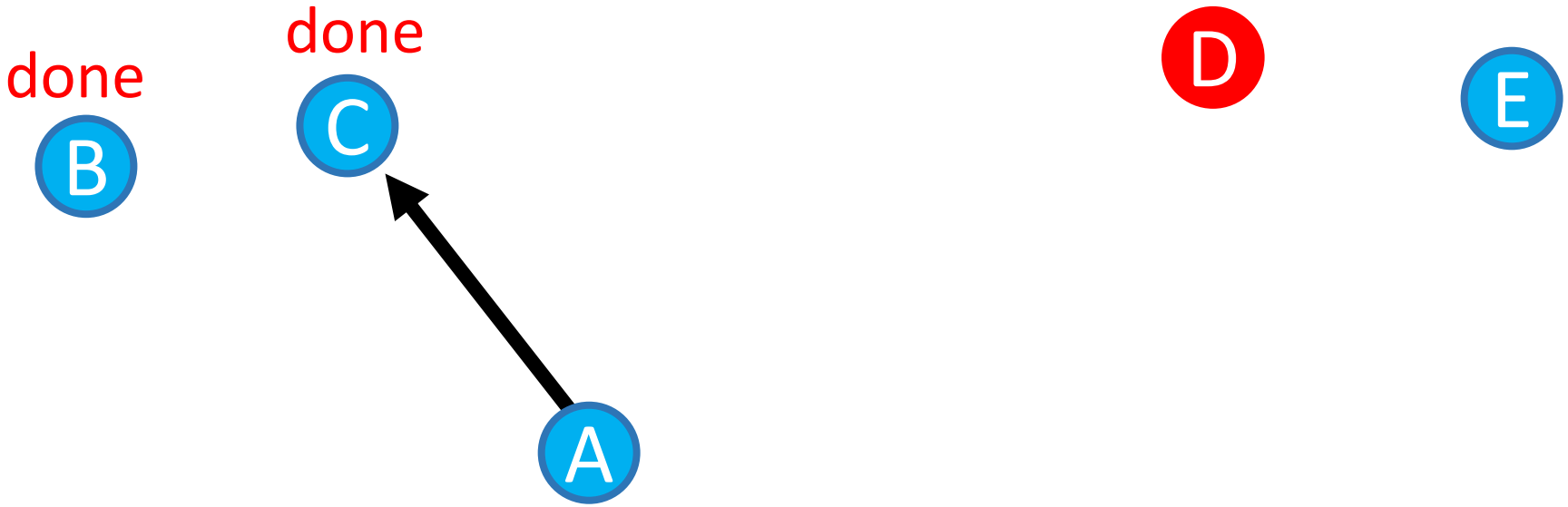
- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

Edge selection: RNG-pruning



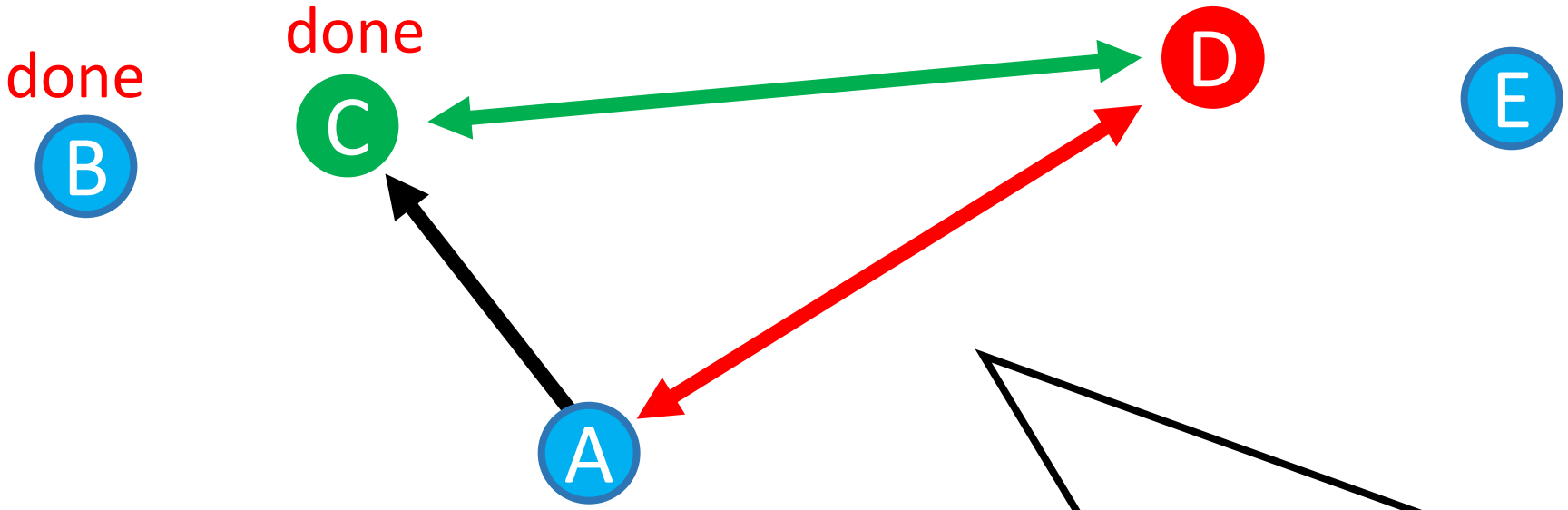
Edge selection: RNG-pruning

Find the 3rd nearest one to A



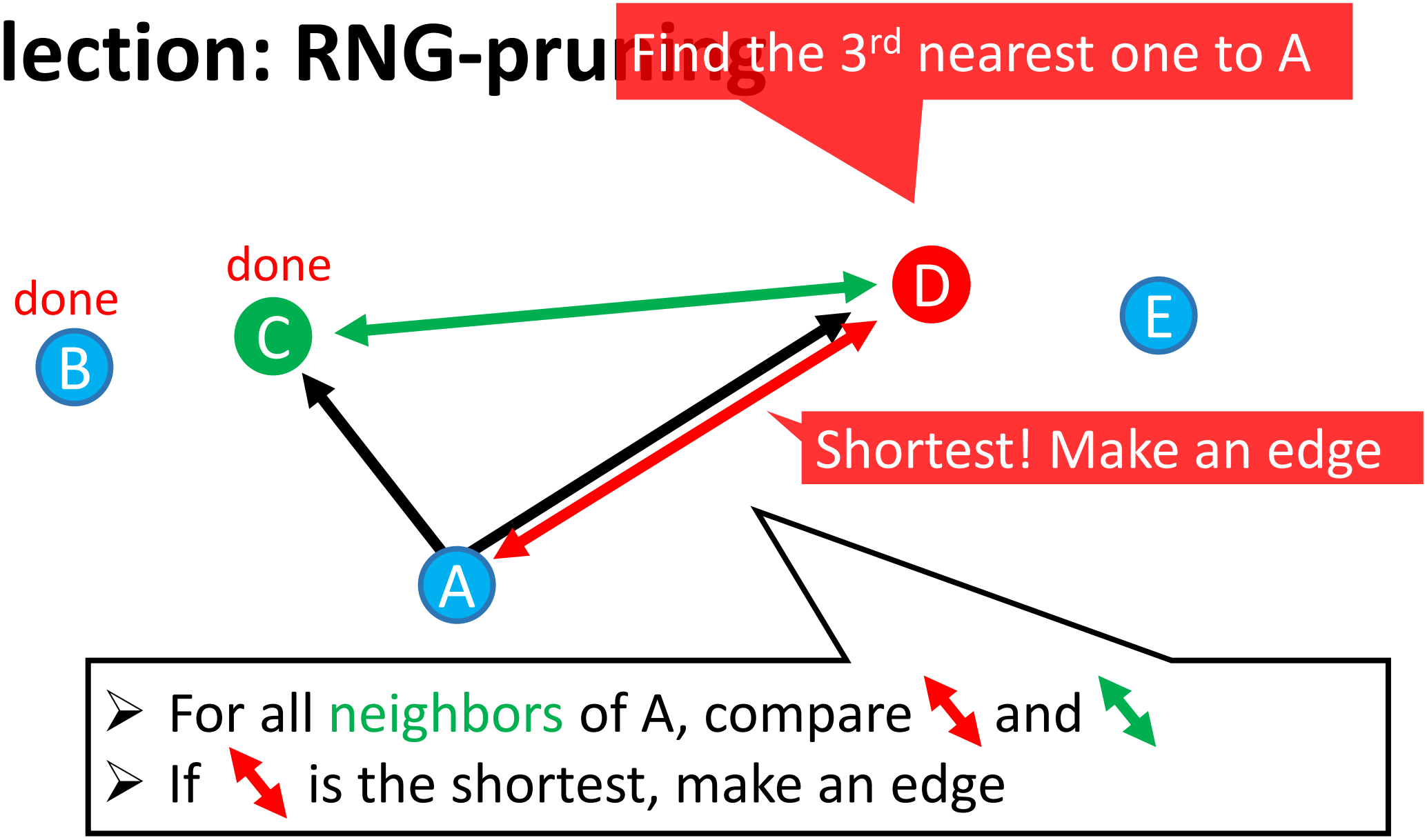
Edge selection: RNG-pruning

Find the 3rd nearest one to A

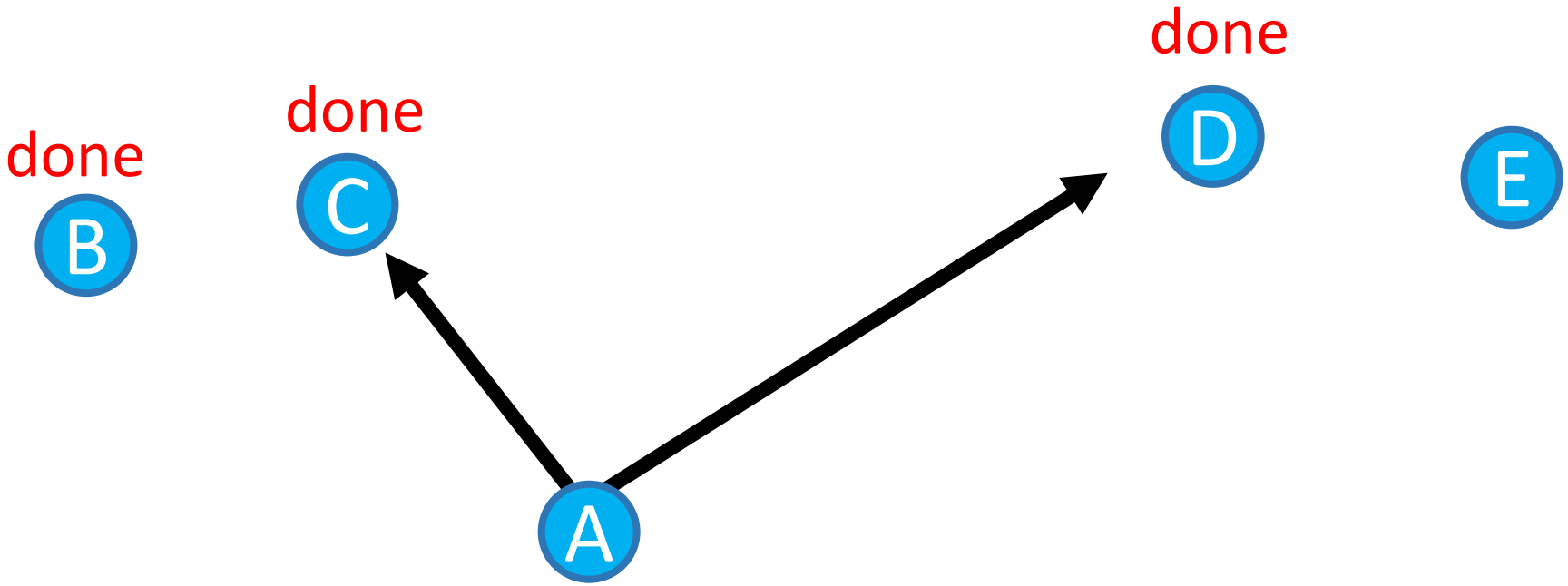


- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

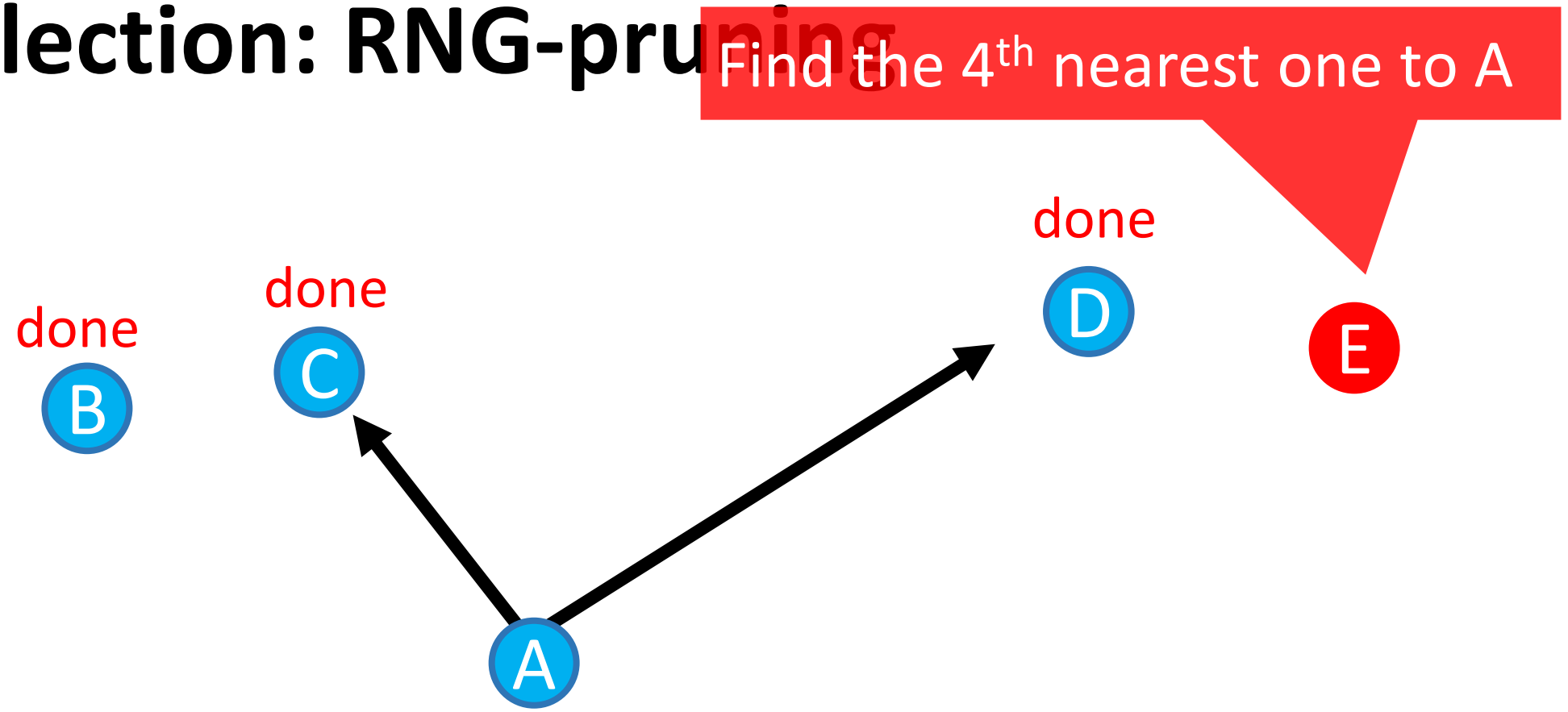
Edge selection: RNG-pruning



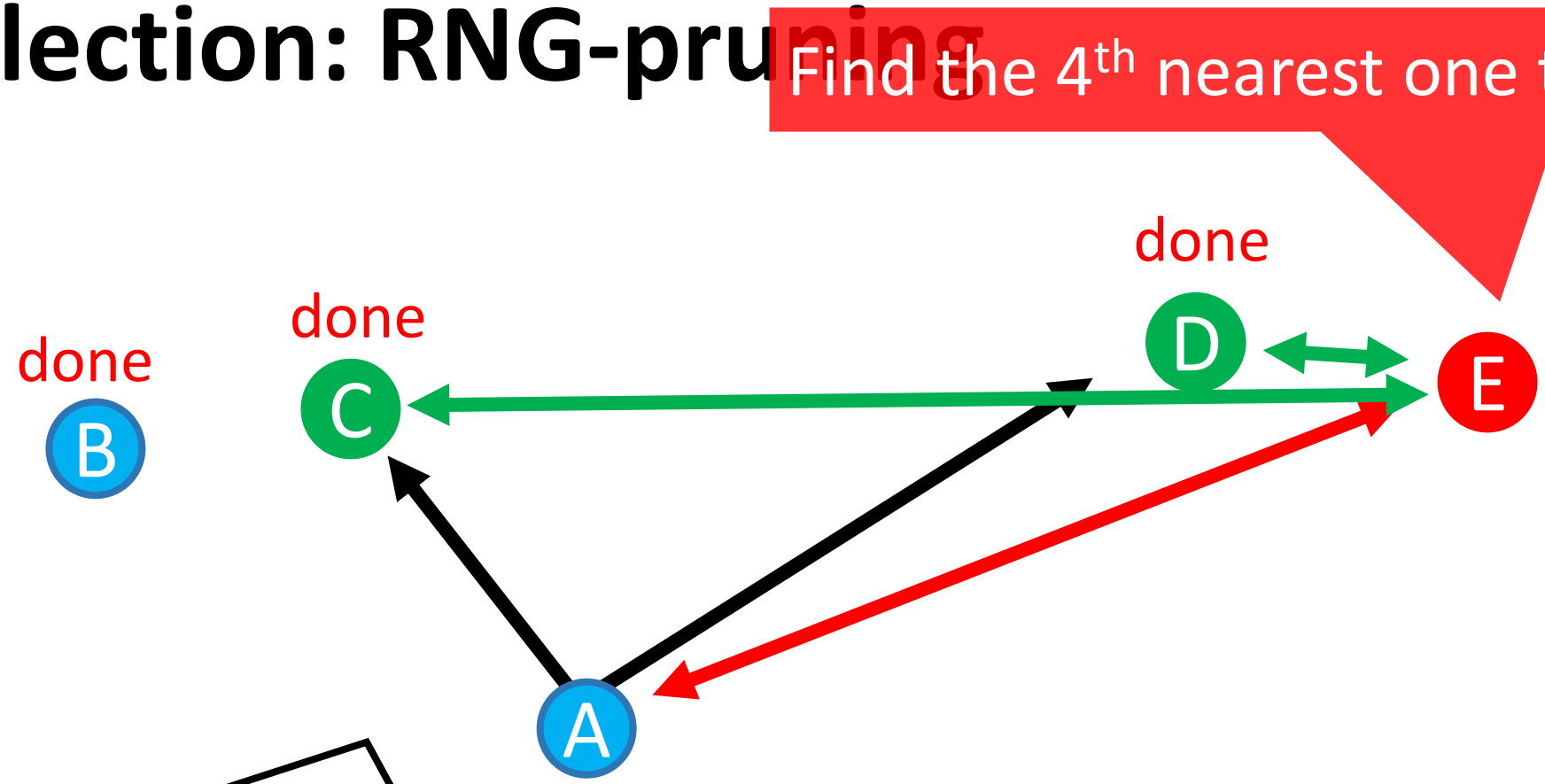
Edge selection: RNG-pruning



Edge selection: RNG-pruning

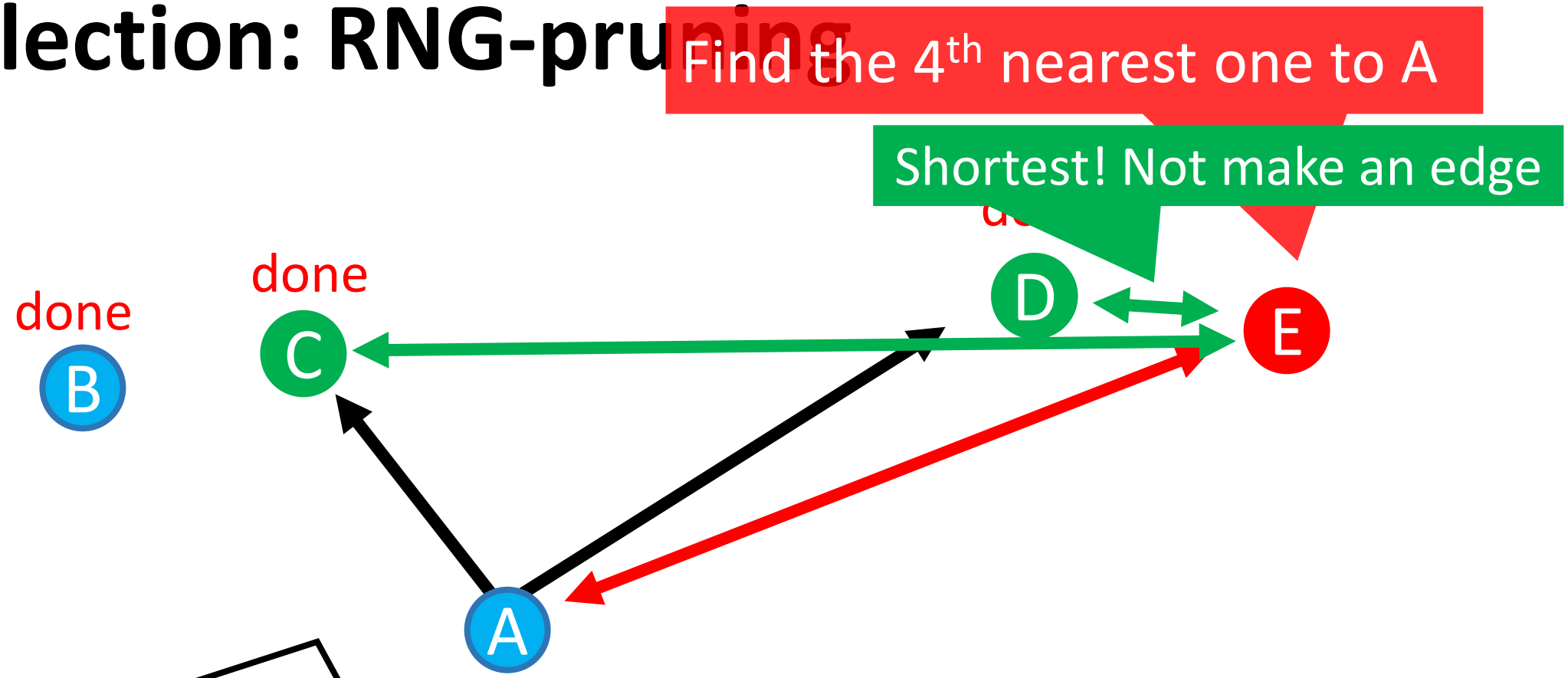


Edge selection: RNG-pruning



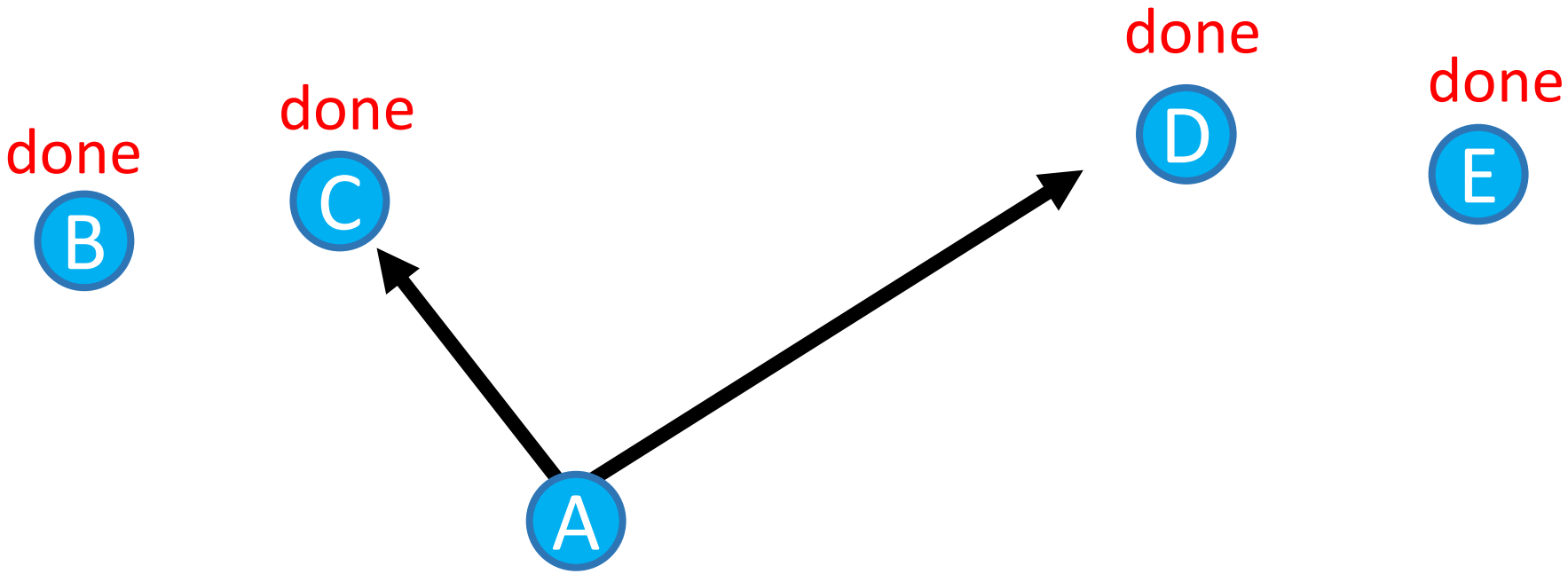
- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

Edge selection: RNG-pruning

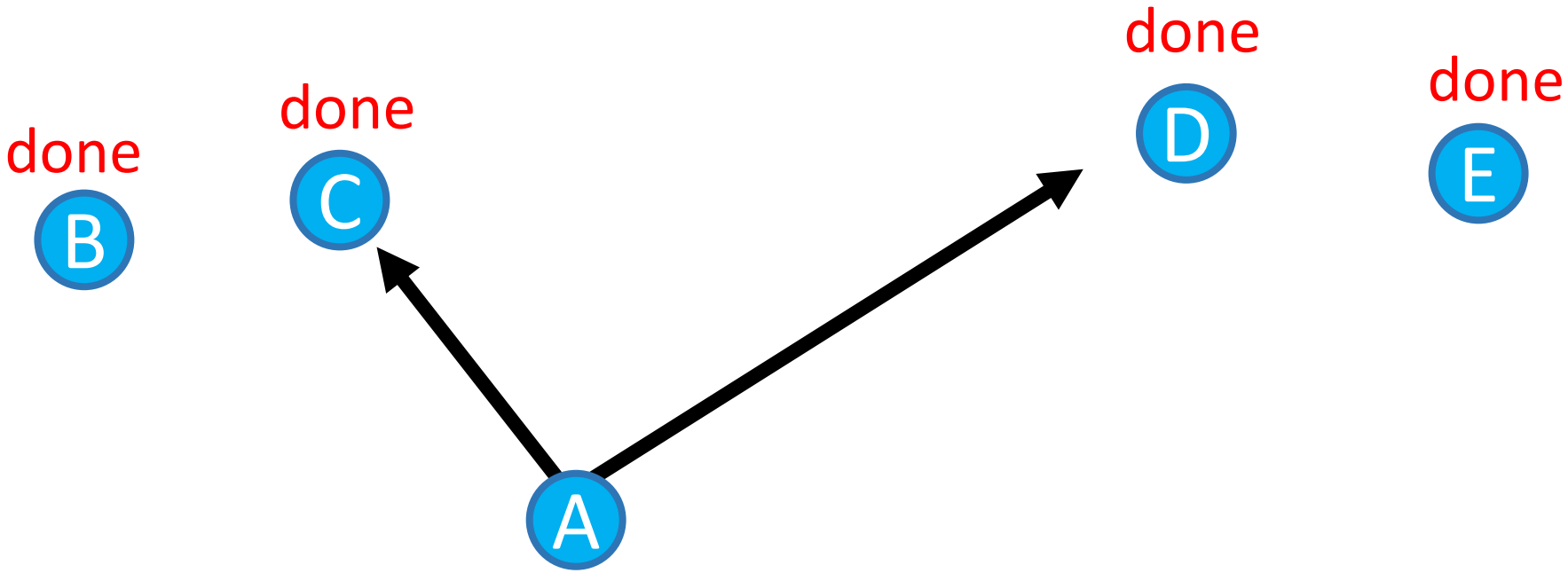


- For all neighbors of A, compare ↘ and ↘
- If ↘ is the shortest, make an edge

Edge selection: RNG-pruning



Edge selection: RNG-pruning



- RNG-pruning is an effective edge-pruning technique, and used in several algorithms

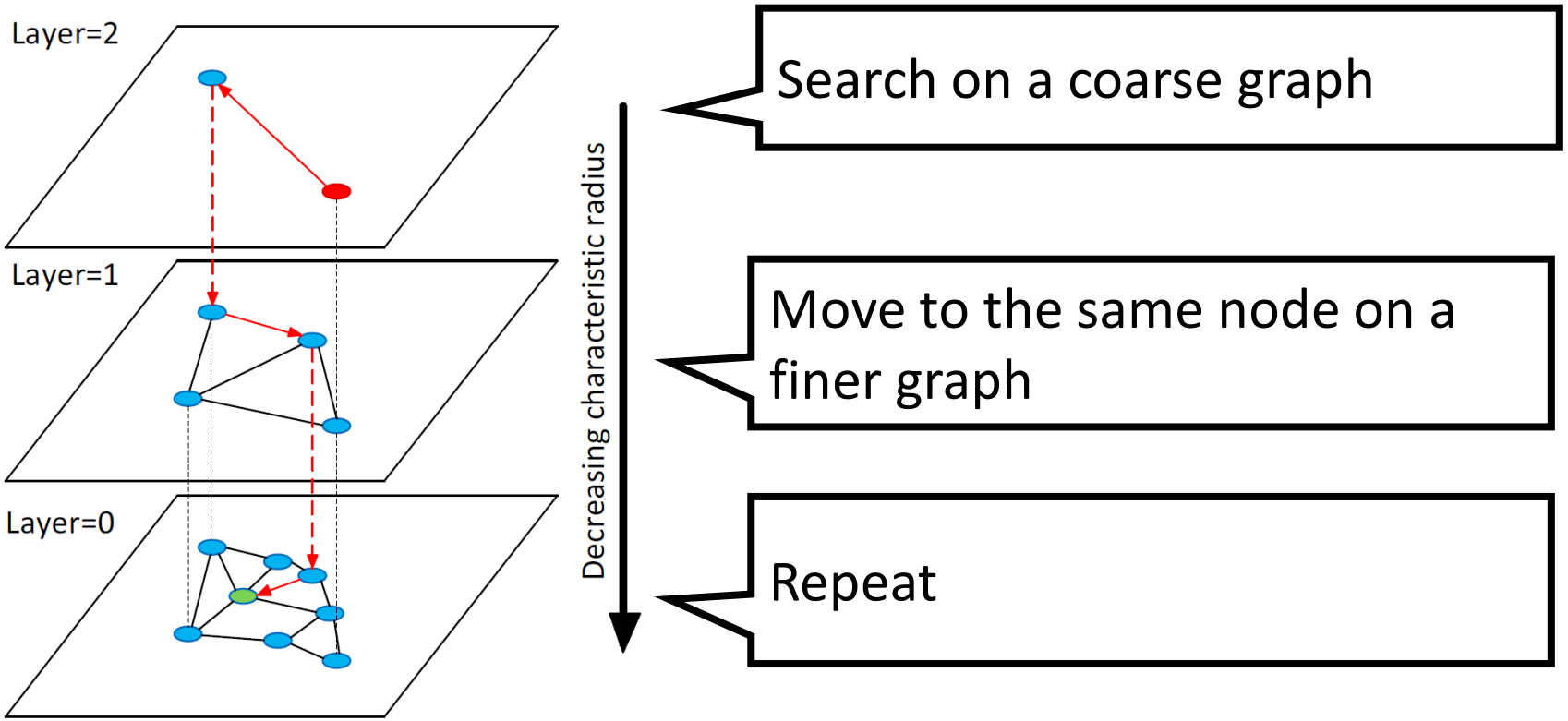
Pros: Implementation is easy

Cons: Require many distance computations

- Background
- Graph-based search
  - ✓ Basic (construction and search)
  - ✓ Observation
  - ✓ Properties
- Representative works
  - ✓ HNSW, NSG, NGT, Vamana
- Discussion

Hierarchical Navigable Small World; HNSW

- Construct the graph hierarchically [Malkov and Yashunin, TPAMI, 2019]
- Fix #edge per node by RNG-pruning
- The most famous algorithm; works very well in real world



[Malkov and Yashunin, TPAMI, 2019]

Hierarchical Navigable Small World; HNSW

- Used in various services
  - ✓ milvus, weaviate, qdrant, vearch, elasticsearch, OpenSearch, vespa, redis, Lucene…

- Three famous implementations
  - ✓ NMSLIB (the original implementation)
  - ✓ hnswlib (light-weight implementation from NMSLIB)
  - ✓ Faiss (re-implemented version by the faiss team)

[NMSLIB] https://github.com/nmslib/nmslib
[hnswlib] https://github.com/nmslib/hnswlib
[Faiss] https://github.com/facebookresearch/faiss/blob/main/faiss/IndexHNSW.h

Discussion from Faiss User Forum in FB

Note that this discussion was in 2020 and the libraries have been updated a lot since then

Any implementation difference between NMSLIB, hnswlib, and faiss-hnsw?

Yury Malkov (the author of HNSW paper)

My view on the implementation differences (I might forgot something):

1) nmslib's HNSW requires internal index conversion step (from nmslib's format to an internal one) to have good performance, and after the conversion the index cannot be updated with new elements. nmslib also has a simple "graph diversification" postprocessing after building the index (controlled by the "post" parameter) and sophisticated queue optimizations which makes it a bit faster compared to other implementations. Another advantage of nmslib is out-of-the box support for large collection of distance functions, including some exotic distances.

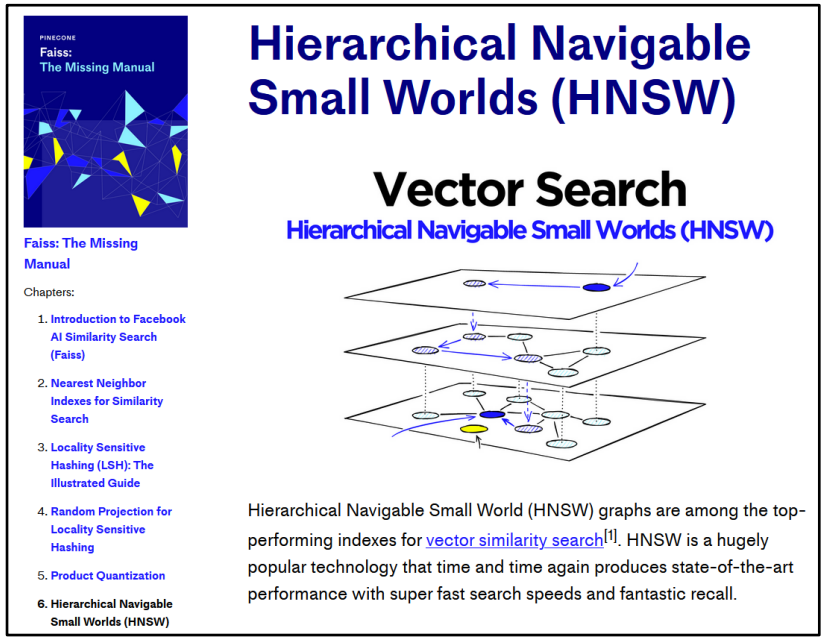
2) hnswlib is a header-only C++ library reimplementation of nmslib's hnsw. It does not have the index conversion step, thus - the Pros (compared to nmslib): much more memory efficient and faster at build time**.** It also supports index insertions, element updates (with incremental graph rewiring - added recently) and fake deletions (mark elements as deleted to avoid returning them during the graph traversal). Cons (compared to nnmslib): It is a tad slower than nmslib due to lack of graph postprocessing and queue optimization; out-of-the box version supports only 3 distance functions, compared to many distance functions in nmslib. Overall, I've tried to keep hnswlib as close as possible to a distributed index (hence no index postprocessing).

3) Faiss hnsw is a different reimplementation. It has its own algorithmic features, like having the first elements in the upper layers on the structure (opposed to random in other implementations). It is a bit more memory efficient compared to hnswlib with raw vectors and optimized for batch processing. Due to the latter it is noticeably slower at single query processing (opposed to nmslib or hnswlib) and generally a bit slower for batch queries (the last time I've tested, but there were exceptions). The implementation also supports incremental insertions (also preferably batched), quantized data and two-level encoding, which makes it much less memory hungry and the overall best when memory is a big concern.

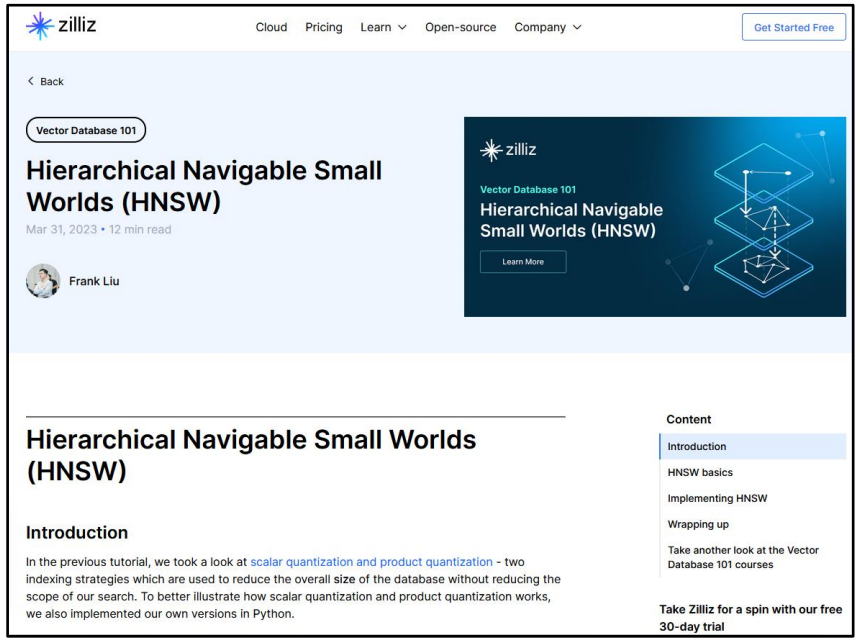
https://www.facebook.com/groups/faissusers/posts/917143142043306/?comment_id=917533385337615&reply_comment_id=920542105036743

Hierarchical Navigable Small World; HNSW

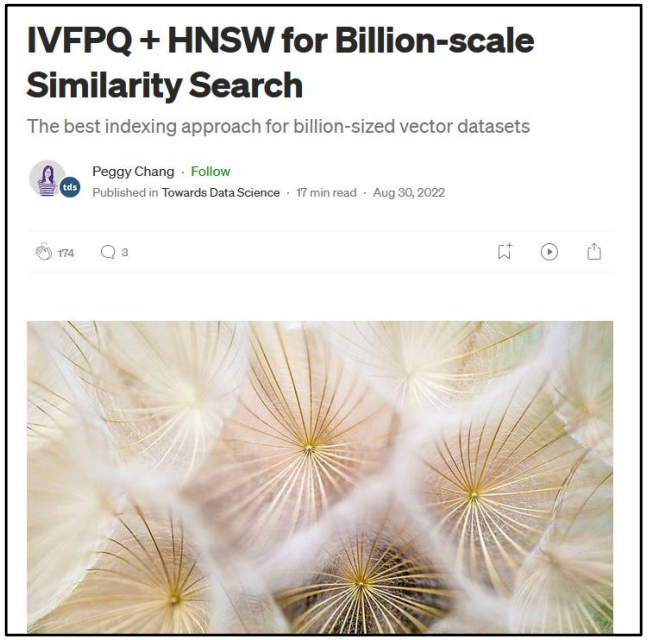
- See the following excellent blog posts for more details



https://www.pinecone.io/learn/hnsw/
James Briggs, PINECONE, Faiss: The Missing Manual, 6. Hierarchical Navigable Small Worlds (HNSW)



https://zilliz.com/blog/hierarchical-navigable-small-worlds-HNSW
Frank Liu, zilliz, Vector Database 101, Hierarchical Navigable Small Worlds (HNSW)



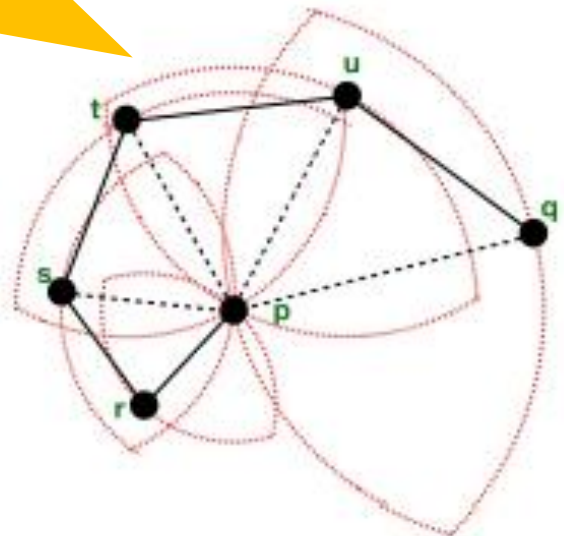
https://towardsdatascience.com/ivfpq-hnsw-for-billion-scale-similarity-search-89ff2f89d90e
Peggy Chang, IVFPQ + HNSW for Billion-scale Similarity Search

Navigating Spreading-out Graph (NSG) [Fu+, VLDB 19]

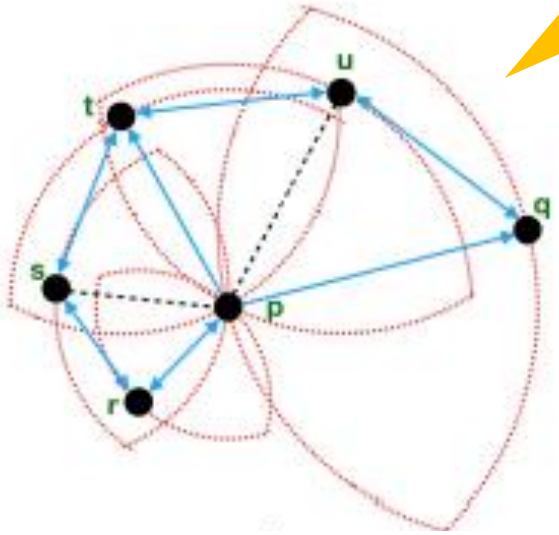
- Monotonic RNG
- In some cases, slightly better than HNSW
- Used in Alibaba's Taobao

- Recall the def. of RNG is "no point in a lune"
- The path "p -> q" is ling

Monotonic RNG can make more edges



RNG



Monotonic RNG

Images are from [Fu+, VLDB 19]

Navigating Spreading-out Graph (NSG) [Fu+, VLDB 19]

- The original implementation: https://github.com/ZJULearning/nsg

- Implemented in faiss as well
- If you're using faiss-hnsw and need a little bit more performance with the same interface, worth trying NSG

```
IndexHNSWFlat(int d, int M, MetricType metric)
IndexNSGFlat(int d, int R, MetricType metric)
```

Neighborhood Graph and Tree (NGT) [Iwasaki+, arXiv 18]

- Make use of range search for construction
- Obtain a seed via VP-tree

- Current best methods in ann-benchmarks are NGT-based algorithms
- Quantization is natively available

- Repository: https://github.com/yahoojapan/NGT
- From Yahoo Japan
- Used in Vald

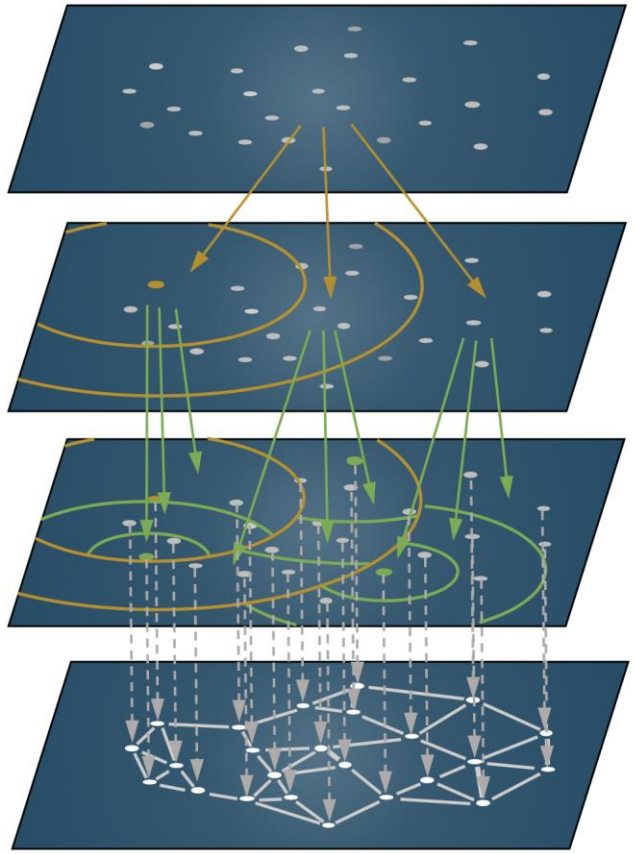
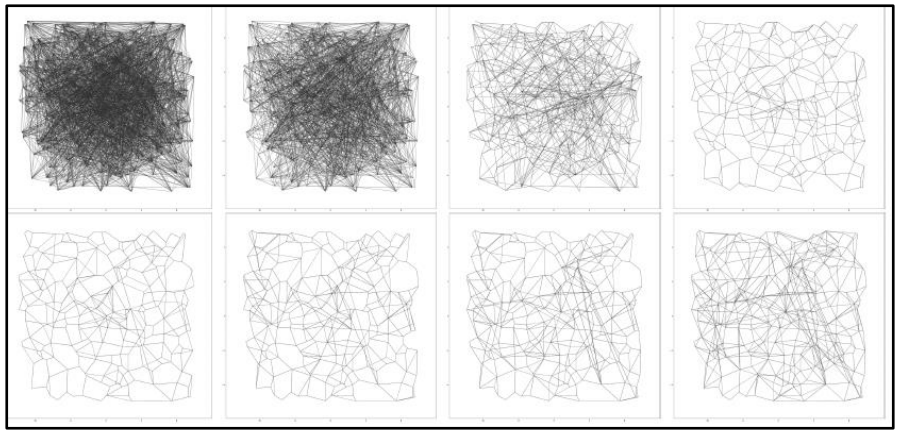


Image are from the original repository

DiskANN (Vamana) [Subramanya+, NeurIPS 19]

- Vamana: Graph-based search algorithm
- DiskANN: Disk-friendly search system using Vamana
- From MSR India https://github.com/microsoft/DiskANN



- Good option for huge data (not the main focus of this talk, though)
- The same team is actively developing interesting functionalites
  - ✓ Data update: FreshDiskANN [Singh+, arXiv 21]
  - ✓ Filter: Filtered-DiskANN [Gollapudi+, WWW 23]